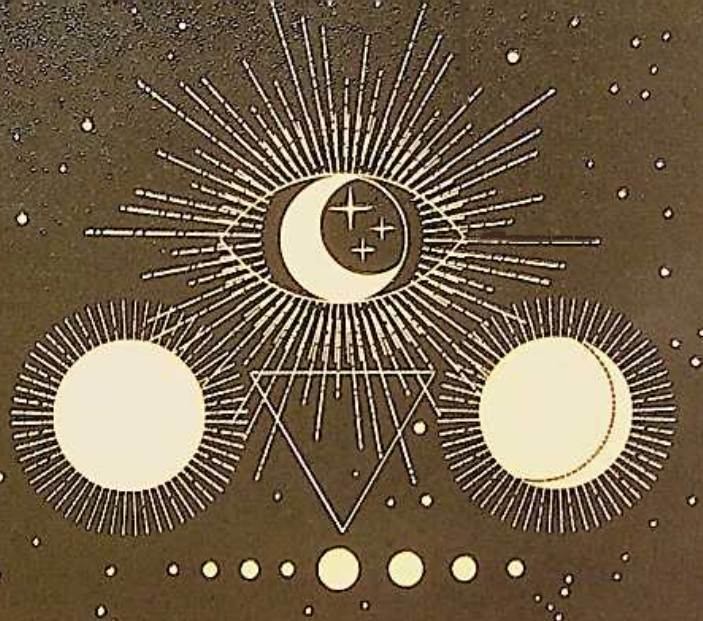




# DEEP LEARNING FOR COMPUTER VISION

컴퓨터 비전과 딥러닝

텐서플로와 케라스를 사용한 전문 가이드

라자링가파 산무가마니 지음 | 테크 트랜스 그룹 T4 옮김



| 지론이 |

## 라자링가파 산무가마니 Rajalingappa Shanmugamani

현재 SAP 싱가포르에서 딥러닝 분야의 리더로 일하고 있다. 이전에는 컴퓨터 비전 제품 개발을 위해 여러 신생 기업에서 근무하고 컨설팅해왔다. 인도 마드라스 공과 대학(Indian Institute of Technology, Madras)에서 석사 학위를 받았으며, 제조 분야의 컴퓨터 비전 애플리케이션 산업에 관한 논문을 저술했다. 저널 및 콘퍼런스에서 동료 논문 검토를 했으며, 머신 러닝 분야에서 몇몇 특허를 보유했다. 여기에는 프로그래밍과 머신 러닝을 학생과 엔지니어에게 가르친다.

| 옮긴이 |

## 테크 트랜스 그룹 T4

최신 IT 테크놀로지에 대한 리서치를 목적으로 하는 스터디 그룹이다. 엔터프라이즈 환경에서 오픈소스를 활용해 프레임워크를 구축하는 데 관심이 많으며, 스프링, React.js, Node.js, OpenCV, ML 등의 기술에 주목하고 있다. 오픈소스 기반의 플랫폼 개발 및 활용도 주요 관심 분야다. 에이콘출판사에서 펴낸 'OpenCV를 위한 머신 러닝'(2017), '추천 엔진을 구축하기 위한 기본서'(2017) 등을 번역했다.

# 컴퓨터 비전과 딥러닝

이 책은 컴퓨터 비전과 딥러닝의 기본 개념과 응용 기술을 소개합니다.

본 책의 주요 내용은 다음과 같습니다.

1. 컴퓨터 비전의 기본 개념과 응용 기술

본 책의 주요 내용은 다음과 같습니다.

1. 컴퓨터 비전의 기본 개념과 응용 기술

2. 딥러닝의 기본 개념과 응용 기술

본 책의 주요 내용은 다음과 같습니다.

1. 컴퓨터 비전의 기본 개념과 응용 기술



Korean edition copyright © 2018 by acom publishing Co. All rights reserved.

Copyright © Packa Publishing 2018.  
First published in the English language under the title  
'Deep Learning for Computer Vision - (9781788295628)'

---

이 책은 Packa Publishing과 에이콘출판(주)가 정식 계약하여 번역한 책이므로  
이 책의 일부나 전체 내용을 무단으로 복사, 복제, 전재하는 것은 저작권법에 저촉됩니다.



# 컴퓨터 비전과 딥러닝

텐서플로와 케라스를 사용한 전문 가이드

라자링가파 산무가미니 지음

테크 트랜스 그룹 T4 옮김

# THE NEW YORK PUBLIC LIBRARY

ASTOR LENOX TILDEN FOUNDATION

500 FIFTH AVENUE

NEW YORK 10017

Acquired from the collection of the New York Public Library

Acquired from the collection of the New York Public Library

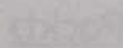
Acquired from the collection of the New York Public Library

Acquired from the collection of the New York Public Library

Acquired from the collection of the New York Public Library

Acquired from the collection of the New York Public Library

Acquired from the collection of the New York Public Library



딥러닝(deep learning)은 인공지능에 혁명을 일으키고 있으며, 앞으로 수십 년 동안 강렬하게 세상을 바꿀 기술이다. 딥러닝을 기반으로 한 인공지능은 산업혁명과 비슷한 수준의 변화를 가져올 수 있다. 딥러닝은 미래에 대한 흥분과 더불어 두려움을 만들어냈다. 그렇지만 딥러닝은 현실상에서 산업혁명 및 기계와 마찬가지로 산업 생산성을 향상시키고 인류의 생활 수준을 높여줄 것이다.

물론, 딥러닝은 직업을 대체하기보다는 더욱 양질의 일자리를 더 많이 만들어낼 것이다. 이것이야말로 이 책이 중요하고 시기적절한 이유이기도 하다. 이 책의 독자들은 컴퓨터 비전을 위한 딥러닝에 대해 파악하게 되고, 딥러닝의 강력한 힘과 많은 애플리케이션에 대해 알게 될 것이다. 이 책은 앞으로 수십 년 동안 급속도로 성장할 거대 산업에 대한 기초 내용을 독자들에게 제공할 것이다.

라자링가파 산무가마니는 컴퓨터 비전과 관련된 여러 프로젝트에서 나와 함께 작업했던 훌륭한 연구원이다. 그는 의류 검색을 위한 복잡한 컴퓨터 비전 및 딥러닝 시스템을 설계하고 제공한 수석 엔지니어였으며, 해당 프로젝트는 실제로도 크게 성공을 거뒀다. 그의 강점 중 하나는 복잡한 문제에 대한 최첨단 연구 내용을 실제 상황에 적용할 수 있다는 것이며, 복잡한 아이디어를 분해하고 간단하게 설명할 수 있다. 라자링가파는 훌륭한 직업 윤리를 가지고 있으며 매우 야심 차다. 그는 이 책을 통해 딥러닝을 사용하는 컴퓨터 비전의 최신 기술 관련 내용을 정리했으며, 이는 결코 아무나 해낼 수 없는 과정이었다. 이 책은 라자링가파의 큰 결과물들을 정리한 것이며, 독자들은 앞으로 수년 동안 이 결과물을 잘 사용함으로써 많은 혜택을 얻게 될 것이다.

스티븐 무어 박사  
싱가포르 이모션리더 최고 기술 책임자(CTO)



## | 지은이 소개 |

### 라자링가파 산무가마니 Rajalingappa Shanmugamani

현재 SAP 싱가포르에서 딥러닝 분야의 리더로 일하고 있다. 이전에는 컴퓨터 비전 제품을 개발하기 위해 여러 신생 기업에서 근무하고 컨설팅해왔다. 인도 마드라스 공과 대학 Indian Institute of Technology, Madras에서 석사 학위를 받았으며, 제조 분야의 컴퓨터 비전 애플리케이션 산업에 관한 논문을 저술했다. 여러 저널 및 콘퍼런스에서 동료 논문 검토를 했으며, 머신 러닝 분야에서 몇몇 특허를 보유했다. 여가를 활용해 프로그래밍과 머신 러닝을 학생과 엔지니어에게 가르친다.

---

아내 에질(Ezhi)과 가족, 친구들의 도움에 감사한다. 많은 가르침을 주신 선생님, 동료, 관리자. 멘토 분들에게도 감사한다. 또한 이 책의 그래픽을 담당한 진오이(Jean Ooi)에게도 감사의 마음을 전한다.

---

## | 기술 감수자 소개 |

**니산스 코간티** Nishanth Koganti

인도 조드푸르 공과 대학 Indian Institute of Technology, Jodhpur에서 전기공학 학사 학위를 받았고, 일본의 NIST Nara Institute of Science and Technology에서 정보과학 석사, 박사 학위를 받았다. 현재 도쿄 대학 University of Tokyo의 박사후 연구원이며, 연구 관심 분야는 보조 로봇, 운동 기술 학습, 머신 러닝이다. 대학원에서는 노인들이 옷을 쉽게 입도록 돕는 의류 원호 로봇 개발에 관한 연구를 진행했다.

## | 옮긴이 소개 |

테크 트랜스 그룹 T4(greg\_kim1002@naver.com)

최신 IT 테크놀로지에 대한 리서치를 목적으로 하는 스터디 그룹이다. 엔터프라이즈 환경에서 오픈소스를 활용해 프레임워크를 구축하는 데 관심이 많으며, 스프링<sup>Spring</sup>, React.js, Node.js, OpenCV, ML 등의 기술에 주목하고 있다. 오픈소스 기반의 플랫폼 개발 및 활용도 주요 관심 분야다. 에이콘출판사에서 펴낸 『OpenCV를 위한 머신 러닝』(2017), 『추천 엔진을 구축하기 위한 기본서』(2017) 등을 번역했다.

컴퓨터 비전은 컴퓨터에 시각을 부여해 이미지를 분석함으로써 유용한 정보를 생성하는 기술이다. 비전 기술은 컴퓨터나 로봇 등을 통해 얼굴, 건물 등의 다양한 객체를 인식하는데 응용되며, 인공지능 기술이 발전하면서 객체 인식 기술의 진화 속도는 빨라지고 있다.

컴퓨터가 사물을 정확하게 인식해 유용한 정보를 제공할 수 있게 되면서 컴퓨터 인터페이스에 변혁이 일어나고 있다. 아이폰 X에는 얼굴을 인식하는 기능인 페이스 ID가 탑재됐고, 스마트폰의 카메라를 이용한 신용카드 스캔 등의 기술이 실제로 적용되고 있다.

컴퓨터 비전을 구현하기 위한 사용자의 프로젝트에서 컴퓨터 비전 알고리즘을 시작하길 원하더라도, 어디서부터 시작해야 할지 다소 막막할 수 있다. 컴퓨터 비전 엔지니어라 해도, 심도 있게 접근하거나 더 많은 내용을 배우려면 많은 기술을 알아야 한다. 이와 같은 목적을 달성하려면 실전 경험이 가장 중요하다. 실제 문제를 해결하는 방법을 통해 현존하는 방법들을 사용자의 요구 사항에 맞춰 수행해야 하며, 샘플 코드들을 통해 컴퓨터 비전 알고리즘의 가능성과 한계를 충분히 이해해 나가야 한다.

이 책은 실제 컴퓨터 비전 작업을 할 때 직접적으로 큰 도움이 될 수 있도록 구성됐다. 텐서플로<sup>TensorFlow</sup>를 사용해 추론, 제품 검색을 위한 시각적 기능의 제공 및 다양한 사용 방법을 학습할 수 있고, 유사도 매칭<sup>similarity matching</sup>과 얼굴 인식을 위한 모델을 훈련시키는 방법도 배울 수 있다. 이러한 내용들을 다룬 이 책은 딥러닝 학습을 위한 많은 내용들을 담고 있으며 이해하기 쉽게 예제 코드와 그 결과물까지 한꺼번에 보여준다. 컴퓨터 비전 학습을 위한 시맨틱 분할<sup>semantic segmentation</sup>, 유사도 학습<sup>similarity learning</sup>, 이미지 캡션<sup>image caption</sup>, 생성 모델<sup>generative model</sup>, 동영상 분류<sup>video classification</sup>에 대해 알기 쉽게 설명해주므로 데이터 기반 컴퓨터 비전 기술에 대한 지식을 한 번에 쉽게 습득할 수 있다.

모든 독자들이 텐서플로/케라스<sup>Keras</sup>와 딥러닝 등에 대한 기본 이론을 이해하고 실제로 구현하는데 많은 도움이 되길 진심으로 바란다. 각각의 심오한 주제와 관련된 내용 전체를 하나씩 천천히 끄집어낸 후 설명하고 있으므로 순서대로 읽다 보면 전반적인 내용을 충분히 파악할 수 있을 것이다.

이 책이 나오기까지 주변에서 묵묵히 많은 도움을 주신 멤버와 가족들에게 감사하며, 번역을 진행하는 과정에서 든든한 버팀목이 돼주신 출판사 관계자 분들께도 감사의 말씀을 드리고 싶다. 같이 고민해주고 처음부터 끝까지 살펴봐주신 편집자 분께도 고마운 마음을 전하고 싶다.



## | 차례 |

|                      |           |
|----------------------|-----------|
| 추천의 글 .....          | 5         |
| 지은이 소개 .....         | 6         |
| 기술 감수자 소개 .....      | 7         |
| 옮긴이 소개 .....         | 8         |
| 옮긴이의 말 .....         | 9         |
| 들어가며 .....           | 21        |
| <br>                 |           |
| <b>1장 시작하기</b> ..... | <b>27</b> |
| 딥러닝 이해하기 .....       | 28        |
| 퍼셉트론 .....           | 28        |
| 활성화 함수 .....         | 29        |
| 시그모이드 함수 .....       | 29        |
| 쌍곡탄젠트 함수 .....       | 30        |
| ReLU 함수 .....        | 31        |
| 인공 신경망 .....         | 32        |
| 원-핫 인코딩 .....        | 33        |
| 소프트맥스 .....          | 34        |
| 교차 엔트로피 .....        | 34        |
| 드롭아웃 .....           | 34        |
| 배치 정규화 .....         | 35        |
| L1 및 L2 정규화 .....    | 35        |
| 신경망 학습 .....         | 36        |
| 오차역전파법 .....         | 36        |
| 경사하강법 .....          | 36        |
| 확률적 경사하강법 .....      | 37        |

|                          |    |
|--------------------------|----|
| 텐서플로 플레이그라운드 살펴보기 .....  | 38 |
| 컨볼루션 신경망 .....           | 38 |
| 커널 .....                 | 39 |
| 최대 풀링 .....              | 40 |
| 순환 신경망 .....             | 41 |
| LSTM .....               | 42 |
| 컴퓨터 비전을 위한 딥러닝 .....     | 43 |
| 분류 .....                 | 43 |
| 검출 또는 로컬라이제이션 및 분할 ..... | 44 |
| 유사도 학습 .....             | 45 |
| 이미지 캡셔닝 .....            | 45 |
| 생성 모델 .....              | 46 |
| 동영상 분석 .....             | 47 |
| 개발 환경 설정하기 .....         | 48 |
| 하드웨어 및 운영체제 .....        | 48 |
| 범용 그래픽 처리 장치 .....       | 48 |
| 소프트웨어 패키지 설치하기 .....     | 51 |
| 파이썬 .....                | 51 |
| OpenCV .....             | 53 |
| 텐서플로 라이브러리 .....         | 53 |
| 케라스 라이브러리 .....          | 60 |
| 요약 .....                 | 60 |

---

## 2장 이미지 분류 61

|                                  |    |
|----------------------------------|----|
| 텐서플로에서 MNIST 모델 훈련하기 .....       | 62 |
| MNIST 데이터셋 .....                 | 62 |
| MNIST 데이터 로드하기 .....             | 63 |
| 퍼셉트론 구축하기 .....                  | 64 |
| 입력 데이터와 타깃을 위한 플레이스홀더 정의하기 ..... | 64 |
| 완전 연결 레이어를 위한 변수 정의하기 .....      | 65 |
| 데이터로 모델 훈련하기 .....               | 66 |

|                                   |     |
|-----------------------------------|-----|
| 다중 레이어 컨볼루션 신경망 구축하기 .....        | 68  |
| 딥러닝에서 텐서보드 활용하기 .....             | 74  |
| 케라스에서 MNIST 모델 훈련시키기 .....        | 79  |
| 데이터셋 준비하기 .....                   | 79  |
| 모델 구축하기 .....                     | 81  |
| 그 외 일반적으로 사용되는 이미지 테스트 데이터셋 ..... | 83  |
| CIFAR 데이터셋 .....                  | 83  |
| Fashion-MNIST 데이터셋 .....          | 84  |
| ImageNet 데이터셋 및 대회 .....          | 85  |
| 더 깊은 딥러닝 모델 .....                 | 87  |
| AlexNet 모델 .....                  | 87  |
| VGG-16 모델 .....                   | 88  |
| 구글 인셉션-V3 모델 .....                | 90  |
| 마이크로소프트 ResNet-50 모델 .....        | 93  |
| SqueezeNet 모델 .....               | 94  |
| 공간 변환 네트워크 .....                  | 95  |
| DenseNet 모델 .....                 | 95  |
| 개와 고양이를 예측하는 모델 훈련시키기 .....       | 97  |
| 데이터 준비하기 .....                    | 97  |
| 간단한 CNN으로 벤치마킹하기 .....            | 98  |
| 데이터셋 확장하기 .....                   | 100 |
| 확장 기법 .....                       | 100 |
| 모델의 전이 학습 또는 미세 조정 .....          | 101 |
| 병목 특징에 대해 훈련시키기 .....             | 102 |
| 딥러닝의 여러 레이어 미세 조정하기 .....         | 104 |
| 실제 애플리케이션 개발하기 .....              | 106 |
| 올바른 모델 선택하기 .....                 | 107 |
| 언더피팅 및 오버피팅 시나리오 해결하기 .....       | 107 |
| 얼굴에서 성별과 나이 검출하기 .....            | 108 |
| 의류 모델 미세 조정하기 .....               | 108 |
| 브랜드 안정성 .....                     | 108 |
| 요약 .....                          | 109 |

|                             |                |
|-----------------------------|----------------|
| <b>3장 이미지 검색</b>            | <b>111</b>     |
| 시각적 특징의 이해                  | 112            |
| 딤러닝 모델 활성화의 시각화             | 113            |
| 임베딩 시각화                     | 114            |
| 유도된 역전파                     | 117            |
| DeepDream                   | 120            |
| 적대적 사례                      | 125            |
| 모델 추론                       | 127            |
| 모델 내보내기                     | 127            |
| 훈련된 모델 사용                   | 129            |
| 콘텐츠 기반 이미지 검색               | 131            |
| 검색 파이프라인 구축                 | 133            |
| 이미지의 병목 특징 추출               | 134            |
| 쿼리 이미지와 대상 데이터베이스 간의 유사도 계산 | 136            |
| 효율적 검색                      | 137            |
| ANNOY를 사용한 매칭 가속화           | 137            |
| Raw 이미지 자동 인코더              | 139            |
| 자동 인코더를 사용한 노이즈 제거          | 141            |
| 요약                          | 145            |
| <br><b>4장 객체 검출</b>         | <br><b>147</b> |
| 이미지에서의 객체 검출                | 148            |
| 데이터셋 탐색하기                   | 149            |
| ImageNet 데이터셋               | 149            |
| 파스칼 VOC 챌린지                 | 149            |
| COCO 객체 검출 챌린지              | 150            |
| 측정 항목을 사용해 데이터 집합 평가하기      | 150            |
| IoU                         | 151            |
| mAP                         | 152            |
| 알고리즘 로컬라이제이션하기              | 153            |
| 슬라이딩 윈도우를 사용해 객체 로컬라이제이션하기  | 153            |

|                                 |            |
|---------------------------------|------------|
| 스케일-공간 개념.....                  | 154        |
| 완전 연결 레이어를 컨볼루션 레이어로 훈련시키기..... | 154        |
| 컨볼루션을 이용한 슬라이딩 윈도우 구현.....      | 155        |
| 로컬라이제이션을 회귀 문제로 생각해보기.....      | 156        |
| 다른 문제에 대해 회귀 분석 적용하기.....       | 157        |
| 회귀 방법과 슬라이딩 윈도우 결합하기.....       | 158        |
| <b>객체 검출.....</b>               | <b>160</b> |
| R-CNN.....                      | 160        |
| Fast R-CNN.....                 | 161        |
| Faster R-CNN.....               | 162        |
| 싱글 샷 다중 박스 검출기.....             | 163        |
| <b>객체 검출 API.....</b>           | <b>163</b> |
| 설치 및 설정.....                    | 164        |
| 사전 훈련된 모델.....                  | 165        |
| 객체 검출 모델 재훈련.....               | 167        |
| Pascal 데이터셋의 데이터 준비.....        | 167        |
| 객체 검출 훈련 파이프라인.....             | 168        |
| 모델 훈련하기.....                    | 169        |
| 테스트를 사용해 손실 및 정확성 모니터링.....     | 169        |
| 자율주행용 보행자 검출 훈련.....            | 170        |
| <b>YOLO 객체 검출 알고리즘.....</b>     | <b>170</b> |
| <b>요약.....</b>                  | <b>173</b> |
| <br>                            |            |
| <b>5장 시맨틱 분할.....</b>           | <b>175</b> |
| 픽셀 예측.....                      | 176        |
| 의료 이미지 진단.....                  | 177        |
| 위성 이미지를 사용해 지구를 살펴보기.....       | 178        |
| 로봇이 볼 수 있도록 허용하기.....           | 178        |
| 데이터셋.....                       | 179        |
| 시맨틱 분할을 위한 알고리즘.....            | 180        |
| 완전 컨볼루션 네트워크.....               | 180        |

|                           |     |
|---------------------------|-----|
| SegNet 아키텍처 .....         | 181 |
| 풀링으로 레이어 업샘플링하기 .....     | 182 |
| 컨볼루션을 사용해 레이어 샘플링하기 ..... | 182 |
| 향상된 훈련을 위한 연결 건너뛰기 .....  | 186 |
| 확장 컨볼루션 .....             | 186 |
| DeepLab .....             | 187 |
| RefiNet .....             | 188 |
| PSPnet .....              | 189 |
| 대형 커널의 문제 .....           | 190 |
| DeepLab v3 .....          | 191 |
| 울트라-신경 분할 .....           | 191 |
| 위성 이미지 분할 .....           | 196 |
| 분할을 위한 FCN 모델링 .....      | 197 |
| 인스턴스 분할 .....             | 200 |
| 요약 .....                  | 201 |

---

## 6장 유사도 학습 203

|                       |     |
|-----------------------|-----|
| 유사도 학습을 위한 알고리즘 ..... | 204 |
| 삼 네트워크 .....          | 204 |
| 대조 손실 .....           | 205 |
| FaceNet .....         | 208 |
| 트리플렛 손실 .....         | 209 |
| DeepNet 모델 .....      | 211 |
| DeepRank .....        | 211 |
| 시각적 추천 시스템 .....      | 213 |
| 인간 얼굴 분석 .....        | 213 |
| 얼굴 검출 .....           | 214 |
| 얼굴 표식 및 속성 .....      | 215 |
| MTFL 데이터셋 .....       | 215 |
| 캐글 키폰트 데이터셋 .....     | 216 |
| MAFL 데이터셋 .....       | 216 |



|                             |     |
|-----------------------------|-----|
| 얼굴 키 포인트 학습 .....           | 217 |
| 얼굴 인식 .....                 | 220 |
| 와일드 데이터셋의 레이블이 부여된 얼굴 ..... | 221 |
| 유튜브의 얼굴 데이터셋 .....          | 221 |
| CelebA .....                | 221 |
| CASIA 웹 얼굴 데이터베이스 .....     | 221 |
| VGGFace2 데이터셋 .....         | 222 |
| 얼굴 간 유사도 계산하기 .....         | 222 |
| 최적의 임계값 찾기 .....            | 224 |
| 얼굴 클러스터링 .....              | 227 |
| 요약 .....                    | 227 |

|                                     |            |
|-------------------------------------|------------|
| <b>7장 이미지 캡션 처리</b> .....           | <b>229</b> |
| 문제 및 데이터셋 이해하기 .....                | 230        |
| 이미지 캡션을 위한 자연어 처리 이해 .....          | 230        |
| 벡터 형태로 단어 표현하기 .....                | 231        |
| 단어를 벡터로 변환 .....                    | 231        |
| 임베딩 훈련 .....                        | 233        |
| 이미지 캡션 및 관련 문제에 대한 접근 방법 .....      | 235        |
| 조건부 무작위 필드를 사용해 이미지와 텍스트 연결하기 ..... | 235        |
| CNN 기능에서 RNN을 사용해 캡션 생성 .....       | 236        |
| 이미지 순위를 사용해 캡션 만들기 .....            | 238        |
| 이미지와 이미지에서 캡션 가져오기 .....            | 239        |
| 밀집 캡션 .....                         | 241        |
| 캡션에 RNN 사용하기 .....                  | 242        |
| 다중 모달 측정 항목 공간 사용하기 .....           | 243        |
| 캡션 작성 시 관심 네트워크 사용하기 .....          | 245        |
| 언제 살펴봐야 할지 파악하기 .....               | 245        |
| 관심 기반 이미지 캡션 방법 구현하기 .....          | 248        |
| 요약 .....                            | 250        |

|                      |            |
|----------------------|------------|
| <b>8장 생성 모델</b>      | <b>251</b> |
| <b>생성 모델의 애플리케이션</b> | <b>252</b> |
| 예술적 스타일 이전 방법        | 252        |
| 동영상의 다음 프레임 예측 방법    | 253        |
| 슈퍼 해상도 이미지           | 254        |
| 대화형 이미지 생성하기         | 254        |
| 이미지를 다른 이미지로 변환하기    | 255        |
| 텍스트로 이미지 생성하기        | 256        |
| 불필요 제거               | 257        |
| 블렌딩                  | 258        |
| 속성 변환하기              | 259        |
| 훈련 데이터 생성            | 260        |
| 새 애니메이션 캐릭터 만들기      | 261        |
| 사진으로부터 3D 모델 생성      | 262        |
| <b>신경 예술 스타일 전송</b>  | <b>262</b> |
| 콘텐츠 손실               | 263        |
| 그램 행렬을 사용한 스타일 손실    | 269        |
| 스타일 전송               | 273        |
| <b>GAN</b>           | <b>277</b> |
| 바닐라 GAN              | 277        |
| 조건부 GAN              | 281        |
| 적대적 손실               | 281        |
| 이미지 변환               | 282        |
| InfoGAN              | 283        |
| GAN의 단점              | 284        |
| <b>VDM</b>           | <b>284</b> |
| VDM 알고리즘             | 284        |
| 생성기                  | 285        |
| 판별기                  | 285        |
| <b>요약</b>            | <b>286</b> |

## 9장 동영상 분류 287

|                              |     |
|------------------------------|-----|
| 동영상의 이해 및 분류                 | 288 |
| 동영상 분류 데이터셋 탐색               | 288 |
| UCF101                       | 288 |
| YouTube-8M                   | 290 |
| 기타 데이터셋                      | 291 |
| 동영상을 프레임으로 분할하기              | 291 |
| 동영상 분류 접근법                   | 292 |
| 동영상 분류를 위한 융합 병렬 CNN         | 293 |
| 장시간에 걸친 동영상 분류               | 295 |
| 액션 인식을 위한 두 개의 CNN 스트리밍 사용하기 | 298 |
| 시간적 학습을 위한 3D 컨볼루션 사용하기      | 299 |
| 분류 동작에 케적 사용하기               | 300 |
| 다중 모드 융합                     | 302 |
| 분류 수행을 위한 주목 영역              | 304 |
| 동영상에 대한 이미지 기반 접근법 확장        | 305 |
| 사람의 포즈도 적용하기                 | 306 |
| 얼굴 표식 추적                     | 307 |
| 동영상 분할                       | 307 |
| 동영상 캡션                       | 308 |
| 동영상 생성                       | 311 |
| 요약                           | 312 |

## 10장 배포 313

|             |     |
|-------------|-----|
| 모델 성능       | 314 |
| 모델 양자화      | 314 |
| MobileNets  | 315 |
| 클라우드에서 배포하기 | 318 |
| AWS         | 318 |
| 구글 클라우드 플랫폼 | 324 |
| 장치에 모델 배포하기 | 328 |

|                 |     |
|-----------------|-----|
| Jetson TX2..... | 328 |
| 안드로이드.....      | 329 |
| 아이폰.....        | 330 |
| 요약.....         | 331 |
| 찾아보기.....       | 333 |

## | 들어가며 |

이 책은 다양한 애플리케이션 프로그램을 위한 컴퓨터 비전 기술을 배우고 싶어 하는 독자를 대상으로 하며, 컴퓨터 비전 기반 제품을 개발할 수 있는 도구와 기술을 소개한다. 이 책의 실용적인 예제들을 통해 비전 관련 이론을 확인할 수 있다.

## ■ 이 책의 대상 독자

분류, 검색, 캡션, 생성, 캡션, 동영상 분류와 같은 컴퓨터 비전 문제에 대한 심층적 학습 방법을 알고 싶은 독자나 적은 데이터, 불균형 클래스, 노이즈와 같은 다양한 제약 조건에서도 높은 정확도를 달성하는 방법을 이해하길 원하는 독자에게 적합한 책이다. 이와 더불어 훈련된 모델을 다양한 플랫폼(AWS, 구글 클라우드, 라즈베리 파이, 휴대전화)에서 사용하는 방법을 알고 싶은 독자에게도 추천한다. 이 책을 모두 읽고 나면 독자는 인물 검색, 얼굴 인식, 제품 검색, 의료 이미지 분할, 이미지 생성, 이미지 캡션, 동영상 분류 등의 문제와 관련된 코드를 개발할 수 있게 될 것이다.

## ■ 이 책에서 다루는 내용

1장. 시작하기 딥러닝의 기초를 소개하며, 초보자가 어휘에 익숙해지도록 돕는다. 이후 장을 수행하는 데 필요한 소프트웨어 패키지 설치에 대한 내용도 다룬다.

2장. 이미지 분류 이미지 전체에 레이블을 붙이는 이미지 분류 문제를 설명한다. 이미지 분류 기술에 대해 배우고, 애완동물 분류에 대한 딥러닝 모델을 훈련할 수 있다. 또한 정확

도를 높이는 방법을 소개하고 다양한 고급 아키텍처에 대한 심도 있는 내용을 살펴본다.

3장. 이미지 검색 심층 특징<sup>deep feature</sup>과 이미지 검색을 다룬다. 모델 시각화, 시각적 기능, 텐서플로를 사용한 추론, 제품 검색을 위한 시각적 기능 제공 및 사용에 대한 다양한 방법을 학습할 수 있다.

4장. 객체 검출 이미지의 객체 검출을 설명한다. 다양한 객체 검출 기술을 배우고, 이를 보행자 검출<sup>pedestrian detection</sup>에 적용할 수 있다. 객체 검출을 위한 텐서플로 API가 이 장에서 활용된다.

5장. 시맨틱 분할 픽셀 단위로 이미지를 분할하는 것을 다룬다. 분할 기법에 대한 지식을 얻고 의료 이미지의 분할 모델을 훈련할 수 있다.

6장. 유사도 학습 유사도 학습에 대해 이야기한다. 유사도 매칭과 얼굴 인식을 위한 모델을 훈련시키는 방법을 소개하고, 얼굴 표식<sup>face landmark</sup>을 훈련시키는 모델을 설명한다.

7장. 이미지 캡션 이미지의 캡션을 생성하거나 선택하는 것을 다룬다. 자연어 처리 기술과 이 기술을 사용해 이미지 캡션을 생성하는 방법을 배운다.

8장. 생성 모델 다양한 목적으로 합성 이미지를 생성하는 방법을 설명한다. 이 장을 통해 생성 모델이 무엇인지 배우고, 스타일 전송과 훈련 데이터 등의 이미지 생성 애플리케이션 프로그램에 적용할 수 있다.

9장. 동영상 분류 동영상 데이터에 대한 컴퓨터 비전 기술을 다룬다. 동영상 문제와 이미지 문제의 주요 차이점을 이해하고 동영상 분류 기술을 구현해본다.

10장. 배포 딥러닝 모델의 배포 단계를 설명한다. 훈련된 모델을 배치하고 다양한 속도에 맞게 최적화하는 방법을 배울 수 있다.



## ■ 이 책을 최대한 활용하는 방법

이 책에서 다루는 예제는 윈도우, 우분투 또는 맥에서 실행할 수 있다. 모든 설치 방법을 설명하며, 파이썬과 머신 러닝에 대한 기본 지식이 필요하다. GPU 하드웨어가 있으면 더욱 좋겠지만, 반드시 필요하지는 않다.

### 예제 코드 다운로드

이 책에 사용된 예제 코드는 [www.packtpub.com](http://www.packtpub.com)에서 자신의 계정으로 로그인한 후 다운로드할 수 있다. 이 책을 다른 곳에서 구매한 경우 [www.packtpub.com/support](http://www.packtpub.com/support)를 방문해 등록하면 파일을 이메일로 직접 받을 수 있다.

코드를 다운로드하려면 다음과 같이 한다.

1. [www.packtpub.com](http://www.packtpub.com)에 로그인하거나 등록한다.
2. SUPPORT 탭을 선택한다.
3. Code Downloads & Errata 항목을 클릭한다.
4. Search 입력란에 책의 이름을 입력하고 화면에 나오는 설명을 따른다.

파일을 다운로드한 후에는 다음과 같은 압축 프로그램을 이용해 파일의 압축을 해제한다.

- 윈도우: WinRAR, 7-Zip
- 맥: Zipeg, iZip, UnRarX
- 리눅스: 7-Zip, PeaZip

이 책의 코드 묶음은 깃허브(<https://github.com/PacktPublishing/Deep-Learning-for-Computer-Vision>)에서도 받을 수 있으며, <https://github.com/PacktPublishing/>에서는 다른 책들의 코드 묶음과 동영상들을 제공한다. 또한 에이콘출판사의 도서정보 페이지인 <http://www.acornpub.co.kr/book/deep-learning-computer-vision>에서도 예제 코드를 다운로드할 수 있다.

## I 편집 규약

이 책에서는 독자의 이해를 돕고자 다루는 정보에 따라 글꼴 스타일을 다르게 적용했다. 이러한 스타일의 예와 의미는 다음과 같다.

텍스트에서 코드 단어는 다음과 같이 표기한다. “그래프는 `summary_writer`를 사용해 1회 저장된다.”

코드 블록은 다음과 같이 표기한다.

```
merged_summary_operation = tf.summary.merge_all ()
train_summary_writer = tf.summary.FileWriter ( '/ tmp / train', session.graph)
test_summary_writer = tf.summary.FileWriter ( '/ tmp / test')
```

명령행 입력이나 출력은 다음과 같이 표기한다.

```
wget http://www.robots.ox.ac.uk/~vgg/data/pets/data/images.tar.gz
wget http://www.robots.ox.ac.uk/~vgg/data/pets /data/annotations.tar.gz
```

화면상에 표시되는 메뉴나 버튼은 다음과 같이 표기한다.

“일단 Actions > Instance State > Terminate를 클릭해 인스턴스를 종료하고 작업을 완료한다.”



경고나 중요한 노트는 이와 같이 나타낸다.



팁과 요령은 이와 같이 나타낸다.

## I 고객 지원

### 일반적인 피드백

메시지 제목에 책 제목을 적어서 [feedback@packtpub.com](mailto:feedback@packtpub.com)으로 이메일을 보내면 된다. 이 책과 관련해 문의 사항이 있다면 [questions@packtpub.com](mailto:questions@packtpub.com)으로 이메일을 보내 주길 바란다. 한국어판에 관한 질문은 이 책의 옮긴이나 에이콘출판사 편집 팀([editor@acornpub.co.kr](mailto:editor@acornpub.co.kr))으로 문의할 수 있다.

### 정오표

내용을 정확하게 전달하기 위해 최선을 다했지만, 실수가 있을 수 있다. 이 책에서 문제 점을 발견했다면 출판사로 알려주길 바란다. [www.packtpub.com/submit-errata](http://www.packtpub.com/submit-errata)에서 책 제목을 선택하고 Errata Submission Form 링크를 클릭한 후 세부 사항을 입력하면 된다. 한국어판의 정오표는 에이콘출판사의 도서정보 페이지 <http://www.acornpub.co.kr/book/deep-learning-computer-vision>에서 찾아볼 수 있다.

### 저작권 침해

인터넷에서 어떤 형태로든 팩트출판사 서적의 불법 복제물을 발견하면 해당 주소나 웹사이트를 알려주길 바란다. 의심되는 불법 복제물의 링크를 [copyright@packtpub.com](mailto:copyright@packtpub.com)으로 보내주면 된다.



# 시작하기

컴퓨터 비전은 이미지와 동영상을 이해하고 처리하는 연구 분야다. 컴퓨터 비전은 자율주행, 산업용 검사, 증강현실과 같이 다양한 분야에 응용된다. 컴퓨터 비전을 위한 딥러닝의 사용 형태를 살펴보면, 이미지 또는 동영상에서의 분류, 검출, 분할, 생성과 같이 다양하다. 이 책에서는 컴퓨터 비전 애플리케이션을 위한 딥러닝 모델을 훈련하고 여러 플랫폼에 배포하는 방법을 학습한다. 이 책 전반에 걸쳐 예제에서 딥러닝을 위한 파이썬 라이브러리로는 유명한 텐서플로를 사용한다. 1장에서는 다음과 같은 주제를 다룬다.

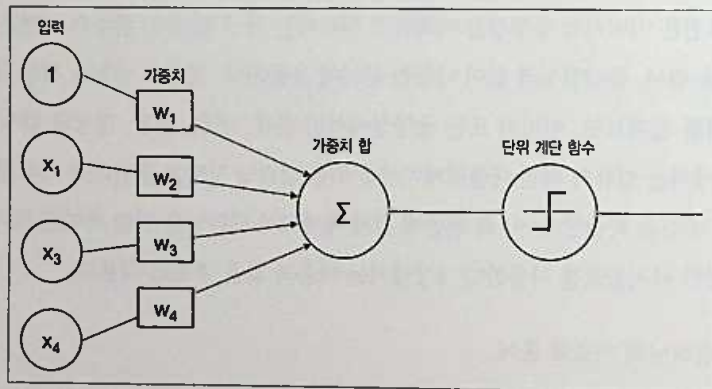
- 딥러닝의 기초와 용어
- 딥러닝을 컴퓨터 비전에서 사용하게 된 이유
- 이 책에서 다루는 예제에 사용될 개발 환경 설정
- 텐서보드<sup>TensorBoard</sup>, 텐서플로 서빙<sup>TensorFlow Serving</sup> 같은 강력한 도구와 함께 텐서

## ■ 딥러닝 이해하기

컴퓨터 비전은 오랜 역사를 가지고 있다. 딥러닝의 출현으로 컴퓨터 비전은 다양한 애플리케이션에 유용하다는 사실이 증명됐다. 딥러닝은 머신 러닝의 한 분야인 인공 신경망(ANN, Artificial Neural Network) 기술의 집합이다. ANN은 인간의 뇌를 모델로 해서 정보를 서로 전달하는 노드들의 연결로 구성된다. 다음 절에서는 일반적으로 사용되는 기본 용어를 이해함으로써 딥러닝이 어떻게 기능하는지 자세히 살펴본다.

### 퍼셉트론

인공 뉴런(artificial neuron) 또는 퍼셉트론(perceptron)은 여러 개의 입력된 정보에 대해 가중치 합을 계산해 출력 정보를 생성한다. 퍼셉트론의 가중치는 훈련 과정에서 결정되며 훈련 데이터를 기반으로 한다. 다음은 퍼셉트론에 대한 그림이다.



위 그림에서 볼 수 있듯이 각 입력에 대해 가중치를 부여하고 합한다. 그런 다음 해당 값은 이 예시의 경우 이진 분류 문제를 위한 단위 계단 함수로 전달된다. 퍼셉트론은 예제로



부터 가중치를 학습함으로써 단순 함수만 학습할 수 있다. 가중치 학습 과정은 훈련이라고 한다. 퍼셉트론에 대한 훈련은 추후 설명할 그래디언트 *gradient* 기반 방법을 통해 수행된다. 퍼셉트론의 출력은 다음 절에서 설명하는 활성화 함수 *activation function* 또는 전달 함수 *transfer function*를 통해 전달된다.

## 활성화 함수

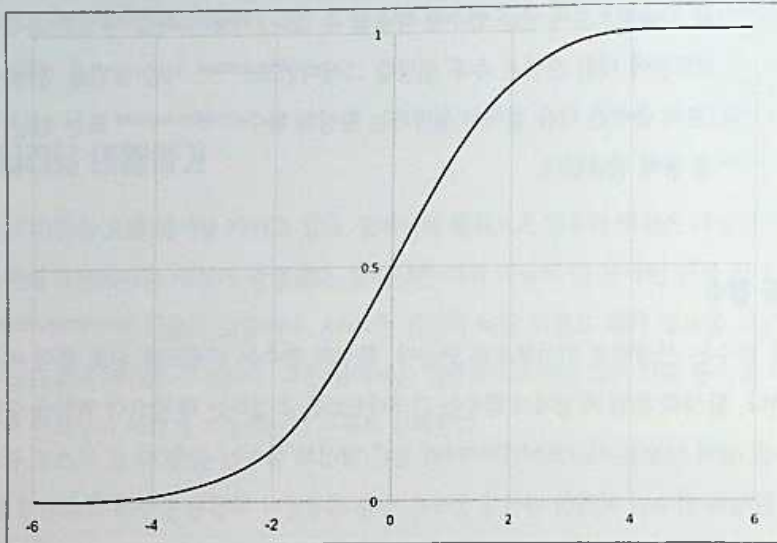
활성화 함수는 신경망을 비선형으로 만든다. 활성화 함수는 퍼셉트론 신호 출력 여부를 결정한다. 활성화 훈련 과정에서 함수는 그래디언트를 조정하는 데 중요한 역할을 수행한다. 다음 절에 나오는 시그모이드 *sigmoid*와 같은 활성화 함수는 값을 더 큰 크기로 감소시킨다. 활성화 함수의 이러한 비선형 동작은 심층 신경망이 복잡한 함수를 학습하게 한다. 대부분의 활성화 함수는 0에서 정류된 단위를 제외하고는 연속 함수며 미분 함수다. 연속 함수는 임의의 입력 값의 작은 변화에 따라 출력 값에 작은 변화가 있다. 미분 함수는 영역 내의 모든 점에서 미분 계수가 존재한다. 신경망을 훈련시키려면 함수는 미분 가능해야 한다. 몇 가지 활성화 함수를 소개한다.



연속 또는 미분과 같은 용어에 대해 자세히 이해하지 못한다고 걱정할 필요는 없다. 각 장을 학습하다 보면 용어의 의미가 명확해질 것이다.

## 시그모이드 함수

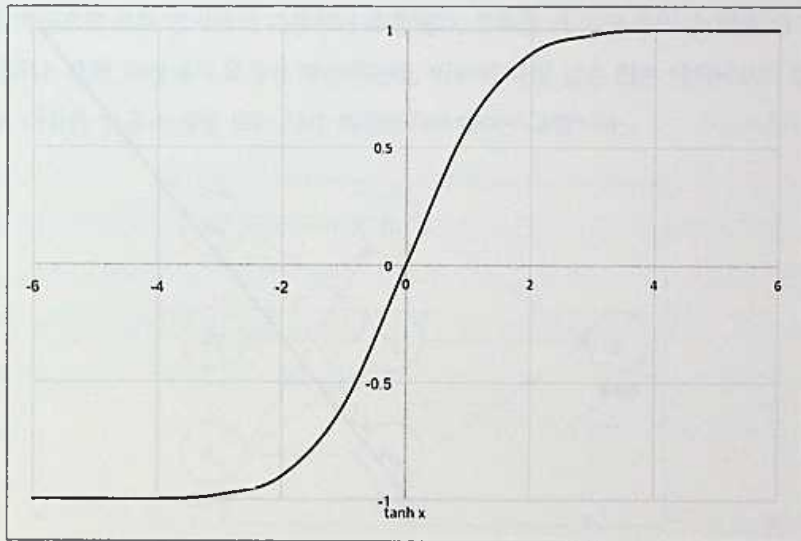
시그모이드는 매끈해진 계단 함수라 할 수 있으며 그렇기 때문에 미분 가능하다. 시그모이드는 모든 값을 확률로 변환하는 데 유용하며 이진 분류에 사용할 수 있다. 시그모이드는 다음 그래프에서 볼 수 있듯이 입력 값에 따라 0에서 1 사이의 출력 값이 발생한다.



X에 대한 Y 값의 변화는 작기 때문에 그래디언트가 소실될 것이다. 어느 정도 학습한 후에 변화는 작을 수 있다. 다음 절에서 설명할 tanh라는 또 다른 활성화 함수는 시그모이드의 크기가 조정된 버전으로 그래디언트 소실 문제를 방지한다.

### 쌍곡탄젠트 함수

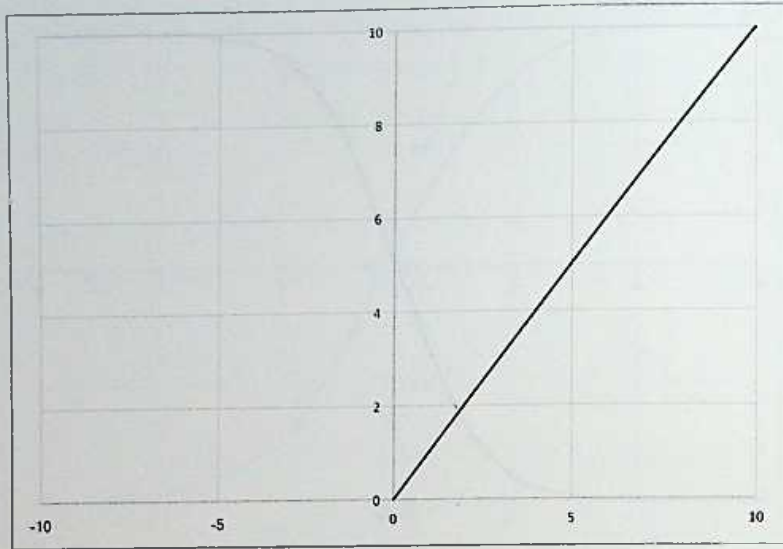
쌍곡탄젠트 함수, 또는 tanh는 시그모이드의 스케일된 버전이다. 시그모이드와 같이 매끈한 형태를 갖고 있으며 미분 가능하다. 다음 그래프와 같이 tanh는 입력 값에 따라 -1에서 1 사이의 출력 값이 발생한다.



그래디언트는 시그모이드에 비해 안정적이므로 그래디언트 소실 문제가 발생할 가능성이 적다. 시그모이드와  $\tanh$  둘 다 항상 신호를 출력하기 때문에 ANN은 매우 무거워진다. 다음 절에서 다룰 ReLU<sup>Rectified Linear Unit</sup> 활성화 함수는 때때로 신호를 출력하지 않기 때문에 이러한 문제를 방지한다.

## ReLU 함수

ReLU는 큰 숫자를 통과시킬 수 있다. ReLU는 일부 뉴런을 무효화하며, 해당 뉴런들은 신호를 출력하지 않는다. 이는 희소성을 증가시키므로, 좋은 결과를 얻을 수 있다. 다음 그래프에서 볼 수 있듯이 ReLU는 입력  $x$ 에 따라  $\max(0, x)$ 를 출력한다. 즉 입력이 0 이하면 0을 출력하고, 0 이상이면 입력 값 그대로 출력한다.



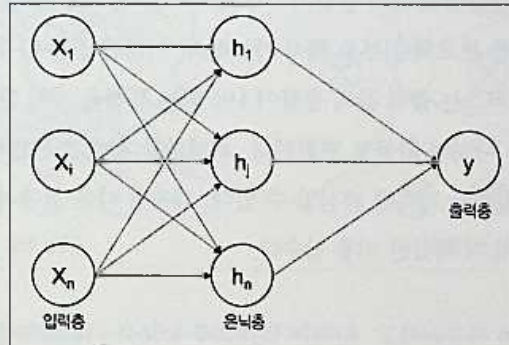
ReLU는 항상 신호를 출력하지 않기 때문에 더 빠르게 훈련 가능하다. 함수는 단순하기 때문에 계산 비용이 가장 적다. 활성화 함수를 선택하는 것은 애플리케이션에 따라 다르지만, 그럼에도 불구하고 ReLU는 광범위한 문제에 대해 효과적으로 동작한다. 다음 절에서는 여러 개의 퍼셉트론을 쌓아서 퍼셉트론보다 복잡한 함수를 학습하는 방법을 배운다.

## 인공 신경망

인공 신경망(ANN, Artificial Neural Network)은 퍼셉트론과 활성화 함수의 집합이다. 연결된 퍼셉트론은 은닉층 또는 은닉 유닛을 형성한다. 은닉 유닛은 입력층을 저차원 공간의 출력층에 매핑하는 비선형 구조를 형성하며, 이는 인공 신경망이라고도 불린다. ANN은 입력과 출력 사이의 지도와도 같다. ANN은 편향과 함께 입력 신호의 가중치 합을 계산해 구한다. 가중치와 편향 값, 구조까지 모두 통틀어서 모델이라고 한다.

훈련 과정은 이러한 가중치와 편향 값을 결정한다. 모델 값은 훈련 시작 시 임의의 값으로 초기화된다. 손실 함수를 이용해 실제 값과 대조함으로써 오류를 계산한다. 계산된 손실

값을 기반으로 모든 단계에서 가중치가 조정된다. 오류를 더 이상 줄일 수 없을 때 훈련은 중지된다. 훈련 과정에서 특징을 학습하는데, 이러한 특징 값은 원본 데이터보다 잘 보여 준다. 다음은 인공 신경망 또는 다층 퍼셉트론의 다이어그램이다.



여러 입력 값  $x$ 는 퍼셉트론의 은닉층을 통해 전달돼 출력으로 합산된다. 보편적 근사 정리[universal approximation theorem]는 이러한 신경망을 이용해서 모든 함수를 근사시킬 수 있다고 한다. 은닉층은 완전 연결 레이어라고도 한다. 모든 레이어는 이전 절에서 설명한 활성화 함수 중 하나를 가질 수 있다. 은닉층과 퍼셉트론의 수는 문제에 따라 달라지며, 다층 퍼셉트론이 다중 클래스 분류 문제를 해결 가능하게 하는 몇 가지 이유들이 더 존재한다. 다중 클래스 분류 문제는 열 개 이상의 카테고리로 구별할 수 있다. 관련 용어들은 다음 절에서 살펴본다.

## 원-핫 인코딩

원-핫 인코딩(one-hot encoding)은 분류 문제가 발생한 경우 대상 변수 또는 클래스를 나타내는 방법이다. 대상 변수는 문자열 레이블에서 인코딩된 원-핫 벡터로 변환될 수 있다. 원-핫 벡터는 대상 클래스의 인덱스 위치만 1이고 나머지는 모두 0으로 채워진 벡터다. 예를 들어 대상 클래스가 고양이와 개라면 각각  $[1, 0]$ ,  $[0, 1]$ 로 나타낼 수 있다. 1,000개의 클래스가 존재하는 경우, 원-핫 벡터는 하나를 제외한 나머지가 모두 0인 1,000개의 정수 크기를 갖는다. 대상 변수의 유사도에 대해서는 생각하지 않는다. 다음 절에서 설명할 소프

트맥스와 원-핫 인코딩을 조합하면 ANN에서 다중 클래스 분류가 가능해진다.

### 소프트맥스

소프트맥스<sup>softmax</sup>는 신경망 출력의 총합이 1이 되도록 한다. 그렇게 함으로써 소프트맥스 함수의 출력 값을 확률 분포의 일부로 해석 가능하다. 이러한 점은 다중 클래스 분류 문제에 유용하다. 소프트맥스는 출력 값의 총합이 1이 되는 특징을 가진 활성화 함수다. 출력을 모든 값의 합으로 나눠서 확률로 변환한다. 최적화를 위해 소프트맥스 확률과 원-핫 인코딩 사이에서 유클리드 거리를 계산할 수 있다. 하지만 다음 절에서 설명할 교차 엔트로피가 최적화하는 데 더 적합한 비용 함수다.

### 교차 엔트로피

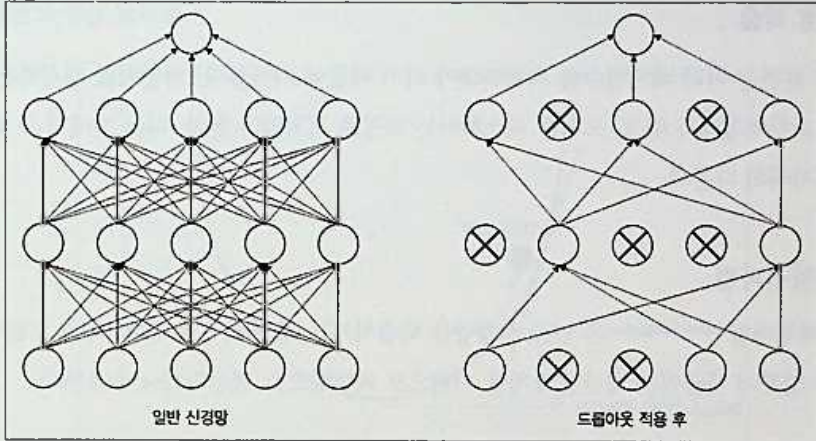
교차 엔트로피는 소프트맥스와 원-핫 인코딩 사이의 출력 간 거리를 비교한다. 교차 엔트로피는 손실 함수로 오류를 최소화해야 한다. 신경망은 모든 클래스에 대해 주어진 데이터의 확률을 추정하며, 올바른 대상 레이블의 확률 값이 최대화돼야 한다. 교차 엔트로피는 음의 로그 확률의 합이다. 로그 값은 수치 안정성을 위해 사용한다. 함수를 최대화하는 것은 동일한 함수의 음수를 최소화하는 것과 동일하다. 지금부터는 ANN의 오버피팅을 방지하는 다음의 정규화 방법을 하나씩 살펴본다.

- 드롭아웃
- 배치 정규화
- L1과 L2 정규화

### 드롭아웃

드롭아웃<sup>dropout</sup>은 ANN의 오버피팅을 방지하기 위해 신경망을 정규화하는 효과적인 방법이다. 다음 그림에서 볼 수 있듯이 훈련 과정에서 드롭아웃 레이어는 은닉 유닛을 임의적으로 삭제함으로써 신경망을 손상시킨다.





뉴런이 무작위로 훈련된다는 사실에 주목하길 바란다. 드롭아웃은 여러 신경망을 결합하는 효율적인 방법이기도 하다. 각 훈련마다 은닉 유닛을 무작위로 선택하기 때문에 매번 다른 모델 구조를 얻게 된다. 이는 배깅(bagging)과 모델 평균화가 과격한 경우다. 드롭아웃 레이어는 데이터 추론 중에는 사용하지 않아도 된다.

## 배치 정규화

배치 정규화(batch normalization)는 신경망 훈련의 안정성과 성능을 향상시킨다. 레이어의 출력은 평균 0과 표준 편차 1로 정규화한다. 이는 오버피팅을 억제하고 신경망 훈련 속도를 향상시킨다. 배치 정규화는 복잡한 신경망을 학습시키는 데 매우 유용하다.

## L1 및 L2 정규화

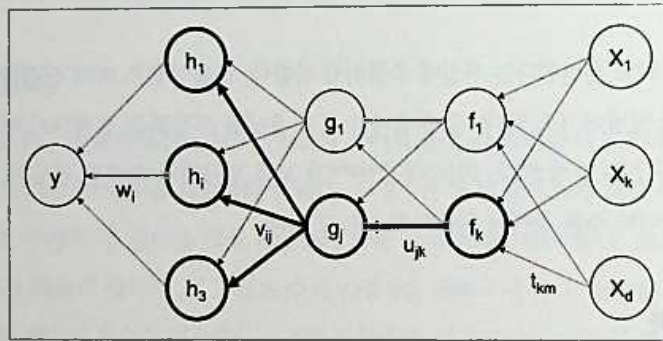
L1은 가중치의 절댓값에 대해 페널티를 부과하고 가중치를 0으로 만든다. L2는 가중치의 제곱 값에 페널티를 부과하고 훈련을 진행하는 중에 가중치의 크기를 작게 만든다. 두 정규화 방법 모두 가중치 크기가 작을수록 더 좋은 모델이라고 여긴다.

## 신경망 학습

ANN 훈련은 여러 매개변수를 최적화해야 하기 때문에 까다롭다. 가중치를 갱신하는 과정을 오차역전파라 하고, 오차를 최소화하는 과정을 최적화라 한다. 다음 절에서 두 과정 모두 자세히 다룬다.

### 오차역전파법

오차역전파법(backpropagation)은 인공 신경망을 학습시키는 데 흔히 사용된다. 다음 그림에서 볼 수 있듯이 가중치는 계산된 오차를 기반으로 역방향으로 갱신되거나 수정된다.



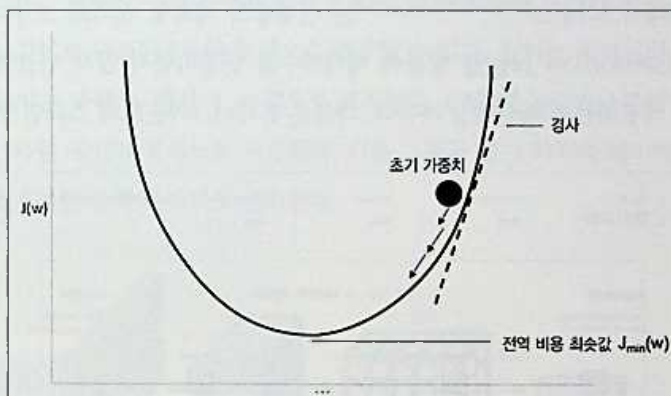
오차 계산 후 가중치 갱신을 위해 다음 절에서 설명할 경사하강법을 사용할 수 있다.

### 경사하강법

경사하강법(gradient descent)은 다차원 최적화를 수행한다. 목표는 전역 최댓값에 도달하는 것이다. 경사하강법은 다양한 머신 러닝 모델에서 흔히 사용되는 최적화 기술이며, 모델 예측을 개선하거나 최적화하는 데 사용된다. 확률적 경사하강법은 경사하강법 중 하나로, 신경망에서 인기가 높아지고 있다(다음 절에서 좀 더 자세히 다룬다).

최적화는 오류 값을 계산하고 해당 값을 최소화하기 위해 가중치를 변경하는 것을 말한다. 최솟값을 찾는 방향은 손실 함수의 기울기가 음수가 되는 방향이다. 다음 그림은 경사

하강법 과정을 보여준다.



학습 속도는 각 단계가 얼마나 커야 할지를 결정한다. 비선형 활성화 함수를 사용하는 ANN은 극솟값을 갖는다. 확률적 경사하강법은 볼록 함수가 아닌 비용 함수를 최적화할 때 실제로 더 잘 동작한다.

### 확률적 경사하강법

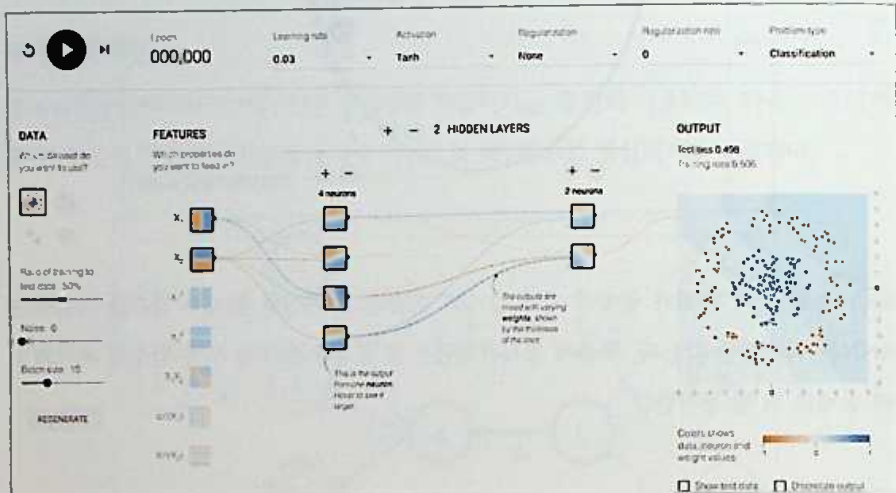
확률적 경사하강법 SGD, Stochastic Gradient Descent은 항상 데이터 일부만 사용해 학습한다는 사실을 제외하고는 경사하강법과 동일하다. 매개변수는 미니 배치라고 한다. 이론적으로, 하나의 예제라도 학습하는 데 사용할 수 있다. 실제로는 다양한 가짓수로 실험하는 것이 좋다. 다음 절에서는 이미지 데이터에 대해 표준 ANN보다 더 잘 동작하는 컨볼루션 신경망을 살펴본다.



<https://yihui.name/animation/example/grad-desc/>에서 볼록 평면과 볼록하지 않은 평면에 대한 경사 하강 관련 시각 자료를 볼 수 있다.

## 텐서플로 플레이그라운드 살펴보기

텐서플로 플레이그라운드 TensorFlow playground는 신경망의 대화형 시각화 도구다. <http://playground.tensorflow.org/>를 방문해 매개변수를 변경하면서 앞서 언급한 용어들이 서로 어떻게 작용하는지 확인하길 바란다. 다음은 플레이그라운드의 스크린샷이다.



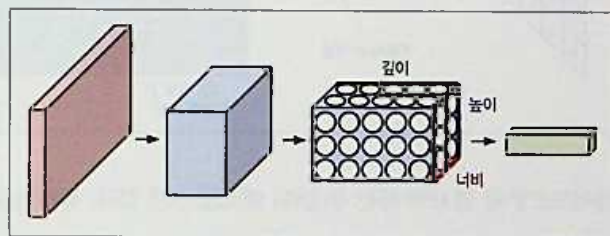
텐서플로 플레이그라운드의 대시보드

앞서 살펴본 바와 같이 학습 속도, 활성화, 정규화, 은닉 유닛, 은닉층을 변경하면서 해당 변경 내용이 학습 과정에 미치는 영향을 확인할 수 있다. 매개변수를 조정하다 보면 다양한 종류의 데이터에 따른 신경망 동작에 관한 직관을 얻을 수 있다.

## 컨볼루션 신경망

컨볼루션 신경망 CNN, Convolutional Neural Network은 이전 절에서 설명한 신경망과 유사하다. CNN은 가중치, 편향, 비선형 활성화를 통한 출력 값을 갖는다. 일반 신경망은 입력 신호와 뉴런이 전부 다음 레이어에 완전하게 연결된다. 동일한 레이어에 존재하는 뉴런은 어떤 연결도 공유하지 않는다. 이미지에 일반 신경망을 사용할 경우 거대한 수의 뉴런으로

인해 크기가 너무 커져서 오버피팅이 발생한다. 이미지는 크기가 너무 크므로 이미지에는 사용할 수 없다. 거대한 수의 뉴런을 필요로 하기 때문에 모델 크기가 커진다. 이미지는 높이, 너비, 깊이로 구성된 3차원 형상으로 간주할 수 있다. 깊이는 이미지의 채널이며 높이, 너비, 깊이는 파란색, 빨간색, 녹색으로 표시된다. CNN의 뉴런은 3차원 형상을 이용하기 위해 3차원 데이터 형식으로 저장된다. 다음 그림과 같이 각각의 레이어는 3차원 입력 데이터를 3차원 출력 데이터로 변환한다.

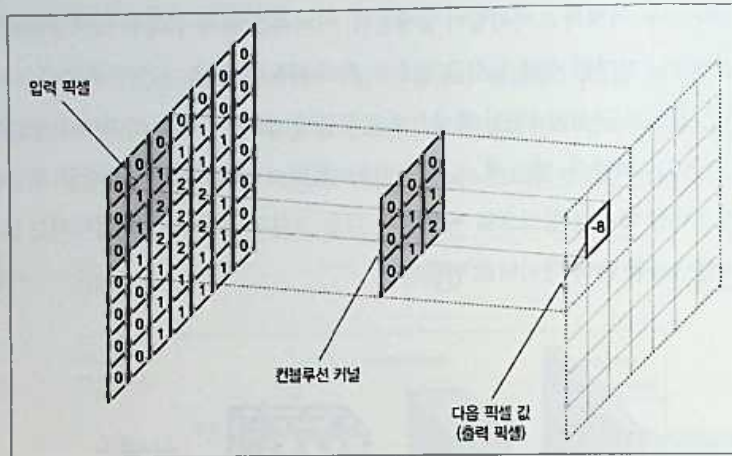


컨볼루션 신경망 필터는 변환으로 인코딩한다. 학습된 필터는 이미지의 특징 또는 패턴을 검출한다. 레이어를 사용해 깊어질수록 패턴은 더 추상적이다. 몇몇 분석 연구에 따르면, 이러한 레이어가 에지, 모서리, 패턴을 검출할 수 있다고 한다. CNN 레이어의 학습 가능한 매개변수는 이전 절에서 언급한 완전 연결 레이어(fully connected layer (또는 밀집 레이어dense layer라고도 한다.)보다 적다.

## 커널

커널(kernel)은 이미지와 컨볼루션 연산을 할 매개변수다. 컨볼루션 연산은 다음 그림과 같이 이뤄진다.





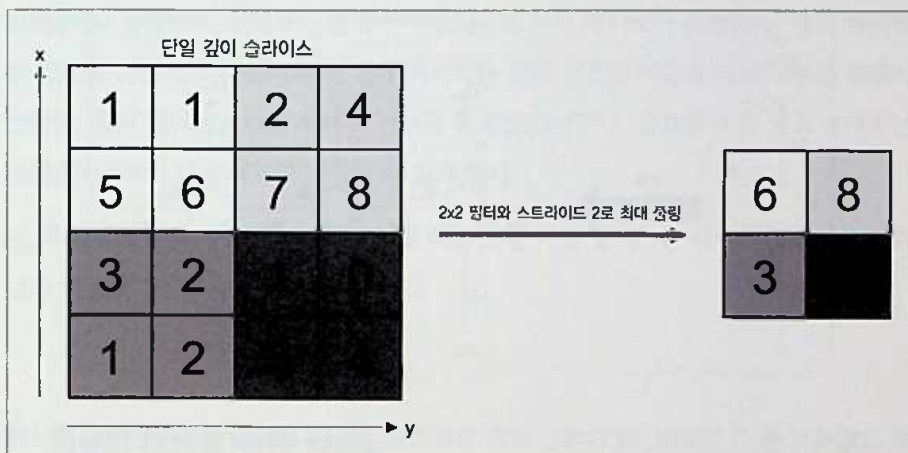
커널에는 스트라이드(stride)와 크기(size)라는 두 개의 매개변수가 있다. 크기는 직사각형의 높이 또는 너비가 된다. 스트라이드는 매번 이동하게 되는 픽셀 수다. 스트라이드가 1인 경우 거의 동일한 크기의 이미지를 생성하며, 2인 경우 절반 크기의 이미지를 생성한다. 이미지 패딩은 입력 이미지와 동일한 크기의 이미지를 얻는 데 도움이 된다.

### 최대 풀링

풀링(pooling) 레이어는 컨볼루션 레이어 사이에 위치한다. 풀링 레이어는 샘플링을 통해 레이어의 이미지 크기를 줄인다. 그리고 영역 내 최댓값을 선택해 샘플링한다. 평균 풀링은 영역의 평균을 구한다. 또한 풀링은 오버피팅이 발생하는 것을 막기 위한 정규화 역할을 한다. 풀링은 특징의 모든 채널에서 수행된다. 또한 풀링은 다양한 크기의 스트라이드로 수행될 수 있다.

영역 크기는 CNN의 수용 영역 크기다. 다음 그림은 최대 풀링(max pooling)의 예를 보여준다.





CNN은 컴퓨터 비전을 위한 모든 딥러닝 모델의 가장 중요한 구성 요소다. CNN 없이 컴퓨터 비전을 가질 수 있는 컴퓨터는 없다고 봐도 무방하다. 다음 절에서는 몇 가지 애플리케이션에서 사용할 수 있는 두 가지 진보된 레이어를 다룬다.



<https://www.youtube.com/watch?v=jajksuQW4mc>에서 CNN과 최대 풀링 연산에 관한 영상을 볼 수 있다.

## 순환 신경망

순환 신경망(RNN, Recurrent Neural Network)은 순차적 정보를 모델링할 수 있다. RNN은 데이터 포인트가 집약적이라고 가정하지 않는다. 일련의 순차적 데이터에 대해 동일한 작업을 수행하지만, 바로 전 데이터의 출력으로부터 영향을 받는다. 이는 메모리로 생각할 수도 있다. 하지만 RNN은 더 긴 순서 또는 시간을 기억할 수 없다. 다음 그림과 같이 RNN은 학습 과정에서 펼쳐진다(unfold).



LSTM에는 망각<sup>forget</sup>, 입력<sup>input</sup>, 출력<sup>output</sup>이라는 세 가지 게이트가 존재한다. 망각 게이트는 정보를 이전 상태로 유지한다. 입력 게이트는 입력 신호를 이용해 현재 상태를 업데이트한다. 출력 게이트는 다음 상태로 전달될 정보를 결정한다. 중요한 것을 잊고 유지하는 능력은 LSTM이 더 오랫동안 기억할 수 있게 한다.

이 책 전반에 걸쳐 사용될 딥러닝 용어를 학습했다. 다음 절에서는 컴퓨터 비전에서 딥러닝을 어떻게 사용하는지 살펴본다.

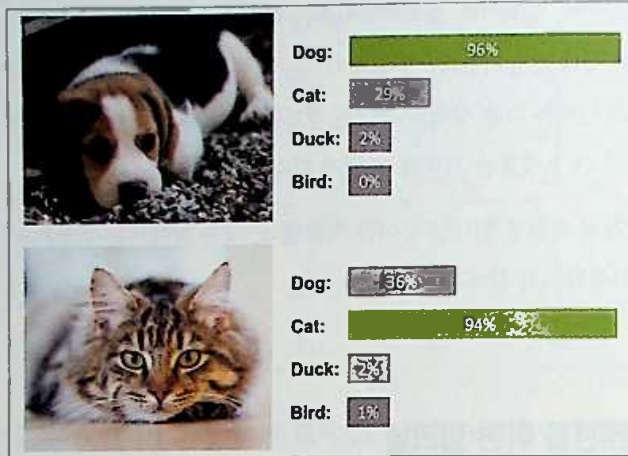
## ■ 컴퓨터 비전을 위한 딥러닝

컴퓨터 비전은 컴퓨터에서 인간의 시각 기능을 활용할 수 있게 한다. 여기서 컴퓨터는 인식을 위한 다양한 종류의 센서를 지닌 스마트폰, 드론, CCTV, MRI 스캐너 등의 형태일 수 있다. 센서는 컴퓨터가 해석해야 하는 디지털 형식의 이미지를 생성한다. 이러한 해석 또는 지능의 기본 구성 요소는 다음 절에서 설명한다. 컴퓨터 비전에서 발생하는 다양한 문제는 딥러닝 기술을 이용해 효과적으로 해결할 수 있다.

### 분류

이미지 분류는 전체 이미지에 객체 또는 콘셉트를 자신 있게 표시하는 작업이다. 애플리케이션에는 주어진 사람 얼굴 사진의 성별 분류하기, 반려 동물 종류 식별하기, 사진 태그 지정하기 등과 같은 것이 있다. 다음은 이러한 분류 작업의 결과다.





2장, '이미지 분류'에서는 분류 작업을 위해 사용되는 방법들을 자세히 다루고, 3장, '이미지 검색'에서는 딥러닝 모델의 시각화를 위해 분류 모델을 사용하고 유사한 이미지를 검색한다.

### 검출 또는 로컬라이제이션 및 분할

검출 또는 로컬라이제이션(localization)은 이미지에서 객체를 찾은 후 해당 객체의 위치를 경계 상자(bounding box)로 표시하는 작업이다. 이러한 작업은 자율주행 차량을 위한 보행자 및 교통 표지판 찾기와 같이 다양한 곳에 응용될 수 있다. 다음 그림은 검출의 실제 사례를 보여준다.



분할(segmentation)은 픽셀 단위 분류 작업이다. 객체를 미세하게 분리할 수 있으며, 의료 영상이나 위성 영상을 처리하는 데 유용하다. 더 많은 예제와 설명은 4장, '객체 검출'과 5장, '이미지 분할'에서 확인할 수 있다.

## 유사도 학습

유사도 학습은 두 이미지가 얼마나 유사한지 학습하는 과정이다. 다음 그림에서 볼 수 있듯이 의미적 유사도를 기반으로 두 이미지 사이의 점수를 계산한다.



유사도 학습은 유사한 제품 찾기부터 얼굴 인식 수행까지 여러 용도로 사용된다. 6장, '유사도 학습'에서 유사도 학습 기술을 다룬다.

## 이미지 캡셔닝

이미지 캡셔닝(image captioning)은 다음과 같이 텍스트로 이미지를 설명하는 작업이다.

| 정확한 캡션                                                                                                                                      | 악감의 오류가 있는 캡션                                                                                                                           | 이미지와 악감 관련 있는 캡션                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
|  <p>A person riding a motorcycle on a dirt road.</p>       |  <p>Two dogs play in the grass.</p>                    |  <p>A skateboarder does a trick on a ramp.</p>          |
|  <p>A group of young people playing a game of frisbee.</p> |  <p>Two hockey players are fighting over the puck.</p> |  <p>A little girl in a pink hat is blowing bubbles.</p> |

비나이스(Vinyls)의 허락하에 재구성 게재

7장, ‘이미지 캡션’에서는 이미지 캡셔닝을 자세히 살펴본다. 이미지 캡셔닝은 자연어 처리 NLP, National Language Processing와 컴퓨터 비전 기술이 결합돼야 하는 특수한 경우다.

## 생성 모델

생성 모델은 이미지를 생성한다는 점에서 매우 흥미롭다. 다음은 이미지의 콘텐츠와 다른 이미지의 스타일을 이용해 새로운 이미지를 생성하는 스타일 전송 애플리케이션의 예다.





게이츠(Gatys)의 허락하에 재구성 게재

이미지는 새로운 학습 예제, 초고해상도 이미지 등과 같이 다른 목적을 위해 생성될 수 있다. 8장, '생성 모델'에서는 생성 모델에 대해 자세히 살펴본다.

## 동영상 분석

동영상 분석은 앞서 살펴본 이미지와 달리 동영상을 전체적으로 처리한다. 스포츠 추적, 불법 침입 검출, 감시 카메라와 같은 여러 가지 애플리케이션이 있다. 9장, '동영상 분류'에서는 동영상용 애플리케이션을 다룬다. 시간 데이터라는 새로운 차원이 추가되면서 다양하고 흥미로운 애플리케이션이 등장한다. 다음 절에서는 개발 환경을 설정하는 방법을 살펴본다.

## ■ 개발 환경 설정하기

이번 절에서는 이 책의 예제들을 따라 하는 데 유용한 프로그래밍 환경을 설정한다. 다음 운영체제 중 하나를 선택할 수 있다.

- 맥, 우분투, 또는 윈도우와 같은 개발 운영체제
- 아마존 웹 서비스AWS, Amazon Web Services, 구글 클라우드 플랫폼GCP, Google Cloud Platform, 애저Azure, 테그라Tegra, 라즈베리파이와 같은 클라우드 플랫폼에 설치된 맥, 윈도우, 안드로이드, iOS, 또는 우분투 운영체제(배포를 통해 제공)

어떤 플랫폼을 사용하든 상관없이 이 책에서 사용된 모든 코드는 문제없이 실행돼야 한다. 이 장에서는 개발 환경 설치 과정을 다룬다. 10장, '배포'에서는 AWS, GCP, 애저, 테그라, 라즈베리파이와 같은 다양한 개발 환경에서의 배포 설치를 다룬다.

### 하드웨어 및 운영체제

개발 환경의 경우 학습하는 데 컴퓨터 리소스를 상당히 많이 사용하기 때문에 충분한 컴퓨팅 성능을 필요로 한다. 맥 사용자는 다소 제한된 컴퓨팅 성능을 갖는다. 윈도우와 우분투 사용자는 더 많은 프로세서와 다음 절에서 설명할 범용 그래픽 처리 장치를 이용해 개발 환경을 보강할 수 있다.

### 범용 그래픽 처리 장치

범용 그래픽 처리 장치GPGPU, General Purpose Graphics Processing Unit는 딥러닝 모델 학습 속도를 가속화하는 특수한 하드웨어다. 엔비디아NVIDIA 사의 GPGPU는 소프트웨어 및 커뮤니티 지원이 활성화돼 있어 딥러닝 학습 및 배포에 매우 많이 사용된다. 좀 더 빠른 학습을 위해 이러한 GPGPU가 있는 머신을 셋업해도 된다. 사용 가능한 선택 사항이 많기 때문에 예산에 따라 선택하면 된다. GPGPU 파워에 적합한 RAM, CPU, 하드디스크를 선택하는 것도 중요하다. 하드웨어를 설치한 후에는 다음의 드라이버와 라이브러리를 설치해야 한

다. 맥을 사용하거나 GPGPU 없이 윈도우/우분투를 사용하는 경우 이 설치 과정은 건너 뛰어도 된다.

다음은 환경 설정이 필요한 라이브러리들을 나타낸다.

- CUDA Computer Unified Device Architecture
- CUDNN CUDA Deep Neural Network

## CUDA

CUDA는 엔비디아에서 제공하는 GPU의 병렬 특성을 이용한 API 레이어다. CUDA를 설치하면 하드웨어 드라이버도 설치된다. 먼저, 엔비디아 포털 사이트 내 <https://developer.nvidia.com/cuda-downloads>에서 CUDA 라이브러리를 다운로드한다.

페이지상의 설명에 따라 드라이버를 다운로드한 후 설치 방법을 따라 한다. 다음은 우분투 CUDA의 설치 방법 화면이다.

**Select Target Platform**

Click on the green buttons that describe your target platform. Only supported platforms will be shown.

Operating System: Windows, Linux, Mac OSX

Architecture: x86\_64, ppc64le

Distribution: Fedora, OpenSUSE, RHEL, CentOS, SLES, Ubuntu

Version: 17.04, 16.04

Installer Type: runfile (local), deb (local), deb (network), cluster (local)

---

**Download Installer for Linux Ubuntu 16.04 x86\_64**

The base installer is available for download below.

> **Base Installer** Download (2.8 KB)

Installation Instructions:

1. `sudo dpkg --i cuda-repo-ubuntu1604_9.0.176-1_amd64.deb`
2. `sudo apt-key adv --fetch-keys https://developer.download.nvidia.com/compute/cuda/repos/ubuntu1604/x86_64/7fa2af80.pub`
3. `sudo apt-get update`
4. `sudo apt-get install cuda`

Other installation options are available in the form of meta-packages. For example, to install all the library packages, replace "cuda" with the "cuda-libraries-9-0" meta package. For more information on all the available meta packages click [here](#).

The CUDA Toolkit contains Open-Source Software. The source code can be found [here](#).  
 The checksums for the installer and patches can be found in [Installer Checksums](#).  
 For further information, see the [Installation Guide for Linux](#) and the [CUDA Quick Start Guide](#).

설치 명령을 수행했다면 cuda-driver와 필요한 다른 CUDA API가 설치됐을 것이다.



드라이버가 제대로 설치됐는지 확인하려면 명령 프롬프트에 `nvidia-smi`를 입력한다.

## CUDNN

CUDNN 라이브러리는 딥러닝 알고리즘을 위한 기본 요소를 제공한다. 엔비디아에서 제공하는 패키지이므로 하드웨어에 최적화돼 있으며 더 빠르게 실행될 수 있다. 이 패키지는 딥러닝을 위한 몇 가지 기본 루틴을 제공한다. 텐서플로, 카페인<sup>cafe</sup> 등과 같은 유명한 딥러닝 라이브러리에서 CUDNN을 사용한다. 다음 절에서는 CUDNN 설치 방법을 보여준다. CUDNN은 엔비디아 포털 사이트 내 <https://developer.nvidia.com/rdp/cudnn->

download에서 다운로드할 수 있다.



사용자 계정이 필요하다(무료 가입 가능).

관련 파일을 CUDA 폴더로 복사하면 GPU에서 더 빨리 실행된다. 우리는 CUDA 및 CUDNN 라이브러리를 직접적으로 사용하지 않는다. 텐서플로는 두 라이브러리를 이용해 최적화된 루틴으로 GPGPU에서 작업한다.

## 소프트웨어 패키지 설치하기

딥러닝 모델 학습에 필요한 몇 가지 라이브러리가 있다. 우리는 다음 라이브러리를 설치하고 다른 패키지 대신 해당 패키지를 선택한 이유를 확인한다.

- 파이썬과 그 외 다른 의존 패키지
- OpenCV
- 텐서플로
- 케라스Keras

### 파이썬

파이썬Python은 모든 데이터 사이언스 애플리케이션을 위한 사실상의 표준이라고 할 수 있으며, 라이브러리 중 가장 큰 커뮤니티와 지원 생태계를 갖고 있다. 파이썬용 텐서플로 API는 가장 완전하므로 당연한 선택이라고 할 수 있다. 파이썬에는 두 가지 버전이 있다. 파이썬 2.x와 파이썬 3.x다. 이 책에서는 파이썬 3.x에 대해 설명할 것이다. 파이썬 3.x를 선택한 이유는 다음과 같다.



- 파이썬 2.x 개발은 2020년부터 중단될 예정이다. 그렇기 때문에 파이썬 3.x는 향후 지속적으로 사용 가능한 파이썬 버전이 된다.
- 파이썬 3.x는 처음 구현할 때부터 많은 설계 결함 방지를 지원한다.
- 일반적으로 생각하는 것과 달리, 파이썬 3.x도 파이썬 2.x처럼 데이터 사이언스를 위한 다양한 라이브러리를 지원한다.

이 책 전반에 걸쳐서는 파이썬 버전 3를 사용한다. <https://www.python.org/downloads/>에서 사용하는 운영체제에 맞는 버전 3를 다운로드한다. 다운로드 링크에서 주어진 순서에 따라 파이썬을 설치한다. 파이썬의 설치를 완료한 후 파이썬 패키지를 쉽게 설치할 수 있도록 pip3를 설치한다. 이어서 이후 OpenCV와 텐서플로를 설치할 수 있도록 다음 명령을 이용해 여러 가지 파이썬 패키지를 설치한다.

---

```
sudo pip3 install numpy scipy scikit-learn pillow h5py
```

---

앞서 설치된 패키지에 대한 설명은 다음과 같다.

- numpy는 고도로 최적화된 수학 연산 패키지다. 강력한 N차원 배열 객체를 갖고 있으며 numpy 라이브러리의 행렬 연산은 속도를 위해 고도로 최적화돼 있다. 이미지는 3차원 배열 객체로 저장할 수 있다.
- scipy에는 과학 및 공학 연산을 위한 여러 가지 루틴이 있다. 추후에 일부 최적화 패키지를 사용할 예정이다.
- scikit-learn은 머신 러닝 라이브러리로 다양한 헬퍼 함수를 사용할 예정이다.
- pillow는 이미지 로딩과 기본적인 연산을 하는 데 유용하다.
- h5py 패키지는 HDF5 바이너리 데이터 포맷을 위한 파이썬 인터페이스다. 이것은 케라스를 이용해 훈련된 모델을 저장하는 포맷이다.



## OpenCV

OpenCV는 매우 유명한 컴퓨터 비전 라이브러리다. 이 라이브러리에는 유용하게 사용할 수 있는 여러 가지 이미지 처리 루틴이 있다. 다음은 우분투에서 OpenCV를 설치하는 방법이다.

---

```
sudo apt-get install python-opencv
```

---

다른 운영체제를 위한 설치 방법은 <https://opencv.org>에서 확인할 수 있다. OpenCV는 크로스 플랫폼 라이브러리며, CPU를 사용하는 애플리케이션에 최적화돼 있다. 또한 여러 가지 프로그래밍 언어를 위한 인터페이스를 가지고 있으며 윈도우, 우분투, 맥에서 지원된다.

## 텐서플로 라이브러리

텐서플로는 딥러닝 모델 개발 및 배포를 위한 오픈소스 라이브러리다. 텐서플로는 데이터 흐름 및 수치 연산을 위한 연산 그래프를 사용한다. 다시 말해 데이터 또는 텐서Tensor는 그래프를 따라 흐르며flow, 그렇기 때문에 텐서플로TensorFlow라고 부른다. 그래프에는 모든 수치 연산을 가능하게 하는 노드가 있으므로 딥러닝 작업에 적합하며, 모든 종류의 플랫폼 및 하드웨어에 대해 단일 API를 제공한다. 텐서플로는 백엔드에서 스케일링 및 최적화를 위한 모든 복잡한 작업을 처리한다. 원래 텐서플로는 구글의 연구를 위해 개발됐다. 큰 규모의 커뮤니티가 있는 가장 유명한 딥러닝 라이브러리며, 제작에 필요한 시각화 및 배포 도구를 제공한다.

## 텐서플로 설치하기

CPU를 사용하는 경우 다음 명령과 같이 pip3를 이용해 텐서플로를 설치한다.

---

```
sudo pip3 install tensorflow
```

---

GPU 하드웨어를 사용하고 있으며 CUDA와 CUDNN을 설치한 경우라면, 다음 명령을 이용해 GPU 버전의 텐서플로를 설치한다.

---

```
sudo pip3 install tensorflow-gpu
```

---

이제 텐서플로가 설치돼 사용할 준비를 마쳤다. 텐서플로가 어떻게 동작하는지 이해하기 위해 몇 가지 예제를 살펴본다.

### Hello, TensorFlow 출력 예제

파이썬 셸에서 직접 텐서플로를 사용해 예제를 시험해본다. 이 예제에서는 텐서플로를 이용해 'Hello, TensorFlow'를 출력할 것이다.

1. 명령 프롬프트에서 다음과 같이 입력해 셸에서 파이썬을 호출한다.

---

```
python3
```

---

2. 다음 명령을 입력해 텐서플로 라이브러리를 가져온다.

---

```
>>> import tensorflow as tf
```

---

3. 그런 다음 문자열 Hello, TensorFlow를 하나의 상수로 정의한다. 아직 값이 초기화되지 않았기 때문에 일반적인 파이썬 할당 연산과 다르다.

---

```
>>> hello = tf.constant('Hello, TensorFlow!')
```

---

4. 연산 그래프를 초기화하기 위해 세션을 생성하고 해당 세션에 이름을 지정한다.

---

```
>>> session = tf.Session()
```

---

세션은 변수 hello를 매개변수로 입력받아 실행된다.

5. 이제 그래프가 실행돼 출력될 특정 변수를 반환한다.

```
>>> print(session.run(hello))
```

다음 내용이 출력돼야 한다.

```
Hello, TensorFlow!
```

세션과 그래프가 어떻게 동작하는지 이해하기 위해 예제를 하나 더 살펴보자.



<https://github.com/rajacheers/DeepLearningForComputerVision>에서 이 책의 모든 예제 코드를 받을 수 있으며, 코드는 장별로 정리돼 있다. 문제가 있을 경우 저장소에서 이슈를 제기하고 도움을 얻을 수 있다.

## 두 개의 숫자 더하기 예제

다음은 텐서플로를 사용해 두 개의 숫자를 더하는 방법에 대한 간단한 예다.

1. 다음 코드를 이용해 파이썬 파일을 생성하고 텐서플로를 가져온다.

```
import tensorflow as tf
```

앞서 본 import 명령은 다른 모든 예제에서도 필요하다. 모든 예제에서 텐서플로 라이브러리를 가져온 것으로 가정한다. 플레이스홀더placeholder는 다음과 같은 방식으로 정의할 수 있다. 플레이스홀더는 선언 당시에 초기화되지 않는다. 여기서 변수는 float32 포맷의 플레이스홀더로 정의했다. 플레이스홀더는 비어있는 선언이며 세션이 실행돼야 값을 얻을 수 있다.

2. 다음 코드와 같이 플레이스홀더를 정의한다.

```
x = tf.placeholder(tf.float32)
y = tf.placeholder(tf.float32)
```

3. 이제 플레이스홀더의 합 연산을 일반적인 합 연산처럼 정의할 수 있다. 여기서 연산은 정의만 하고 실행되지는 않는다.

```
z = x + y
```

4. 이전 예제와 같이 세션을 생성할 수 있다. 다음과 같이 정의하면 그래프는 연산을 수행할 준비가 된다.

```
session = tf.Session()
```

5. 딕셔너리 포맷으로 플레이스홀더의 값을 정의한다.

```
values = {x: 5.0, y: 4.0}
```

6. 변수 `c`와 값을 이용해 세션을 실행한다. 그래프는 적절한 플레이스홀더에 값을 입력하고 변수 `c`의 값을 반환한다.

```
result = session.run([z], values)
print(result)
```

이 프로그램은 딕셔너리 결과로 `[9.0]`을 출력한다.

이것이 두 개의 숫자를 더하는 가장 좋은 방법이 아니라는 사실은 알고 있다. 이 예제는 텐서플로에서 텐서와 연산이 어떻게 정의되는지 이해하기 위한 것이다. 1조 개의 숫자를 사용해 더하는 것이 얼마나 어려울지는 상상해볼 수 있다. 텐서플로는 해당 규모의 연산을

동일한 API를 이용해 쉽게 할 수 있게 해준다. 다음 절에서는 텐서보드 및 텐서플로 서버를 설치하고 사용하는 방법을 살펴본다.

## 텐서보드

텐서보드(TensorBoard)는 텐서플로를 사용한 딥러닝 기반 모델 학습을 위한 시각화 도구 세트다. 텐서보드에서는 다음 데이터를 시각화한다.

- 그래프: 연산 그래프, 디바이스 위치, 텐서 정보
- 스칼라: 반복 중 발생하는 손실 및 정확도와 같은 측정 항목
- 이미지: 해당 레이블이 있는 이미지를 확인하기 위해 사용
- 오디오: 학습된 또는 생성된 오디오를 듣기 위해 사용
- 분포: 일부 스칼라의 분포를 확인하기 위해 사용
- 히스토그램: 가중치 및 편향의 히스토그램 포함
- 프로젝터: 데이터를 3차원 공간에서 시각화
- 텍스트: 학습 텍스트 데이터 출력
- 프로파일: 학습에 사용되는 하드웨어 리소스 확인

텐서보드는 텐서플로와 함께 설치된다. 텐서보드 사용을 시작하기 위해 파이썬 3 프롬프트에서 이전 예제와 유사한 다음 명령을 입력한다.

---

```
x = tf.placeholder(tf.float32, name='x')
y = tf.placeholder(tf.float32, name='y')
z = tf.add(x, y, name='sum')
```

---

플레이스홀더 및 연산에 인수명이 추가 매개변수로 제공됐다는 점에 주목하길 바란다. 그래프를 시각화하면 이 인수명을 볼 수 있다. 이제 텐서보드에서 다음 명령을 이용해 특정 폴더에 그래프를 작성한다.

```
session = tf.Session()
summary_writer = tf.summary.FileWriter('/tmp/1', session.graph)
```

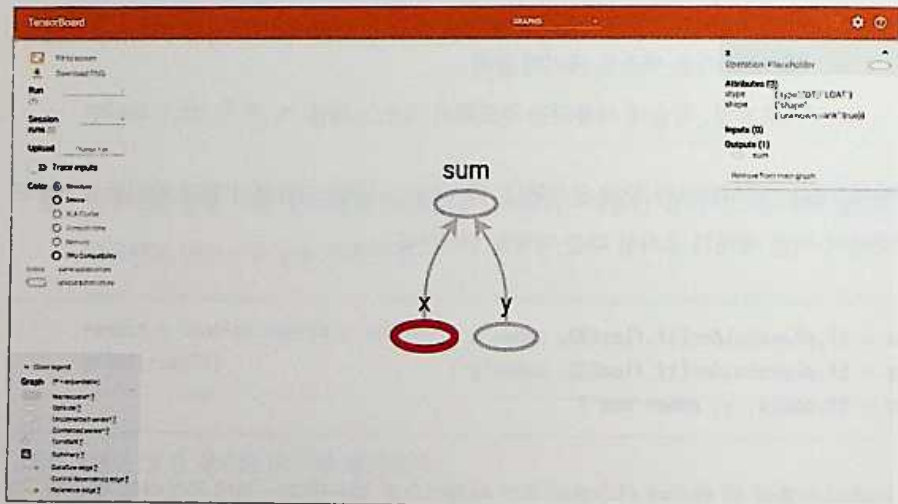
이 명령은 인수에 명시된 디스크의 특정 폴더에 그래프를 저장한다. 이제 다음 명령을 사용하여 텐서보드를 호출한다.

```
tensorboard --logdir=/tmp/1
```

파일이 저장될 위치인 logdir 옵션의 인수로 어떤 디렉터리든 상관없이 전달할 수 있다. 시각화를 위해 텐서보드를 시작하려면 브라우저에서 다음 URL로 접속한다.

<http://localhost:6006>

브라우저는 다음과 같은 화면을 보여준다.



브라우저 윈도우에서의 텐서보드

덧셈 그래프가 플레이스홀더에 주어진 이름과 함께 표시된다. 플레이스홀더를 클릭하면 우측에서 해당 연산에 대한 텐서의 모든 세부 사항들을 확인할 수 있다. 이제 탭과 옵션에



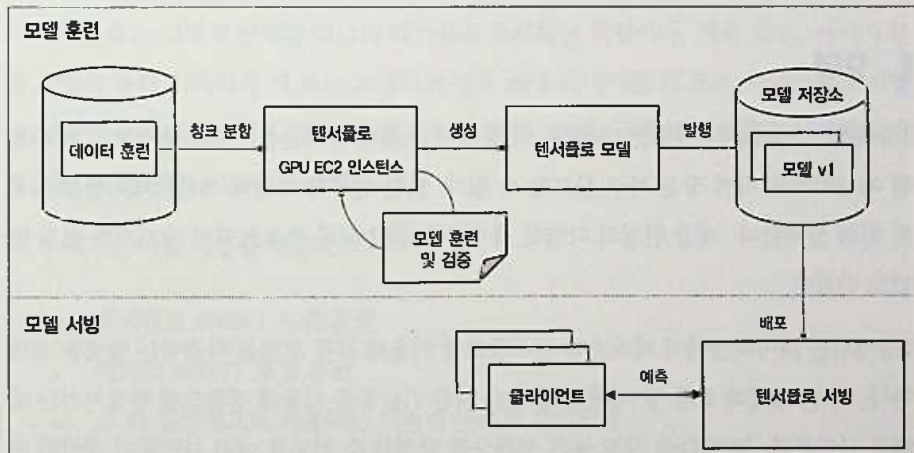
익숙해지도록 한다. 이 윈도우에서는 여러 가지 부분이 있다. 각각에 대해서는 다른 장에서 학습할 예정이다. 텐서보드는 텐서플로에서 가장 뛰어난 도구 중 하나며 그 어떤 딥러닝 프레임워크보다도 훌륭하다.

## 텐서플로 서빙 도구

텐서플로 서빙(TensorFlow Serving)은 유연한 배포 환경을 위해 개발된 텐서플로의 도구며 높은 지연 시간 및 처리량 환경을 제공한다. 텐서플로로 학습된 모든 딥러닝 모델은 서빙으로 배포할 수 있다. 다음 명령을 이용해 서빙을 설치한다.

```
sudo apt-get install tensorflow-model-server
```

서빙의 사용 순서는 3장, '이미지 검색'에서 설명한다. 서빙은 우분투에서만 설치하기가 쉽다. 다른 운영체제의 경우 <https://www.tensorflow.org/serving/setup>을 참고하길 바란다. 다음 그림은 제품 환경에서 텐서플로 서빙과 텐서플로가 상호작용하는 방법을 보여준다.



학습 과정에서 수많은 모델이 생성되며, 서빙은 모델이 정지되는 일 없이 원활하게 전환되도록 해준다. 텐서플로 서빙은 3장, '이미지 검색'과 10장, '배포'를 제외한 나머지 장에서는 필요하지 않다.

## 케라스 라이브러리

케라스(Keras)는 파이썬으로 작성된 딥러닝을 위한 오픈소스 라이브러리다. 케라스는 텐서플로를 백엔드로 사용할 수 있도록 쉬운 인터페이스를 제공한다. 또한 케라스는 백엔드로 테아노(Theano), 딥러닝 4j, 또는 CNTK를 사용할 수도 있다. 케라스는 친숙할 뿐만 아니라 모듈성과 확장성에 초점을 맞춰 설계돼 쉽고 빠른 작업에 적합하다. 또한 필요한 요소를 모두 갖춘 프레임워크며 CPU와 GPU 사이에서 원활하게 동작한다. 케라스만 따로 설치 사용할 수도 있지만 텐서플로 안에서 tf.keras API를 이용해 사용할 수도 있다. 이 책에서는 tf.keras API를 사용하며, 지금까지 개발 환경에 필요한 라이브러리의 설치 과정을 살펴봤다. 다음 장으로 넘어가려면 CUDA, CUDNN, OpenCV, 텐서플로, 케라스가 문제없이 설치되고 실행돼야 한다.

## I 요약

1장에서는 딥러닝의 기초를 다뤘다. 이 장에서 소개된 용어들은 이 책 전반에 걸쳐 사용될 예정이므로 이번 장을 자주 참고할 수 있다. 또한 컴퓨터 비전의 애플리케이션을 예제와 함께 살펴봤다. 개발 환경의 다양한 플랫폼을 위한 모든 소프트웨어 패키지의 설치 방법도 다뤘다.

2장에서는 데이터셋에서 케라스와 텐서플로를 이용해 분류 모델을 학습하는 방법을 설명한다. 더 큰 모델과 보강 및 미세 조정 같은 다른 기술들을 사용해 정확도를 향상시키는 방법을 살펴본다. 그런 다음 가장 높은 정확도를 달성할 수 있도록 여러 사람들이 제안한 여러 가지 고급 모델을 살펴본다.

# 이미지 분류

이미지 분류는 이미지 전체를 하나의 레이블로 분류하는 작업이다. 예를 들면, 이미지 분류 작업을 통해 이미지가 개 또는 고양이인 경우 해당 이미지를 개 또는 고양이로 표시할 수 있다. 2장에서는 텐서플로를 이용해 이미지 분류 모델을 구축하는 방법을 살펴보고 정확도를 향상시키는 기법을 학습한다.

이 장에서는 다음 내용을 다룬다.

- 텐서플로 MNIST 모델 훈련
- 케라스 MNIST 모델 훈련
- 그 외 일반적으로 사용되는 이미지 테스트 데이터셋
- 더 깊은 딥러닝 모델
- 개와 고양이를 분류하는 모델 훈련

- 실제 애플리케이션 개발

## ■ 텐서플로에서 MNIST 모델 훈련하기

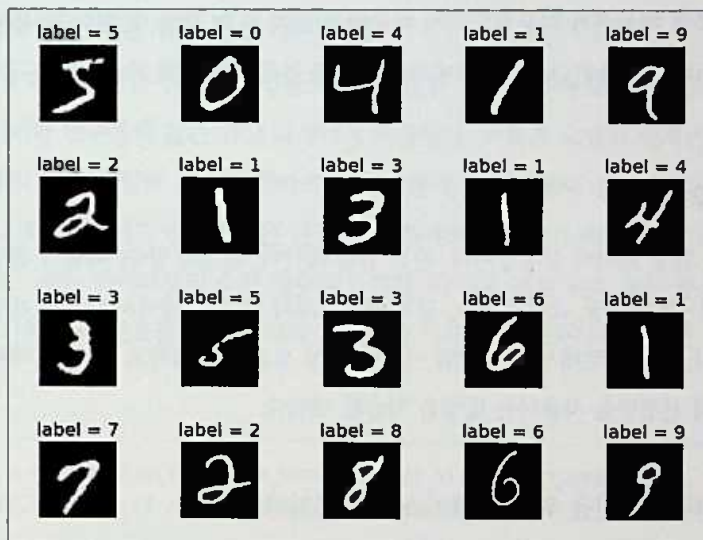
이 절에서는 MNIST 데이터베이스 데이터<sup>1</sup>에 대해 학습하고 간단한 분류 모델을 생성한다. 이번 절의 목적은 딥러닝을 위한 일반적인 프레임워크를 학습하고 딥러닝을 위해 텐서플로를 사용하는 것이다. 먼저, 퍼셉트론 또는 로지스틱 회귀 모델을 구축한다. 이어서 정확도 향상을 위해 CNN을 훈련한다. 또한 텐서보드를 이용해 훈련 프로세스를 시각화하고 매개변수를 이해한다.

### MNIST 데이터셋

MNIST 데이터는 손으로 쓰여진 숫자 0-9로, 훈련을 위한 60,000개의 이미지와 테스트를 위한 10,000개의 이미지로 이뤄져 있다. MNIST 데이터베이스는 전처리 과정이 최소화된 알고리즘에 널리 사용된다. 머신 러닝 알고리즘을 학습하기 좋은 간편한 데이터베이스며, 이미지 분류 문제를 위한 가장 유명한 데이터베이스다. 몇 개의 예시 이미지는 다음과 같다.

---

<sup>1</sup> MNIST는 Modified National Institute of Standards and Technology의 약자로 NIST의 기존 데이터셋의 샘플을 재조합해 만들어졌다. - 옮긴이



위 그림에서 볼 수 있듯이 손으로 작성된 문자들에 대해 열 개의 레이블이 있다. 이미지는  $28 \times 28$  이미지 픽셀 크기로 정규화됐고 그레이 스케일로 변환됐으며 중앙 정렬돼 있다. 이는 알고리즘을 신속하게 테스트할 수 있는 작은 데이터셋이다. 다음 절에서는 텐서플로에서 사용할 수 있도록 데이터셋을 로드하는 방법을 살펴본다.

## MNIST 데이터 로드하기

MNIST 데이터는 텐서플로에서 바로 로드한다. 데이터를 로드할 때 원-핫 인코딩을 인수로 지정한다는 점에 주목하길 바란다. 레이블은 정수 형태로 저장되지만 훈련을 위해서는 원-핫 인코딩으로 로드돼야 한다. 이 시점부터 tf로 가져오기를 한 텐서플로를 이용해 편집기에서 코드를 실행한다고 가정한다. 다음은 MNIST\_data를 로드하는 코드다.

---

```
from tensorflow.examples.tutorials.mnist import input_data
mnist_data = input_data.read_data_sets('MNIST_data', one_hot=True)
```

---



처음 실행하면 데이터가 다운로드되기 때문에 시간이 다소 걸릴 수 있다. 다음 실행부터 캐싱된 데이터를 이용한다. 다음 절에서는 숫자를 분류하기 위해 퍼셉트론을 구축한다.

## 퍼셉트론 구축하기

퍼셉트론은 단일 레이어 신경망이다. 완전 연결 레이어, 활성화 함수, 확률적 경사하강법, 로짓<sup>logit</sup>, 원-핫 인코딩, 소프트맥스, 교차 엔트로피와 같이 이 장에서 다루는 개념은 여기에 유용하다. 텐서플로에서 신경망의 이러한 구성 요소를 정의하고 MNIST 데이터를 훈련하기 위해 신경망을 사용하는 방법을 학습할 것이다.

### 입력 데이터와 타겟을 위한 플레이스홀더 정의하기

플레이스홀더는 데이터가 전달될 텐서다. 플레이스홀더는 특정 값은 아니지만 계산 과정에서 입력을 받는다. 먼저 퍼셉트론의 입력 크기, 클래스 수, 배치 크기, 반복 또는 배치의 총 개수를 선언한다. `x_input`은 이후 이미지가 저장되는 입력 값이다. `y_input`은 다음과 같이 원-핫 레이블 또는 타겟이 제공되는 플레이스홀더다.

---

```
input_size = 784
no_classes = 10
batch_size = 100
total_batches = 200

x_input = tf.placeholder(tf.float32, shape=[None, input_size])
y_input = tf.placeholder(tf.float32, shape=[None, no_classes])
```

---

`shape` 인수의 `None`은 아직 배치 크기를 정의하지 않았으므로 어떠한 크기든 될 수 있음을 의미한다. `x_input`의 두 번째 인수는 텐서의 크기며 `y_input`의 두 번째 인수는 클래스 수다. 플레이스홀더의 타입에 따르면, 데이터를 실수 형태로 전달했다. 이제 퍼셉트론을 정의할 수 있다.



## 완전 연결 레이어를 위한 변수 정의하기

가중치 및 편향과 같은 변수를 설명함으로써 단순한 선형 분류자 또는 퍼셉트론을 정의해보자. 이러한 변수들의 값은 연산 과정에서 학습된다. 이들은 모델의 매개변수라고도 한다. 가중치 변수는 입력 크기의 세이프<sup>shape</sup> 및 클래스 수의 정규 분포를 따르는 임의 값으로 초기화된다. 여기서 이미지는 단일 벡터 형태로 변환됐기 때문에 입력 크기는 784다. 클래스 수는 데이터셋의 숫자 개수와 동일한 10이다. 편향 변수 또한 클래스 수와 동일한 크기의 정규 분포를 따르는 임의 값으로 초기화된다. 가중치와 편향은 다음과 같이 정의된다.

---

```
weights = tf.Variable(tf.random_normal([input_size, no_classes]))
bias = tf.Variable(tf.random_normal([no_classes]))
```

---

변수의 초기화된 값은 0일 수 있지만 `random_normal` 함수를 통해 꾸준히 훈련된다. 이어서 입력 값에 가중치가 부여되고 편향 값이 추가돼 다음과 같이 로짓을 생성한다.

---

```
logits = tf.matmul(x_input, weights) + bias
```

---

퍼셉트론에 의해 생성된 로짓은 원-핫 레이블 `y_input`과 비교돼야 한다. 1장, '시작하기'에서 학습했듯이 로짓과 원-핫 레이블을 비교할 때 교차 엔트로피와 함께 소프트맥스를 사용하는 것이 좋다.

텐서플로의 `tf.nn.softmax_cross_entropy_with_logits` API가 이 작업을 수행한다. 교차 엔트로피의 평균을 계산해 손실을 구할 수 있다. 그러면 교차 엔트로피는 경사하강법 최적화를 수행하는 `tf.train.GradientDescentOptimizer`에 전달된다. 최적화 알고리즘은 학습 속도 0.5로 손실을 최소화한다. 소프트맥스, 교차 엔트로피, 손실, 최적화의 계산은 다음과 같다.

```
softmax_cross_entropy = tf.nn.softmax_cross_entropy_with_logits(  
    labels=y_input, logits=logits)  
loss_operation = tf.reduce_mean(softmax_cross_entropy)  
optimiser = tf.train.GradientDescentOptimizer(  
    learning_rate=0.5).minimize(loss_operation)
```

소프트맥스와 교차 엔트로피는 여러 가지 다른 유용한 메소드를 가진 tf.nn 패키지에서 동시에 계산된다. tf.train에는 여러 최적화 알고리즘이 있으며, 여기서는 바닐라 경사 하강법을 사용한다. 그 외 다른 선택적 매개변수에 대해 궁금하다면 텐서플로 API 문서를 확인하자. 지금까지 플레이스홀더, 변수, 연산 작업은 정의됐지만 아직 텐서로 채워진 것은 아니다.



텐서플로에서 사용할 수 있는 최적화 알고리즘의 리스트는 [https://www.tensorflow.org/api\\_guides/python/train](https://www.tensorflow.org/api_guides/python/train)에서 확인하길 바란다. 아담 최적화 알고리즘은 특히 컴퓨터 비전 애플리케이션에 유용하다. 대체로 더 빠르게 수렴하며 학습 속도를 정의하지 않아도 된다. 최적화 알고리즘에 대한 간단한 이론 내용은 <http://ruder.io/optimizing-gradient-descent>에서 확인한다.

## 데이터로 모델 훈련하기

모델과 훈련 작업을 정의했다. 다음 단계는 데이터로 모델 훈련하기다. 훈련 과정에서 기울기를 구하고 가중치가 업데이트된다. 변수는 아직 초기화되지 않았다. 이어서 세션을 시작하고 전역 변수 초기화 함수를 이용해 변수를 초기화한다.

```
session = tf.Session()  
session.run(tf.global_variables_initializer())
```

위 두 줄의 코드는 이 책의 대부분의 예제에서 필수적으로 사용된다. 필요할 때마다 해당 코드를 사용한다고 가정한다. 이제 그래프에 데이터를 입력하고 훈련을 시작할 준비를 마

쳤다. 배치의 데이터를 반복해서 읽고 모델을 훈련시킨다. 필요한 텐서를 사용해 세션을 실행함으로써 모델 훈련을 실행한다. 그래프가 가중치를 업데이트할 수 있도록 최적화 알고리즘이 호출돼야 한다.

---

```
for batch_no in range(total_batches):
    mnist_batch = mnist_data.train.next_batch(batch_size)
    _, loss_value = session.run([optimiser, loss_operation], feed_dict={
        x_input: mnist_batch[0],
        y_input: mnist_batch[1]
    })
    print(loss_value)
```

---

run 메소드의 첫 번째 인수는 출력 요청 값으로 이뤄진 배열이 될 수 있다. 손실 출력은 모델이 훈련을 받았는지 여부를 알려주기 때문에 손실을 전달할 수 있다. 손실을 최소화하려고 하기 때문에 손실이 감소할 것으로 예상된다. feed\_dict는 파이썬 딕셔너리로 플레 이스홀더에 입력 레이블과 타깃 레이블을 직접적으로 입력하기 위해 사용된다. 이 루프가 끝나면 보통 손실은 0.46보다 낮아야 한다. 그런 다음 다음과 같이 정확도를 계산해 모델이 얼마나 잘 동작하는지 확인할 수 있다.

---

```
predictions = tf.argmax(logits, 1)
correct_predictions = tf.equal(predictions, tf.argmax(y_input, 1))
accuracy_operation = tf.reduce_mean(tf.cast(correct_predictions,
                                             tf.float32))
test_images, test_labels = mnist_data.test.images, mnist_data.test.labels
accuracy_value = session.run(accuracy_operation, feed_dict={
    x_input: test_images,
    y_input: test_labels
})
print('Accuracy : ', accuracy_value)
session.close()
```

---

예측값은 가장 큰 활성화 값의 인덱스가 돼야 한다. 예측값과 MNIST 레이블의 실제 정

보를 비교해 정확한 예측값을 얻는다. 앞서 얻은 정확한 예측값의 평균을 구함으로써 정확도를 계산한다. 테스트 데이터로 이뤄진 피드 디서너리로 세션을 실행함으로써 데이터의 정확도를 평가할 수 있다. 프로그램이 전부 실행되면 최종적으로 대략 90%의 정확도를 보여줘야 한다. 좀 더 간단한 훈련 및 테스트 API가 없다면 모델의 정의가 너무 장황하게 보일 수 있다. 이러한 수준의 기본 정의는 텐서플로의 풍부한 표현력을 보여준다. 다음 절에서는 좀 더 높은 수준의 API를 살펴볼 것이다. 퍼셉트론을 통해 얻은 정확도는 그다지 좋은 편이 아니다. 정확도 향상을 위해 컨볼루션 레이어가 있는 더 깊은 네트워크를 사용해보자.

## 다중 레이어 컨볼루션 신경망 구축하기

이번 절에서는 텐서플로에서 다중 레이어 컨볼루션 신경망을 구축하는 방법을 살펴보고 심층 신경망이 분류 정확도를 어떻게 향상시키는지 확인한다. 레이어를 완전히 처음부터 새롭게 정의하지 않고 텐서플로 레이어 API에서 제공하는 메소드를 이용해 정의할 예정이다. 모범 사례의 메소드들이 텐서플로 API 메소드에 담겨 있다. 라이브러리, 데이터셋, 플레이스홀더 가져오기는 이전 절에서 이미 수행했다. 이번에는 텐서보드를 사용해 훈련 프로세스를 시각화한다. 변수의 통계를 시각화하려면 `tf.summary`에 변수의 통계 값이 추가돼야 한다. 요약은 텐서보드에서 해석 가능한 폴더에 작성된다. 텐서보드를 시각화하는데 사용할 수 있도록 요약을 작성하는 함수를 정의해보자.

```
def add_variable_summary(tf_variable, summary_name):
    with tf.name_scope(summary_name + '_summary'):
        mean = tf.reduce_mean(tf_variable)
        tf.summary.scalar('Mean', mean)
        with tf.name_scope('standard_deviation'):
            standard_deviation = tf.sqrt(tf.reduce_mean(
                tf.square(tf_variable - mean)))
            tf.summary.scalar('StandardDeviation', standard_deviation)
        tf.summary.scalar('Maximum', tf.reduce_max(tf_variable))
```

```
tf.summary.scalar('Minimum', tf.reduce_min(tf_variable))
tf.summary.histogram('Histogram', tf_variable)
```

---

변수 요약 함수는 변수의 요약 데이터를 작성한다. 요약에 추가된 통계 값은 다섯 가지로 평균, 표준 편차, 최대, 최소, 히스토그램이다. 요약 데이터는 스칼라 또는 히스토그램이 될 수 있다. 여러 가지 변수가 기록될 때 이러한 값들이 텐서보드에서 어떻게 시각화되는지 살펴본다. 이전 모델과 달리, MNIST 데이터를 정사각형 형태로 크기를 변경하고 2차원 이미지 형태로 사용한다. 다음은 이미지를 28×28 이미지 픽셀 크기로 변형하는 명령이다.

---

```
x_input_reshape = tf.reshape(x_input, [-1, 28, 28, 1],
                             name='input_reshape')
```

---

차원 값이 -1이라는 것은 배치 크기가 임의의 수가 될 수 있음을 의미한다. name이라는 인수는 텐서보드 그래프에 반영돼 쉽게 이해할 수 있게 한다. 입력, 필터, 커널, 활성화가 정의된 2D 컨볼루션 레이어를 정의한다. 이 메소드는 추후 다른 예제에서도 언제든지 호출 가능하며 활성화 함수에 ReLU 활성화가 필요한 경우에 유용하다. 컨볼루션 레이어 함수는 다음과 같이 정의된다.

---

```
def convolution_layer(input_layer, filters, kernel_size=[3, 3],
                    activation=tf.nn.relu):
    layer = tf.layers.conv2d(
        inputs=input_layer,
        filters=filters,
        kernel_size=kernel_size,
        activation=activation,
    )
    add_variable_summary(layer, 'convolution')
    return layer
```

---



kernel\_size와 activation에 대한 디폴트 값이 있다. 요약 데이터는 함수 내에서 레이어에 추가되고 해당 레이어가 반환된다. 함수를 호출할 때마다 매개변수로 input\_layer가 주입되어야 한다. 이 정의는 다른 코드를 단순하고 간결하게 만든다. 매우 유사하게 풀링 레이어에 대한 함수를 다음과 같이 정의할 수 있다.

---

```
def pooling_layer(input_layer, pool_size=[2, 2], strides=2):
    layer = tf.layers.max_pooling2d(
        inputs=input_layer,
        pool_size=pool_size,
        strides=strides
    )
    add_variable_summary(layer, 'pooling')
    return layer
```

---

이 레이어는 pool\_size와 strides의 디폴트 값으로 각각 [2, 2]와 2를 갖는다. 이러한 매개변수는 대체로 잘 동작하지만 필요한 경우에는 변경할 수 있다. 이 레이어에도 요약 데이터가 추가된다. 밀집 레이어는 다음과 같이 정의한다.

---

```
def dense_layer(input_layer, units, activation=tf.nn.relu):
    layer = tf.layers.dense(
        inputs=input_layer,
        units=units,
        activation=activation
    )
    add_variable_summary(layer, 'dense')
    return layer
```

---

앞서 정의한 밀집 레이어는 activation에 대한 기본값을 갖고 있으며 변수 요약 데이터 또한 추가된다. pooling\_layer는 컨볼루션 레이어로부터 특징 지도를 가져온 후 풀 크기와 스트라이드를 이용해 건너뛰으로써 특징 지도를 절반 크기로 줄인다. 이러한 모든 레이어는 그래프로 연결됐으며, 정의만 한 상태로 값은 아직 초기화되기 전이다. 첫 번째 컨



볼루션 레이어로부터 샘플링된 특성을 좀 더 나은 특성으로 변환하기 위해 새로운 컨볼루션 레이어를 추가할 수 있다. 풀링 후, 밀집 레이어로 주입될 수 있도록 활성화를 선형적으로 변형할 수 있다.

---

```
convolution_layer_1 = convolution_layer(x_input_reshape, 64)
pooling_layer_1 = pooling_layer(convolution_layer_1)
convolution_layer_2 = convolution_layer(pooling_layer_1, 128)
pooling_layer_2 = pooling_layer(convolution_layer_2)
flattened_pool = tf.reshape(pooling_layer_2, [-1, 5 * 5 * 128],
                             name='flattened_pool')
dense_layer_bottleneck = dense_layer(flattened_pool, 1024)
```

---

컨볼루션 레이어 간의 유일한 차이점은 필터 크기며, 크기가 레이어마다 적절하게 변경되는 것이 중요하다. 커널과 스트라이드의 매개변수 값을 선택하는 것은 임의적이며, 경험에 의해 선택된다. 두 개의 컨볼루션 레이어가 정의되면, 그다음에는 완전 연결 레이어가 있다. 밀집 레이어 API는 단일 차원의 벡터를 가져와서 특정 개수의 숨겨진 유닛<sup>hidden unit</sup>으로 매핑시킨다(이 경우에는 1,024개다). 숨겨진 레이어<sup>hidden layer</sup>는 ReLU 활성화 함수로 이어져 비선형 연산이 된다. 변수 요약 데이터는 이 레이어에도 추가된다. 다음으로 드롭아웃 확률을 가진 드롭아웃 레이어로 이어진다. 훈련 모드는 드롭아웃 적용 여부에 따라 True 또는 False로 설정할 수 있다. 훈련을 위해 True로 설정한다(디폴트 값은 False다). 정확도가 계산될 때 해당 값은 변경돼야 한다. 그러므로 부울 값을 저장해둬야 하며, 훈련할 때 적용될 수 있다.

---

```
dropout_bool = tf.placeholder(tf.bool)
dropout_layer = tf.layers.dropout(
    inputs=dense_layer_bottleneck,
    rate=0.4,
    training=dropout_bool
)
```

---

드롭아웃 레이어는 로짓이라 불리는 밀집 레이어에 주입된다. 로짓은 클래스 개수로 이어지는 활성화 함수를 가진 마지막 레이어다. 활성화 함수는 타깃 클래스인 특정 클래스에 대해 활성화되며 최대 10회의 활성화를 얻을 수 있다.<sup>2</sup>

---

```
logits = dense_layer(dropout_layer, no_classes)
```

---

로짓의 출력 결과는 이전 절에서 생성한 모델과 매우 유사하다. 이제 로짓은 이전과 같이 소프트맥스 레이어를 통과한 후 교차 엔트로피 계산을 수행한다. 여기서는 텐서보드에서 좀 더 나은 시각화를 위해 이름 범주<sup>name scope</sup>를 추가했다.

---

```
with tf.name_scope('loss'):
    softmax_cross_entropy = tf.nn.softmax_cross_entropy_with_logits(
        labels=y_input, logits=logits)
    loss_operation = tf.reduce_mean(softmax_cross_entropy, name='loss')
    tf.summary.scalar('loss', loss_operation)
```

---

손실 함수는 tf.train API의 메소드를 이용해 최적화할 수 있으며, 여기서는 그중 AdamOptimizer를 사용한다. 이는 학습 속도를 정의할 필요가 없으며, 대부분의 경우 잘 동작한다.

---

```
with tf.name_scope('optimiser'):
    optimiser = tf.train.AdamOptimizer().minimize(loss_operation)
```

---

정확도는 이전과 동일하게 계산되지만 정확한 예측값과 정확도 계산을 위해 이름 범주를 추가했다.

---

<sup>2</sup> 숫자 0-9까지로 열 개의 유닛을 갖고 있기 때문이다. - 옮긴이

---

```

with tf.name_scope('accuracy'):
    with tf.name_scope('correct_prediction'):
        predictions = tf.argmax(logits, 1)
        correct_predictions = tf.equal(predictions, tf.argmax(y_input, 1))
    with tf.name_scope('accuracy'):
        accuracy_operation = tf.reduce_mean(
            tf.cast(correct_predictions, tf.float32))
tf.summary.scalar('accuracy', accuracy_operation)

```

---

정확도에 대한 스칼라 요약 데이터도 추가된다. 다음 단계에서는 이전 절과 동일하게 세션을 시작하고 변수를 초기화한다. 그러므로 코드는 생략한다. 요약 데이터는 모두 합쳐야 하며, 훈련 요약 데이터와 테스트 요약 데이터를 작성하기 위한 파일도 정의돼야 한다.

---

```

merged_summary_operation = tf.summary.merge_all()
train_summary_writer = tf.summary.FileWriter('/tmp/train', session.graph)
test_summary_writer = tf.summary.FileWriter('/tmp/test')

```

---

1장에서는 summary\_writer로 그래프를 한번 작성해봤다. 훈련하는 도중에 정확도를 계산하고 값이 요약 데이터에 추가되는 것을 제외하면, 이번 훈련 동작은 이전에 했던 훈련과 거의 다르지 않다. 그리고 배치에서 데이터가 로드되고 훈련을 시작할 수 있다.

---

```

test_images, test_labels = mnist_data.test.images, mnist_data.test.labels

for batch_no in range(total_batches):
    mnist_batch = mnist_data.train.next_batch(batch_size)
    train_images, train_labels = mnist_batch[0], mnist_batch[1]
    _, merged_summary = session.run([optimiser, merged_summary_operation],
                                    feed_dict={
                    x_input: train_images,
                    y_input: train_labels,
                    dropout_bool: True
                })
    train_summary_writer.add_summary(merged_summary, batch_no)

```

```

if batch_no % 10 == 0:
    merged_summary, _ = session.run([merged_summary_operation,
                                     accuracy_operation], feed_dict={
        x_input: test_images,
        y_input: test_labels,
        dropout_bool: False
    })
    test_summary_writer.add_summary(merged_summary, batch_no)

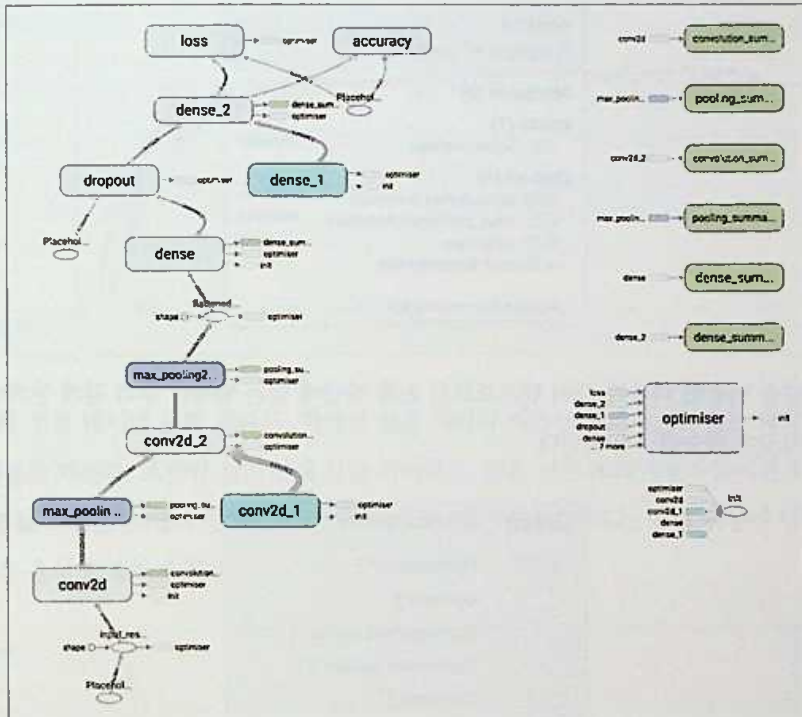
```

---

요약 데이터는 데이터 훈련이 진행되는 동안 for문마다 반환되고 추가되는데, 매 열 번째 반복마다 테스트 요약 데이터가 추가된다. 드롭아웃은 훈련 중에만 적용되고 테스트 중에는 적용되지 않는다. 신경망의 정의와 요약을 마쳤으니 이제 실행할 수 있다. 훈련 프로세스를 보려면 1장, '시작하기'에서 설명한 텐서보드로 이동한다.

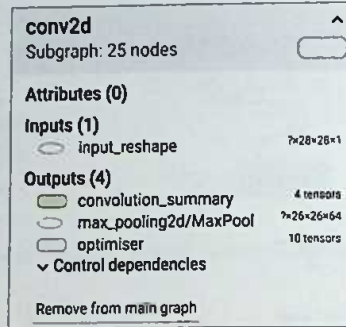
### 딥러닝에서 텐서보드 활용하기

브라우저에서 텐서보드를 연 다음 Graph 탭으로 이동한다. 앞서 정의하고 훈련시킨 그래프가 표시돼야 한다. 노드를 마우스 오른쪽 버튼으로 클릭하면, 메인 그래프로부터 삭제할 연산을 선택할 수 있다. 정렬 후 그래프는 다음과 같다.

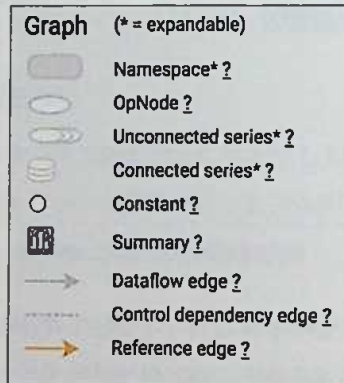


이전 사례에서 훈련되고 정의된 그래프

앞서 정의된 모든 레이어가 얼마나 잘 그려지고 있는지 확인하길 바란다. 이는 아키텍처의 정의를 확인하는 데 매우 유용하다. 그래프의 방향이 모든 세부 사항과 함께 꼼꼼하게 그려졌다. 각각의 노드를 클릭하면, 다음과 같이 입력 및 출력 텐서 노드 셰이프와 같은 세부 사항을 볼 수 있다.

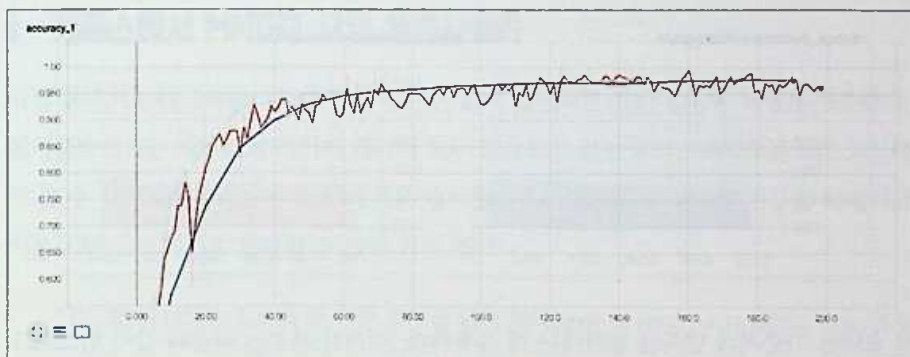


해당 값을 이용해 다음과 같이 텐서보드의 왼쪽 하단에 있는 범례를 주의 깊게 살펴보고, 이 페이지에 익숙해지길 바란다.

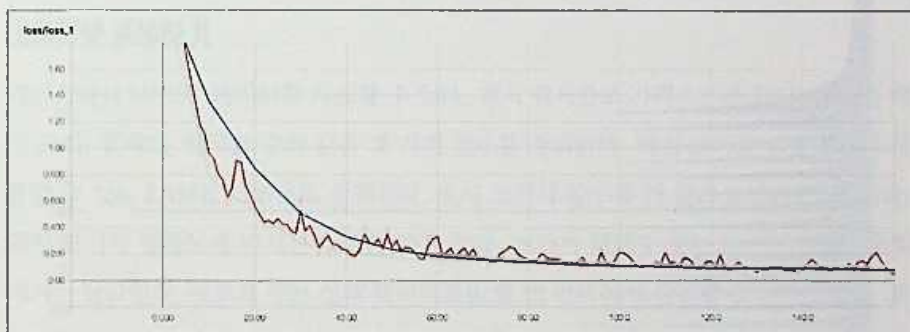


이름 범주는 그룹화돼 있으며, 노드의 더하기 기호를 클릭하면 각각의 구성 요소를 볼 수 있다. 노드는 색상별로 정렬된다. 이제 Scalar 페이지로 이동한다. 페이지에서 다음 그림과 같은 정확도 그래프를 볼 수 있다.



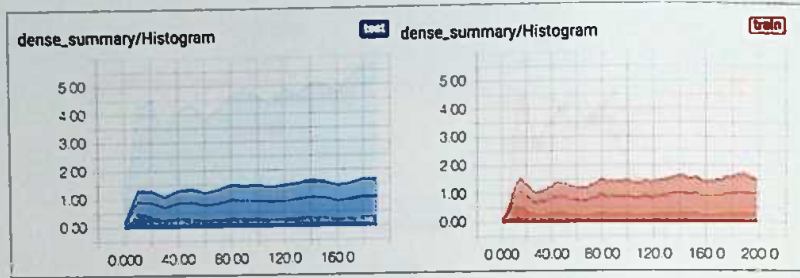


주황색 선은 데이터 훈련 결과고, 파란색 선은 데이터 테스트 결과다. 두 선은 대략 동일한 패턴을 따른다. 흐릿한 선은 본래 값을 의미하고, 밝은 선은 매끄럽게 다듬어진 곡선이다. 평활 계수는 UI에서 선택할 수 있다. 테스트 데이터의 정확도는 97% 이상에 달했다. 다음은 손실 요약의 수치다.

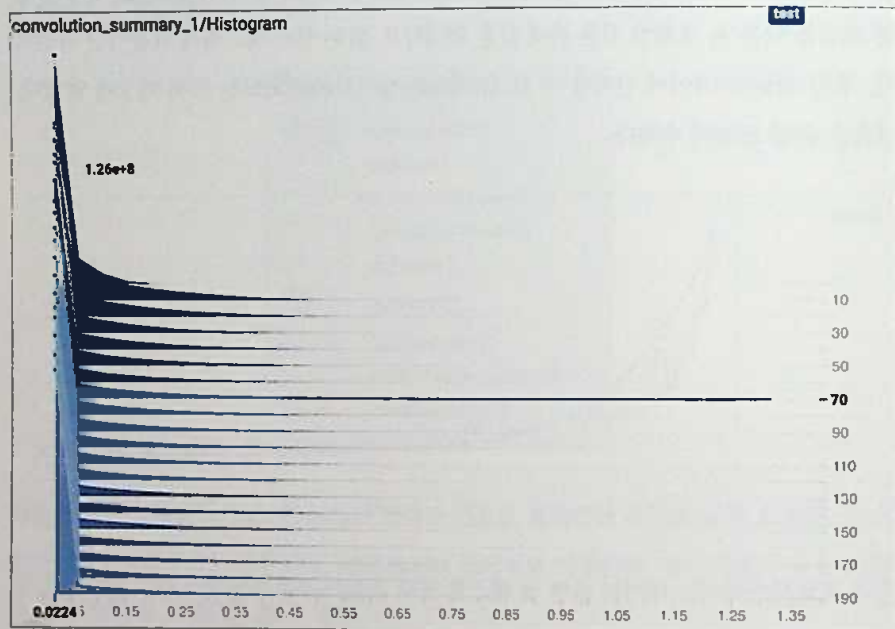


훈련 프로세스에서는 데이터 훈련 및 테스트 모두 손실이 꾸준히 감소하고 있다(좋은 징조다). 모든 요약 데이터는 훈련 진행 중에 업데이트되며, 손실이 감소하고 정확도가 증가하면서 97.38%라는 훌륭한 수치의 테스트 정확도를 얻게 된다.

이렇게 하면 모델이 더 나은 결과를 얻을 수 있는 방향으로 학습이 진행되고 있는지 알 수 있다. 그 외에 최소, 최대, 평균, 표준 편차와 같은 다른 요약 데이터도 유용하다. 다음은 밀집 레이어의 그래프다.



이 요약은 가중치의 변화를 알아내는 데 유용하다. 이러한 분포는 다음과 같이 히스토그램으로 시각화할 수도 있다.



이는 로짓의 가중치를 확장시킨 것이다. 텐서보드를 이용해 할 수 있는 훌륭한 시각화며 훈련에 매우 유용하다. 모델이 더 깊어지면 정확도가 크게 향상하는 것을 확인할 수 있다. 다음 절에서는 케라스 API를 이용해 동일한 모델을 훈련시키는 방법을 살펴본다. 이제 딥러닝 모델 및 훈련 프로세스를 점검하는 데 텐서보드에 얼마나 유용한지 알게 됐다.

## ■ 케라스에서 MNIST 모델 훈련시키기

이번 절에서는 `tf.keras` API를 이용해 이전 절에서 정의한 것과 동일한 모델을 사용한다. 텐서플로에서는 케라스와 레이어 패키지 모두 학습하는 것이 좋다. 여러 오픈소스 코드에서 볼 수 있기 때문이다. 이 책의 목적은 텐서플로에서 제공하는 다양한 기능을 이해하고 이를 기반으로 제품을 개발할 수 있게 하는 것이다.

“코드를 작성하는 것보다 더 자주 코드를 읽게 된다(`code is read more often than it is written`).”

위 인용문을 옆두에 두고, 다양한 API를 이용해 동일한 모델을 구현하는 방법을 살펴보자. 최신 알고리즘을 구현한 모든 오픈소스 코드에는 이러한 API가 혼합돼 있다. 케라스 구현부터 시작해보자.

### 데이터셋 준비하기

케라스에서 MNIST 데이터를 사용할 수 있다. 먼저 텐서플로 가져오기를 한다. 이어서 배치 크기, 클래스, 에포크 수와 같은 몇 가지 상수를 정의한다. 배치 크기는 컴퓨터에서 사용할 수 있는 RAM을 기반으로 선택된다. 배치 크기가 클수록 더 많은 RAM이 필요하다. 배치 크기가 정확도에 미치는 영향은 아주 적다. 여기서 클래스 수 수는 10이며, 다른 문제에서는 달라진다. 에포크 수는 전체 데이터셋을 몇 번 반복해서 훈련할 것인지 정한다. 모든 에포크를 마친 후에 손실이 감소된다면 더 높은 수로 설정할 수 있다. 경우에 따라 훈련이 길어지면 더 좋은 정확도를 얻는다. 이제 데이터셋을 생성하기 위한 단계를 살펴보자.

1. 다음과 같이 입력 이미지의 크기를 설정한다.

---

```
batch_size = 128
no_classes = 10
epochs = 2
image_height, image_width = 28, 28
```

---

2. 케라스 유틸리티를 이용해 데이터를 디스크에서 메모리로 로드한다.

---

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
```

---

3. 주어진 코드를 이용해 벡터를 이미지 형식으로 변경하고 컨볼루션을 위한 입력 크기를 정의한다.

---

```
x_train = x_train.reshape(x_train.shape[0], image_height, image_width, 1)
x_test = x_test.reshape(x_test.shape[0], image_height, image_width, 1)
input_shape = (image_height, image_width, 1)
```

---

4. 다음과 같이 데이터 타입을 float으로 변환한다.

---

```
x_train = x_train.astype('float32')
x_test = x_test.astype('float32')
```

---

5. 데이터를 정규화한다.

---

```
x_train /= 255
x_test /= 255
```

---

6. 카테고리 레이블을 원-핫 인코딩으로 변환한다.

---

```
y_train = tf.keras.utils.to_categorical(y_train, no_classes)
y_test = tf.keras.utils.to_categorical(y_test, no_classes)
```

---

텐서플로 코드 작성 방식과는 매우 다르다. 데이터가 이미 메모리에 로드됐으며 플레이스 홀더와 같은 개념이 없다.

## 모델 구축하기

이 절에서는 완전 연결 레이어로 이어지는 몇 개의 컨볼루션 레이어를 사용해 앞서 본 데이터셋을 훈련시킬 예정이다. 풀링, 드롭아웃, 밀집 레이어가 뒤따르는 두 개의 컨볼루션 레이어가 있는 간단한 순차 모델을 구성한다. 순차 모델에는 레이어를 차례로 쌓아 올리 기 위한 add 메소드가 있다. 첫 번째 레이어에는 64개의 필터가 있고, 두 번째 레이어에는 128개의 필터가 있다. 커널 크기는 모든 필터에 대해 3이다. 컨볼루션 레이어 다음에 최대 풀링을 적용한다. 컨볼루션 레이어의 출력은 1차원으로 변경돼 드롭아웃으로 연결된 두 개의 완전 연결 레이어로 연결된다.

다중 클래스 분류 문제이므로 마지막 레이어는 소프트맥스로 연결된다. 다음 코드는 모델을 정의하는 방법을 보여준다.

```
def simple_cnn(input_shape):
    model = tf.keras.models.Sequential()
    model.add(tf.keras.layers.Conv2D(
        filters=64,
        kernel_size=(3, 3),
        activation='relu',
        input_shape=input_shape
    ))
    model.add(tf.keras.layers.Conv2D(
        filters=128,
        kernel_size=(3, 3),
        activation='relu'
    ))
    model.add(tf.keras.layers.MaxPooling2D(pool_size=(2, 2)))
    model.add(tf.keras.layers.Dropout(rate=0.3))
    model.add(tf.keras.layers.Flatten())
    model.add(tf.keras.layers.Dense(units=1024, activation='relu'))
    model.add(tf.keras.layers.Dropout(rate=0.3))
    model.add(tf.keras.layers.Dense(units=no_classes, activation='softmax'))
    model.compile(loss=tf.keras.losses.categorical_crossentropy,
                  optimizer=tf.keras.optimizers.Adam(),
                  metrics=['accuracy'])
```

```
return model
simple_cnn_model = simple_cnn(input_shape)
```

---

이 모델은 정의만 된 상태며 컴파일돼야 한다. 컴파일 중 손실 함수, 최적화 알고리즘, 측정 항목(메트릭<sup>metric</sup>)이 정의돼야 한다. 손실 함수는 아담 알고리즘에 의해 최적화된 교차 엔트로피가 되며, 측정 항목으로 정확도를 보고한다. 로드된 데이터를 이용해 데이터를 훈련시키고 평가한다. 훈련 파라미터와 함께 훈련 데이터를 로드하고 모델을 학습시킨다.

---

```
simple_cnn_model.fit(x_train, y_train, batch_size, epochs, (x_test, y_test))
train_loss, train_accuracy = simple_cnn_model.evaluate(x_train, y_train,
verbose=0)
print('Train data loss:', train_loss)
print('Train data accuracy:', train_accuracy)
```

---

케라스 API를 사용하면 세션은 생성되지 않는다. 이어서 다음과 같이 테스트 데이터를 평가할 수 있다.

---

```
test_loss, test_accuracy = simple_cnn_model.evaluate(
    x_test, y_test, verbose=0)
print('Test data loss:', test_loss)
print('Test data accuracy:', test_accuracy)
```

---

또한 세션의 명시적 생성 없이도 평가 데이터가 생성된다. 실행이 끝나면 결과는 다음과 유사하다.

---

```
Loss for train data: 0.0171295607952
Accuracy of train data: 0.995016666667
Loss for test data: 0.0282736890309
Accuracy of test data: 0.9902
```

---



이렇게 하면 테스트 데이터에 대해 99%의 정확도를 얻게 된다. 훈련 정확도가 테스트 데이터 정확도보다 높으며, 항상 두 정확도를 출력해보는 것이 좋다. 정확도가 다른 이유는 반복 횟수 때문이다. 텐서플로에서 생성한 이전 모델보다 정확도가 좀 더 높은 이유는 데이터셋의 차이 때문이다.

## ■ 그 외 일반적으로 사용되는 이미지 테스트 데이터셋

MNIST 데이터셋은 알고리즘을 테스트하는 데 가장 일반적으로 사용되는 데이터셋이다. 하지만 이미지 분류 알고리즘을 테스트하는 데 사용되는 다른 데이터셋도 있다.

### CIFAR 데이터셋

CIFAR<sup>Canadian Institute for Advanced Research-10</sup> 데이터셋에는 50,000개의 훈련용 이미지와 10,000개의 테스트용 이미지로 구성된 총 60,000개의 이미지가 있다. 클래스 수는 10이며, 이미지 크기는 32×32 픽셀이다. 다음은 각 클래스로부터 임의로 선택된 이미지다.

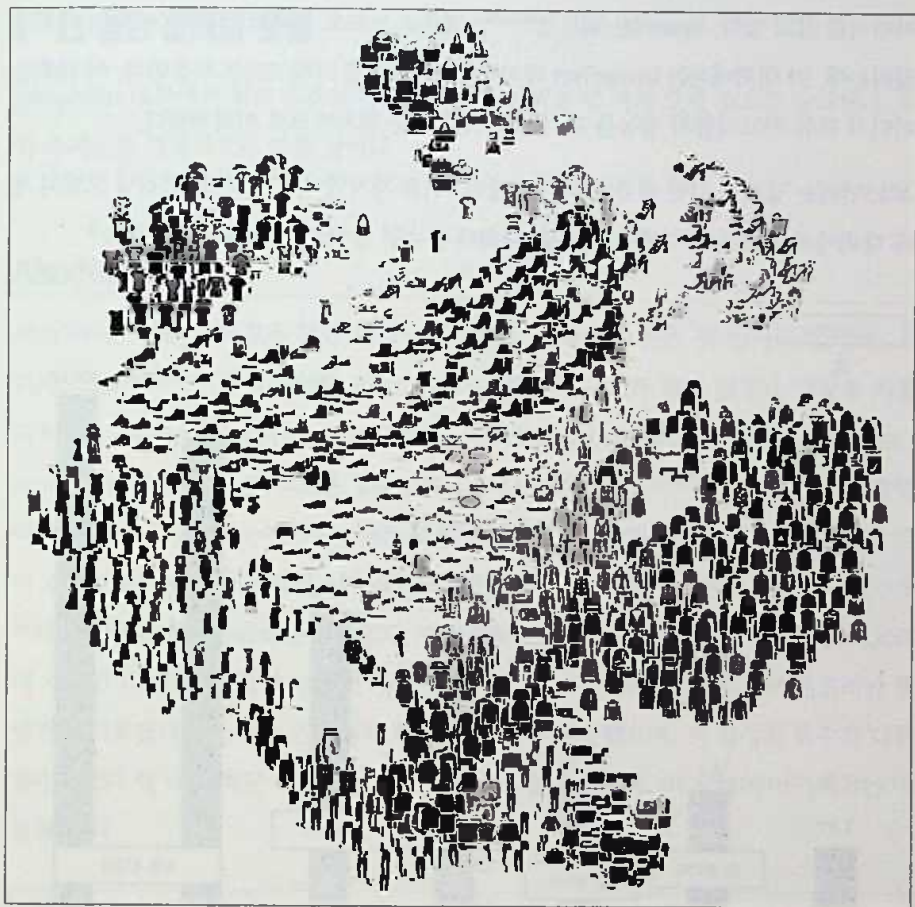


이미지는 작으며, 한 가지 객체만 포함한다. CIFAR-100 데이터셋은 동일한 수의 이미지를 갖고 있지만 100개의 클래스를 포함한다. 그러므로 클래스당 600개의 이미지만 있다. 각각의 이미지에는 슈퍼 레이블(super label)과 파인 레이블(fine label)이 있다. CIFAR 데이터셋을 시험 삼아 사용해보고 싶다면 `tf.keras.datasets`에서 이용할 수 있다.

## Fashion-MNIST 데이터셋

Fashion-MNIST는 MNIST 데이터셋의 대안으로 만들어진 데이터셋이다. MNIST로 생성된 데이터셋이 너무 쉽다고 생각해서 만들어진 데이터셋이며, MNIST를 바로 대체할 수 있다.

다음은 주성분 분석(PCA, Principal Component Analysis)이 수행된 후 데이터셋에서 임의로 선택된 예시 이미지다.



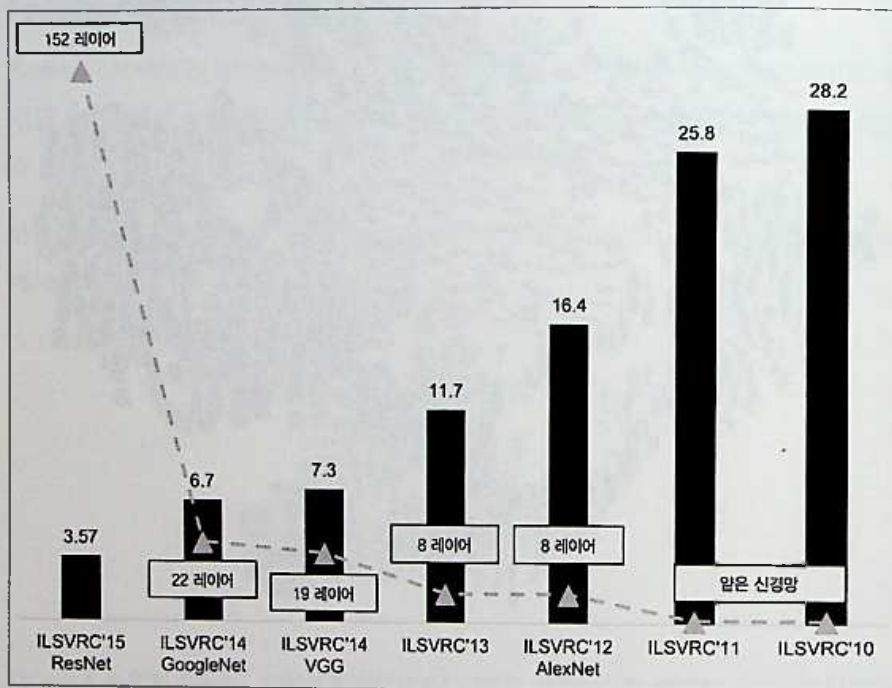
데이터셋 크기, 레이블 수, 이미지 크기는 MNIST와 유사하다. 자세한 내용은 <https://github.com/zalandoresearch/fashion-mnist>에서 확인할 수 있다. 앞서 학습한 모델을 실행하면, 정확도를 확인할 수 있다.

## ImageNet 데이터셋 및 대회

ImageNet은 컴퓨터 비전 데이터셋으로 21,841개의 Synset으로 색인된 14,197,122개의

이미지를 갖고 있다. Synset은 워드넷<sup>WordNet</sup> 계층의 노드로 동의어 집합이다. 매년 대회가 열리는데, 이 대회에서는 ImageNet 데이터셋의 1,000개 카테고리를 사용한다. 이 대회는 이미지 분류 알고리즘의 성능을 평가하기 위한 표준 벤치마크로 여겨져왔다.

2013년에는 딥러닝 기반 컴퓨터 비전 모델이 1위를 차지했다. 그 이후로 딥러닝 모델이 계속 대회에서 우승을 차지했다. 다음은 대회에서 수년 동안의 오류율을 보여준다.<sup>3</sup>



수년간 정확도가 증가할 뿐만 아니라 레이어의 수 또한 증가하고 있음을 알 수 있다. 다음으로 그래프에 있는 모델을 살펴본다.

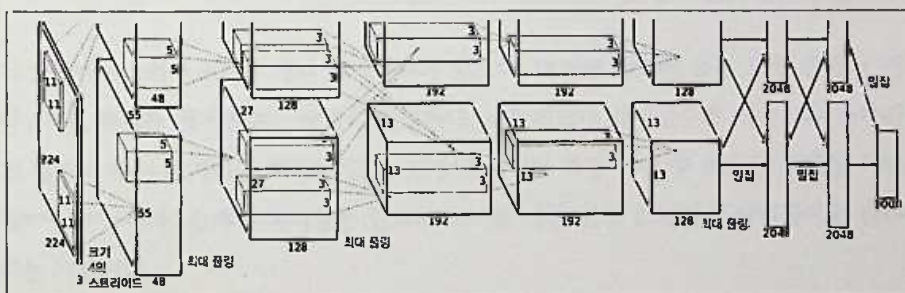
3 여기서 사용하는 오류율은 top-5 error rate로 모델이 가장 높은 확률로 예측한 다섯 가지 예측이 정답이 아닐 확률을 의미한다.  
- 옮긴이

## I 더 깊은 딥러닝 모델

ImageNet 대회에서 최고 수준의 기술적 성과를 보여준 몇 가지 모델 정의를 살펴본다. 다음 주제들을 개별적으로 다룰 것이다.

### AlexNet 모델

AlexNet은 컴퓨터 비전을 위한 딥러닝이 열풍을 일으키게 만든 첫 번째 논문이다. 크리제브스키(Krizhevsky) 등이 제안한 AlexNet은 이 분야의 시초가 됐고 엄청난 영향을 미쳤다(<https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf>). 이 모델은 'ImageNet 2013 챌린지(ImageNet 2013 challenge)'에서 우승했다. 오류율은 15.4%였는데, 이는 다른 모델들보다 훨씬 우수한 수치였다. 모델은 다섯 개의 컨볼루션의 레이어를 가진 상대적으로 간단한 아키텍처였다. 챌린지 과제는 1,000개 카테고리의 객체를 분류하는 것이었다. 이미지와 데이터에는 22,000개가 넘는 카테고리에 포함된 1,500만 개의 주석 달린 이미지가 있었고, 그중 오직 1,000개의 카테고리만 챌린지에 사용됐다. AlexNet은 활성화 함수로 ReLU를 사용했으며, 이 활성화 함수가 다른 함수들보다 몇 배나 빠른 속도로 훈련시킬 수 있음을 확인했다. 이 모델의 아키텍처는 다음과 같다.



크리제브스키 외의 허락하에 재구성 게재



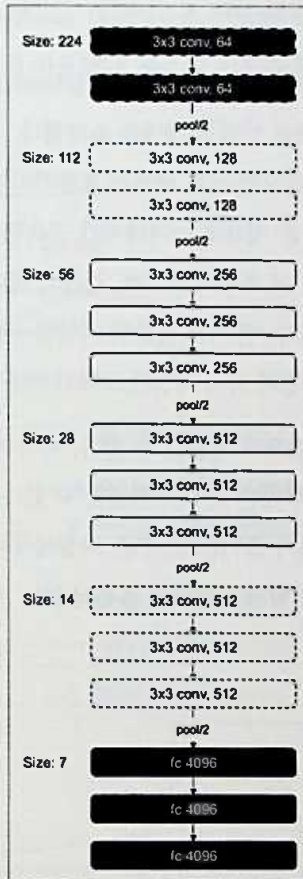
또한 이 논문은 이미지 변환, 수평 반전, 무작위로 이미지 잘라내기와 같은 데이터 확장 기법을 사용했다. 드롭아웃 레이어는 오버피팅을 방지한다. 이 모델은 훈련을 위해 바닐라 확률적 경사하강법<sup>56)</sup>을 사용했다. SGD의 매개변수는 훈련을 위해 신중하게 선택된다. 고정된 세트의 훈련이 반복됨에 따라 학습 속도는 바뀐다. 모멘텀 및 가중치 감소<sup>weight decay</sup>는 훈련을 위해 고정 값을 사용한다. 이 논문에서는 LRN<sup>Local Response Normalization</sup>이라는 개념을 소개한다. LRN 레이어는 특정 필터에서 크게 활성화되는 것을 방지하기 위해 필터의 모든 픽셀을 정규화한다.

최근 연구에서 LRN으로 인한 많은 개선이 없다고 제안하게 되면서 해당 레이어는 더 이상 사용하지 않는다. AlexNet에는 총 6,000만 개의 매개변수가 있다.

## VGG-16 모델

VGG 모델은 옥스포드의 비주얼 지오메트리 그룹<sup>Visual Geometry Group</sup>을 의미한다. 이 모델은 매우 간단하고 AlexNet보다 더 깊다. 이 논문은 16과 19 레이어 깊이의 두 가지 모델을 다룬다. 모든 CNN 레이어는 스트라이드와 패딩 크기가 1인  $3 \times 3$  크기의 필터와 스트라이드가 2인  $2 \times 2$  크기의 최대 풀링을 사용한다. 이로 인해 매개변수 개수가 감소한다. 최대 풀링 때문에 크기는 줄어들었지만 필터의 수는 레이어와 함께 증가한다. 16개 레이어의 모델 구조는 다음과 같다.

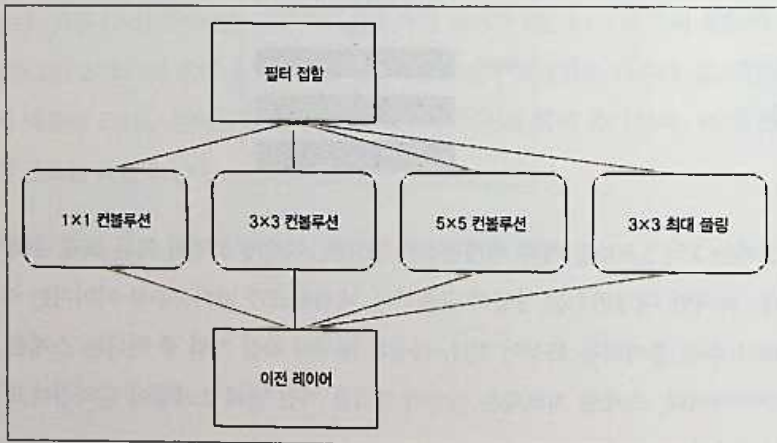




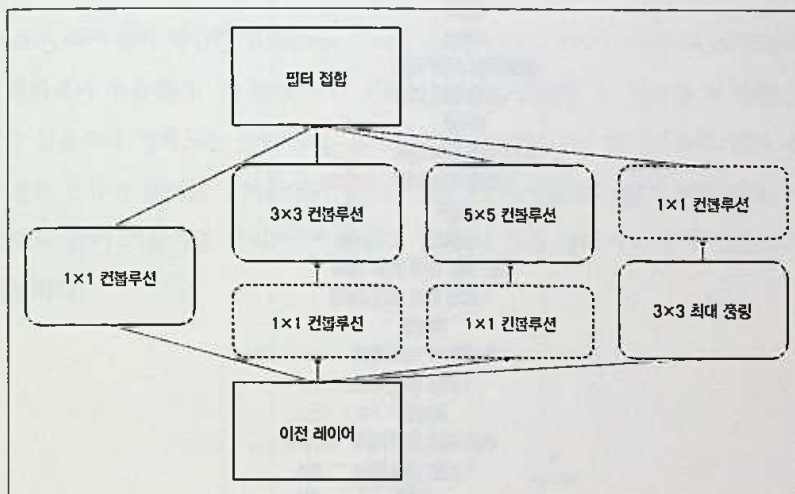
이 모델에는 1억 3,800만 개의 매개변수가 있으며, 여기에 설명된 모든 모델 중에서 가장 크다. 하지만 매개변수는 상당히 균일하다. 네트워크가 깊어질수록 이미지는 작아지고 필터의 수는 증가하는 특성이 있다. 사용된 데이터 확장 기법 중 하나는 스케일 지터링(scale jittering)이다. 스케일 지터링은 임의의 크기를 가진 먼의 스케일이 달라진다고 간주하는 기법이다.

## 구글 인셉션-V3 모델

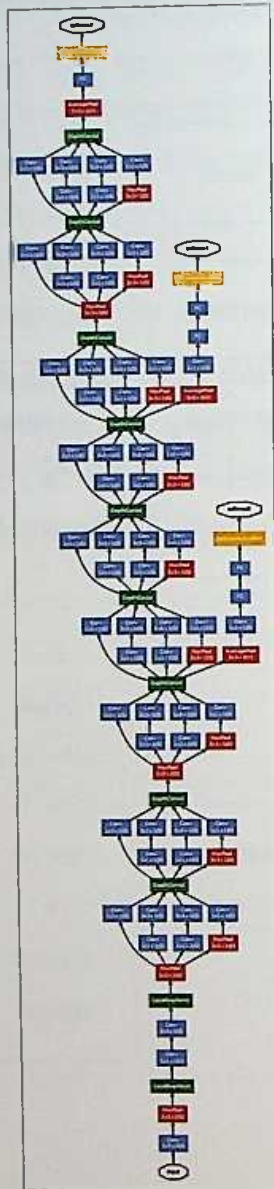
인셉션-V3 Inception-V3는 세계적 Szegedy 등이 제안한 모델(<https://arxiv.org/pdf/1409.4842.pdf>)이며 더 나은 일반화 방법을 가진 인셉션 개념을 소개했다. 이 모델은 2014년 ImageNet 대회에서 우승한 아키텍처로, 속도와 크기 면에서 효율적이며 AlexNet보다 12배 적은 매개변수를 가지고 있다. 인셉션은 매크로 아키텍처가 구축된 마이크로 아키텍처다. 각 숨겨진 레이어에는 고수준의 이미지 표현이 있으며, 각각의 레이어에서 풀링 또는 다른 레이어를 사용할 수 있는 옵션이 있다. 한 가지 종류의 커널만 사용하는 대신에 인셉션은 여러 가지 커널을 사용한다. 평균 풀링에 이어 다양한 크기의 컨볼루션이 이어지면서 연결된다. 커널 매개변수는 데이터를 기반으로 학습할 수 있다. 여러 가지 커널을 사용하기 때문에 모델은 높은 추상화뿐만 아니라 작은 특징도 검출할 수 있다.  $1 \times 1$  컨볼루션은 특징을 감소시키며 그에 따라 연산 작업이 줄어든다. 추론 과정에서는 RAM 사용량이 적다. 다음은 다양한 커널 크기 및 풀링과 함께 컨볼루션 옵션이 있는 가장 간단한 형태의 인셉션 모델이다.



AlexNet 또는 VGG와 달리 연산은 병렬적으로 진행된다. 출력 볼륨이 크기 때문에 차원 수 감소를 위해  $1 \times 1$  필터를 사용했다. 축소된 차원이 아키텍처에 추가되면 다음과 같이 된다.



모든 부가적인 것들이 있는 모델의 전체 아키텍처는 다음과 같다.

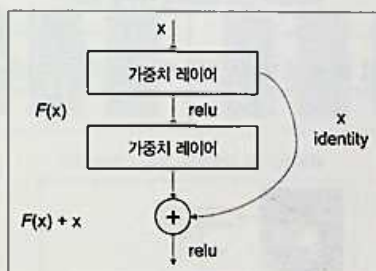


구글 인셉션 V3 모델 아키텍처(세게디 외의 허락하에 재구성 게재)

총 100개의 레이어를 가진 아홉 개의 인셉션 모듈이 있으며 성능이 우수하다.

## 마이크로소프트 ResNet-50 모델

ResNet은 허<sup>16</sup> 등이 제안한 모델(<https://arxiv.org/pdf/1512.03385.pdf>)이며 2015년 Image Net 대회에서 우승했다. 이 방법은 더 깊은 신경망도 훈련될 수 있음을 보여줬다. 네트워크가 깊을수록 정확도는 높아진다. 오버피팅이나 많은 수의 매개변수로 인한 것이 아니라 훈련 오류가 줄어들기 때문이며, 이는 기울기가 역전파되지 않기 때문이다. 이는 다음 그림과 같이 기울기를 잔여(residual) 블록과 함께 더 깊은 레이어로 바로 전송하기 때문에 가능하다.



두 개의 레이어가 연결돼 하나의 잔여 블록을 형성하며, 레이어 간에 훈련이 전달되는 것을 볼 수 있다. 이러한 기법에 의해 역전파는 에러를 이전 레이어로 전달할 수 있다.

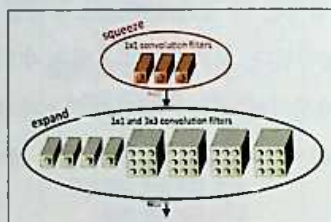


[https://github.com/tensorflow/tensorflow/tree/r1.4/tensorflow/python/keras/\\_impl/keras/applications](https://github.com/tensorflow/tensorflow/tree/r1.4/tensorflow/python/keras/_impl/keras/applications)의 모델 정의를 사용할 수 있다. 이 모델의 모든 레이어가 정의돼 있으며, ImageNet 데이터셋의 사전 훈련된 가중치를 사용할 수 있다.

## SqueezeNet 모델

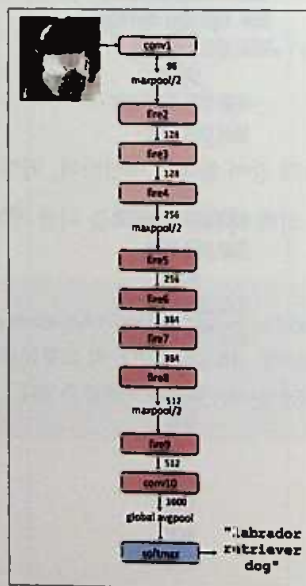
SqueezeNet 모델은 이나돌라<sup>1)</sup> 등이 소개한 모델(<https://arxiv.org/pdf/1602.07360.pdf>)이며 모델 크기와 매개변수의 수를 줄인다.

$3 \times 3$  필터를  $1 \times 1$  필터로 대체함으로써 다음과 같이 네트워크를 더 작게 만든다.



이나돌라 외의 허락하에 재구성 게재

$3 \times 3$  필터의 입력 수는 상위 레벨에서 발생할 때 레이어의 다운샘플링을 줄여 크기가 큰 활성화 맵을 제공한다.

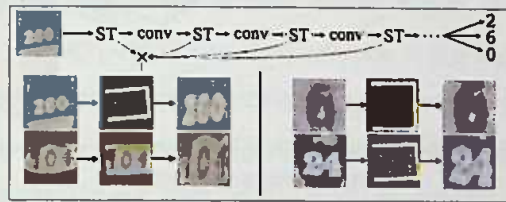


이나돌라 외의 허락하에 재구성 게재



## 공간 변환 네트워크

공간 변환 네트워크(Spatial Transformer Network)는 자데르버그(Jaderberg) 등이 제안한 모델(<https://arxiv.org/pdf/1506.02025.pdf>)이며 CNN을 통과하기 전에 이미지를 변환하려고 한다. 컨볼루션 전에 이미지를 수정하려고 하기 때문에 다른 네트워크와 다르다. 이 네트워크는 이미지를 변환하기 위해 매개변수를 학습한다. 또한, 매개변수는 아핀 변환(Affine transformation)을 하기 위해 학습된다. 아핀 변환을 적용함으로써 공간 불변성(spatial invariance)이 이뤄진다. 이전 네트워크에서는 공간 불변성이 최대 풀링 레이어에 의해 이뤄졌다. 공간 변환 네트워크의 배치는 다음과 같다.

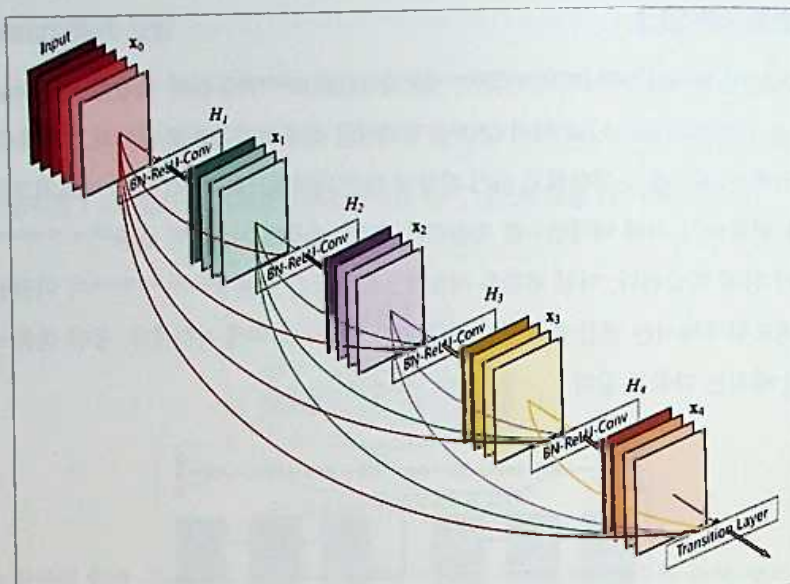


자데르버그 외의 허락하에 재구성 게재

## DenseNet 모델

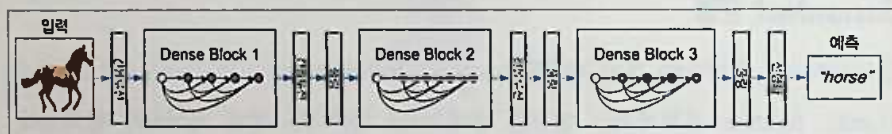
DenseNet은 후양(Huang) 등이 제안한 ResNet의 확장판이다(<https://arxiv.org/pdf/1608.06993.pdf>). ResNet 블록에서는 합산을 통해 이전 레이어와 현재 레이어가 합쳐졌다. DenseNet의 경우, 연결을 통해 이전 레이어와 현재 레이어가 합쳐진다. DenseNet은 모든 레이어를 이전 레이어와 연결하고 현재 레이어를 다음 레이어에 연결한다.

다음 다이어그램에서 특징 지도가 어떻게 다른 레이어로 입력되는지 확인할 수 있다.



후방 외의 허락하에 재구성 계재

이런 방법은 더 매끄러운 기울기, 특징 변환 등과 같은 여러 가지 이점을 제공한다. 또한 매개변수의 개수가 줄어든다.



후방 외의 허락하에 재구성 계재

이미지 분류 작업을 위한 모든 최신 알고리즘을 다뤘다. 학습한 아키텍처 중 하나를 이미지 분류 작업에 사용할 수 있다. 다음 절에서는 이러한 고급 아키텍처를 이용해 반려 동물을 예측하는 모델을 훈련시키는 방법과 정확도를 향상시키는 방법을 확인한다.

## 1 개와 고양이를 예측하는 모델 훈련시키기

이 절에서는 개와 고양이를 예측하는 모델을 준비하고 훈련시키고 정확도를 향상시키는 몇 가지 기법을 살펴본다. 대부분의 이미지 분류 문제는 이러한 패러다임에 해당된다. 확장과 전이 학습 같은 이번 절에서 다루는 기법들은 여러 가지 문제에 유용하다.

### 데이터 준비하기

분류를 위해 <sup>kaggle</sup>캐글로부터 데이터를 다운로드한 후 적합한 포맷으로 저장한다. [www.kaggle.com](https://www.kaggle.com)에서 가입하고 로그인한 후 <https://www.kaggle.com/c/dogs-vs-cats/data>로 이동한다. 해당 페이지에서 train.zip과 test1.zip 파일을 다운로드한다. train.zip 파일에는 25,000개의 반려 동물 이미지 데이터가 있다. 여기서는 모델을 훈련시키는 데 일부 데이터만 사용할 것이다. GPU와 같은 더 우수한 컴퓨팅 환경을 갖췄다면 제안한 것보다 더 많은 데이터를 사용해도 된다. 다음 스크립트를 실행해서 이미지를 재정렬하고 필요한 폴더를 생성한다.

---

```
import os
import shutil

work_dir = '' # 정확한 디렉터리 지정
image_names = sorted(os.listdir(os.path.join(work_dir, 'train'))))

def copy_files(prefix_str, range_start, range_end, target_dir):
    image_paths = [os.path.join(work_dir, 'train', prefix_str + '.' + str(i) +
    '.jpg')
                    for i in range(range_start, range_end)]
    dest_dir = os.path.join(work_dir, 'data', target_dir, prefix_str)
    os.makedirs(dest_dir)
    for image_path in image_paths:
        shutil.copy(image_path, dest_dir)

copy_files('dog', 0, 1000, 'train')
```

```
copy_files('cat', 0, 1000, 'train')
copy_files('dog', 1000, 1400, 'test')
copy_files('cat', 1000, 1400, 'test')
```

---

실습을 위해 고양이와 개 이미지를 1,000개만 사용한다. 그래서 다운로드한 폴더로부터 이미지 0-1,000을 새로 생성한 train 폴더의 cat 폴더 안으로 복사한다. 마찬가지로 이미지 1,000-1,400은 data/test/cat으로, 이미지 0-1,000은 train/dog으로, 이미지 1,000-1,400은 data/test/dog으로 복사해 각 클래스에 대해 1,000개의 훈련용 이미지와 400개의 검증용 이미지를 제공한다.

## 간단한 CNN으로 벤치마킹하기

이 데이터셋에 대해 이전의 simple\_cnn 모델을 실행해보고 어떻게 동작하는지 살펴보자. 이 모델의 성능은 다른 기술과 비교해 판단할 수 있는 기준점이 된다. 다음과 같이 데이터 로드 및 훈련을 위한 몇 가지 변수를 정의한다.

---

```
image_height, image_width = 150, 150
train_dir = os.path.join(work_dir, 'train')
test_dir = os.path.join(work_dir, 'test')
no_classes = 2
no_validation = 800
epochs = 2
batch_size = 200
no_train = 2000
no_test = 800
input_shape = (image_height, image_width, 3)
epoch_steps = no_train // batch_size
test_steps = no_test // batch_size
```

---

이 상수는 이 절에서 논의된 고양이와 개를 예측하기 위한 모델을 훈련시키는 기술에 사용된다. 여기서 훈련과 테스트를 위해 2,800개의 이미지를 사용하는데, 이는 개인용 컴퓨터의 RAM에 적당하다. 하지만 더 큰 규모의 데이터셋에 대해서는 유지할 수 없다. 훈련 및 테스트를 위해서는 한 번에 이미지 묶음 하나씩만 로드하는 것이 좋다. 이를 위해 `tf.keras`에는 `ImageDataGenerator`라 불리는 클래스가 있는데, 이 클래스는 필요할 때마다 이미지를 읽는다. `simple_cnn` 모델은 이전 절로부터 가져오기를 했다고 가정한다.

다음은 이미지를 로드하기 위해 `ImageDataGenerator`를 사용한 예다.

---

```
generator_train = tf.keras.preprocessing.image.ImageDataGenerator(rescale=1. / 255)
generator_test = tf.keras.preprocessing.image.ImageDataGenerator(rescale=1. / 255)
```

---

또한 이 정의는 이미지가 로드될 때 이미지의 크기를 재조정한다. 이어서 디렉터리로부터 이미지를 읽어오기 위해 다음과 같이 `flow_from_directory` 메소드를 이용한다.

---

```
train_images = generator_train.flow_from_directory(
    train_dir,
    batch_size=batch_size,
    target_size=(image_width, image_height))

test_images = generator_test.flow_from_directory(
    test_dir,
    batch_size=batch_size,
    target_size=(image_width, image_height))
```

---

이미지를 로드하기 위한 디렉터리, 배치 크기, 이미지의 타겟 크기가 인수로 전달된다. 이 메소드는 크기 재조정을 수행하고 데이터를 배치 형태로 전달한다. 생성기(generator)는 모델을 학습시키기 위해 직접적으로 사용할 수 있다. 이 모델의 `fit_generator` 메소드는 다음과 같이 사용할 수 있다.

```
simple_cnn_model.fit_generator(  
    train_images,  
    steps_per_epoch=epoch_steps,  
    epochs=epochs,  
    validation_data=test_images,  
    validation_steps=test_steps)
```

이 모델은 훈련용 이미지 생성기의 데이터에 맞춰졌다. 에포크 개수는 훈련을 통해 정의되며 모델 훈련 성능을 얻기 위해 검증용 데이터가 전달된다. `fit_generator`는 데이터 및 모델 훈련의 병렬 처리를 가능하게 한다. CPU가 크기 재조정을 수행하는 동안 GPU는 모델 훈련을 수행할 수 있다. 이는 컴퓨팅 리소스의 효율성을 높여준다. 50 에포크 이후 이 모델의 정확도는 60% 정도가 돼야 한다. 이어서 개선된 성능을 얻기 위해 데이터셋을 확장하는 방법을 살펴본다.

## 데이터셋 확장하기

데이터 확장(`data augmentation`)은 데이터셋 크기를 늘리는 방법을 제공한다. 데이터 확장은 훈련 중 노이즈를 유발해 다양한 입력에 대해 모델이 더욱 견고해지게 만든다. 이 기술은 데이터셋이 작은 경우에 유용하며 다른 기술과 결합해 사용할 수 있다. 다음으로 여러 종류의 확장 기법을 살펴본다.

### 확장 기법

다음과 같이 이미지를 확장하는 다양한 방법들이 존재한다.

- 플립핑(`flipping`): 이미지가 수평 또는 수직 방향으로 대칭 또는 반전된다.
- 무작위 잘라내기(`random cropping`): 이미지 일부를 잘라내어 모델이 가려진 부분을 처리할 수 있다.
- 시어링(`shearing`): 이미지 모양이 변형돼 객체의 모양에 영향을 준다.



- 확대/축소: 이미지가 확대/축소된 부분은 다양한 크기의 이미지를 처리하도록 훈련한다.
- 회전: 객체의 다양한 각도 변화를 처리할 수 있도록 객체를 회전시킨다.
- 화이트닝whitening: 중요한 데이터만 보존하는 주성분 분석을 통해 화이트닝 작업을 수행한다.
- 정규화: 평균 및 분산을 표준화해 픽셀을 정규화한다.
- 채널 시프팅channel shifting: 다양한 아티팩트artifact로 인해 발생한 색상 변화에 대해 모델이 견고해지도록 색상 채널을 이동한다.

이러한 모든 기법들은 ImageDataGenerator에서 구현돼 데이터셋 크기를 늘린다. 다음은 앞서 언급한 몇 가지 확장 기법을 사용하는 수정 버전의 generator\_train이다.

---

```
generator_train = tf.keras.preprocessing.image.ImageDataGenerator(
    rescale=1. / 255,
    horizontal_flip=True,
    zoom_range=0.3,
    shear_range=0.3,)
```

---

앞 코드의 generator\_train 대신 사용하면 정확도가 90%로 향상된다. 확장 매개변수를 변경하고 그 변화에 주목하길 바란다. 다음 절에서는 전이 학습이라는 기술을 논의하는데, 이 기술은 적은 수의 데이터로 더 큰 모델을 훈련시키는 데 도움이 된다.

## 모델의 전이 학습 또는 미세 조정

전이 학습(transfer learning)이란 더 큰 데이터셋으로 사전 훈련된 모델을 학습하는 프로세스다. 무작위 초기화로 모델을 훈련하는 것은 종종 결과를 얻는 데 많은 시간과 에너지를 필요로 한다. 사전 훈련된 모델로 초기화하면 좀 더 빠르게 수렴하게 돼 시간과 에너지를 아낄 수 있다. 사전 훈련된 모델은 신중하게 선택된 초매개변수(hyperparameter)로 훈련시키는

경우가 많다.

사전 훈련된 모델의 여러 레이어를 수정하지 않은 채 사용할 수도 있고, 변경 사항에 맞춰질 수 있도록 조금 훈련시킬 수도 있다. 이 절에서는 수백만 개의 클래스가 있는 ImageNet 데이터셋으로 학습된 모델을 미세 조정(fine-tuning)하거나 전이 학습하는 방법을 살펴본다.

### 병목 특징에 대해 훈련시키기

이전 절에서 다른 모델은 간단하므로 정확도가 떨어질 수 있다. 좀 더 복잡한 모델을 만들어야 하는데 아예 처음부터 새롭게 만들 수는 없다. 그러므로 추출한 병목 특징(bottleneck feature)을 이용해 분류 모델을 훈련시킨다. 병목 특징은 수백만 개의 이미지를 학습하는 복잡한 아키텍처로 인해 발생하는 특성이다. 이미지는 정방향으로 처리 완료되고 마지막 단계 앞의 레이어가 저장된다. 이로부터 간단한 로지스틱 분류 모델이 훈련된다. 병목 레이어는 다음과 같이 추출한다.

---

```
generator = tf.keras.preprocessing.image.ImageDataGenerator(rescale=1. / 255)

model = tf.keras.applications.VGG16(include_top=False)

train_images = generator.flow_from_directory(
    train_dir,
    batch_size=batch_size,
    target_size=(image_width, image_height),
    class_mode=None,
    shuffle=False
)
train_bottleneck_features = model.predict_generator(train_images, epoch_steps)

test_images = generator.flow_from_directory(
    test_dir,
    batch_size=batch_size,
    target_size=(image_width, image_height),
    class_mode=None,
    shuffle=False
```

)

```
test_bottleneck_features = model.predict_generator(test_images, test_steps)
```

---

이미지 예측을 위해 VGG 모델을 사용한다. 레이블은 다음과 같이 지정된다.

```
train_labels = np.array([0] * int(no_train / 2) + [1] * int(no_train / 2))  
test_labels = np.array([0] * int(no_test / 2) + [1] * int(no_test / 2))
```

---

두 개의 레이어가 있는 순차 모델이 구축되고 컴파일되며 병목 특징으로 훈련된다. 이는 다음 코드를 사용해 구현할 수 있다.

```
model = tf.keras.models.Sequential()  
model.add(tf.keras.layers.Flatten(input_shape=train_bottleneck_features.  
shape[1:]))  
model.add(tf.keras.layers.Dense(1024, activation='relu'))  
model.add(tf.keras.layers.Dropout(0.3))  
model.add(tf.keras.layers.Dense(1, activation='softmax'))  
model.compile(loss=tf.keras.losses.categorical_crossentropy,  
              optimizer=tf.keras.optimizers.Adam(),  
              metrics=['accuracy'])
```

---

병목 특징은 다음과 같은 코드를 이용해 훈련된다.

```
model.fit(  
    train_bottleneck_features,  
    train_labels,  
    batch_size=batch_size,  
    epochs=epochs,  
    validation_data=(test_bottleneck_features, test_labels))
```

---

모델을 학습하는 다른 접근법을 제공하며 훈련 데이터가 적을 때 유용하다. 종종 모델을

더 빠르게 훈련시키는 방법이기도 하다. 새로운 작업에 적응시키는 데 사전 훈련된 모델의 최종 활성화 함수만 사용한다. 다음과 같이 이 아이디어는 여러 개의 레이어를 미세 조정하는 것으로 확장된다.

## 딥러닝의 여러 레이어 미세 조정하기

사전 훈련된 모델을 로드할 수 있으며 몇 개의 레이어만 훈련시킬 수 있다. 이 접근법은 주어진 문제가 모델을 학습시킨 이미지와 매우 다른 경우에 더 효과적이다. 미세 조정은 딥러닝에서 흔히 사용된다. 데이터셋이 작을 때 이점이 있으며, 빠르게 최적화할 수도 있다.

작은 데이터셋으로 훈련된 딥 네트워크는 오버피팅이 발생한다. 이러한 종류의 오버피팅 또한 미세 조정 작업을 통해 방지할 수 있다. 더 큰 데이터셋으로 훈련된 모델 또한 활성화 및 특성이 작은 데이터셋과 유사할 것으로 기대하고 있기 때문에 비슷할 것이다. 다음과 같이 저장된 가중치 경로로 시작할 수 있다.

---

```
top_model_weights_path = 'fc_model.h5'
```

---

VGG 모델을 로드하고 초기 레이어는 훈련되지 않도록 설정한다. VGG 모델은 다음 절에서 자세히 다룰 예정이다. 지금은 VGG를 이미지 데이터에 대해 잘 동작하는 커다란 딥러닝 모델이라고 생각하면 된다. 다음 코드를 이용해 완전 연결 레이어를 새로운 훈련 가능한 레이어로 교체한다.

---

```
model = tf.keras.applications.VGG16(include_top=False)
```

---

다음과 같이 작은 2~레이어 피드포워드 네트워크를 숨겨진 유닛, 활성화, 드롭아웃과 함께 VGG 모델 최상위에 구축할 수 있다.

---

```
model_fine_tune = tf.keras.models.Sequential()
model_fine_tune.add(tf.keras.layers.Flatten(input_shape=model.output_shape))
model_fine_tune.add(tf.keras.layers.Dense(256, activation='relu'))
model_fine_tune.add(tf.keras.layers.Dropout(0.5))
model_fine_tune.add(tf.keras.layers.Dense(no_classes, activation='softmax'))
```

---

최상위 모델 또한 이미 완전히 훈련된 가중치와 함께 로드할 수 있다. 그런 다음 최상위 모델은 컨볼루션 레이어에 추가된다.

---

```
model_fine_tune.load_weights(top_model_weights_path)
model.add(model_fine_tune)
```

---

상위 25개 레이어를 훈련할 수 없도록 설정해 가중치를 업데이트하지 않도록 할 수 있다. 나머지 레이어만 업데이트된다.

---

```
for vgg_layer in model.layers[:25]:
    vgg_layer.trainable = False
```

---

차수가 4인 느린 학습 속도의 경사 하강 최적화 알고리즘과 함께 모델을 컴파일한다.

---

```
model.compile(loss='binary_crossentropy',
              optimizer=tf.keras.optimizers.SGD(lr=1e-4, momentum=0.9),
              metrics=['accuracy'])
```

---

앞서 다룬 확장 기법을 시어링, 확대/축소, 플립핑과 결합할 수 있다. 생성기는 훈련 및 검증 데이터셋을 가진 `flow_from_directory`와 함께 추가할 수 있다. 이제 이 모델은 데이터 확장과 함께 미세 조정을 할 수 있다. 이 훈련 방법은 모든 기존 메소드보다 우수한 정확도를 제공한다. 다음은 전이 학습에 대한 가이드다.

| 데이터 크기 | 유사한 데이터셋         | 다른 데이터셋          |
|--------|------------------|------------------|
| 작은 데이터 | 출력 레이어를 미세 조정한다. | 깊은 레이어를 미세 조정한다. |
| 큰 데이터  | 전체 모델을 미세 조정한다.  | 처음부터 훈련시킨다.      |

데이터 크기에 따라 미세 조정 레이어 수를 정할 수 있다. 데이터가 적을수록 미세 조정 레이어 수가 적다. 전이 학습 기법을 이용해서 모델의 정확도를 향상시키는 방법을 살펴봤다.

## ■ 실제 애플리케이션 개발하기

고양이와 개를 인식하는 것은 좋은 문제지만 그렇게 중요한 문제는 아니다. 제품에 실제로 적용된 이미지 분류 애플리케이션은 다를 수 있으며, 다른 데이터와 타깃 등이 있을 수 있다. 이 절에서는 이러한 다른 설정을 해결할 수 있는 팁과 요령을 살펴본다. 새로운 문제에 접근할 때 고려해야 하는 요인은 다음과 같다.

- 타깃의 수. 열 개의 클래스 문제입니까, 아니면 10,000개의 클래스 문제입니까?
- 클래스 내 차이는 얼마나 광대합니까? 예를 들면, 다른 종류의 고양이는 하나의 클래스 레이블로 식별돼야 합니까?
- 클래스 간 차이는 얼마나 광대합니까? 예를 들면, 다른 고양이인지 식별돼야 합니까?
- 데이터는 얼마나 큼니까?
- 데이터의 균형은 어떻습니까?
- 이미 많은 이미지로 훈련된 모델이 있습니까?
- 적용 유추 시간과 모델 크기에 맞는 필요 조건은 무엇입니까? 아이폰에서는 50밀리초입니까? 또는 구글 클라우드 플랫폼에서는 10밀리초입니까? 모델을 저장하기 위해 소모되는 RAM의 크기는 얼마입니까?



이미지 분류 문제에 접근할 때 위 질문을 던져보길 바란다. 질문에 대한 답에 따라 다음 절에서 설명한 것과 같이 훈련 아키텍처를 설계하고 정확도를 높일 수 있다.

## 올바른 모델 선택하기

아키텍처에는 다양한 옵션이 있으며, 적용 유연성에 따라 모델을 선택할 수 있다. 컨볼루션 레이어는 작고 느리지만 밀집 레이어는 크고 빠르다는 사실을 기억하길 바란다. 크기, 런타임, 정확도 사이에는 트레이드오프가 발생한다. 최종 결정을 하기 전에 모든 아키텍처를 테스트하는 것이 좋다. 애플리케이션에 따라 일부 모델이 다른 모델보다 잘 작동할 수 있다. 예측을 빠르게 하기 위해 입력 크기를 줄일 수 있다. 아키텍처는 다음 절에서 설명한 것과 같이 측정 항목(메트릭)을 기반으로 선택할 수 있다.

## 언더피팅 및 오버피팅 시나리오 해결하기

문제에 비해 모델이 너무 크거나 작은 경우가 있다. 그로 인해 언더피팅되거나 오버피팅될 수 있다. 모델이 너무 작아서 훈련 정확도가 낮은 경우 언더피팅이 발생한다. 모델이 너무 크고 훈련 정확도와 테스트 정확도 사이에 큰 차이가 있을 때는 오버피팅이 발생한다.

언더피팅은 다음과 같은 방법으로 해결할 수 있다.

- 추가 데이터 얻기
- 더 큰 모델을 사용해보기
- 데이터가 작은 경우 전이 학습 기법이나 데이터 확장을 시도해보기

오버피팅은 다음과 같은 방법으로 해결할 수 있다.

- 드롭아웃과 배치 정규화 같은 기법을 사용해 정규화하기
- 데이터셋 확장하기

항상 손실에 주의하길 바란다. 손실은 반복을 통해 감소해야 한다. 손실이 감소하지 않으면 훈련이 중지됐다는 의미다. 한 가지 해결책은 다른 옵티마이저를 사용하는 것이다. 클래스 불균형은 손실 함수에 가중치를 줌으로써 처리할 수 있다. 요약 데이터를 확인하기 위해 항상 텐서보드를 사용할 것을 권한다. 어느 정도의 데이터가 필요한지는 가늠하기 어렵다. 이 절은 모든 딥러닝 모델 훈련에 대한 가장 좋은 수업이다. 다음으로 몇 가지 애플리케이션별 가이드를 제공한다.

## 얼굴에서 성별과 나이 검출하기

애플리케이션에서 얼굴로부터 성별과 나이를 검출해야 할 수 있다. 얼굴 이미지는 얼굴 검출기로 얻을 수 있다. 잘라낸 얼굴 이미지를 훈련용 데이터로 제공할 수 있으며 유사한 얼굴 이미지가 테스트를 위해 제공되어야 한다. 필요한 추론 시간을 기반으로 OpenCV나 CNN 얼굴 검출 알고리즘을 선택할 수 있으며, 훈련을 위해 인셉션이나 ResNet을 사용할 수 있다. 데이터가 영상이라서 허용된 추론 시간이 적은 경우 세 개의 컨볼루션 레이어와 두 개의 완전 연결 레이어를 사용하는 것이 좋다. 나이 데이터셋의 경우 클래스 불균형이 심하므로 정밀도와 재현율 같은 다른 측정 항목을 사용하는 것이 좋다.

## 의류 모델 미세 조정하기

의류 모델을 미세 조정하는 것은 좋은 선택이다. 이 경우에는 속성을 분류하는 여러 개의 소프트맥스 레이어를 사용하는 것이 좋다. 속성으로는 패턴, 색상 등이 있다.

## 브랜드 안정성

SVM(Support Vector Machine(서포트 벡터 머신))으로 병목 레이어를 훈련하는 것은 이미지가 클래스 사이에서 상당히 다를 수 있기 때문에 좋은 옵션이다. 이는 노골적인 이미지를 방지하는 데 유용하므로 일반적으로 콘텐츠 규제에 사용된다. 우리는 이미지 분류에서 새로운

문제에 접근하는 방법을 학습했다.

## ■ 요약

분류 작업을 훈련하는 데 기본적인지만 유용한 모델을 다뤘다. 케라스 및 텐서플로 API와 함께 MNIST 데이터셋의 간단한 모델을 살펴봤다. 또한 훈련 프로세스를 보기 위해 텐서보드를 이용하는 방법도 살펴봤다. 그런 다음 최신 기술의 아키텍처를 몇 가지 특정 애플리케이션과 함께 논의했다. 또한 데이터 확장, 병목 레이어 훈련, 사전 훈련된 모델 미세 조정과 같은 몇 가지 정확도 향상 방법을 다뤘으며, 새로운 모델을 훈련시키는 팁과 요령도 소개했다.

다음 장에서는 딥러닝 모델을 시각화하는 방법을 살펴본다. 또한 추론을 위해 이 장에서 훈련된 모델을 적용한다. 애플리케이션을 통해 이미지 검색 애플리케이션을 위해 훈련된 레이어를 사용하는 방법도 학습한다. 이어서 자동 인코더의 개념을 이해하고 특성의 차원에 사용한다.



## 이미지 검색

딥러닝은 모델의 특징이나 표현이 훈련 중에 학습되기 때문에 표현 학습이라고 한다. 훈련 과정 중에 숨겨진 레이어에서 생성된 시각적 특징은 거리 계산에 사용할 수 있다. 딥러닝에서 사용하는 모델은 분류 작업에 따라 다양한 레이어에서 에지, 패턴 등을 검출하는 방법을 학습한다. 3장에서는 다음 내용들을 살펴본다.

- 분류를 목적으로 훈련된 모델에서의 특징 추출법
- 생산 시스템에서 빠른 추론을 위한 텐서플로 서빙 사용법
- 특징을 사용해 쿼리 이미지와 대상 집합 간의 유사도를 계산하는 방법
- 순위를 지정하기 위한 분류 모델 사용법
- 검색 시스템의 속도를 높이는 방법
- 전체 시스템 아키텍처 살펴보기

- 대상 이미지가 너무 많은 경우에 사용하는 자동 인코더(autoencoder)를 통해 간소화된 기술자 이해하기
- 노이즈를 제거하는 자동 인코더 훈련

## ■ 시각적 특징의 이해

딥러닝 모델은 해석이 불가능하다는 점에서 비판을 받기도 한다. 신경망 기반 모델은 딥러닝 모델의 동작을 사람이 추론하기 힘들기 때문에 블랙박스로 간주된다. 딥러닝 모델의 레이어를 통한 이미지 변형은 활성화 함수로 인해 비선형적이기 때문에 시각화에 용이하지는 않다. 딥 네트워크의 레이어를 시각화해서 해석 불가능성을 비판하는 목소리에 대응하고자 개발된 여러 가지 방법들이 있다. 이번 절에서는 모델의 동작을 이해하기 위해 딥 레이어를 시각화하는 방법을 알아본다.

시각화는 모델의 활성화 및 경사도 값을 사용해 수행된다. 활성화는 다음 기법을 사용해 시각화된다.

- 최근접 이웃(nearest neighbour): 이미지의 레이어 활성화가 수행되며, 해당 활성화에 가장 근접한 이미지들을 함께 볼 수 있다.
- 차원 축소(dimensionality reduction): 활성화의 차원은 시각화를 위해 PCA(Principal Component Analysis)나 t-SNE(t-Distributed Stochastic Neighbor Embedding)를 통해 2차원이나 3차원으로 축소할 수 있다. PCA는 최대 변량 방향으로 값을 프로젝션해 차원을 축소한다. t-SNE는 최근접 점을 3차원으로 매핑해 차원을 축소한다. 차원 축소의 사용과 기술은 이 책에서 다루지 않는다. 차원 축소에 대해 더 알고 싶다면 머신 러닝에 대한 기본 자료를 참조하자.





위키피디아에는 차원 축소 기술을 이해하는 데 도움이 되는 좋은 자료가 많다. 다음은 참조할 만한 링크 목록이다.

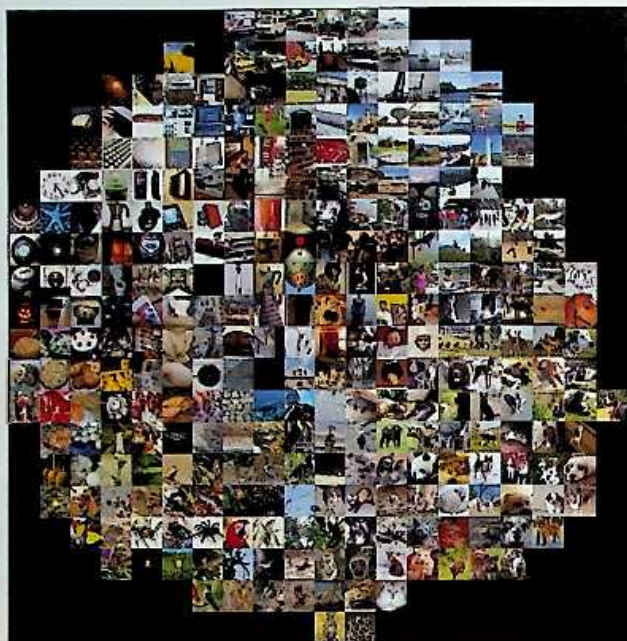
- [https://en.wikipedia.org/wiki/Dimensionality\\_reduction](https://en.wikipedia.org/wiki/Dimensionality_reduction)
- [https://en.wikipedia.org/wiki/Principal\\_component\\_analysis](https://en.wikipedia.org/wiki/Principal_component_analysis)
- [https://en.wikipedia.org/wiki/T-distributed\\_stochastic\\_neighbor\\_embedding](https://en.wikipedia.org/wiki/T-distributed_stochastic_neighbor_embedding)
- [https://en.wikipedia.org/wiki/Locality-sensitive\\_hashing](https://en.wikipedia.org/wiki/Locality-sensitive_hashing)

- **최댓값 패치**: 하나의 뉴런이 활성화되고 활성화가 최대인 해당 패치를 캡처한다.
- **페색occlusion**: 다양한 위치에서 이미지가 페색되며, 활성화는 이미지에서 중요한 부분을 파악하기 위해 히트 맵으로 표시된다.

다음 절에서는 이러한 특징을 시각화하는 구현 방법을 살펴본다.

## 딥러닝 모델 활성화의 시각화

모델 아키텍처는 레이어의 필터로 시각화된다. 이 기술로는 초기 레이어만 이해 가능하며, 마지막 레이어는 최근접 이웃 방식에 유용하다. ImageNet의 데이터셋을 최근접 이웃 방식으로 정렬하면 다음과 같다.



이 이미지를 보면 동일한 객체가 함께 표시됨을 알 수 있다. 재미있는 것 중 하나는 개, 원숭이, 치타와 같은 동물은 단일 레이블로 훈련하지는 않았지만 함께 표시될 수 있다는 점이다. 이미지의 최근접 이웃 시각화는 객체가 유사할 때 유용하고, 모델 예측 방법을 이해할 수 있다. 여기의 마지막 레이어는 주성분 분석(PCA)이나 t-SNE와 같은 차원 축소 기술로 시각화할 수 있다. 다음 절에서는 차원 축소를 사용한 시각화의 구현을 살펴본다.

## 임베딩 시각화

임베딩 레이어(embedding layer)는 최종 레이어의 직전 레이어로 텐서보드(TensorBoard)를 사용해 2차원이나 3차원으로 시각화된다. 이번 절의 코드는 이미지 분류에 대한 장에서 훈련된 컨볼루션 신경망 모델을 따르는 것을 가정한다. 먼저 탭으로 구분된 메타데이터 파일이 필요하다. 메타데이터 파일의 모든 행에는 시각화되는 이미지의 레이블이 있어야 한다. 다음 코드와 같이 세션 생성과 초기화 사이에 정의된 임베딩을 저장할 새로운 변수가 필요하다.

---

```
no_embedding_data = 1000
embedding_variable = tf.Variable(
    tf.stack(mnist.test.images[:no_embedding_data], axis=0),
    trainable=False
)
```

---

다음 코드에서는 MNIST 테스트 데이터를 가져와 시각화를 위한 메타데이터 파일을 생성한다.

---

```
metadata_path = '/tmp/train/metadata.tsv'

with open(metadata_path, 'w') as metadata_file:
    for i in range(no_embedding_data):
        metadata_file.write('{}\n'.format(np.nonzero(
            mnist.test.labels[:,1])[1:][0][i]
        ))
```

---

앞 코드에서와 같이 매개변수를 설정해 임베딩 변수를 훈련이 불가능하게 설정한다. 이어서 프로젝터의 설정을 정의한다. 여기에는 임베딩 변수명인 `tensor_name`, 메타데이터 파일의 경로, 스프라이트 이미지가 필요하다. 스프라이트 이미지는 작은 이미지들을 포함하는 단일 이미지며, 임베딩을 통해 시각화되는 레이블을 나타낸다. 다음은 임베딩의 프로젝트 정의 코드다.

---

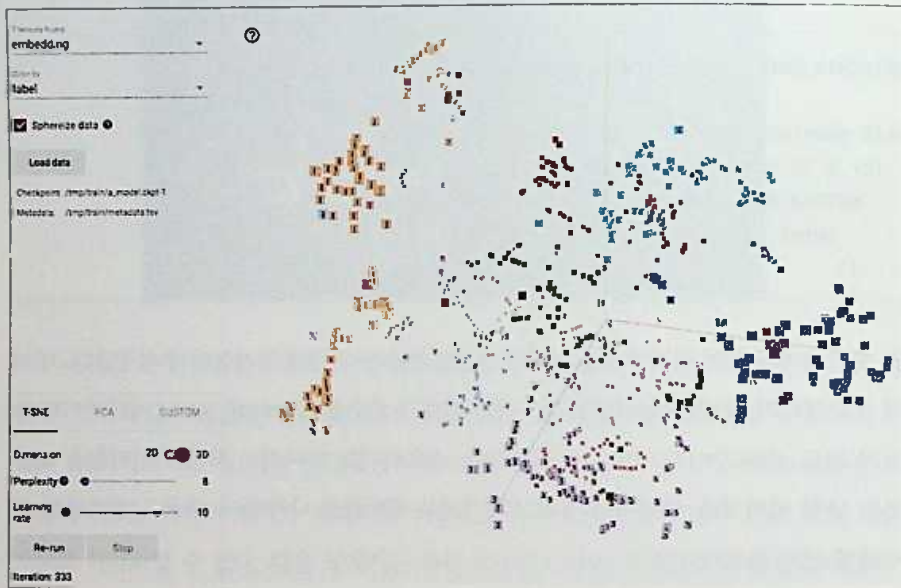
```
from tensorflow.contrib.tensorboard.plugins import projector
projector_config = projector.ProjectorConfig()
embedding_projection = projector_config.embeddings.add()
embedding_projection.tensor_name = embedding_variable.name
embedding_projection.metadata_path = metadata_path
embedding_projection.sprite.image_path = os.path.join(work_dir +
    '/mnist_10k_sprite.png')
embedding_projection.sprite.single_image_dim.extend([28, 28])
```

---

이제 스프라이트 이미지의 차원을 지정하고, 다음 코드와 같이 앞의 설정과 요약 작성자를 프로젝터와 사용해 임베딩을 시각화한다.

```
projector.visualize_embeddings(train_summary_writer, projector_config)
tf.train.Saver().save(session, '/tmp/train/model.ckpt', global_step=1)
```

이어서 모델이 세션과 함께 저장되며, 텐서보드로 이동해 다음의 시각화를 확인할 수 있다.



코드의 출력을 보여주는 텐서보드

유사한 시각화를 얻으려면 스크린샷과 같이 버튼으로 T-SNE 및 색상을 선택해야 하며, 숫자가 함께 표시되는 것을 확인할 수 있다. 이 시각화는 데이터와 훈련된 임베딩 검사에 유용한 텐서보드의 강력한 기능이다. 다음 절에서는 시각화를 위한 유도된 역전파를 구현한다.

## 유도된 역전파

특징을 직접 시각화하면 정보성이 떨어진다. 따라서 더 나은 시각화를 위한 필터를 활성화하기 위해 역전파의 훈련 과정을 거친다. 여기서 역전파로 활성화될 뉴런을 고르기 때문에 유도된 역전파(guided backpropagation)라고 부른다. 이번 절에서는 특징의 시각화를 위한 유도된 역전파를 구현한다.

다음과 같이 크기를 정의하고 VGG 모델을 불러온다.

---

```
image_width, image_height = 128, 128
vgg_model = tf.keras.applications.vgg16.VGG16(include_top=False)
```

---

해당 레이어들은 레이어명이 키로 된 딕셔너리와 접근이 용이한 키 값인 가중치가 있는 모델의 레이어로 구성된다. 이제 시각화 연산을 위한 다섯 번째 블록의 첫 번째 컨볼루션 레이어인 block5\_conv1을 가져온다. 이에 대한 입력과 출력은 다음과 같이 정의된다.

---

```
input_image = vgg_model.input
vgg_layer_dict = dict([(vgg_layer.name, vgg_layer) for vgg_layer in vgg_model.
                        layers[1:]])
vgg_layer_output = vgg_layer_dict['block5_conv1'].output
```

---

이제 손실 함수를 정의해야 한다. 손실 함수는 특정 레이어의 활성화를 최대화한다. 손실 함수를 최대화하기 위해서는 일반적인 경사 하강이 아닌 경사 상승 과정이 되어야 한다. 경사 상승 과정에서는 경사를 부드럽게 만드는 것이 중요하다. 따라서 여기서는 픽셀 경사를 정규화해 경사를 부드럽게 만들어준다. 또한 여기의 손실 함수는 꽤 빠르게 수렴될 수 있다.

이미지의 출력은 다시 시각화를 위해 정규화되며, 경사 상승은 최적화 과정에서 최댓값을 얻기 위해 사용한다. 이제 다음과 같이 평가자와 경사를 정의해 경사 상승 최적화를 시작한다. 다음으로 손실 함수를 정의하고 경사를 계산한다. 반복자는 다음과 같이 반복을 통



해 손실과 경사 값을 계산한다.

---

```
filters = []
for filter_idx in range(20):
    loss = tf.keras.backend.mean(vgg_layer_output[:, :, :, filter_idx])
    gradients = tf.keras.backend.gradients(loss, input_image)[0]
    gradient_mean_square = tf.keras.backend.mean(tf.keras.backend.
square(gradients))
    gradients /= (tf.keras.backend.sqrt(gradient_mean_square) + 1e-5)
    evaluator = tf.keras.backend.function([input_image], [loss, gradients])
```

---

입력은 약간의 노이즈를 추가한 임의의 그레이 이미지다. 임의의 이미지는 다음과 같이 생성되고 스케일링된다.

---

```
gradient_ascent_step = 1.
input_image_data = np.random.random((1, image_width, image_height, 3))
input_image_data = (input_image_data - 0.5) * 20 + 128
```

---

손실 함수의 최적화를 시작하자. 다음과 같이 일부 필터의 손실 값이 0인 경우에는 무시한다.

---

```
for i in range(20):
    loss_value, gradient_values = evaluator([input_image_data])
    input_image_data += gradient_values * gradient_ascent_step
    # print('Loss :', loss_value)
    if loss_value <= 0.:
        break
```

---

최적화한 후에는 평균을 빼고 표준 편차를 조정하는 정규화를 수행한다. 이어서 다음과 같이 필터의 스케일을 원래대로 돌려놓고 경사 값으로 잘라낸다(클리핑).



---

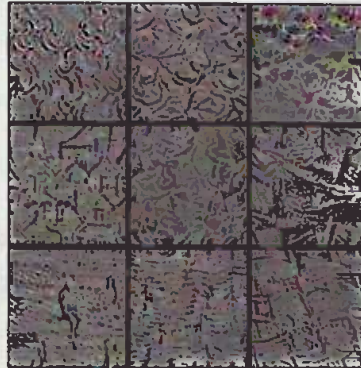
```

if loss_value > 0:
    filter = input_image_data[0]
    filter -= filter.mean()
    filter /= (filter.std() + 1e-5)
    filter *= 0.1
    filter += 0.5
    filter = np.clip(filter, 0, 1)
    filter *= 255
    filter = np.clip(filter, 0, 255).astype('uint8')
    filters.append((filter, loss_value))

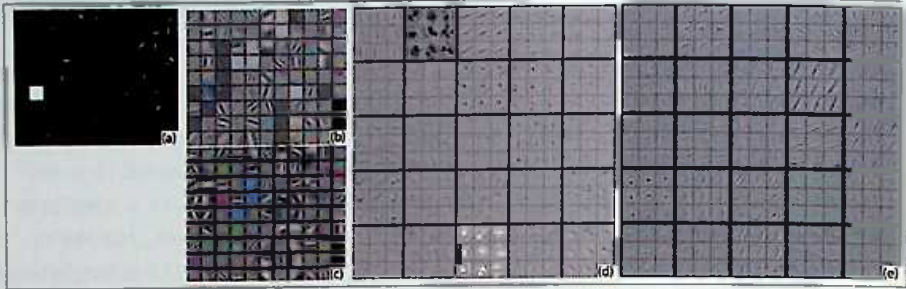
```

---

여기의 필터들은 무작위로 선택돼 시각화된다.



이미지를 스티칭<sup>stitching</sup>해 다음의 출력을 생성하는 코드는 코드 묶음에서 확인할 수 있다. 시각화 결과는 인식되는 부분이 점점 더 넓어지기 때문에 다음 레이어로 갈수록 복잡해진다. 일부 비슷한 필터는 회전만 한다. 여기서는 시각화의 계층이 자일러<sup>Zeiler</sup>(<https://arxiv.org/pdf/1412.6572.pdf>)에 의해 보여진 것처럼 명확하게 표시된다. 다른 레이어의 직접 시각화가 다음과 같이 표시된다.



자일리의 허락하에 재구성 제거

처음 두 개 레이어는 에지 검출과 코너 검출을 수행한 결과다. Gabor 필터 적용은 세 번째 레이어에서 보여준다. Gabor 필터는 선형이며 전통적으로 텍스처 분석에 사용된다. 지금까지 유도된 역전파를 통한 직접적인 특징 시각화를 살펴보았다. 다음으로, 시각화를 위한 DeepDream의 구현을 살펴보자.

## DeepDream

뉴런 활성화는 이미지를 합성하는 대신 네트워크의 일부 레이어에서 증폭되기도 한다. 특징의 영향을 보기 위해 원본 이미지를 증폭하는 개념을 DeepDream이라 한다. DeepDream은 다음 과정을 거쳐 생성한다.

1. 이미지를 가져오고 CNN에서 레이어를 선택한다.
2. 특정 레이어에서 활성화를 가져온다.
3. 경사와 활성화가 같아지도록 경사를 수정한다.
4. 이미지의 경사를 계산하고 역전파를 진행한다.
5. 정규화(regularization)를 사용해 이미지를 지터(jitter) 처리하고 정규화(normalization)한다.
6. 픽셀 값을 클리핑한다.
7. 프랙탈(fractal) 효과를 위해 이미지를 다중 스케일 처리한다.

필요한 패키지 가져오기는 다음과 같이 수행한다.

```
import os
import numpy as np
import PIL.Image
import urllib.request from tensorflow.python.platform
import gfile import zipfile
```

ImageNet 데이터셋으로 사전 훈련된 인셉션 모델 파일은 구글에서 제공한다. 여기서는 제공되는 모델을 다운로드해 예제에 활용한다. 다음과 같이 ZIP으로 압축된 모델 파일을 다운로드한 후 폴더에 압축 해제한다.

```
model_url =
    'https://storage.googleapis.com/download.tensorflow.org/models/inception5h.zip'
file_name = model_url.split('/')[-1]

file_path = os.path.join(work_dir, file_name)

if not os.path.exists(file_path):
    file_path, _ = urllib.request.urlretrieve(model_url, file_path)
zip_handle = zipfile.ZipFile(file_path, 'r')
zip_handle.extractall(work_dir)
zip_handle.close()
```

여기서 사용된 명령어는 작업 디렉터리에 세 개의 파일을 생성하는 것이다. 다음과 같은 코드를 사용해서 세션을 통해 훈련된 모델을 불러온 후 사용한다.

```
graph = tf.Graph()
session = tf.InteractiveSession(graph=graph)
model_path = os.path.join(work_dir, 'tensorflow_inception_graph.pb')
with gfile.GFile(model_path, 'rb') as f:
    graph_defnition = tf.GraphDef()
    graph_defnition.ParseFromString(f.read())
```

세션은 그래프를 초기화하면서 시작한다. 그다음에는 다운로드된 모델의 그래프 정의를 메모리에 적재한다. 다음과 같이 전처리 단계에서는 입력에서 ImageNet 평균을 빼준다. 이어서 전처리된 이미지는 그래프의 입력으로 사용된다.

---

```
input_placeholder = tf.placeholder(np.float32, name='input')
imagenet_mean_value = 117.0
preprocessed_input = tf.expand_dims(input_placeholder - imagenet_mean_value, 0)
tf.import_graph_def(graph_definition, {'input': preprocessed_input})
```

---

이제 추론을 위한 세션과 그래프가 준비됐다. 이중 선형 보간을 위해 `resize_image` 함수가 필요하다. 다음과 같이 텐서플로 세션에 이미지 크기를 조정하는 `resize` 함수 메소드를 추가한다.

---

```
def resize_image(image, size):
    resize_placeholder = tf.placeholder(tf.float32)
    resize_placeholder_expanded = tf.expand_dims(resize_placeholder, 0)
    resized_image = tf.image.resize_bilinear(
        resize_placeholder_expanded,
        size
    )[0, :, :, :]
    return session.run(resized_image, feed_dict={resize_placeholder: image})
```

---

다음과 같이 작업 디렉터리의 이미지를 메모리에 적재하고 부동 소수점 값으로 변환한다.

---

```
image_name = 'mountain.jpg'
image = PIL.Image.open(image_name)
image = np.float32(image)
```

---

참조하기 위해 적재된 이미지는 다음과 같다.



옥타브 수, 크기, 스케일 공간의 스케일은 다음과 같이 정의된다.

---

```
no_octave = 4  
scale = 1.4  
window_size = 51
```

---

여기의 값은 해당 예제에서 잘 동작하기 때문에 다른 이미지를 사용한다면 크기에 따라 조정이 필요하다. DeepDream을 위해 레이어를 선택한다. 다음과 같이 해당 레이어의 평균은 objective 함수가 된다.

---

```
score = tf.reduce_mean(objective_fn)  
gradients = tf.gradients(score, input_placeholder)[0]
```

---

최적화를 위해 이미지의 경사를 계산한다. 다음과 같이 다양한 스케일로 크기를 조절한 옥타브 이미지로 차이를 계산한다.



---

```

octave_images = []
for i in range(no_octave - 1):
    image_height_width = image.shape[:2]
    scaled_image = resize_image(
        image, np.int32(np.float32(image_height_width) / scale)
    )
    image_difference = image -
        resize_image(scaled_image, image_height_width)
    image = scaled_image
    octave_images.append(image_difference)

```

---

이제 모든 옥타브 이미지를 사용해 최적화를 실행한다. 다음과 같이 윈도우가 이미지를 지나가면서 DeepDream 이미지를 생성하기 위한 경사 활성화를 계산한다.

---

```

for octave_idx in range(no_octave):
    if octave_idx > 0:
        image_difference = octave_images[-octave_idx]
        image = resize_image(image, image_difference.shape[:2]) +
            image_difference

    for i in range(10):
        image_height, image_width = image.shape[:2]
        sx, sy = np.random.randint(window_size, size=2)
        shifted_image = np.roll(np.roll(image, sx, 1), sy, 0)
        gradient_values = np.zeros_like(image)

        for y in range(0, max(image_height - window_size // 2, window_size), window_size):
            for x in range(0, max(image_width - window_size // 2, window_size), window_size):
                sub = shifted_image[y:y + window_size, x:x + window_size]
                gradient_windows = session.run(gradients, {input_placeholder: sub})
                gradient_values[y:y + window_size, x:x + window_size] = gradient_windows
                gradient_windows = np.roll(np.roll(gradient_values, -sx, 1), -sy, 0)
                image += gradient_windows * (1.5 / (np.abs(gradient_windows).mean() + 1e-7))

```

---



이제 DeepDream을 생성하기 위한 최적화가 완료됐다. 다음과 같이 값을 클리핑해 저장한다.

```
image /= 255.0
image = np.uint8(np.clip(image, 0, 1) * 255)
PIL.Image.fromarray(image).save('dream_' + image_name, 'jpeg')
```

이번 절에서는 DeepDream을 만드는 과정을 살펴보고 결과는 다음과 같다.



보다시피, ‘개와 민달팽이’ 형태가 모든 곳에서 나타난다. 다양한 레이어에 대해 이를 시도하고 결과를 확인해보자. 이러한 결과들은 예술적 목적으로 사용된다. 비슷한 방식으로, 다른 아티팩트를 생성하기 위해 다른 레이어가 활성화될 수 있다. 다음 절에서는 딥러닝 모델을 속일 수 있는 몇 가지 적대적 사례를 살펴본다.

## 적대적 사례

이미지 분류 알고리즘은 여러 데이터셋에 대해 이미 인간 수준의 정확도에 도달했다. 하지만 여전히 적대적 사례들을 통해 이를 속일 수 있다. 적대적 사례란 필요한 결과물을 생성하기 위해 모델을 속이는 합성 이미지를 말한다. 아무 이미지나 가져와서 임의로 틀린 대상의 클래스를 선택하자. 굿펠로우<sup>Goodfellow</sup>(<https://arxiv.org/pdf/1412.6572.pdf>)가 보여

준 것처럼 네트워크를 속일 수 있을 때까지 이미지를 수정해볼 수 있다. 다음은 모델에 대한 적대적 공격의 예다.



굿펠로우의 허락하에 재구성 게재

이 그림에서 왼쪽의 이미지는 특정 레이블에 대해 58%의 신뢰도를 보여준다. 왼쪽 이미지를 가운데 있는 노이즈와 결합해 오른쪽 이미지를 생성한다. 인간의 눈으로는 약간의 노이즈가 있더라도 동일하게 이미지를 인식할 수 있다. 하지만 노이즈가 있는 이미지는 97%의 신뢰도로 다른 레이블로 예측된다. 특정 예제에 매우 높은 신뢰도를 부여하고 있지만, 사실 전혀 다른 이미지를 포함하고 있다. 이는 딥러닝 모델이 갖는 문제이므로, 사용처에 대해 이해해야 한다.

- 적대적 사례는 모델에 접근하지 않아도 생성할 수 있다. 자체적인 모델을 훈련시킬 수 있고, 적대적 사례를 만들어 다른 모델을 속일 수 있다.
- 실제 사례로는 거의 발생하지 않지만, 누군가에 의해 시스템에서 악용되면 심각한 문제를 일으킬 수 있다.
- 딥러닝 모델뿐만 아니라 모든 머신 러닝 모델이 해당 문제에 취약하다.

안전이 중요한 시스템에 딥러닝 모델을 적용할 경우 적대적 예제를 고려한 결과를 이해하는 것이 매우 중요하다. 다음 절에서는 빠른 추론을 위해 텐서플로 서빙을 활용하는 방법을 알아본다.

## ■ 모델 추론

새로운 데이터를 모델로 전달해 결과를 얻을 수 있다. 이렇게 이미지로부터 분류 결과나 특징을 얻는 과정을 추론이라 한다. 훈련과 추론은 일반적으로 서로 다른 컴퓨터에서 다른 시기에 수행된다. 여기서는 모델을 저장하고, 추론을 수행하고, 우수한 반응성과 높은 처리율을 갖춘 서버를 위해 텐서플로 서빙을 사용하는 방법을 알아본다.

### 모델 내보내기

훈련이 끝나면 모델을 내보내고 저장해야 한다. 이때 가중치, 편향, 그래프는 추론을 위해 저장된다. 여기서는 MNIST 모델을 훈련시켜서 저장해본다. 먼저 다음 코드를 사용해 필요한 상수를 정의해보자.

---

```
work_dir = '/tmp'
model_version = 9
training_iteration = 1000
input_size = 784
no_classes = 10
batch_size = 100
total_batches = 200
```

---

`model_version`은 내보내져서 제공되는 모델을 지정하기 위한 숫자다. 특징 설정 값<sup>feature config</sup> 플레이스홀더명과 이에 해당하는 데이터타입으로 된 딕셔너리로 저장된다. 예측 클래스와 해당 레이블을 매핑해야 한다. 식별을 위한 플레이스홀더를 API와 함께 사용할 수 있다.

---

```
tf_example = tf.parse_example(tf.placeholder(tf.string, name='tf_example'),
    {'x': tf.FixedLenFeature(shape=[784], dtype=tf.float32), })
x_input = tf.identity(tf_example['x'], name='x')
```

---

다음 코드를 사용해 가중치, 편향, 로짓, 옵티마이저를 통해 간단한 분류자를 정의할 수 있다.

---

```
y_input = tf.placeholder(tf.float32, shape=[None, no_classes])
weights = tf.Variable(tf.random_normal([input_size, no_classes]))
bias = tf.Variable(tf.random_normal([no_classes]))
logits = tf.matmul(x_input, weights) + bias
softmax_cross_entropy = tf.nn.softmax_cross_entropy_with_logits(labels=y_input,
logits=logits)
loss_operation = tf.reduce_mean(softmax_cross_entropy)
optimiser = tf.train.GradientDescentOptimizer(0.5).minimize(loss_operation)
```

---

다음 코드와 같이 모델을 훈련한다.

---

```
mnist = input_data.read_data_sets('MNIST_data', one_hot=True)
for batch_no in range(total_batches):
    mnist_batch = mnist.train.next_batch(batch_size)
    _, loss_value = session.run([optimiser, loss_operation], feed_dict={
        x_input: mnist_batch[0],
        y_input: mnist_batch[1]
    })
    print(loss_value)
```

---

예측 시그니처(prediction signature)를 정의하고 모델을 내보낸다. 이제 모델을 영구 저장소에 저장해 이후 추론을 수행할 때 사용할 수 있다. 이는 역직렬화를 통해 데이터를 내보내고, 다른 시스템에서 이해할 수 있는 형태로 저장한다. 다양한 변수와 플레이스홀더가 있는 여러 그래프를 내보낼 수 있다. 또한 signature\_defs와 애셋을 지원한다. signature\_defs는 외부 클라이언트에서 입출력에 접근하기 때문에 입력과 출력이 지정돼 있다. 애셋은 어휘 등과 같이 추론에 활용하는 비그래프 구성 요소를 말한다.

분류 시그니처(classification signature)는 텐서플로의 분류 API에 대한 접근을 사용한다. 입력은 필수며, 예측 클래스나 예측 확률의 두 가지 출력 옵션에서 하나 이상은 필수로 사용

해야 한다. 반면 예측 시그니처는 입력 및 출력의 수에 더 유연하다. 다중 출력도 정의할 수 있으며, 클라이언트 측에서 명시적으로 쿼리를 수행할 수 있다. `signature_def`는 다음과 같다.

---

```
signature_def = (  
    tf.saved_model.signature_def_utils.build_signature_def(  
        inputs={'x': tf.saved_model.utils.build_tensor_info(x_input)},  
        outputs={'y': tf.saved_model.utils.build_tensor_info(y_input)},  
        method_name="tensorflow/serving/predict"))
```

---

마지막으로 예측 시그니처를 사용해 빌더에 메타그래프와 변수를 추가하자.

---

```
model_path = os.path.join(work_dir, str(model_version))  
saved_model_builder = tf.saved_model.builder.SavedModelBuilder(model_path)  
saved_model_builder.add_meta_graph_and_variables(  
    session,  
    [tf.saved_model.tag_constants.SERVING],  
    signature_def_map={  
        'prediction': signature_def  
    },  
    legacy_init_op=tf.group(tf.tables_initializer(), name='legacy_init_op'))  
saved_model_builder.save()
```

---

빌더가 저장되면 서버에서 이를 사용할 수 있게 된다. 위의 예제는 모든 모델에 대해 내보내기를 수행할 때 적용할 수 있다. 다음 절에서는 내보내진 모델을 사용해 쿼리를 수행해본다.

## 훈련된 모델 사용

이전 절에서 내보낸 모델은 다음의 명령을 사용해 텐서플로 서빙을 사용할 수 있다.



---

```
tensorflow_model_server --port=9000 --model_name=mnist --model_base_path=/tmp/  
mnist_model/
```

---

model\_base\_path는 내보낸 모델의 디렉터리를 가리킨다. 이제 클라이언트로 서버를 테스트할 수 있다. 여기서 말하는 서버는 HTTP 서버가 아니므로 HTTP 클라이언트 대신 다음에 나와 있는 클라이언트가 필요하다. 먼저 필요한 라이브러리를 가져온다.

---

```
from grpc.beta import implementations  
import numpy import tensorflow as tf  
from tensorflow.examples.tutorials.mnist import input_data  
from tensorflow_serving.apis import predict_pb2  
from tensorflow_serving.apis import prediction_service_pb2
```

---

동시성, 테스트 수, 작업 디렉터리에 대한 상수를 추가한다. 그리고 반환된 결과를 계산하기 위한 클래스를 정의한다. 다음과 같이 원격 프로시저 호출(RPC) 콜백은 예측 수를 계산할 수 있는 카운터로 정의한다.

---

```
concurrency = 1  
num_tests = 100  
host = ''  
port = 8000  
work_dir = '/tmp'  
  
def _create_rpc_callback():  
    def _callback(result):  
        response = numpy.array(result.result().outputs['y'].float_val)  
        prediction = numpy.argmax(response)  
        print(prediction)  
    return _callback
```

---

요구 사항에 맞춰 host와 port를 수정한다. \_callback 메소드는 응답이 서버에서 다시 수신될 때 필요한 단계를 정의한다. 여기서는 확률의 최댓값을 계산한다. 이제 서버를 호출



해 추론을 수행하자.

---

```
test_data_set = mnist.test
test_image = mnist.test.images[0]

predict_request = predict_pb2.PredictRequest()
predict_request.model_spec.name = 'mnist'
predict_request.model_spec.signature_name = 'prediction'
predict_channel = implementations.insecure_channel(host, int(port))
predict_stub = prediction_service_pb2.beta_create_PredictionService_stub(
    predict_channel
)

predict_request.inputs['x'].CopyFrom(
    tf.contrib.util.make_tensor_proto(test_image, shape=[1, test_image.size])
)
result = predict_stub.Predict.future(predict_request, 3.0)
result.add_done_callback(_create_rpc_callback())
```

---

반복적으로 추론을 수행해 정확도, 반응성, 처리 속도를 측정한다. 추론 오차는 90%고, 동시성도 우수해야 한다. 내보내기와 클라이언트 메소드를 모든 모델에서 사용해 결과 및 특징을 얻을 수 있다. 다음 절에서는 검색 파이프라인을 구축해보자.

## ■ 콘텐츠 기반 이미지 검색

CBIR(Content-based Image Retrieval) 기술은 쿼리 이미지를 입력받아 대상 이미지 데이터베이스에 있는 이미지들의 순위를 정해 출력을 생성한다. CBIR은 특정 목표가 있는 이미지 검색 엔진이다. 검색을 위해서는 대상 이미지를 저장한 데이터베이스가 필요하며, 쿼리 이미지와 가장 근접한 대상 이미지를 반환한다. 유사도를 계산하기 위해 직접 이미지를 사용할 수 있지만, 다음과 같은 문제들이 있다.

- 이미지는 거대한 차원이다.
- 중복된 픽셀이 많다.
- 픽셀 정보에는 의미가 내포돼 있지 않다.

그래서 검색을 위해 특징을 사용한 객체 분류 모델을 훈련한다. 다음으로 동일한 모델에 쿼리 이미지와 데이터베이스를 전달해 특징을 얻는다. 여기의 모델은 특정 작업을 위해 이미지의 정보를 인코딩하기 때문에 인코더라고도 부른다. 인코더는 전역 및 지역 특징을 잡아낼 수 있어야 한다. 여기서는 이미지 분류에 대한 내용을 다루면서 분류 작업을 위해 훈련시켰던 모델을 사용한다. 무차별 대입<sup>brute-force</sup>이나 선형 스캔 방식은 느리기 때문에 이미지를 검색하는 데 많은 시간이 걸린다. 따라서 빠른 검색을 위한 몇 가지 방법이 필요하다. 다음은 빠른 매칭을 위한 몇 가지 방법들이다.

- 지역 민감 해싱<sup>LSH, Locality Sensitive Hashing</sup>: LSH는 해당 부분 공간에 특징을 프로젝션해 후보 목록을 제공하고 우수한 특징에 대해 순위를 정한다. 이 방식은 앞서 언급했던 PCA와 t-SNE 같은 차원 축소 기법으로 더 낮은 차원의 특징 벡터를 갖는다.
- 다중 인덱스 해싱: 이 방법은 특징을 해시해 핀홀에 맞추는 것과 같이 속도를 빠르게 하며, 계산을 빠르게 하기 위해 해밍 거리를 사용한다. 해밍 거리는 이진으로 표현된 숫자에서 위치의 차이 수에 불과하다.

이러한 방식은 정확도를 다소 낮추는 대신 빠르고 메모리를 적게 사용한다. 여전히 이 방법들이 의미를 잡아내지는 못한다. 쿼리를 기반으로 더 좋은 결과를 얻기 위해 결과의 순위를 다시 지정할 수도 있다. 순위의 재지정은 반환되는 대상 이미지들의 순서를 재정렬하는 것으로 결과를 개선할 수 있다. 순위를 재지정하기 위해서는 다음의 기법 중 하나를 사용할 수 있다.

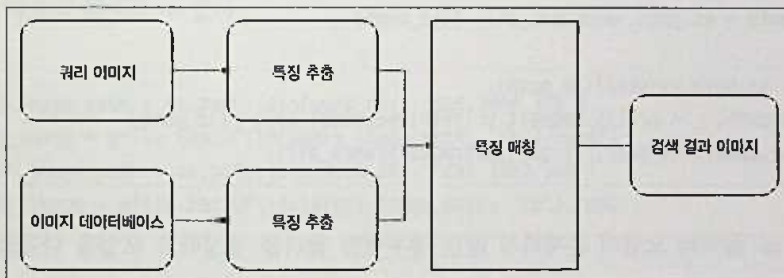
- 기하학 검증: 이 방법은 목표 이미지의 기하학적 특징을 반환된 이미지의 기하학적 특징과 비교한다.

- 쿼리 확장: 대상 이미지의 목록을 확장해 완전하게 검색한다.
- 적합성 피드백: 이 방법은 사용자로부터 피드백을 얻고 결과를 반환한다. 사용자의 입력에 따라 순위가 다시 결정된다.

이러한 기술은 텍스트를 대상으로 개발돼 있으며 이미지에도 활용할 수 있다. 이번 장에서는 특징을 추출해 CBIR에서 사용하는 방법에 초점을 맞춘다. 다음 절에서는 모델 추론 방법을 알아보자.

## 검색 파이프라인 구축

쿼리 이미지와 가장 잘 일치하는 대상 이미지를 얻는 일련의 과정을 검색 파이프라인이라고 한다. 검색 파이프라인은 여러 요소와 단계로 구성돼 있다. 이미지 데이터베이스의 특징들은 오프라인 상태에서 추출돼 데이터베이스로 저장돼야 한다. 모든 쿼리 이미지에 대해 특징을 추출해 모든 대상 이미지에 대해 유사도를 계산한다. 그리고 나서 최종 결과로 이미지에 순위를 지정한다. 검색 파이프라인은 다음과 같다.



특징 추출 단계는 빨라야 하기 때문에 텐서플로 서빙TensorFlow Serving을 사용하며, 애플리케이션에 따라 사용할 특징을 선택할 수 있다. 예를 들어 텍스처 기반 매칭이 요구되면 초기 레이어를 사용하고, 객체 수준의 매칭이 필요하면 마지막 레이어를 사용할 수 있다. 다음 절에서는 사전 훈련된 인셉션 모델inception model에서 특징을 추출하는 방법을 알아본다.

## 이미지의 병목 특징 추출

병목 특징은 사전 분류 레이어에서 계산된 값이다. 이번 절에서는 텐서플로를 사용해 사전 훈련된 모델에서 병목 특징을 추출하는 방법을 알아본다. 먼저 다음의 코드로 필요한 라이브러리를 가져오자.

---

```
import os
import urllib.request
from tensorflow.python.platform
import gfile import tarfile
```

---

이어서 그래프 정의와 가중치를 포함한 사전 훈련된 모델을 다운로드한다. 텐서플로는 인셉션 아키텍처와 제공된 모델을 사용해 ImageNet 데이터셋에 대해 모델을 훈련한다. 여기서는 다음 코드를 사용해 이 모델을 다운로드하고 로컬 폴더에 압축 해제한다.

---

```
model_url = 'http://download.tensorflow.org/models/image/imagenet/
inception-2015-12-05.tgz'
file_name = model_url.split('/')[-1]
file_path = os.path.join(work_dir, file_name)

if not os.path.exists(file_path):
    file_path, _ = urllib.request.urlretrieve(model_url, file_path)
    tarfile.open(file_path, 'r:gz').extractall(work_dir)
```

---

이것으로 폴더와 모델이 존재하지 않는 경우에만 폴더를 생성하고 모델을 다운로드한다. 코드를 반복적으로 실행해도 매번 모델을 다운로드하지는 않는다. 그래프는 프로토콜 버퍼(Protocol Buffers(<sup>1</sup>protohub)) 형식으로 파일에 저장된다. 이 형식을 문자열로 읽어들이어 `tf.Graphdef()` 객체에 전달하고 메모리로 가져온다.

---

```
model_path = os.path.join(work_dir, 'classify_image_graph_def.pb')
```

---

```
with gfile.FastGFile(model_path, 'rb') as f:
    graph_definition = tf.GraphDef()
    graph_definition.ParseFromString(f.read())
```

---

인셉션 모델에서 병목 레이어명은 pool\_3/\_reshape:0이고 2,048차원이다. 플레이스홀더명은 DecodeJpeg/contents:0이고 크기 조절 텐서명은 ResizeBilinear:0이다. 그래프 정의를 이후의 작업을 위해 요구되는 반환 텐서와 tf.import\_graph\_def로 가져올 수 있다.

---

```
bottleneck, image, resized_input = (
    tf.import_graph_def(
        graph_definition,
        name='',
        return_elements=['pool_3/_reshape:0',
                        'DecodeJpeg/contents:0',
                        'ResizeBilinear:0'])
)
```

---

쿼리와 대상 이미지를 가져와서 메모리에 적재한다. gfile 함수를 사용해 이미지를 메모리에 빠르게 적재할 수 있다.

---

```
query_image_path = os.path.join(work_dir, 'cat.1000.jpg')
query_image = gfile.FastGFile(query_image_path, 'rb').read()
target_image_path = os.path.join(work_dir, 'cat.1001.jpg')
target_image = gfile.FastGFile(target_image_path, 'rb').read()
```

---

session과 이미지를 사용해 이미지로부터 병목 특징을 추출하는 함수를 정의하자.

---

```
def get_bottleneck_data(session, image_data):
    bottleneck_data = session.run(bottleneck, {image: image_data})
    bottleneck_data = np.squeeze(bottleneck_data)
    return bottleneck_data
```

---

사전 훈련된 모델에서 병목 값을 얻기 위해 세션을 초기화하고 이미지를 전달해 추론을 진행하자.

---

```
query_feature = get_bottleneck_data(session, query_image)
print(query_feature)
target_feature = get_bottleneck_data(session, target_image)
print(target_feature)
```

---

위 코드를 실행하면 다음과 같이 출력된다.

---

```
[ 0.55705792 0.36785451 1.06618118 ..., 0.6011821 0.36407694
 0.0996572 ]
[ 0.30421323 0.0926369 0.26213276 ..., 0.72273785 0.30847171
 0.08719242]
```

---

특징을 계산하는 절차는 더 많은 대상 이미지에 맞춰 조절할 수 있다. 다음 절에서는 위의 값을 사용해 쿼리 이미지와 대상 데이터베이스 간의 유사도를 계산하는 방법을 설명한다.

### 쿼리 이미지와 대상 데이터베이스 간의 유사도 계산

NumPy의 `linalg.norm`은 유클리드 거리를 계산하는 데 유용하다. 쿼리 이미지와 대상 데이터베이스 간의 이미지 유사도는 다음과 같이 특징 사이의 유클리드 거리를 통해 계산된다.

---

```
dist = np.linalg.norm(np.asarray(query_feature) - np.asarray(target_feature))
print(dist)
```

---

이 명령을 실행하면 다음과 같이 출력된다.

---

**16.9965**

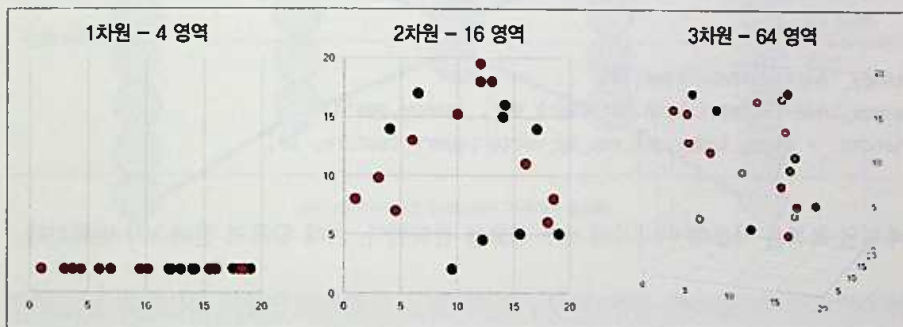
---



위의 값은 유사도 계산에 사용되는 측정 항목이다. 쿼리와 대상 이미지 사이의 유클리드 거리가 작을수록 이미지는 더 유사해진다. 따라서 유클리드 거리의 계산은 유사도를 측정하는 것이다. 유클리드 거리 계산에 특징을 사용하는 것은 모델을 훈련하는 동안 특징을 습득한다는 가정에 기반한다. 수백만 개의 이미지에 대해 이 계산을 확장하는 것은 비효율적이다. 상품화 시스템은 밀리초 단위로 결과를 반환해야 한다. 다음 절에서는 검색을 효율적으로 만드는 방법을 다룬다.

## 효율적 검색

무차별 대입 방법의 검색 속도는 느리다. 근사 최근접 이웃을 사용하면 매칭을 더 빠르게 수행할 수 있다. 다음 그림과 같이 '차원의 저주'도 개입한다.



2차원에서 3차원으로 차원이 증가할 때마다 복잡도 역시 마찬가지로 증가한다. 따라서 거리 계산도 느려진다. 거리 탐색을 빠르게 수행하기 위한 대략적인 방법은 다음 절에서 설명한다.

## ANNOY를 사용한 매칭 가속화

ANNOY(Approximate nearest neighbour oh yeah)는 최근접 이웃을 빠르게 검색하는 방법이다. ANNOY는 임의의 프로젝션을 통해 트리를 생성한다. 트리 구조는 가장 근접하게 매칭되

는 값을 쉽게 찾을 수 있다. 다음과 같이 빠른 검색을 위해 ANNOYIndex를 생성하자.

```
def create_annoy(target_features):
    t = AnnoyIndex(layer_dimension)
    for idx, target_feature in enumerate(target_features):
        t.add_item(idx, target_feature)
    t.build(10)
    t.save(os.path.join(work_dir, 'annoy.ann'))
```

```
create_annoy(target_features)
```

인덱스를 생성하기 위해 특징의 차원이 요구된다. 다음으로 인덱스에 항목이 추가돼 트리가 생성된다. 트리 수가 많을수록 시간과 공간의 복잡성은 증가하지만 결과는 더 정확해진다. 생성된 인덱스는 메모리에 불러올 수 있다. ANNOY는 다음과 같이 쿼리한다.

```
annoy_index = AnnoyIndex(10)
annoy_index.load(os.path.join(work_dir, 'annoy.ann'))
matches = annoy_index.get_nns_by_vector(query_feature, 20)
```

매칭된 목록을 사용해 이미지의 세부 정보를 검색한다. 이때 항목의 인덱스가 반환된다.



정확도와 속도 면에서 ANNOY의 완벽한 구현 및 다른 근사 최근접 이웃 알고리즘과의 벤치마크 비교는 <https://github.com/spotify/annoy>에서 확인할 수 있다.

## ANNOY의 장점

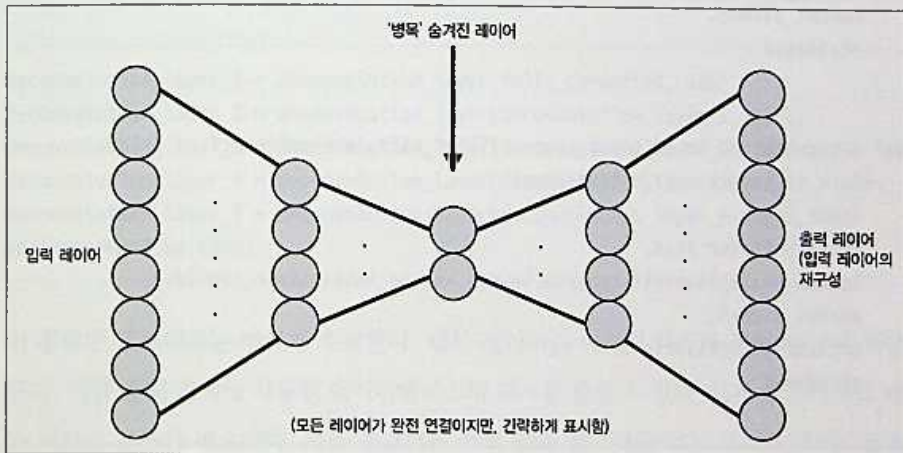
ANNOY를 사용하는 많은 이유가 존재하며, 주요 장점은 다음과 같다.

- 메모리-맵 데이터 구조를 갖기 때문에 RAM의 집중도가 낮다. 이를 통해 동일한 파일을 여러 프로세스에서 공유할 수 있다.

- 맨해튼, 코사인, 유클리디안과 같은 다양한 거리를 사용해 쿼리 이미지와 대상 데이터베이스 간의 유사도를 계산할 수 있다.

## Raw 이미지 자동 인코더

자동 인코더(autoencoder)는 효율적인 인코딩을 생성하는 자율 학습 알고리즘이다. 입력 레이어와 대상 출력이 일반적으로 동일하다. 레이어 사이의 증감은 다음과 같다.



병목 현상 레이어는 차원이 축소된 중간 레이어를 말한다. 병목 층의 왼쪽을 인코더라 하고, 오른쪽을 디코더라 한다. 일반적으로 인코더는 데이터의 차원을 축소시키고, 디코더는 차원을 증가시킨다. 이러한 인코더와 디코더의 조합을 자동 인코더라 한다. 전체 네트워크는 오류 복원으로 훈련된다. 이론적으로 병목 레이어는 저장되고 원본 데이터는 디코더 네트워크로 복원된다. 이런 식으로 차원이 축소돼 다음과 같이 쉽게 프로그래밍된다. 다음 코드를 사용해 `fully_connected_layer`, `convolution_layer`, `deconvolution_layer`를 정의한다.

```
def fully_connected_layer(input_layer, units):
    return tf.layers.dense(
```

```

        input_layer,
        units=units,
        activation=tf.nn.relu
    )

def convolution_layer(input_layer, filter_size):
    return tf.layers.conv2d(
        input_layer,
        filters=filter_size,
        kernel_initializer=tf.contrib.layers.xavier_initializer_conv2d(),
        kernel_size=3,
        strides=2
    )

def deconvolution_layer(input_layer, filter_size, activation=tf.nn.relu):
    return tf.layers.conv2d_transpose(
        input_layer,
        filters=filter_size,
        kernel_initializer=tf.contrib.layers.xavier_initializer_conv2d(),
        kernel_size=3,
        activation=activation,
        strides=2
    )

```

---

다음 코드와 같이 다섯 개의 컨볼루션 레이어로 수렴되는 인코더를 정의한다.

```

input_layer = tf.placeholder(tf.float32, [None, 128, 128, 3])
convolution_layer_1 = convolution_layer(input_layer, 1024)
convolution_layer_2 = convolution_layer(convolution_layer_1, 512)
convolution_layer_3 = convolution_layer(convolution_layer_2, 256)
convolution_layer_4 = convolution_layer(convolution_layer_3, 128)
convolution_layer_5 = convolution_layer(convolution_layer_4, 32)

```

---

다섯 번째 컨볼루션 레이어를 평면화해 병목 레이어를 연산한다. 병목 레이어는 다음과 같이 컨볼루션 레이어에 맞춰 다시 변형된다.

---

```
convolution_layer_5_flattened = tf.layers.flatten(convolution_layer_5)
bottleneck_layer = fully_connected_layer(convolution_layer_5_flattened, 16)
c5_shape = convolution_layer_5.get_shape().as_list()
c5f_flat_shape = convolution_layer_5_flattened.get_shape().as_list()[1]
fully_connected = fully_connected_layer(bottleneck_layer, c5f_flat_shape)
fully_connected = tf.reshape(fully_connected, [-1, c5_shape[1], c5_shape[2], c5_shape[3]])
```

---

다음 코드와 같이 발산되는 부분이나 이미지를 복원하는 디코더 부분을 계산한다.

---

```
deconvolution_layer_1 = deconvolution_layer(fully_connected, 128)
deconvolution_layer_2 = deconvolution_layer(deconvolution_layer_1, 256)
deconvolution_layer_3 = deconvolution_layer(deconvolution_layer_2, 512)
deconvolution_layer_4 = deconvolution_layer(deconvolution_layer_3, 1024)
deconvolution_layer_5 = deconvolution_layer(deconvolution_layer_4, 3,
activation=tf.nn.tanh)
```

---

이 훈련된 네트워크는 빠르게 수렴한다. 병목 레이어는 이미지 특징과 함께 전달돼 저장된다. 이를 통해 검색에 사용될 데이터베이스의 크기를 줄일 수 있다. 특징을 인덱싱할 때는 인코더 부분만 필요하다. 자동 인코더는 손실 압축 알고리즘이다. 그리고 데이터로부터 압축 패턴을 학습하는 점은 다른 압축 알고리즘과 차별되며, 이는 자동 인코더 모델이 데이터에 따라 고유함을 의미한다. 자동 인코더를 t-SNE와 결합하면 시각화를 개선할 수 있다. 자동 인코더로 학습된 병목 레이어는 다른 작업에 유용하지 않다. 또한 병목 레이어의 크기는 이전의 레이어보다 커질 수 있다. 발산 및 수렴의 경우 연결은 희박한 자동 인코더가 된다. 다음 절에서는 자동 인코더를 사용한 다른 애플리케이션에 대해 배워보자.

## 자동 인코더를 사용한 노이즈 제거

자동 인코더는 이미지 노이즈를 제거하는 데도 사용할 수 있다. 노이즈 제거 인코더는 자율 방식으로 훈련된다. 노이즈는 일반 이미지에 도입되며, 자동 인코더는 원본 이미지들에



대해 학습한다. 그리고 나서 완전한 자동 인코더를 사용하면 노이즈가 없는 이미지를 생성할 수 있다. 이번 절에서는 MNIST 이미지의 노이즈를 제거하는 과정을 단계별로 소개한다. 먼저 필요한 라이브러리를 가져오고 다음과 같이 플레이스홀더를 정의한다.

---

```
x_input = tf.placeholder(tf.float32, shape=[None, input_size])
y_input = tf.placeholder(tf.float32, shape=[None, input_size])
```

---

x\_input과 y\_input은 모두 자동 인코더에서와 같은 shape를 갖는다. 이어서 기본 활성화 함수로 tanh 활성화 함수를 설정해 dense\_layer를 다음과 같이 정의한다. add\_variable\_summary 메소드는 이미지 분류 관련 장의 예제에서 사용한 것을 가져온다. dense\_layer의 정의는 다음과 같다.

---

```
def dense_layer(input_layer, units, activation=tf.nn.tanh):
    layer = tf.layers.dense(
        inputs=input_layer,
        units=units,
        activation=activation
    )
    add_variable_summary(layer, 'dense')
    return layer
```

---

다음으로 자동 인코더 레이어를 정의한다. 이 자동 인코더는 완전 연결 레이어만 포함한다. 인코더 부분에는 차원을 축소시키는 세 개의 레이어가 있고, 디코더 부분에는 차원을 확대시키는 세 개의 레이어가 있다. 따라서 인코더와 디코더는 다음과 같이 대칭 형태를 갖는다.

---

```
layer_1 = dense_layer(x_input, 500)
layer_2 = dense_layer(layer_1, 250)
layer_3 = dense_layer(layer_2, 50)
layer_4 = dense_layer(layer_3, 250)
layer_5 = dense_layer(layer_4, 500)
```

---



```
layer_6 = dense_layer(layer_5, 784)
```

---

숨겨진 레이어의 크기는 임의로 선택한다. 이어서 loss와 optimise를 정의한다. 여기서는 다음과 같이 소프트맥스 대신 시그모이드를 사용해 분류한다.

---

```
with tf.name_scope('loss'):
    softmax_cross_entropy = tf.nn.sigmoid_cross_entropy_with_logits(labels=y_input,
logits=layer_6)
    loss_operation = tf.reduce_mean(softmax_cross_entropy, name='loss')
    tf.summary.scalar('loss', loss_operation)

with tf.name_scope('optimiser'):
    optimiser = tf.train.AdamOptimizer().minimize(loss_operation)
```

---

텐서보드는 다른 종류의 요약으로 image를 제공하며, 이는 이미지를 시각화하는 데 유용하다. 다음과 같이 두 개의 입력과 layer\_6를 가져와서 형태를 변경하고 요약에 추가한다.

---

```
x_input_resaped = tf.reshape(x_input, [-1, 28, 28, 1])
tf.summary.image("noisy_images", x_input_resaped)

y_input_resaped = tf.reshape(y_input, [-1, 28, 28, 1])
tf.summary.image("original_images", y_input_resaped)

layer_6_resaped = tf.reshape(layer_6, [-1, 28, 28, 1])
tf.summary.image("reconstructed_images", layer_6_resaped)
```

---

모든 이미지를 요약 폴더에 저장하는 것을 방지하기 위해 기본 설정으로 이미지의 수가 세 개로 제한되지만 변경 가능하다. 다음으로 모든 요약이 병합되고, 요약 작성자에 그래프가 추가된다.

---

```
merged_summary_operation = tf.summary.merge_all()
train_summary_writer = tf.summary.FileWriter('/tmp/train', session.graph)
```

---

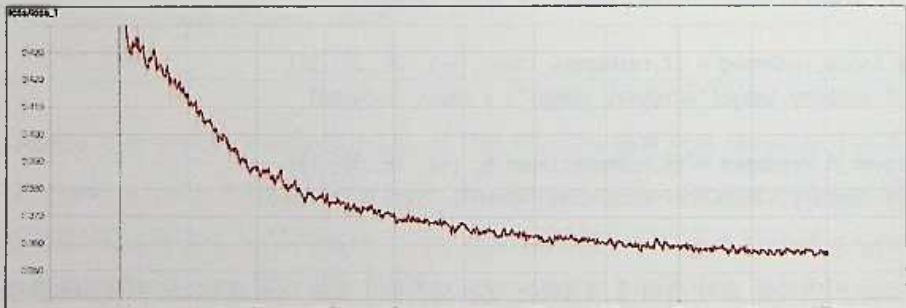
일반적인 임의의 노이즈를 이미지에 추가해 입력 텐서로 제공한다. 노이즈가 추가되면 초과되는 값들은 클리핑clipping한다. 여기서 대상은 원본 이미지가 된다. 노이즈의 추가와 훈련 절차는 다음과 같다.

---

```
for batch_no in range(total_batches):
    mnist_batch = mnist_data.train.next_batch(batch_size)
    train_images, _ = mnist_batch[0], mnist_batch[1]
    train_images_noise = train_images + 0.2 * np.random.normal(size=train_images.
shape)
    train_images_noise = np.clip(train_images_noise, 0., 1.)
    _, merged_summary = session.run([optimiser, merged_summary_operation],
    feed_dict={
        x_input: train_images_noise,
        y_input: train_images,
    })
    train_summary_writer.add_summary(merged_summary, batch_no)
```

---

훈련을 시작하면 텐서보드에서 결과를 볼 수 있다. 다음과 같이 손실이 표시된다.

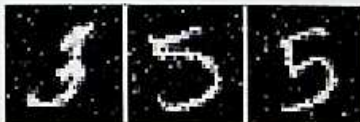


출력 플롯을 보여주는 텐서보드

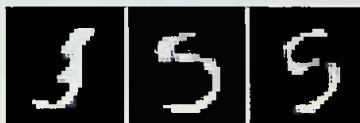
손실은 반복을 통해 지속적으로 천천히 감소하며 자동 인코더가 빠르게 수렴한다. 이어서 원본 이미지로부터 세 자리 숫자가 표시된다.



다음은 노이즈를 추가한 동일한 이미지다.



이미지에 노이즈가 추가된 것을 확인할 수 있다. 이 이미지는 입력으로 주어진다. 노이즈 제거 자동 인코더로 동일한 숫자 이미지가 복원된다.



노이즈 제거 자동 인코더가 깔끔하게 노이즈를 제거하는 것을 확인할 수 있다. 이를 테스트 이미지에서 실행해 품질이 유지되는 것을 확인할 수 있다. 더 복잡한 데이터셋의 경우 컨볼루션 신경망을 사용해 더 좋은 결과를 얻을 수 있다. 이 예제는 자율 방식으로 훈련됐음을 전제로 해서 컴퓨터 비전의 딥러닝 능력을 보여준다.

## **I 요약**

이 장에서는 이미지에서 특징을 추출해 CBIR에 적용하는 방법을 배웠다. 또한 텐서플로 서빙을 사용해 이미지 특징에 대해 추론하는 방법을 알아봤다. 그리고 빠른 매칭을 위해 선형 스캔 대신에 근사 최근접 이웃을 활용해봤고, 해시를 사용해 결과를 개선하는 방법을 살펴봤다. 자동 인코더를 소개하면서 검색을 위해 작은 특징 벡터를 훈련하는 방법을

다웠다. 마지막으로 자동 인코더를 사용해 이미지의 노이즈를 제거하는 예제를 살펴봤고, 이를 수십억 개의 이미지까지 확대할 수 있는 비트 기반 비교의 사용 가능성을 확인했다. 다음 장에서는 객체 검출 문제를 위한 모델 훈련 방법을 알아본다. 여기서는 정확도가 높은 오픈소스 모델을 활용해 내부에서 사용하는 모든 알고리즘을 이해해본다. 또한 모든 내용들은 보행자 검출 모델을 훈련시키는 데 활용할 것이다.

## 객체 검출

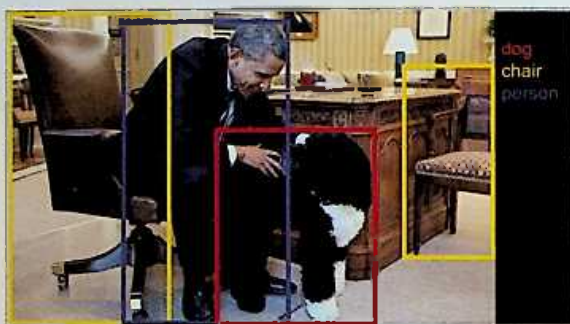
객체 검출은 이미지에서 물체의 위치를 찾는 동작이다. 이 장에서는 다음 항목을 이해해 객체 검출과 보행자 검출 기술을 학습한다.

- 로컬라이제이션과 검출의 차이점
- 다양한 데이터셋 및 설명
- 객체 로컬라이제이션 및 검출에 사용되는 알고리즘
- 객체 검출을 위한 텐서플로 API
- 새로운 객체 검출 모델 훈련
- YOLO 알고리즘을 사용해 움직이는 자동차에서 보행자 검출하기

## ■ 이미지에서의 객체 검출

최근 몇 년 동안 객체 검출은 애플리케이션과 연구 분야 모두에서 폭발적인 사건이었다. 객체 검출은 컴퓨터 비전 분야에서 중요한 사항이다. 이미지 분류 작업과 비슷하게, 네트워크가 깊어지면 검출 성능이 향상되는 것을 볼 수 있다. 현재 이 기술의 정확성은 매우 뛰어나기 때문에 많은 애플리케이션 분야에서 이미 적용됐다.

이미지 분류는 이미지 전체 영역에서 레이블을 붙이는 작업이다. 객체에 레이블을 지정하는 것 외에 객체의 위치를 찾는 것도 있다. 이를 객체 로컬라이제이션이라고 한다. 일반적으로 객체의 위치는 직각 좌표로 정의된다. 직각 좌표로 이미지에서 여러 객체를 찾는 것을 검출이라 한다. 다음은 객체 검출의 한 예다.



이미지에서는 경계 박스로 표현되는 네 개의 객체를 보여준다. 여기서는 박스 내의 객체를 찾는 작업을 수행할 수 있는 알고리즘을 배운다. 이러한 알고리즘이 들어간 애플리케이션은 자가 운전 차량 및 산업용 물체와 같은 로봇 비전 분야에 막대한 영향을 미칠 수 있다. 로컬라이제이션 및 검출 작업은 다음과 같이 요약할 수 있다.

- 로컬라이제이션은 레이블 내의 이미지에서 하나의 객체를 검출한다.
- 검출은 레이블과 함께 이미지 내의 모든 객체를 찾는다.

이 둘 간의 차이점은 객체의 수다. 검출할 때는 다양한 개수의 객체를 찾을 수 있다. 이 작



은 차이는 로컬라이제이션 또는 검출과 관련해 딥러닝 모델을 위한 아키텍처를 설계할 때 큰 차이를 만든다. 이어서 작업에 사용할 수 있는 다양한 데이터셋을 살펴본다.

## ■ 데이터셋 탐색하기

객체 로컬라이제이션 및 검출에 사용할 수 있는 데이터셋은 많다. 이 절에서는 연구 커뮤니티가 알고리즘을 평가하는 데 사용한 데이터셋을 이용한다. 이 데이터셋에는 애노테이션(annotation)이 달린 20개에서 200개에 이르는 다양한 객체가 있으므로 여기서 객체를 정확히 검출해내기는 쉽지 않다. 일부 데이터셋은 한 이미지에 너무 많은 객체를 가지고 있으며, 이미지당 하나의 객체만 가진 다른 데이터셋과 비교할 때 쉽지 않다. 지금부터 해당 데이터셋을 자세히 보게 될 것이다.

### ImageNet 데이터셋

ImageNet에는 분류, 로컬라이제이션, 검출 작업을 평가하기 위한 데이터가 있다. 2장, ‘이미지 분류’에서 분류 데이터셋을 자세히 설명했다. 분류 데이터와 마찬가지로 로컬라이제이션 작업에는 1,000개의 클래스가 존재한다. 정확도는 상위 다섯 개의 검출 결과를 기준으로 계산된다. 모든 이미지에는 최소한 하나의 경계 박스가 있다. 이미지당 평균 1.1개의 객체를 가지는 이미지 470,000개 중에서 검색 평가를 위해 객체 200개를 사용한다.

### 파스칼 VOC 챌린지

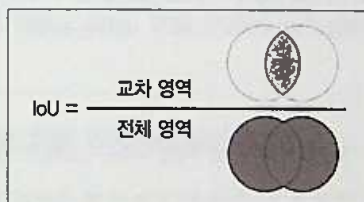
파스칼 VOC 챌린지는 2005년부터 2012년까지 진행됐다. 이 챌린지는 객체 검출 기술의 벤치마크로 간주되고 있다. 데이터 집합에는 20개의 클래스가 존재한다. 이 데이터셋에는 훈련 및 검증을 위해 11,530개의 이미지가 있으며 관심 영역에 대한 27,450개의 애노테이션이 있다. 다음은 데이터 집합에 있는 20개의 클래스다.



측값이라 불리는 에노테이션 처리된 박스를 사용한다. 실측값이 절대 진리일 필요는 없다. 또한 박스는 인간과 다른 인간 간에 몇 픽셀의 차이를 가질 수 있다. 따라서 알고리즘으로 인간에 의해 그려지는 정확한 경계 박스를 검출하기가 더 어려워진다.  $\text{IoU}^{\text{Intersection over Union}}$ 는 로컬라이제이션 작업의 값들을 평가하는 데 사용된다.  $\text{mAP}^{\text{Mean Precision Average}}$ 는 검출 작업에서의 값을 평가하는 데 사용된다. 다음 절에서는 측정 항목에 대한 설명을 볼 수 있다.

## IoU

IoU는 실측값 면적과 예상 면적의 교차 영역 $\text{area of overlap}$  및 전체 영역 $\text{area of union}$ 의 비율이다. 측정 항목에 대한 시각적 설명은 다음과 같다.



두 개의 사각형은 실측값과 예측값의 경계 박스를 나타낸다. IoU는 전체 영역에 대한 교차 영역의 비율로 계산된다. IoU를 계산하는 스크립트는 다음과 같다.

```
def calculate_iou(gt_bb, pred_bb):
    ...
    :param gt_bb: ground truth bounding box
    :param pred_bb: predicted bounding box
    ...
    gt_bb = tf.stack([
        gt_bb[:, :, :, :, 0] - gt_bb[:, :, :, :, 2] / 2.0,
        gt_bb[:, :, :, :, 1] - gt_bb[:, :, :, :, 3] / 2.0,
        gt_bb[:, :, :, :, 0] + gt_bb[:, :, :, :, 2] / 2.0,
        gt_bb[:, :, :, :, 1] + gt_bb[:, :, :, :, 3] / 2.0])
    gt_bb = tf.transpose(gt_bb, [1, 2, 3, 4, 0])
```

```

pred_bb = tf.stack([
    pred_bb[:, :, :, :, 0] - pred_bb[:, :, :, :, 2] / 2.0,
    pred_bb[:, :, :, :, 1] - pred_bb[:, :, :, :, 3] / 2.0,
    pred_bb[:, :, :, :, 0] + pred_bb[:, :, :, :, 2] / 2.0,
    pred_bb[:, :, :, :, 1] + pred_bb[:, :, :, :, 3] / 2.0])
pred_bb = tf.transpose(pred_bb, [1, 2, 3, 4, 0])
area = tf.maximum(
    0.0,
    tf.minimum(gt_bb[:, :, :, :, 2:], pred_bb[:, :, :, :, 2:]) -
    tf.maximum(gt_bb[:, :, :, :, :2], pred_bb[:, :, :, :, :2]))
intersection_area= area[:, :, :, :, 0] * area[:, :, :, :, 1]
gt_bb_area = (gt_bb[:, :, :, :, 2] - gt_bb[:, :, :, :, 0]) * \
    (gt_bb[:, :, :, :, 3] - gt_bb[:, :, :, :, 1])
pred_bb_area = (pred_bb[:, :, :, :, 2] - pred_bb[:, :, :, :, 0]) * \
    (pred_bb[:, :, :, :, 3] - pred_bb[:, :, :, :, 1])
union_area = tf.maximum(gt_bb_area + pred_bb_area - intersection_area, 1e-10)
iou = tf.clip_by_value(intersection_area / union_area, 0.0, 1.0)
return iou

```

실측값과 예상 경계 박스는 함께 스택으로 저장된다. 그리고 영역은 음수 영역의 경우를 처리하면서 계산된다. 음수 영역은 경계 박스 좌표가 올바르지 않은 경우 발생할 수 있다. 박스의 오른쪽 좌표는 왼쪽 좌표로 많이 이동할 수 있다. 경계 박스의 구조가 유지되지 않으므로, 음수 영역이 발생할 수 있다. 최종 IoU 계산을 통해 유니온 및 교차 영역이 계산되며, 실측값 및 예측 영역의 교차 영역과 전체 영역의 비율이 된다. IoU 계산은 로컬라이제이션 문제를 훈련시키는 알고리즘과 결합될 수 있다.

## mAP

mAP(평균 정확률)는 검출 알고리즘을 평가하는 데 사용된다. mAP 측정 항목은 검출된 경계 박스의 정밀도와 재현율 결과다. mAP 값의 범위는 0에서 100 사이다. 숫자가 높을수록 좋다. mAP는 각 클래스에 대해 별도로 평균 정밀도(AP)를 계산한 후 클래스에 대한 평균을 계산해 구할 수 있다. 검출은 mAP가 0.5 이상일 때만 참양성(true-positive)으로 간주된다. 테스트 이미지의 모든 검출은 각 클래스에 대한 정밀도/재현율 곡선을 그려서 결합

할 수 있다. 곡선 아래의 최종 영역은 알고리즘 비교에 사용될 수 있다. mAP는 많은 잘못된 경보를 발생시키지 않으면서 네트워크의 민감도를 측정하는 좋은 방법이다. 여기서 는 데이터 집합에 대한 평가 알고리즘을 배웠다. 이어서 로컬라이제이션 작업을 위한 알고리즘을 살펴본다.

## ■ 알고리즘 로컬라이제이션하기

로컬라이제이션 알고리즘은 2장, '이미지 분류'와 3장, '이미지 검색'에서 배운 자료를 확장한 것이다. 이미지 분류에서 이미지는 CNN(Convolutional Neural Network)의 여러 레이어를 통과한다. CNN의 최종 레이어는 각 레이블에 속하는 확률 값을 출력한다. 이 값은 객체를 로컬라이제이션하기 위해 확장될 수 있다. 다음 절에서 이와 관련된 아이디어를 볼 수 있다.

### 슬라이딩 윈도우를 사용해 객체 로컬라이제이션하기

직관적인 로컬라이제이션 방법은 객체로 이미지의 여러 부분을 예측하는 것이다. 이미지 자르기는 이미지에서 창을 이동하고 모든 창을 예측해 수행할 수 있다. 이미지보다 작은 창을 이동하고 창 크기에 따라 이미지를 자르는 방법을 슬라이딩 윈도우라 한다. 이미지의 모든 잘린 윈도우에 대해 예측이 이뤄질 수 있으며, 이를 슬라이딩 윈도우 객체 검출(sliding window object detection)이라 한다.

예측은 정확하게 잘라낸 이미지에 대한 이미지 분류 문제를 해결하기 위해 훈련된 딥러닝 모델을 사용할 수 있다. 정확하게 이미지 자르기(close cropping)는 전체 이미지에서 하나의 객체만 찾는 것을 의미한다. 윈도우의 움직임은 이미지 전체에 걸쳐 균일해야 한다. 이미지의 각 부분에서 분류를 수행하기 위해 모델을 사용한다. 이 접근법에는 두 가지 문제점이 있다.



- 윈도우와 동일한 크기의 객체만 찾을 수 있다. 객체 크기가 윈도우 크기보다 크면 슬라이딩 윈도우에서 객체가 누락될 수 있다. 이를 극복하기 위해 스케일 공간이라는 개념을 사용할 것이다.
- 또 다른 문제는 픽셀 위로 윈도우를 이동하면 몇 개의 객체가 누락될 수 있다는 것이다. 모든 픽셀 위에서 윈도우를 이동하면 추가적인 많은 계산이 발생하므로 시스템 속도가 느려진다. 이를 방지하기 위해 컨볼루션 레이어에 트릭을 포함시킨다.

다음 절에서는 두 기술을 다룰 것이다.

### 스케일-공간 개념

스케일-공간(scale-space)은 다양한 크기의 이미지를 사용하는 개념이다. 이미지가 더 작은 크기로 축소되므로 동일한 크기의 창으로 더 큰 객체를 검출할 수 있다. 이미지 크기가 줄어들면서 크기가 조정될 수 있다. 대체 픽셀을 제거하거나 보간법을 사용해 이미지 크기를 조정하면 일부 아티팩트가 남을 수 있다. 따라서 이미지가 부드럽게 만들어지고 반복적으로 크기가 조절된다. 부드럽게 하고 크기를 조정해 얻은 이미지가 스케일-공간이 된다.

윈도우는 객체의 로컬라이제이션을 위해 모든 단일 스케일로 구분될 수 있다. 여러 스케일을 사용하는 것은 이미지에 더 큰 창을 적용하는 것과 같다. 여러 개의 스케일을 사용하면 계산상의 복잡성이 매우 높아진다. 로컬라이제이션은 정확성을 위해 절충하면서 더 빠르게 움직여 가속화될 수 있다. 복잡성이 높은 경우에는 솔루션을 제품 적용에 사용할 수 없게 된다. 슬라이딩 윈도우에 대한 아이디어는 슬라이딩 윈도우를 컨볼루션 방법을 사용해 적용함으로써 효율적으로 만들 수 있다.

### 완전 연결 레이어를 컨볼루션 레이어로 훈련시키기

슬라이딩 윈도우의 문제점은 계산상으로 복잡하다는 것이다. 예측은 모든 윈도우에 대해 이뤄지기 때문에 복잡성 문제가 발생한다. 모든 윈도우의 겹치는 영역에 대해 딥러닝 기



능이 적용된다. 크기가 변경된 윈도우를 사용하면 겹치는 영역에 대한 특징 계산을 줄일 수 있다. 이를 위한 해결책은 한 번만 특징을 계산하는 완전 컨볼루션 그물을 사용하는 것이다. 완전 컨볼루션 그물을 이해하기 위해 먼저 완전 연결 레이어를 `convolution_layer`로 변환하는 방법을 살펴본다. 커널은 뉴런 수와 동일한 수의 필터를 사용해 동일한 크기로 변경된다. 이는 다른 레이어에도 반복해서 적용할 수 있다. 커널 크기를 변경하면 완전 연결 레이어를 `convolution_layer`로 쉽게 변환할 수 있다.

---

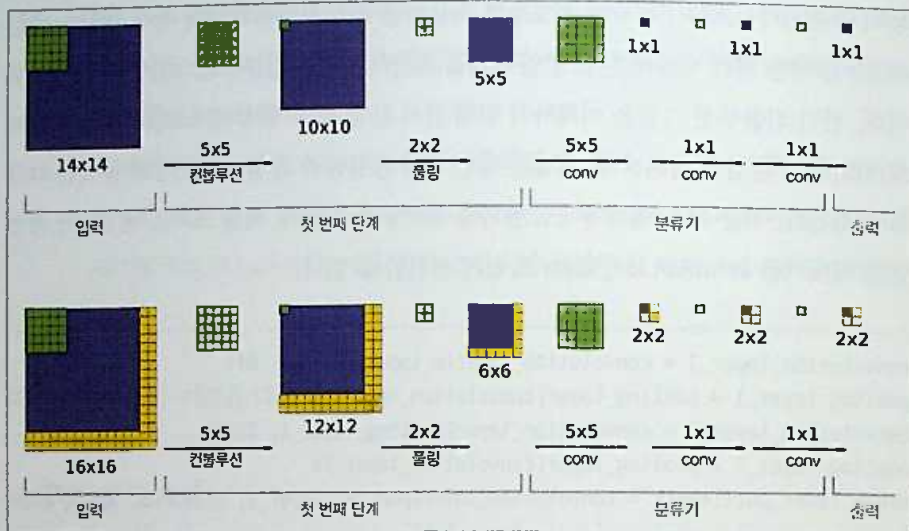
```
convolution_layer_1 = convolution_layer(x_input_reshape, 64)
pooling_layer_1 = pooling_layer(convolution_layer_1)
convolution_layer_2 = convolution_layer(pooling_layer_1, 128)
pooling_layer_2 = pooling_layer(convolution_layer_2)
dense_layer_bottleneck = convolution_layer(pooling_layer_2, 1024, [5, 5])
logits = convolution_layer(dense_layer_bottleneck, no_classes, [1, 1])
logits = tf.reshape(logits, [-1, 10])
```

---

밀집 레이어 `dense layer`는 컨볼루션 레이어로 표현된다. 이 아이디어는 다양한 시나리오에서 강력하고 유용하게 사용된다. 이 아이디어를 확장해 슬라이딩 윈도우를 완전한 컨볼루션 네트워크로 표현할 수 있다.

### 컨볼루션을 이용한 슬라이딩 윈도우 구현

이 기술에서는 슬라이딩 대신 윈도우로 박스 개수와 깊이를 지정해 최종으로 일부 타깃을 만든다. 서먼트 <sup>Sermanet</sup>는 슬라이딩 윈도우의 이러한 문제를 극복하기 위해 완전한 컨볼루션 구현을 사용하는 논문(<https://arxiv.org/pdf/1312.6229.pdf>)을 발표했다. 다음은 슬라이딩 윈도우의 컨볼루션 구현 예다.



서면트의 허락하에 재구성 게재

예제의 위 부분에서 정상 분류는 완전 컨볼루션 레이어로 표현된다. 그림의 아래 부분에서는 동일한 커널이 큰 이미지에 적용돼 1 대신 2×2를 만들게 된다. 최종 레이어는 해당 경계 박스의 출력 중 네 가지를 나타낸다. 예측을 위한 볼륨을 확보하면 효율성은 향상되지만, 박스에서는 정확한 위치 지정에 여전히 문제가 발생한다. 따라서 슬라이딩 윈도우를 사용하지 않으면, 복잡성이 해결된다. 종횡비<sup>aspect ratio</sup>는 항상 변하고 여러 스케일로 살펴봐야 한다. 완전 컨볼루션 방법으로 만들어진 경계 박스는 실제로 그다지 정확하지 않으며, 추가 영역에 대해 추가 계산을 수행한다. 상상하듯이, 박스는 오히려 훈련된 박스의 수로 제한된다. 다음으로 경계 박스 위치를 좀 더 정확하게 검출하는 방법을 살펴본다.

## 로컬라이제이션을 회귀 문제로 생각해보기

로컬라이제이션을 수행하기 위한 한 가지 기본적인 방법은 회귀 문제로 모델링해 문제를 푸는 것이다. 경계 박스는 네 개의 숫자를 가지고 있으므로 회귀 방식을 사용해 설정하고, 직접적으로 예측할 수 있다. 문제를 해결하기 위해서는 분류 문제에 대한 레이블을 예측

해야 한다.

경계 박스를 정의하는 데 사용할 수 있는 다양한 매개변수가 있으며, 일반적으로 경계 박스에는 네 개의 숫자가 있다. 표현 중 하나는 경계 박스의 높이와 너비가 있는 좌표의 중심이다. 완전 연결 레이어를 제거하고 회귀 인코더로 교체하면 사전 학습된 모델을 사용할 수 있다. 회귀는 이상치<sup>outlier</sup>로 적용하기 어려운 L2 손실로 정규화돼야 한다. 정규화할 때는 L1보다 L1 손실을 사용한다. 회귀는 잘 만들어진 정규화 방식으로 변경할 수 있다. 모델을 미세 조정하면 정확도가 높아지고 네트워크 전체를 훈련하면 성능이 향상될 수 있다. 이 점을 훈련 시간과 정확성 사이의 절충점으로 사용할 수 있다. 지금부터는 컨볼루션 네트워크를 사용해 다양한 회귀 애플리케이션을 살펴볼 것이다.

## 다른 문제에 대해 회귀 분석 적용하기

이미지 좌표를 회귀를 통해 구하는 방식은 포즈 검출, 기준점 검출과 같은 여러 가지 다른 애플리케이션 분야에서 적용할 수 있다. 포즈 검출 방법을 사용해 다음과 같이 사람의 관절 위치를 찾을 수 있다.



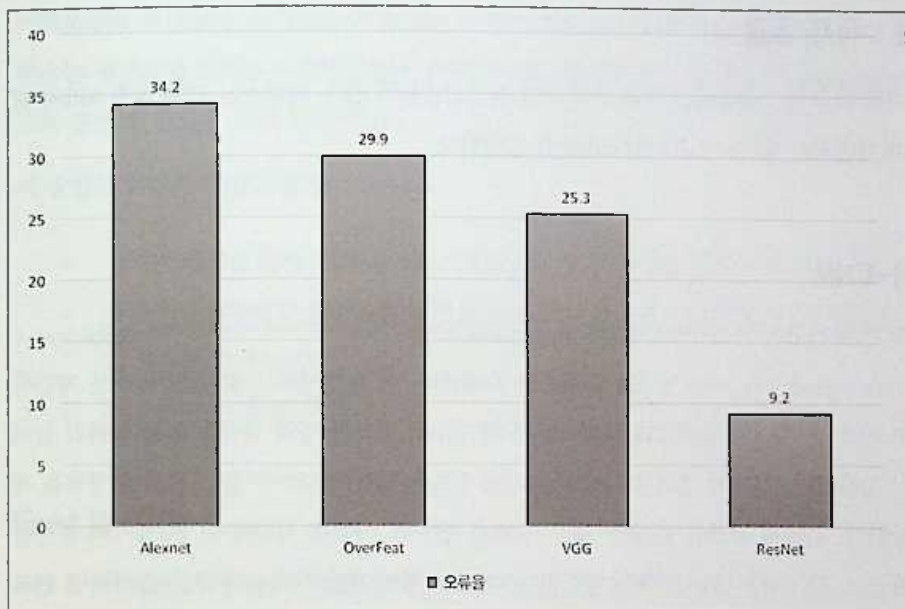
앞의 이미지에서는 머리, 목, 어깨, 발목, 손 같은 여러 위치가 검출됐다. 이는 사람의 다른 신체 부분으로 확장될 수 있으며, 회귀 방법을 사용해 이 애플리케이션에 사용될 수 있다. 다음은 기준점 검출의 예다.



기점은 눈, 코, 입술의 위치와 같은 얼굴의 표식이다. 이러한 표식을 찾는 것은 얼굴 기반의 증강현실 애플리케이션에서 매우 중요하다. 얼굴에는 사용 가능한 더 많은 표식이 있으며, 얼굴 인식과 관련된 내용은 6장, '유사도 학습'에서 자세히 다룰 예정이다.

### 회귀 방법과 슬라이딩 윈도우 결합하기

분류 점수는 슬라이딩 윈도우 접근법 또는 완전 컨볼루션 접근법에서 모든 윈도우를 대상으로 계산돼 어떤 객체가 해당 윈도우에 존재하는지 알려준다. 객체를 검출하기 위해 모든 윈도우에 대한 분류 점수를 예측하는 대신, 각 윈도우 자체는 분류 점수로 예측될 수 있다. 슬라이딩 윈도우, 스케일 공간, 완전 컨볼루션, 회귀와 같은 모든 아이디어를 결합하면 개별적인 접근 방식보다 우수한 결과를 얻을 수 있다. 다음은 회귀 방식을 사용하는 다양한 네트워크에 의해 얻은 ImageNet 데이터셋의 상위 다섯 개 로컬라이제이션의 오류율이다.



앞의 그래프는 네트워크가 깊어질수록 결과가 좋음을 보여준다. AlexNet의 로컬라이제이션 방법은 여기서는 다루지 않는다. OverFeat는 박스 병합(box merging)과 함께 다중 스케일 컨볼루션 회귀(convolutional regression)를 사용했다. VGG는 로컬라이제이션을 사용했지만 규모와 위치는 더 작다. 이러한 이득은 심층 특징(deep feature)에 기인한다. ResNet은 다른 로컬라이제이션 방법과 더 심층화된 특징들을 사용한다.

회귀 인코더와 분류 인코더는 독립적으로 작동한다. 따라서 경계 박스에 대해 잘못된 레이블을 예측할 가능성이 있다. 하지만 이 문제는 회귀 인코더를 다른 레이어에 연결함으로써 극복될 수 있다. 이 방법은 여러 객체에 사용돼 객체 검출 문제를 해결할 수도 있다. 주어진 이미지에서 그 안에 있는 모든 인스턴스를 찾아야 한다. 출력 개수는 가변적이므로 회귀 방법으로 검출하기가 어려울 수 있다. 하나의 이미지는 두 개의 객체를 가질 수 있고, 다른 이미지는 세 개 이상의 객체를 가질 수도 있다. 다음 절에서는 검출을 좀 더 효과적으로 처리하는 알고리즘을 살펴본다.



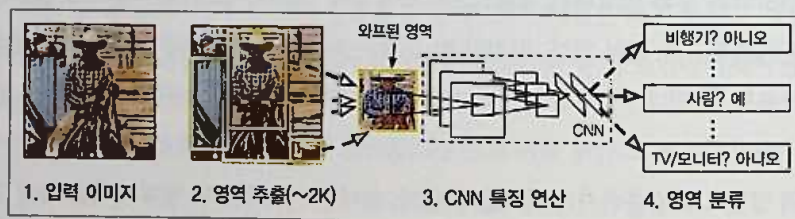
## ■ 객체 검출

객체 검출 알고리즘에는 여러 가지 변형 알고리즘들이 있다. 여기서는 객체 검출 API와 함께 제공되는 몇 가지 주요 알고리즘을 설명한다.

### R-CNN

첫 번째 연구는 거쉬(Girshick)이 제안한 R-CNN(Regions of the convolutional neural network) 논문(<https://arxiv.org/pdf/1311.2524.pdf>)을 바탕으로 수행한다. 이 논문에서는 몇 개의 박스를 제안하고 박스 중 하나가 실측값과 일치하는지 확인한다. 이러한 영역 제안을 위해 선택적 검색이 사용됐다. 선택적 검색은 다양한 크기의 윈도우 색상/질감(Texture)을 그룹화해 영역을 제안한다. 선택적 검색은 블롭(blob) 같은 구조를 찾는다. 픽셀로 시작해 더 큰 크기의 블롭을 만들고, 약 2,000개의 영역을 제안한다. 이러한 영역 제안은 가능한 모든 슬라이딩 윈도우와 비교할 때 적다는 특징을 가진다.

제안한 논문에서는 크기가 조정되고, AlexNet/VGG/인셉션/ResNet과 같은 표준 CNN 아키텍처를 사용한다. CNN의 마지막 레이어에서는 객체가 없는 클래스를 사용하는 객체 식별 SVM으로 훈련한다. 박스는 이미지 주변의 박스를 최적화해 추가로 향상된다. 더 가까운 경계 박스를 예측하는 선형 회귀 모델은 객체 영역 제안을 통해 학습된다. R-CNN의 아키텍처는 다음과 같다.



거쉬의 허락하에 재구성 게재

인코더는 표준 딥러닝 모델의 사전 훈련된 모델이 될 수 있다. 특징은 모든 영역의 학습 데



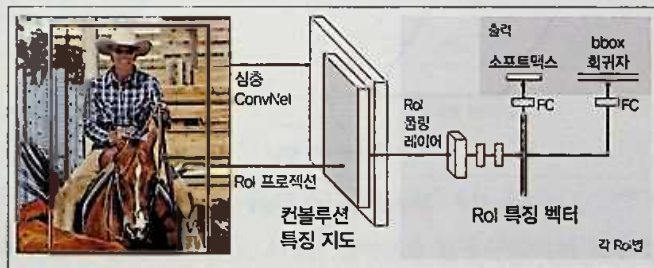
이터로부터 계산된다. 특징들은 저장되고, SVM이 특징을 사용해 학습한다. 이어서 경계 박스는 정규화된 좌표로 트레이닝된다. 이미지 좌표 외부에 몇 가지 제안이 있을 수 있으므로 훈련 및 추론을 위해 정규화된다.

이 방법의 단점은 다음과 같다.

- 선택적 검색을 통해 여러 제안이 이뤄지므로 많은 추론을 계산해야 한다.
- 대개 약 2,000번의 훈련을 받아야 하는 세 개의 분류기가 있으며, 이는 매개변수의 개수를 증가시킨다.
- 종단<sup>end-to-end</sup> 훈련<sup>1</sup>이 없다.

## Fast R-CNN

Fast R-CNN은 거석<sup>2</sup>에 의해 제안됐으며, CNN 추론을 한 번만 수행하기 때문에 연산량을 줄일 수 있다. CNN의 결과는 네트워크를 제안하거나 경계 박스를 선택하기 위해 사용한다. 이러한 방법을 관심 풀링 영역<sup>RoI, Region of Interest pooling</sup>이라 한다. 관심 풀링 영역은 CNN 특징을 사용하고 영역에 맞춰서 특징들을 같이 풀링<sup>2</sup>할 수 있다. 다음 그림에서 나타낸 바와 같이, CNN을 사용해 추론한 이후에 얻는 특징들은 풀링되고 영역들이 선택될 수 있다.



거석의 허락하에 재구성 게재

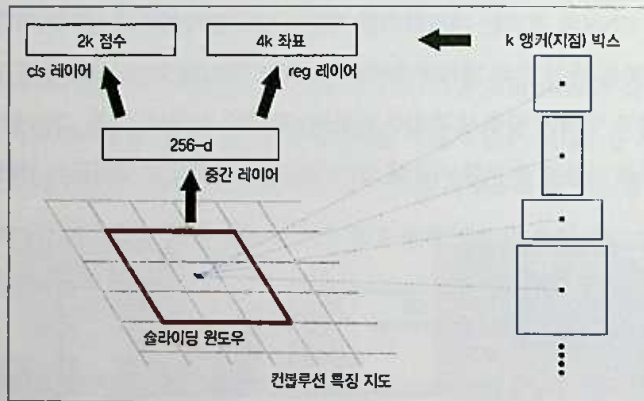
1 전체급 모델로 한 번에 학습하는 훈련을 말한다. - 옮긴이

2 네트워크의 파라미터 개수나 연산량을 줄이기 위해 사이즈를 줄이고 오버피팅을 조절한다. - 옮긴이

이렇게 하면 다중 분류자를 피하면서 종단 간 훈련이 수행된다. SVM이 소프트맥스 레이어로 대체되고 박스 회귀 변수가 경계 박스 회귀 변수로 바뀐다. 여전히 남아있는 단점은 선택 검색(selective search)에 시간이 걸린다는 점이다.

## Faster R-CNN

Faster R-CNN은 Ren<sup>ren</sup>에 의해 제안된 방법(<https://arxiv.org/pdf/1506.01497.pdf>)이다. Faster R-CNN과 Fast R-CNN 방법의 차이점은 Faster R-CNN이 선택 검색 대신 제안 영역 제안 네트워크를 통해 추가로 전달된다. 슬라이딩 윈도우는 잠재적 경계 박스와 점수(score)를 가지는 특징을 결과로 전달하며, 직관적 중형비도 전달한다. 모델은 바운드 박스와 점수를 결과로 출력한다.



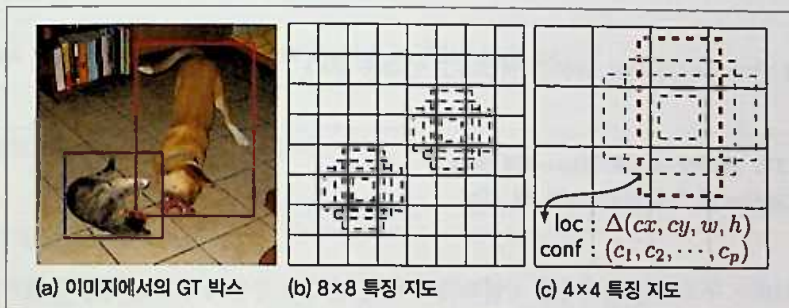
렌의 허락하에 재구성 게재

Faster R-CNN은 특징을 한 번에 계산해 연산 과정을 절약하기 때문에 Fast R-CNN보다 빠르다.

## 싱글 샷 다중 박스 검출기

SSD(Single Shot multi-box Detector)는 모든 방법 중에서 가장 빠른 방법이다. 이 방법은 객체를 예측하고 경계 박스를 찾는 작업을 동시에 한다. 훈련 도중에 부정(negative) 값이 많아서 클래스의 불균형이 심하게 발생할 수 있으며, CNN은 다양한 크기의 결과 출력을 가진다. 경계 박스를 예측하기 위해 CNN 결과를  $3 \times 3$  컨볼루션 필터에 전달한다.

이 단계에서는 객체와 경계 박스를 예측할 수 있다.



리우(Liu)의 허락하에 재구성 게재

다음은 객체 검출에 사용할 수 있는 알고리즘이며, 다음 절에서 이를 구현하는 방법을 학습한다.

## ■ 객체 검출 API

구글은 공개적으로 사용될 수 있도록 COCO 데이터셋에서 훈련된 다양한 알고리즘을 사용해 사전 훈련된 모델을 제공한다. API는 텐서플로 위에 구축되며 객체 검색 모델을 구성, 훈련, 배포하는 용도로 활용된다. 또한 객체 검출 및 로컬라이제이션 작업을 모두 지원하고, 사전 훈련된 모델을 사용할 수 있어 새로운 데이터를 미세 조정할 수 있으므로 훈련 시간을 단축할 수 있다. 이러한 서로 다른 모델은 속도와 정확성 간에 절충점을 가진다.

## 설치 및 설정

다음 명령을 사용해 프로토콜 버퍼(Protocol Buffers([protobuf](https://github.com/google/protobuf))) 컴파일러를 설치한다. `protobuf` 용 디렉터리를 만들고 라이브러리를 직접 다운로드한다.

---

```
mkdir protoc_3.3
cd protoc_3.3
wget https://github.com/google/protobuf/releases/download/v3.3.0/protoc-3.3.0-
linux-x86_64.zip
```

---

다음과 같이 폴더의 사용 권한을 변경하고 압축을 푼다.

---

```
chmod 775 protoc-3.3.0-linux-x86_64.zip
unzip protoc-3.3.0-linux-x86_64.zip
```

---

`protobuf`는 구조화된 데이터를 직렬화하기 위한 언어에 중립적이고 플랫폼 중립적이며 확장 가능한 구글의 메커니즘이다. XML을 사용하지만 훨씬 간단하고 빠르다. 모델은 일반적으로 텐서플로에서 이 형식으로 내보낸다. 데이터 구조를 정의하고, 다양한 언어를 사용해 읽고 쓸 수 있다. 이어서 `protobuf`를 컴파일하려면 다음 명령을 실행한다. 작업 폴더로 다시 이동해 <https://github.com/tensorflow/models.git>에서 repo를 클론(clone)하고 해당 폴더로 이동한다.

---

```
git clone https://github.com/tensorflow/models.git
```

---

이제 다음 코드를 사용해 `research` 폴더로 모델을 이동한다.

---

```
cd models/research/
~/protoc_3.3/bin/protoc object_detection/protos/*.proto --python_out=.
```

---

텐서플로 객체 검출 API는 모델 가중치 및 훈련 매개변수를 내보낼 때 `protobuf`를 사용

한다. TensorFlow, model, research, slim 디렉터리는 다음 명령을 사용해 PYTHONPATH에 추가한다.

```
export PYTHONPATH =. : ./slim/
```

위의 명령으로 파이썬 경로에 추가하면 되고, 한 번만 실행하면 된다. 이후 이 명령을 다시 실행할 수 있다. 다음 코드를 실행해 설치가 제대로 됐는지 테스트할 수 있다.

```
python object_detection/builders/model_builder_test.py
```

이 코드의 실행 결과는 다음과 같다.

```
Ran 7 tests in 0.022s  
OK
```



설치에 대한 자세한 내용은 from [https://github.com/tensorflow/models/blob/master/research/object\\_detection/g3doc/installation.md](https://github.com/tensorflow/models/blob/master/research/object_detection/g3doc/installation.md)에서 얻을 수 있다. 이제 설치가 완료됐다. 그리고 테스트도 완료됐다.

## 사전 훈련된 모델

미리 훈련되고 사용 가능한 여러 모델이 있다. 이 모든 모델은 COCO 데이터셋에서 학습되며 인간 및 자동차와 같은 COCO 데이터셋에서 사용 가능한 객체를 검색하는 데 사용할 수 있다. 이 모델은 교통 표지판 검출과 같은 새로운 작업을 위한 전승 학습에도 유용하다. 사전 훈련된 모델 표에서는 COCO 데이터셋의 상대 속도 및 mAP가 함께 표시된다. 다양한 알고리즘이 다른 CNN을 사용해 훈련되고, 그 내용을 이름에 묘사한다.

| 모델명                                         | 속도 | COCO mAP |
|---------------------------------------------|----|----------|
| ssd_mobilenet_v1_coco                       | 빠름 | 21       |
| ssd_inception_v2_coco                       | 빠름 | 24       |
| rfcn_resnet101_coco                         | 중간 | 30       |
| faster_rcnn_resnet101_coco                  | 중간 | 32       |
| faster_rcnn_inception_resnet_v2_atrous_coco | 느림 | 37       |

요구 사항에 따라 모델에서 선택할 수 있다. Mobilenet에서 훈련받은 SSD 모델을 다운로드하고 작업 디렉터리로 이동해 여기에 표시된 대로 검출할 수 있다.

---

```
mkdir Chapter04 && cd Chapter04
wget
http://download.tensorflow.org/models/object_detection/ssd_mobilenet_v1_
coco_11_06_2017.tar.gz
tar -xvzf ssd_mobilenet_v1_coco_11_06_2017.tar.gz
```

---

Chapter04 폴더에는 다음과 같은 다양한 파일이 있다.

- 그래프의 원형<sup>proto</sup> 정의를 나타낸 파일: graph.pbtxt
- 그래프의 가중치가 고정돼 추론에 사용하기 위한 파일: frozen\_inference\_graph.pb
- 체크포인트 파일
  - model.ckpt.data-00000-of-00001
  - model.ckpt.meta
  - model.ckpt.index

이 모델은 다음 절에서 검색 작업에 사용된다.



## 객체 검출 모델 재훈련

동일한 API를 사용해 맞춤 데이터 집합에 대한 모델을 재훈련할 수 있다. 맞춤 데이터 훈련에는 데이터셋 준비, 알고리즘 선택, 미세 조정 수행이 포함된다. 전체 파이프라인을 매개변수로 전달해 스크립트로 전달한다. 훈련 데이터는 텐서플로 레코드로 변환해야 한다. 텐서플로 레코드는 구글이 일반 파일보다 빠르게 데이터를 읽도록 제공하는 파일 형식이다. 이제 우리는 훈련 단계를 밟을 것이다.

### Pet 데이터셋의 데이터 준비

이 예제에서는 Oxford-IIIT Pet 데이터 집합이 사용된다. Chapter04 디렉터리에서 이 명령으로 이미지와 애노테이션을 다운로드한다.

---

```
wget http://www.robots.ox.ac.uk/~vgg/data/pets/data/images.tar.gz
wget http://www.robots.ox.ac.uk/~vgg/data/pets /data/annotations.tar.gz
```

---

다음과 같이 이미지와 애노테이션을 추출한다.

---

```
tar -xvf images.tar.gz
tar -xvf annotations.tar.gz
```

---

객체 검출 트레이너에 대한 필수 입력으로 필요하기 때문에 tf 레코드에서 데이터 집합을 생성하기 위한 pet\_tf 레코드 파일을 만들어야 한다. Pet 데이터셋의 label\_map은 object\_detection/data/pet\_label\_map.pbtxt에서 찾을 수 있다. research 폴더로 이동하고 다음 명령을 실행한다.

---

```
python object_detection/create_pet_tf_record.py \
  --label_map_path=object_detection/data/pet_label_map.pbtxt \
  --data_dir=~/.chapter4/ \
  --output_dir=~/.chapter4/.
```

---

pet\_train.record와 pet\_val.record라는 이름을 가진 research 디렉터리에서 두 개의 .record 파일을 볼 수 있다.

### 객체 검출 훈련 파이프라인

훈련을 위한 protobuf는 훈련용으로 구성돼야 한다. 이 과정에서 고려해야 할 다섯 가지 중요한 사항을 정리하면 다음과 같다.

- 모델 유형을 사용한 모델 설정
- 표준 훈련 매개변수용 train\_config
- 보고해야 하는 측정 항목용 eval\_config
- 데이터셋용 train\_input\_config
- 평가 데이터셋용 eval\_input\_config

그리고 [https://github.com/tensorflow/models/blob/master/research/object\\_detection/samples/configs/ssd\\_mobilenet\\_v1\\_pets.config](https://github.com/tensorflow/models/blob/master/research/object_detection/samples/configs/ssd_mobilenet_v1_pets.config)의 설정 파일을 사용할 것이다. 다음 명령어를 사용해 다운로드한 후 Chapter04 폴더에 넣는다. config 파일을 열고 다음과 같이 편집한다.

---

```
fine_tune_checkpoint:
  "~/Chapter04/ssd_mobilenet_v1_coco_11_06_2017/model.ckpt"
train_input_reader: {
  tf_record_input_reader {
    input_path: "~/Chapter04/pet_train.record"
  }
  label_map_path:
    "~/model/research/object_detection/data/pet_label_map.pbtxt"
}
eval_input_reader: {
  tf_record_input_reader {
    input_path: "~/Chapter04/pet_val.record"
  }
}
```

```
label_map_path:
"~/model/research/object_detection/data/pet_label_map.pbtxt"
}
```

---

config 파일을 저장한다. 파일에는 모델의 정확성에 영향을 주는 다양한 매개변수가 있다.

## 모델 훈련하기

이제 다시 훈련하기 위해 API, 데이터, 설정 파일을 사용할 준비를 해야 한다. 다음 명령을 사용해 훈련을 시작할 수 있다.

```
PYTHONPATH=../../slim/. python object_detection/train.py \
--logtostderr \
--pipeline_config_path=~/chapter4/ssd_mobilenet_v1_pets.config \
--train_dir=~/Chapter04
```

---

훈련에서 손실 값은 약 140 정도로 시작하며, 그 값은 계속 감소할 것이다. 훈련은 영원히 계속되며 Ctrl + C 명령을 사용해 수동으로 종료해야 한다. 트레이닝 중에 생성된 체크포인트는 이후 추론을 위해 사용될 수 있다.

## 텐서보드를 사용해 손실 및 정확성 모니터링

훈련 손실 및 정확도는 텐서보드를 사용해 모니터링할 수 있다. 다음 명령을 사용해 텐서보드를 실행한다.

```
tensorboard --logdir=/home/ubuntu/Chapter04
```

---

훈련과 평가는 텐서보드에서 시각화할 수 있다.

## 자율주행용 보행자 검출 훈련

보행자 객체 검출 훈련을 위한 데이터셋은 <http://pascal.inrialpes.fr/data/human/>에서 얻을 수 있다. 보행자를 검출하는 단계는 [https://github.com/diegocavalca/machine-learning/blob/master/supervised/object\\_detection\\_tensorflow/simple\\_detection.ipynb](https://github.com/diegocavalca/machine-learning/blob/master/supervised/object_detection_tensorflow/simple_detection.ipynb)에서 확인할 수 있다. 교통 표지판 검출 훈련을 하기 위한 데이터셋은 [http://www.vision.ee.ethz.ch/~timofter/traffic\\_signs/](http://www.vision.ee.ethz.ch/~timofter/traffic_signs/)와 <http://btsd.ethz.ch/shareddata/>에서 다운로드할 수 있다. 자율주행 자동차의 경우 보행자, 자동차, 오토바이, 배경 등의 네 가지 클래스를 이미지 레이블링을 위해 사용한다. 배경 클래스는 클래스가 하나도 없을 때 검출된다. 딥러닝 분류 모델을 훈련할 때는 적어도 하나의 객체가 이미지에 존재한다고 가정한다. 백그라운드 클래스를 추가하면 이러한 문제를 극복할 수 있다. 신경망은 또한 레이블로부터 객체의 경계 박스를 생성할 수 있다.

## ■ YOLO 객체 검출 알고리즘

객체 검출을 위한 최근의 알고리즘은 YOLO(You look only once)다. 이미지는 여러 그리드로 나뉜다. 이미지의 각 그리드 셀에는 동일한 알고리즘을 적용한다. 초기화 방법(initializer)을 사용해 레이어를 정의하고 구현해보자.

---

```
def pooling_layer(input_layer, pool_size=[2, 2], strides=2,
padding='valid'):
    layer = tf.layers.max_pooling2d(
        inputs=input_layer,
        pool_size=pool_size,
        strides=strides,
        padding=padding
    )
    add_variable_summary(layer, 'pooling')
    return layer
```

```

def convolution_layer(input_layer, filters, kernel_size=[3, 3],
padding='valid', activation=tf.nn.leaky_relu):
    layer = tf.layers.conv2d(
        inputs=input_layer,
        filters=filters,
        kernel_size=kernel_size,
        activation=activation,
        padding=padding,
        weights_initializer=tf.truncated_normal_initializer(0.0, 0.01),
        weights_regularizer=tf.l2_regularizer(0.0005)
    )
    add_variable_summary(layer, 'convolution')
    return layer

def dense_layer(input_layer, units, activation=tf.nn.leaky_relu):
    layer = tf.layers.dense(
        inputs=input_layer,
        units=units,
        activation=activation,
        weights_initializer=tf.truncated_normal_initializer(0.0, 0.01),
        weights_regularizer=tf.l2_regularizer(0.0005)
    )
    add_variable_summary(layer, 'dense')
    return layer

```

---

활성화 레이어(activation layer)는 leaky\_relu며, 가중치(weight)는 절단(truncated) 정규 분포로 초기화된다는 것을 알 수 있다. 이 수정된 레이어는 모델을 작성하는 데 사용할 수 있다. 모델은 다음과 같이 생성된다.

---

```

yolo = tf.pad(images, np.array([[0, 0], [3, 3], [3, 3], [0, 0]]), name='pad_1')
yolo = convolution_layer(yolo, 64, 7, 2)
yolo = pooling_layer(yolo, [2, 2], 2, 'same')
yolo = convolution_layer(yolo, 192, 3)
yolo = pooling_layer(yolo, 2, 'same')
yolo = convolution_layer(yolo, 128, 1)
yolo = convolution_layer(yolo, 256, 3)

```

```

yolo = convolution_layer(yolo, 256, 1)
yolo = convolution_layer(yolo, 512, 3)
yolo = pooling_layer(yolo, 2, 'same')
yolo = convolution_layer(yolo, 256, 1)
yolo = convolution_layer(yolo, 512, 3)
yolo = convolution_layer(yolo, 256, 1)
yolo = convolution_layer(yolo, 512, 3)
yolo = convolution_layer(yolo, 256, 1)
yolo = convolution_layer(yolo, 512, 3)
yolo = convolution_layer(yolo, 256, 1)
yolo = convolution_layer(yolo, 512, 3)
yolo = convolution_layer(yolo, 512, 1)
yolo = convolution_layer(yolo, 1024, 3)
yolo = pooling_layer(yolo, 2)
yolo = convolution_layer(yolo, 512, 1)
yolo = convolution_layer(yolo, 1024, 3)
yolo = convolution_layer(yolo, 512, 1)
yolo = convolution_layer(yolo, 1024, 3)
yolo = convolution_layer(yolo, 1024, 3)
yolo = tf.pad(yolo, np.array([[0, 0], [1, 1], [1, 1], [0, 0]]))
yolo = convolution_layer(yolo, 1024, 3, 2)
yolo = convolution_layer(yolo, 1024, 3)
yolo = convolution_layer(yolo, 1024, 3)
yolo = tf.transpose(yolo, [0, 3, 1, 2])
yolo = tf.layers.flatten(yolo)
yolo = dense_layer(yolo, 512)
yolo = dense_layer(yolo, 4096)
dropout_bool = tf.placeholder(tf.bool)
yolo = tf.layers.dropout(
    inputs=yolo,
    rate=0.4,
    training=dropout_bool
)
yolo = dense_layer(yolo, output_size, None)

```

---

여러 개의 컨볼루션 레이어가 스택돼 YOLO 네트워크를 생성한다. 이 네트워크는 실시간 검출을 위한 객체 검출 알고리즘을 생성하는 데 사용된다.



## I 요약

이 장에서는 객체 로컬라이제이션과 검출 작업의 차이점을 살펴보았다. 여러 데이터셋과 평가 기준들이 논의됐으며, 검출을 위한 R-CNN 및 SSD 모델의 변종과 같은 로컬라이제이션 문제 및 알고리즘에 대한 다양한 접근 방법도 논의됐다. 그리고 오픈소스 저장소에서 검출 구현을 다뤘다. 기술을 사용해 보행자 검출 모델을 훈련했으며, 그러한 모델을 훈련할 때의 여러 가지 절충점에 대해서도 배웠다.

다음 장에서는 시맨틱 분할 알고리즘을 학습한다. 의료 영상 및 위성 이미지 문제에 대한 분할 알고리즘을 구현하기 위해 이 지식을 사용할 것이다.



## 시맨틱 분할

이 장에서는 다양한 시맨틱 분할(semantic segmentation) 기법과 그 모델을 학습한다. 분할은 픽셀 단위의 분류 작업이다. 분할 문제를 해결하기 위한 아이디어는 객체 검출 문제의 연장이 된다. 분할은 의료 및 위성 이미지 분석과 같은 애플리케이션에 매우 유용하다.

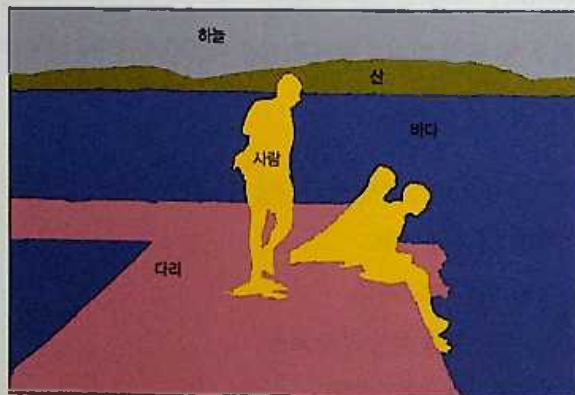
이 장에서 다룰 내용은 다음과 같다.

- 시맨틱 분할과 인스턴스 분할의 차이점 배우기
- 분할 데이터셋 및 측정 항목
- 시맨틱 분할을 위한 알고리즘
- 의료 및 위성 이미지에 대한 분할 적용
- 인스턴스 분할을 위한 알고리즘

## 픽셀 예측

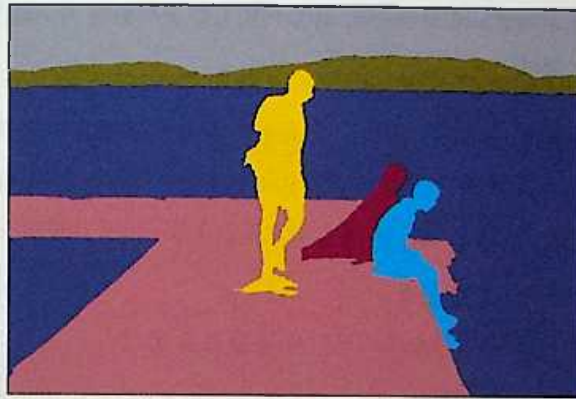
이미지 분류는 레이블이나 카테고리를 예측하는 작업이다. 객체 검출은 해당 경계 박스를 사용해 몇 가지 딥러닝 기반 알고리즘의 목록을 예측하는 작업이다. 경계 박스에는 검출된 객체 이외의 객체가 있을 수 있다. 일부 애플리케이션 프로그램에서는 레이블에 모든 픽셀에 레이블을 지정하는 것이 여러 객체를 가질 수 있는 경계 박스보다는 중요하다. 시맨틱 분할은 픽셀 단위 레이블을 예측하는 작업이다.

다음은 이미지 및 해당 시맨틱 분할의 예다.



이미지에 표시된 것처럼 입력 이미지는 모든 픽셀에 대해 레이블을 사용해 예측한다. 레이블은 하늘, 나무, 사람, 산, 다리일 수 있다. 전체 이미지에 레이블을 할당하는 대신 각 픽셀 단위로 할당한다. 시맨틱 분할은 픽셀을 독립적으로 레이블링한다. 실제로는 모든 사람들이 각각 구별되지 않음을 알게 될 것이다. 이미지의 모든 사람은 같은 방법으로 레이블이 지정된다.

다음은 동일한 레이블의 모든 인스턴스가 구별되는 예다.



모든 인스턴스를 픽셀 단위의 레이블로 분할하는 작업을 인스턴스 분할이라 한다. 인스턴스 분할은 픽셀 수준 레이블을 사용하는 객체 검색의 확장으로 생각할 수 있다. 시맨틱 분할과 인스턴스 분할의 애플리케이션 프로그램은 엄청나기 때문에 애플리케이션 프로그램 중 일부만 다음 절에서 제공된다.

## 의료 이미지 진단

의학 이미지는 분할 기술을 사용해 진단할 수 있다. 자기 공명 영상(MRI, Magnetic Resonance Imaging), CT(Computed Tomography), 망막 병증에 대한 최신 의료 영상 기술은 고품질 이미지를 만든다. 이러한 기술로 만든 이미지는 다양한 영역으로 분할할 수 있어서 뇌 스캔이나 망막 스캔을 할 때 종양을 검출해낼 수 있다. 일부 장치는 부피 측정 이미지를 제공하며 분할

이미지로도 분석할 수 있다. 로봇 수술을 위해 동영상을 분할하면 의사는 로봇 보조 수술에서 해당 영역을 주의 깊게 볼 수 있다. 이 장의 뒷부분에서는 의료 영상을 분할하는 방법을 알아본다.

## 위성 이미지를 사용해 지구를 살펴보기

최근에는 일반인들도 많은 위성 이미지를 볼 수 있게 됐다. 위성으로 캡처한 이미지는 지구의 전체 표면을 고해상도로 보여준다. 위성 이미지를 분석하면 지구와 관련된 몇 가지 사항을 이해할 수 있다.

- 경제 성장과 관련된 국가의 건설물 측정
- 오일 탱크 측정
- 교통 계획 및 구성
- 삼림 벌채 및 그 효과 산정
- 동물 수를 세고 움직임을 추적해 야생 동물 보호 돕기
- 고고학적 유적지 발견
- 자연 재해로 인한 피해 영역 매핑

위성 이미지에는 더 많은 애플리케이션을 적용할 수 있다. 언급된 대부분의 문제에 대한 솔루션은 위성 이미지의 분할로 시작된다. 이 장의 뒷부분에서 위성 이미지를 분할하는 방법을 살펴본다.

## 로봇이 볼 수 있도록 허용하기

어떤 장면을 분할하는 것은 로봇이 주변 세계를 보고 처리하고자 할 때 매우 중요하다. 산업용 및 가정용 로봇은 물체를 구분해낼 수 있어야 한다. 로봇에서 비전 처리가 가능해지면, 물체를 다루는 처리가 가능해진다. 로봇과 관련해 언급할 만한 몇 가지 활용 가능 애플리케이션 프로그램들을 다음과 같이 들 수 있다.



- 결합 분류 도구의 산업적 검사
- 패션 산업에서의 색상 진단(다양한 패션 객체로 이미지를 분할해 컬러 분류에 사용 가능)
- 전경과 배경을 구분해 인물 효과에 적용

다음 절에서는 분할 알고리즘을 평가하기 위해 몇 가지 공개 데이터 집합을 학습한다.

## I 데이터셋

4장, '객체 검출'에서 언급한 파스칼 및 COCO 데이터셋은 분할 작업에도 사용할 수 있다. 애노테이션은 픽셀 단위로 레이블이 지정돼 있기 때문에 다르다. 새로운 알고리즘은 일반적으로 COCO 데이터셋을 사용해 벤치마킹한다. COCO에는 잔디, 벽, 하늘 등의 사물과 관련된 데이터셋이 있다. 픽셀 정확도와 같은 특성은 알고리즘 평가를 위한 척도로 사용될 수 있다.



언급된 것과는 별도로 의료 이미지 및 위성 이미지 분야의 여러 데이터셋이 있다. 참조용으로 다음과 같은 몇 가지 링크가 제공된다.

- <http://www.cs.bu.edu/~belke/BiomedicalImageSegmentation>
- <https://www.kaggle.com/c/intel-mobileodt-cervical-cancer-screening/data>
- <https://www.kaggle.com/c/diabetic-retinopathy-detection>
- [https://grand-challenge.org/all\\_challenges](https://grand-challenge.org/all_challenges)
- <http://www.via.cornell.edu/databases>
- <https://www.kaggle.com/c/dstl-satellite-imagery-feature-detection>
- <https://aws.amazon.com/public-datasets/spacenet>
- <https://www.iarpa.gov/challenges/1mow.html>
- <https://www.kaggle.com/c/planet-understanding-the-amazon-from-space>

분할 작업을 위한 훈련 데이터를 생성하려면 많은 비용이 든다. 데이터셋에 애노테이션을 달 수 있는 온라인 도구가 있다. MIT에서 제공하는 LabelMe 모바일 애플리케이션은

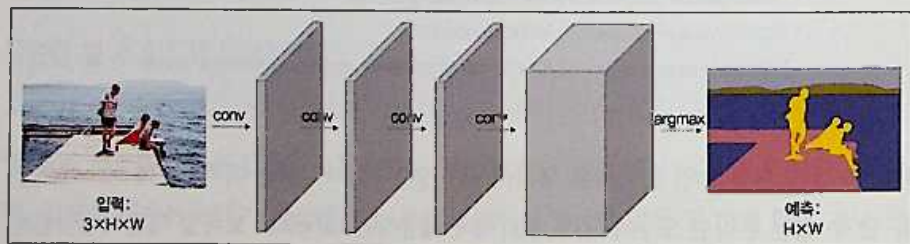
애노테이션 처리에 적합하며 <http://labelme.csail.mit.edu/Release3.0>에서 다운로드 할 수 있다.

## ■ 시맨틱 분할을 위한 알고리즘

이미지 분할 작업을 해결하기 위해 제안된 몇 가지 딥러닝 기반 알고리즘이 있다. 슬라이딩 윈도우 방식은 세그먼트화를 위해 픽셀 레벨에서 적용될 수 있다. 슬라이딩 윈도우 방식은 이미지를 가져와서 이미지를 작게 나눈다. 이미지의 모든 자르기는 레이블 단위로 분류된다. 이 방법은 겹치는 패치 간에 공유 기능을 다시 사용하지 않기 때문에 비용이 많이 들고 비효율적이다. 다음 절에서는 이 문제를 극복할 수 있는 몇 가지 알고리즘을 설명한다.

### 완전 컨볼루션 네트워크

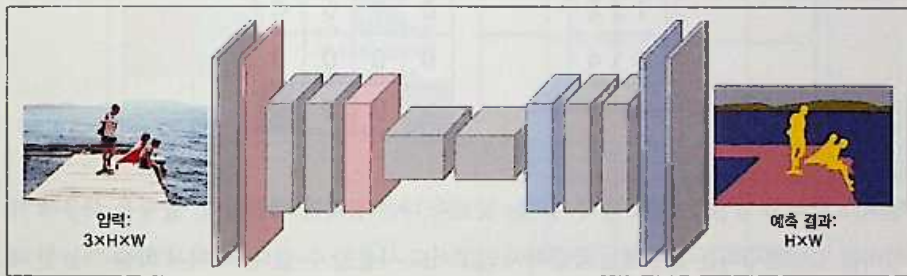
완전 컨볼루션 네트워크 FCN, Fully Convolutional Network는 종단 간 컨볼루션 네트워크 아이디어를 도입했다. 완전 연결 레이어를 제거해 표준 CNN 아키텍처를 FCN에 사용할 수 있으며, 동일한 구현 방법은 4장, ‘객체 검출’에서 설명했다. 완전 연결 레이어는 컨볼루션 레이어로 대체된다. 깊이는 최종 레이어에서 더 깊고 크기는 더 작다. 따라서 1D 컨볼루션을 수행해 원하는 수의 레이블에 도달할 수 있다. 그러나 분할의 경우 공간 차원을 보존해야 한다. 따라서 완전 컨볼루션 네트워크는 다음과 같이 최대 풀링 없이 구성된다.



이 네트워크의 손실은 모든 픽셀과 미니 배치의 교차 엔트로피 손실을 평균해 계산된다. 최종 레이어의 깊이는 클래스 수와 같다. FCN은 공간 차원이 보존된다는 점을 제외하고는 객체 검색과 비슷하다. 일부 픽셀이 잘못 예측될 수 있으므로 아키텍처에서 생성되는 출력은 아주 정확하지 않을 수 있다. 계산량이 높을 것이며, 다음 절에서 이 문제를 해결하는 방법을 살펴본다.

## SegNet 아키텍처

SegNet에는 인코더 및 디코더 방식이 있다. 인코더에는 다양한 컨볼루션 레이어들이 있고, 디코더에는 다양한 디컨볼루션<sup>deconvolution</sup> 레이어들이 있다. SegNet은 FCN에 의해 생성된 출력 결과를 향상시킨다. 이 때문에 메모리에 과도하게 집중적이지는 않다. 특징의 크기가 축소되면 디컨볼루션으로 이미지 크기를 다시 업샘플링해 컨볼루션 효과를 역전시킨다. 디컨볼루션에서는 업샘플링을 위해 매개변수를 학습한다. 이러한 아키텍처를 사용한 출력 결과는 풀링 레이어의 정보 손실로 인해 결과가 거칠어진다.



이제 업샘플링<sup>upsampling</sup>, 구멍 컨볼루션<sup>atrous convolution</sup>, 전치 컨볼루션<sup>transpose convolution</sup>이라는 새로운 개념을 배울 것이므로, 네트워크를 더 잘 이해하는 데 도움이 될 것이다.

## 풀링으로 레이어 업샘플링하기

1장, '시작하기'에서는 최대 풀링에 대해 논의했다. 최대 풀링은 윈도우에서 최댓값을 선택하는 샘플링 전략이다. 최대 풀링은 업샘플링에 의해 반대로 사용될 수 있다. 각 값은 다음과 같이 레이어를 업샘플링하기 위해 0으로 둘러쌀 수 있다.

|   |   |  |   |   |   |   |
|---|---|--|---|---|---|---|
|   |   |  | 1 | 0 | 2 | 0 |
|   |   |  | 0 | 0 | 0 | 0 |
| 1 | 2 |  | 3 | 0 | 4 | 0 |
| 3 | 4 |  | 0 | 0 | 0 | 0 |

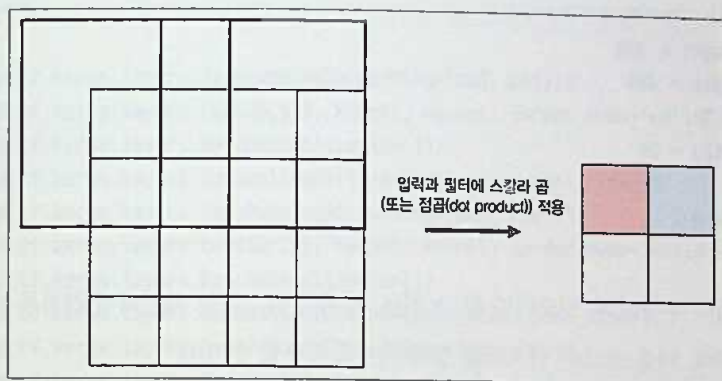
0은 업샘플링된 숫자와 동일한 위치에 추가된다. 언풀링(unpooling)은 다운샘플링(downsampling) 위치를 기억해 업샘플링할 때 사용할 수 있으며, 이를 통해 성능이 개선될 수 있다.

|   |   |  |   |   |   |   |
|---|---|--|---|---|---|---|
|   |   |  | 0 | 0 | 2 | 0 |
|   |   |  | 0 | 1 | 0 | 0 |
| 1 | 2 |  | 0 | 0 | 0 | 0 |
| 3 | 4 |  | 3 | 0 | 0 | 4 |

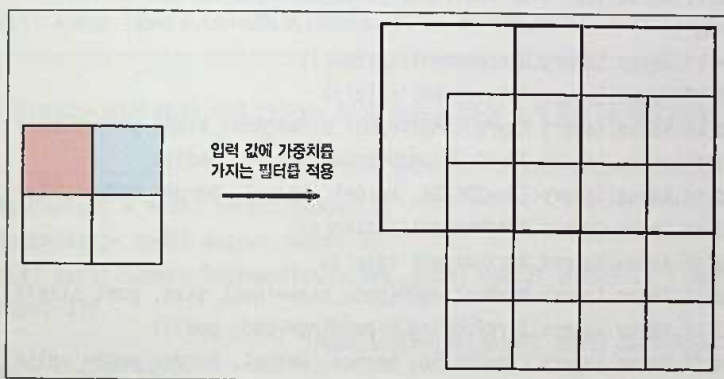
인덱스 방식의 업샘플링은 0을 추가하는 것보다 나은 결과를 제공한다. 풀링을 사용해 레이어를 업샘플링하는 경우에는 학습하지 않고서도 사용할 수 있다. 이어서 학습 가능한 때 개변수를 가지고 업샘플링과 다운샘플링을 어떻게 하는지 살펴본다.

## 컨볼루션을 사용해 레이어 샘플링하기

컨볼루션을 사용해 레이어를 업샘플링하거나 다운샘플링할 수 있다. 컨볼루션에 사용되는 간격은 다음과 같이 다운샘플링할 수 있도록 증가될 수 있다.



컨볼루션에 의한 다운샘플링은 구멍 컨볼루션, 확장 컨볼루션 *dilated convolution*, 또는 간격 컨볼루션 *strided convolution*이라 불린다. 마찬가지로 다음과 같이 커널을 학습하면 업샘플링을 수행할 수 있다.<sup>1</sup>



컨볼루션을 사용해 직접 업샘플링하는 것은 전치 컨볼루션이라 할 수 있다. 일부 다른 동의어는 디컨볼루션 *deconvolution*, 부분적 간격 컨볼루션 *fractionally strided convolution*, 또는 업컨볼루션 *up-convolution*을 사용한다. 이제 업샘플링 프로세스가 이해될 것이다. 이전 알고리즘을 간단히 설명하는 예제 코드는 다음과 같다.

<sup>1</sup> 확장 컨볼루션에서 구멍(*atrous*) 알고리즘을 사용했기 때문에 거의 동일한 알고리즘으로 볼 수 있다(<https://laonp.e.blog.me/220991967450>). - 옮긴이



---

```
input_height = 360
input_width = 480
kernel = 3
filter_size = 64
pad = 1
pool_size = 2
```

---

입력 작업이 수행된 후 인코더라 할 수 있는. 크기가 감소하는 일반적인 컨볼루션 신경망을 사용한다. 다음 코드는 인코더를 정의하는 데 사용할 수 있다.

---

```
model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Layer(input_shape=(3, input_height, input_width)))
# 인코더
model.add(tf.keras.layers.ZeroPadding2D(padding=(pad, pad)))
model.add(tf.keras.layers.Conv2D(filter_size, kernel, kernel, border_
mode='valid'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.Activation('relu'))
model.add(tf.keras.layers.MaxPooling2D(pool_size=(pool_size, pool_size)))
model.add(tf.keras.layers.ZeroPadding2D(padding=(pad, pad)))
model.add(tf.keras.layers.Conv2D(128, kernel, kernel, border_mode='valid'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.Activation('relu'))
model.add(tf.keras.layers.MaxPooling2D(pool_size=(pool_size, pool_size)))
model.add(tf.keras.layers.ZeroPadding2D(padding=(pad, pad)))
model.add(tf.keras.layers.Conv2D(256, kernel, kernel, border_mode='valid'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.Activation('relu'))
model.add(tf.keras.layers.MaxPooling2D(pool_size=(pool_size, pool_size)))
model.add(tf.keras.layers.ZeroPadding2D(padding=(pad, pad)))
model.add(tf.keras.layers.Conv2D(512, kernel, kernel, border_mode='valid'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.Activation('relu'))
```

---

인코더의 출력은 다음 코드를 통해 크기가 증가하는 디코더로 공급될 수 있다.



---

# 디코더

```
model.add(tf.keras.layers.ZeroPadding2D(padding=(pad, pad)))
model.add(tf.keras.layers.Conv2D(512, kernel, kernel, border_mode='valid'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.UpSampling2D(size=(pool_size, pool_size)))
model.add(tf.keras.layers.ZeroPadding2D(padding=(pad, pad)))
model.add(tf.keras.layers.Conv2D(256, kernel, kernel, border_mode='valid'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.UpSampling2D(size=(pool_size, pool_size)))
model.add(tf.keras.layers.ZeroPadding2D(padding=(pad, pad)))
model.add(tf.keras.layers.Conv2D(128, kernel, kernel, border_mode='valid'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.UpSampling2D(size=(pool_size, pool_size)))
model.add(tf.keras.layers.ZeroPadding2D(padding=(pad, pad)))
model.add(tf.keras.layers.Conv2D(filter_size, kernel, kernel,
border_mode='valid'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.Conv2D(nClasses, 1, 1, border_mode='valid', ))
```

---

디코딩된 이미지는 입력과 크기가 같으며, 전체 모델은 다음 코드를 사용해 학습할 수 있다.

---

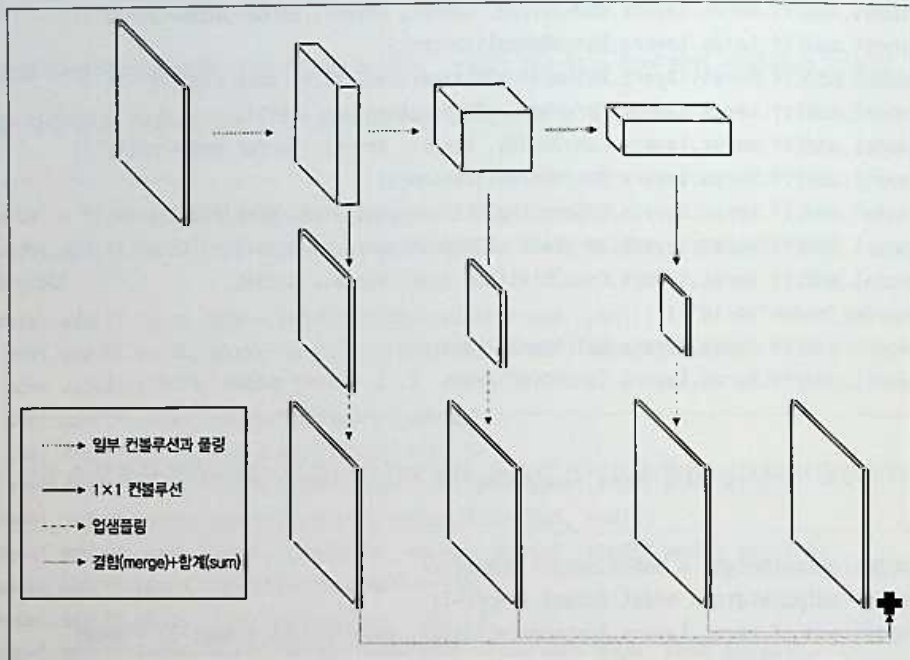
```
model.outputHeight = model.output_shape[-2]
model.outputWidth = model.output_shape[-1]
model.add(tf.keras.layers.Reshape((nClasses, model.output_shape[-2] * model.
output_shape[-1]),
                                input_shape=(nClasses, model.output_shape[-2],
model.output_shape[-1])))
model.add(tf.keras.layers.Permute((2, 1)))
model.add(tf.keras.layers.Activation('softmax'))
model.compile(loss="categorical_crossentropy",
optimizer=tf.keras.optimizers.Adam, metrics=['accuracy'])
```

---

이러한 이미지 인코딩 및 디코딩 방식은 FCN 기반 모델의 단점을 극복한다. 다음 내용은 확장 컨볼루션과는 다른 개념이다.

## 향상된 훈련을 위한 연결 건너뛰기

분할 출력의 거칠기(coarseness)는 건너뛰기 아키텍처에 의해 제한될 수 있으며 더 높은 해상도를 얻을 수 있다. 또 다른 방법은 마지막 세 개의 레이어를 확장해 여기에 표시된 대로 평균화하는 것이다.

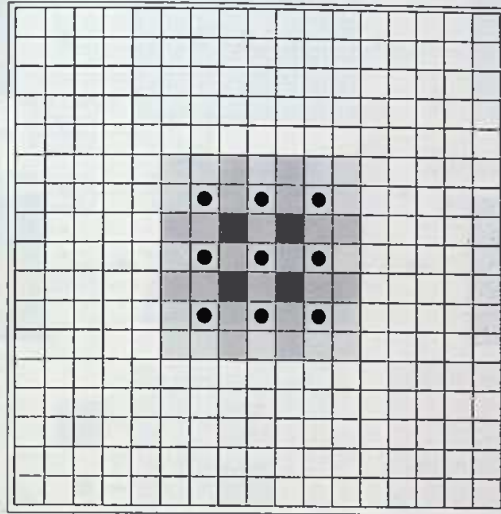


이 알고리즘은 다음 절의 위성 이미지 예에서 사용된다.

## 확장 컨볼루션

픽셀 단위 분류와 이미지 분류는 구조적으로 다르다. 따라서 정보를 줄이는 풀링 레이어에서는 거친 분할이 이뤄진다. 그러나 풀링은 더 넓은 뷰(view)를 확보하고 샘플링을 허용하기 위해 필수적임을 기억해야 한다. 확장 컨볼루션이라 하는 새로운 아이디어는 큰 손실이 없는 샘플링에 대해 이 문제를 해결하고자 도입됐다. 확장 컨볼루션은 본질적으로 원

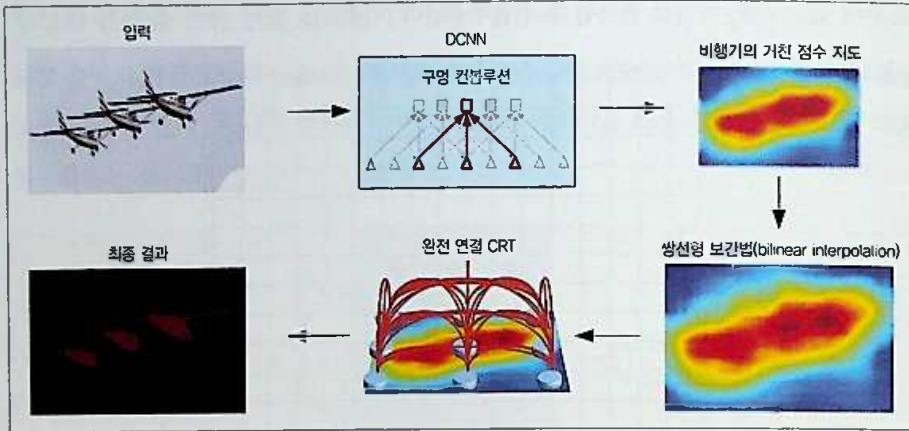
도우에 있는 모든 픽셀을 건너뛰 후 컨볼루션한다.



확장 거리 *dilation distance*는 레이어에 따라 다르다. 이러한 분할 결과의 출력은 더 미세한 해상도로 업스케일된다. 여러 가지 스케일 집합을 사용하기 위해 별도 네트워크를 훈련시킨다.

## DeepLab

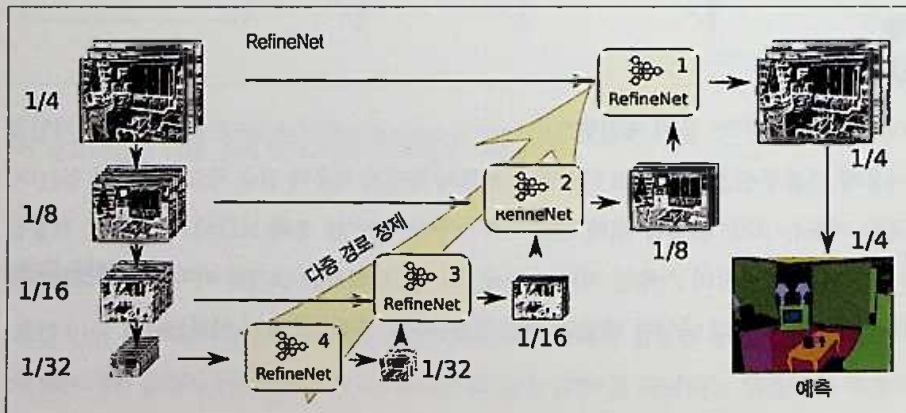
DeepLab은 첸<sup>Chen</sup> 등이 제안했으며(<https://arxiv.org/pdf/1606.00915.pdf>), 여러 스케일을 사용해 컨볼루션을 수행하고 다양한 스케일의 특징을 사용해 점수 지도 *score map*를 얻는다. 점수 지도는 최종 분할을 위해 CRF<sup>Conditional Random Field</sup>를 통해 보간되고 *interpolated* 전달된다. 이러한 이미지의 스케일 처리는 자체 CNN으로 다양한 크기의 이미지를 처리하거나 확장된 컨볼루션의 다양한 레벨로 병렬 컨볼루션을 수행함으로써 이뤄진다.



첸의 허락하에 재구성 게재

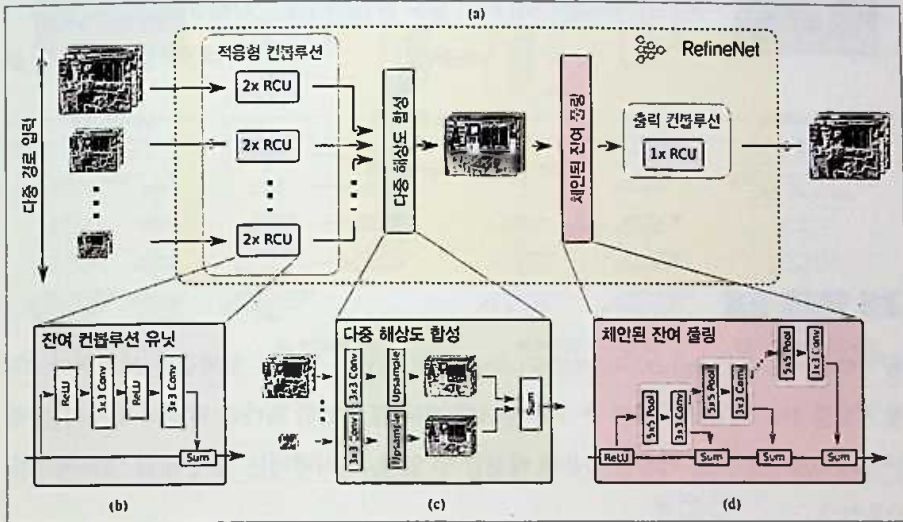
## RefiNet

확장 컨볼루션은 더 큰 입력을 필요로 하므로 메모리를 많이 사용한다. 따라서 고해상도 그림에서 계산하고자 할 때 문제가 발생할 수 있다. 리드<sup>Reid</sup> 등이 문제를 해결하기 위해 RefiNet(<https://arxiv.org/pdf/1611.06612.pdf>)을 개발했다.



리드의 허락하에 재구성 게재

RefiNet은 인코더와 디코더(인코더 뒤에 위치)를 사용한다. 인코더는 CNN의 결과물들을 사용하고, 디코더는 다양한 크기의 특징 값들을 연결한다.



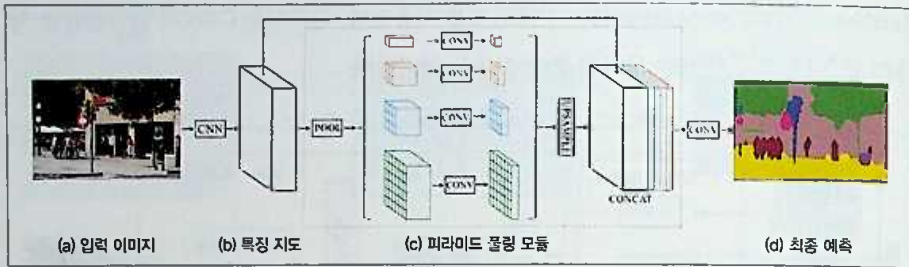
리드의 허락하에 재구성 개제

연결 기능을 사용해 저차원 특징들을 업스케일링한다.

## PSPnet

글로벌 콘텐츠(global content)는 PSPnet에서 조이<sup>Zhou</sup>가 도입한 방법(<https://arxiv.org/pdf/1612.01105.pdf>)이며, 풀링 레이어의 커널 크기를 늘렸다. 풀링에서는 피라미드 방식을 사용했다. 피라미드는 이미지의 다양한 부분과 크기를 동시에 커버한다. 중간 수준의 지도 supervision가 가능하며, 이로 인해 아키텍처 간에 손실이 발생한다.

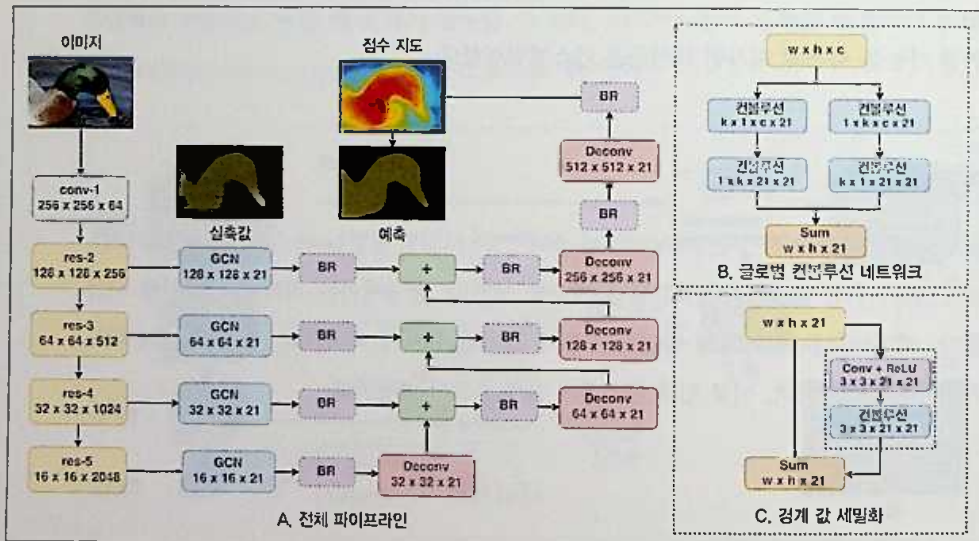




조아의 허락하에 재구성 게재

## 대형 커널의 문제

펑Peng(<https://arxiv.org/pdf/1703.02719.pdf>)은 대형 커널이 가지는 문제점을 보여줬다. 대형 커널은 작은 커널보다 훨씬 큰 수용 필드를 가지며, 이러한 커다란 커널에서 가지는 계산상의 복잡성은 작은 커널을 사용해 해결할 수 있다. 마지막에는 경계 분할 네트워크를 사용한다.

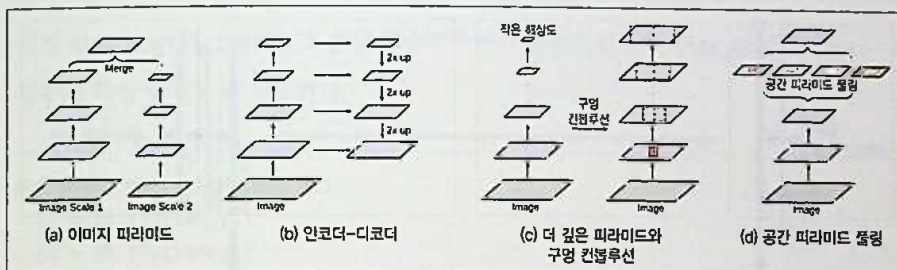


펑의 허락하에 재구성 게재

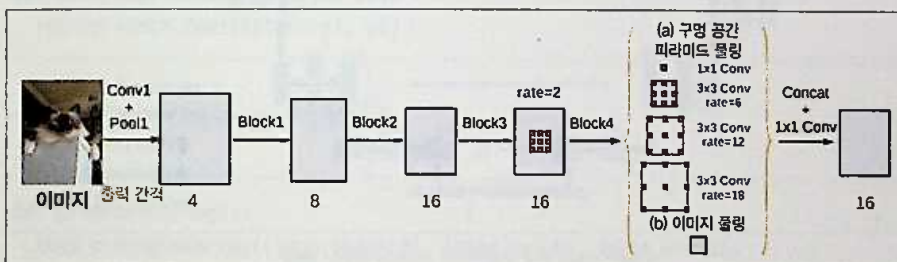


## DeepLab v3

일괄 정규화(batch normalization)는 성능 향상을 위한 첸Chen의 논문(<https://arxiv.org/pdf/1706.05587.pdf>)에서 사용됐다. 특징에 대한 다중 스케일은 캐스케이드(cascade) 방식으로 인코딩돼 성능을 향상시킨다.



첸의 허락하에 재구성 게재



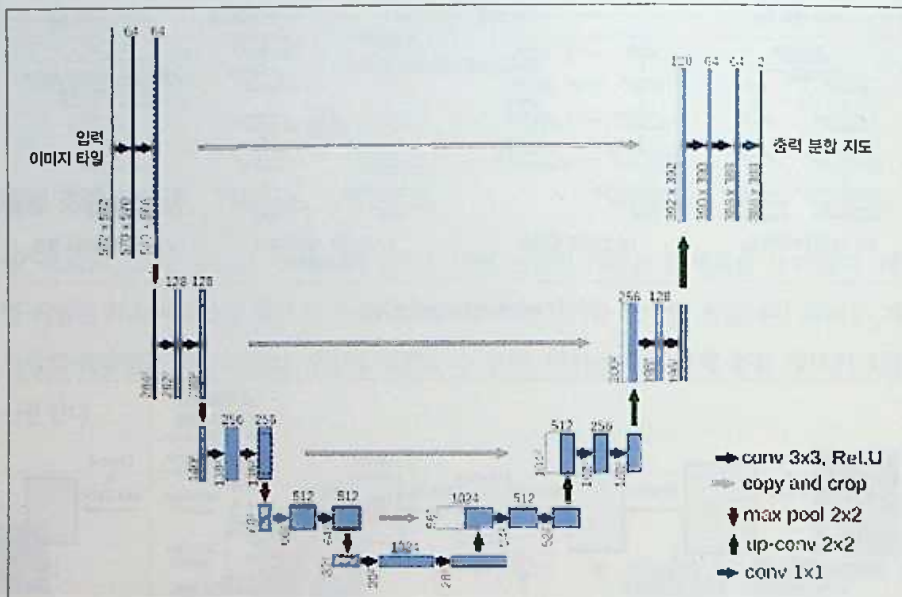
첸의 허락하에 재구성 게재

몇 가지 아키텍처에서 딥러닝을 사용해 이미지 분할의 정확성을 향상시키는 것을 살펴봤다. 이어서 의료 이미징 분야에 활용되는 애플리케이션을 살펴볼 것이다.

## Ⅰ 울트라-신경 분할

캐글러Kaggle는 예측 모델링 및 분석에 대한 대회를 진행하는 조직이다. 캐글러는 예전에 목(neck)의 초음파 이미지에서 신경 구조를 분할하는 데 어려움을 겪고 있었다. 분할할 때

사용한 동일한 데이터는 <https://www.kaggle.com/c/ultrasound-nerve-segmentation>에서 다운로드할 수 있다. 로젠버거Ronneberger가 제안한 UNET 모델(<https://arxiv.org/pdf/1505.04597.pdf>)은 자동 인코더와 유사하지만 완전 연결 레이어 대신 컨볼루션을 사용하며, 감소 차원 컨볼루션을 사용하는 인코딩 부분과 차원 증가 디코딩 부분이 있다.



UNET 모델의 구조(로젠버거의 허락하에 재구성 게재)

유사한 크기의 인코더 및 디코더 부분의 컨볼루션은 스킵 연결(skip connection)에 의해 학습된다. 모델의 출력은 0과 1 사이의 범위를 가지는 마스크다. 먼저 다음 코드를 사용해 사용하고자 하는 함수를 호출한다.

```
import os
from skimage.transform import resize
from skimage.io import imsave
import numpy as np
from data import load_train_data, load_test_data
```

모든 불러오기가 끝나면 다음 코드를 사용해 크기를 정의한다.

---

```
image_height, image_width = 96, 96
smoothness = 1.0
work_dir = ''
```

---

이제 `dice_coefficient`와 그 손실 함수<sup>loss function</sup>를 정의해보자. `dice_coefficient`는 이 경우에 측정 항목으로 사용한다.

---

```
def dice_coefficient(y1, y2):
    y1 = tf.flatten(y1)
    y2 = tf.flatten(y2)
    return (2. * tf.sum(y1 * y2) + smoothness) / (tf.sum(y1) + tf.sum(y2) +
smoothness)
def dice_coefficient_loss(y1, y2):
    return -dice_coefficient(y1, y2)
```

---

UNET 모델은 다음과 같이 정의된다.

---

```
def preprocess(imgs):
    imgs_p = np.ndarray((imgs.shape[0], image_height, image_width),
dtype=np.uint8)
    for i in range(imgs.shape[0]):
        imgs_p[i] = resize(imgs[i], (image_width, image_height),
preserve_range=True)
    imgs_p = imgs_p[..., np.newaxis]
    return imgs_p
def covolution_layer(filters, kernel=(3,3), activation='relu',
input_shape=None):
    if input_shape is None:
        return tf.keras.layers.Conv2D(
            filters=filters,
            kernel=kernel,
            activation=activation)
```

---

```

else:
    return tf.keras.layers.Conv2D(
        filters=filters,
        kernel=kernel,
        activation=activation,
        input_shape=input_shape)

def concatenated_deconvolution_layer(filters):
    return tf.keras.layers.concatenate([
        tf.keras.layers.Conv2DTranspose(
            filters=filters,
            kernel=(2, 2),
            strides=(2, 2),
            padding='same'
        )],
        axis=3
    )

```

---

다음 코드에서는 모든 레이어가 연결돼 사용된다.

---

```

unet = tf.keras.models.Sequential()
inputs = tf.keras.layers.Input((image_height, image_width, 1))
input_shape = (image_height, image_width, 1)
unet.add(covolution_layer(32, input_shape=input_shape))
unet.add(covolution_layer(32))
unet.add(pooling_layer())
unet.add(covolution_layer(64))
unet.add(covolution_layer(64))
unet.add(pooling_layer())
unet.add(covolution_layer(128))
unet.add(covolution_layer(128))
unet.add(pooling_layer())
unet.add(covolution_layer(256))
unet.add(covolution_layer(256))
unet.add(pooling_layer())
unet.add(covolution_layer(512))
unet.add(covolution_layer(512))

```

---

레이어는 연결되고 디컨볼루션 레이어가 사용된다.

---

```
unet.add(concatenated_de_convolution_layer(256))
unet.add(covolution_layer(256))
unet.add(covolution_layer(256))
unet.add(concatenated_de_convolution_layer(128))
unet.add(covolution_layer(128))
unet.add(covolution_layer(128))
unet.add(concatenated_de_convolution_layer(64))
unet.add(covolution_layer(64))
unet.add(covolution_layer(64))
unet.add(concatenated_de_convolution_layer(32))
unet.add(covolution_layer(32))
unet.add(covolution_layer(32))
unet.add(covolution_layer(1, kernel=(1, 1), activation='sigmoid'))
unet.compile(optimizer=tf.keras.optimizers.Adam(lr=1e-5),
             loss=dice_coefficient_loss,
             metrics=[dice_coefficient])
```

---

다음 모델에서는 다음 코드를 사용해 이미지를 학습할 수 있다.

---

```
x_train, y_train_mask = load_train_data()
x_train = preprocess(x_train)
y_train_mask = preprocess(y_train_mask)
x_train = x_train.astype('float32')
mean = np.mean(x_train)
std = np.std(x_train)
x_train -= mean
x_train /= std
y_train_mask = y_train_mask.astype('float32')
y_train_mask /= 255.
unet.fit(x_train, y_train_mask, batch_size=32, epochs=20, verbose=1,
        shuffle=True,
        validation_split=0.2)
x_test, y_test_mask = load_test_data()
x_test = preprocess(x_test)
```

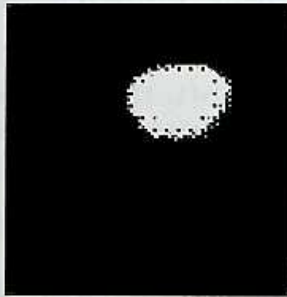
```

x_test = x_test.astype('float32')
x_test -= mean
x_test /= std
y_test_pred = unet.predict(x_test, verbose=1)
for image, image_id in zip(y_test_pred, y_test_mask):
    image = (image[:, :, 0] * 255.).astype(np.uint8)
    imsave(os.path.join(work_dir, str(image_id) + '.png'), image)

```

---

이미지는 사전 처리 후 사용할 수 있다. 지금 단계에서는 이미지 훈련 및 테스트가 가능하다. 모델을 훈련하고 나면 분할을 사용했을 때 다음과 같은 좋은 결과를 얻을 수 있다.



지금까지는 의료 영상을 분할할 수 있는 모델을 훈련했다. 이 알고리즘은 여러 사례에 사용될 수 있다. 다음 절에서는 위성 이미지를 분할하는 방법을 살펴본다.

## **■ 위성 이미지 분할**

이 절에서는 국제 학회 ISPRS(International Society for Photogrammetry and Remote Sensing)에서 제공한 데이터셋을 사용한다. 이 데이터셋에는 5cm 해상도의 독일 포츠담 위성 이미지가 포함되어 있다. 이 이미지에는 이미지의 적외선 및 높이 윤곽선에 대한 추가 데이터가 제공된다. 이 이미지와 관련된 레이블은 다음과 같이 여섯 가지다.



- 건물
- 초목
- 나무
- 택시
- 어지럽게 흩어져 있는 사물clutter
- 불투과 사물impervious

총 38개의 이미지가  $6,000 \times 6,000$  패치로 제공된다. <http://www2.isprs.org/commissions/comm3/wg4/data-request-form2.html> 페이지로 이동해 필요한 양식 내용을 채운다. 양식을 채운 후에는 양식 중에서 다음 옵션을 선택한다.

In which dataset(s) are you interested?

☐ Vaihingen
☐ Toronto
☒ Potsdam

Are you interested in the detection/reconstruction benchmark or the semantic labeling benchmark?

☐ Detection or Reconstruction
☒ Semantic Labeling
☐ both

양식을 선택하는 것이 완료되면, 데이터를 다운로드할 수 있는 이메일이 전송된다.

## 분할을 위한 FCN 모델링

라이브러리를 가져온 후에는 입력을 어떠한 모양으로 가져올지 결정할 수 있다. 레이블 수는 6으로 정의할 수 있다.

---

```

from .resnet50 import ResNet50
nb_labels = 6
img_height, img_width, _ = input_shape
input_tensor = tf.keras.layers.Input(shape=input_shape)
weights = 'imagenet'

```

---

ImageNet에서 사전 훈련된 ResNet 모델이 기본 모델로 사용된다. 다음 코드는 ResNet을 사용해 기본 모델을 정의하는 데 사용할 수 있다.

---

```
resnet50_model = ResNet50 (  
    include_top = False, weights = 'imagenet', input_tensor = input_tensor)
```

---

이제 다음 코드를 사용해 ResNet에서 마지막 세 개의 레이어를 가져온다.

---

```
final_32 = resnet50_model.get_layer('final_32').output  
final_16 = resnet50_model.get_layer('final_16').output  
final_x8 = resnet50_model.get_layer('final_x8').output
```

---

각 스킵<sup>skip</sup> 연결은 레이블 수와 동일한 채널과 일치하도록 압축되어야 한다.

---

```
c32 = tf.keras.layers.Conv2D(nb_labels, (1, 1))(final_32)  
c16 = tf.keras.layers.Conv2D(nb_labels, (1, 1))(final_16)  
c8 = tf.keras.layers.Conv2D(nb_labels, (1, 1))(final_x8)
```

---

압축된 스킵 연결의 출력은 쌍선형 보간법을 사용해 크기를 조정할 수 있다. 보간법은 텐서플로 연산을 할 수 있는 Lamda 레이어를 사용해 구현할 수 있다. 다음 코드는 Lamda 레이어를 사용한 보간에 사용할 수 있다.

---

```
def resize_bilinear(images):  
    return tf.image.resize_bilinear(images, [img_height, img_width])  
r32 = tf.keras.layers.Lambda(resize_bilinear)(c32)  
r16 = tf.keras.layers.Lambda(resize_bilinear)(c16)  
r8 = tf.keras.layers.Lambda(resize_bilinear)(c8)
```

---

여기서 정의한 세 개의 레이어에서는 다음 코드를 사용해 세 개의 값을 추가 병합할 수 있다.

---

```
m = tf.keras.layers.Add()([r32, r16, r8])
```

---

소프트맥스 활성화 방법을 사용해 모델의 정확도를 높일 수 있다. 소프트맥스를 적용하기 전과 후에 모델 크기는 조정된다.

---

```
x = tf.keras.layers.Reshape((img_height * img_width, nb_labels))(m)
x = tf.keras.layers.Activation('softmax')(x)
x = tf.keras.layers.Reshape((img_height, img_width, nb_labels))(x)
fcn_model = tf.keras.models.Model(input=input_tensor, output=x)
```

---

간단한 FCN 레이어가 정의됐으며 훈련받은 결과를 다음과 같이 확인할 수 있다.



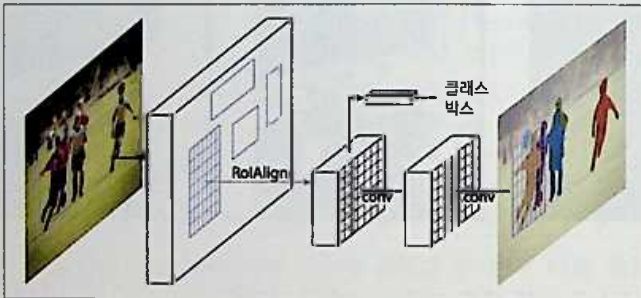
여섯 개의 레이블에 대한 예측 결과가 상당히 정확함을 알 수 있다. 이어서 인스턴스 분할을 배워보자.

## I 인스턴스 분할

이미지를 분석하는 동안 많은 관심이 이미지의 특정 인스턴스에 집중된다. 따라서 이미지의 나머지 부분에서 이러한 인스턴스를 구분해야 했다. 필요한 정보를 나머지에서 분리하는 이 프로세스는 세그먼트를 분할하는 것으로 널리 알려져 있다. 이 과정에서 입력 이미지를 먼저 가져온 다음 경계 박스가 객체와 함께 로컬라이제이션되고 마침내 픽셀 단위 마스크가 각 클래스에 대해 예측된다. 각 객체에 대해 픽셀 단위의 정확도가 계산될 수 있다.

인스턴스를 분할하기 위한 여러 가지 알고리즘이 존재하며, 최근의 알고리즘 중 하나는 He가 제안한 마스크 R-CNN 알고리즘(<https://arxiv.org/pdf/1703.06870.pdf>)이다.

다음 그림은 마스크 R-CNN의 아키텍처를 나타낸다.



히의 허락하에 재구성 게재

이 아키텍처는 분할이 추가된 R-CNN과 유사하다. 이것은 중단 간 훈련을 이용한 다단계 네트워크다. 영역을 찾는 방법은 학습을 통해 이뤄진다. 네트워크는 검출용과 분류용, 이렇게 두 가지로 분할된다. 이를 통해 다음과 같이 뛰어난 결과를 얻을 수 있다.



세그먼트 인스턴스 처리를 설명하는 그림. 객체는 정확하게 검출되고 그 결과에 따라 세그먼트화된다(히의 허락하에 재구성 게재).

동일한 네트워크를 사용해 사람들의 움직임을 예측할 수도 있다. 분할 및 검출의 두 가지 태스크는 병렬로 처리할 수 있다.

## **I 요약**

5장에서는 다양한 세그먼트 알고리즘에 대해 학습했다. 또한 벤치마킹에 사용되는 데이터셋과 측정 항목을 살펴봤다. 위성 및 의료 영상을 분할하기 위해 배운 기술을 적용했으며, 인스턴스 분할을 위해 마스크 R-CNN 알고리즘을 사용했다.

다음 장에서는 유사도 학습을 배운다. 유사도 학습 모델은 두 이미지 간의 비교 메커니즘을 사용해 학습한다. 유사도 학습 모델은 얼굴 인식과 같은 여러 애플리케이션 프로그램에 유용하다. 6장에서는 유사도를 위해 사용할 수 있는 몇 가지 모델 아키텍처를 살펴볼 것이다.







## 유사도 학습

6장에서는 유사도 학습(similarity learning)을 배우고 유사도 학습에 사용되는 다양한 손실 함수(loss function)를 학습한다. 유사도는 클래스에서 데이터셋이 작은 경우에 유용하다. 이번 장에서는 얼굴 분석에 사용할 수 있는 다양한 데이터셋을 이해하고 얼굴 인식, 표식(landmark) 검출을 위한 모델을 구축할 것이다. 지금부터 다룰 내용은 다음과 같다.

- 유사도 학습을 위한 다양한 알고리즘
- 유사도 학습에 사용되는 다양한 손실 함수
- 손실 모델을 사용할 수 있는 다양한 시나리오
- 완벽한 얼굴 인식 프로세스

## ■ 유사도 학습을 위한 알고리즘

유사도 학습은 두 엔티티(entity) 간의 유사도를 계산하기 위해 측정 항목을 학습하는 프로세스다. 여기서는 유사도가 학습되므로 측정 항목 학습이라고도 할 수 있다. 측정 항목은 유클리드(Euclidean), 코사인 또는 기타 사용자 정의 거리 함수(custom distance function)일 수 있다. 엔티티는 이미지, 동영상, 텍스트 또는 표와 같은 모든 데이터일 수 있다. 측정 항목을 계산하려면 이미지의 벡터 표현이 필요하다. 이 표현은 3장, '이미지 검색'에서 설명한 CNN에 의해 계산된 특징이 사용될 수 있다. 객체 분류를 위해 배운 CNN은 측정 항목을 계산하는 벡터로 사용될 수 있다. 이미지 분류를 위해 얻어진 특징 벡터가 현재의 작업을 가장 잘 표현한 것은 아니다. 유사도 학습에서는 유사도 학습 과제를 위해 훈련된 특징을 생성하는 CNN을 고려해야 한다. 유사도 학습이 적용될 수 있는 일부 애플리케이션 프로그램은 다음과 같다.

- 두 얼굴을 비교해 바이오메트릭스(biometrics)<sup>1</sup>를 위한 얼굴 확인하기
- 실제 제품을 시각적으로 검색해 유사한 제품을 온라인에서 찾기
- 일부 속성과 유사한 제품에 대한 시각적 추천을 하기

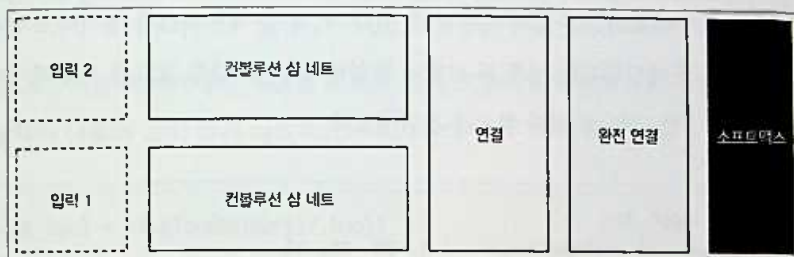
이 장에서는 얼굴 확인 방법을 자세히 설명한다. 이제 유사도 학습에 사용할 수 있는 알고리즘부터 살펴본다.

## 삼 네트워크

이름에서 알 수 있듯이 삼 네트워크(Siamese network)는 네트워크가 두 입력을 구분할 수 있도록 훈련된 신경망 모델이다. 삼 네트워크는 CNN을 훈련해 두 개의 인코더가 임베딩(embedding)(끼워 넣기)되도록 한다. 각 인코더에는 양 또는 음 쌍의 이미지 중 하나가 공급된다. 삼 네트워크는 다른 딥러닝 알고리즘보다 적은 데이터를 필요로 한다. 삼 네트워크는 원래 서

1 하나 이상의 고유한 신체적, 행동적 형질에 기반해 사람을 인식하는 방식을 두루 가리킨다. 생체 인증, 바이오 인증, 생물 측정학, 바이오 인식, 생체 인식, 생체측량 등 다양한 용어로 번역될 수 있다(<https://ko.wikipedia.org/wiki/바이오메트릭스>). - 옮긴이

명을 비교하기 위해 도입됐다. 다음 그림은 삼 네트워크를 보여주며, 삼 네트워크의 가중치는 네트워크 간에 공유된다.



삼 네트워크의 다른 용도는 원샷<sup>oneshot</sup> 학습이다. 원샷 학습은 단지 한 가지 예를 사용해 학습하는 기술이다. 이 경우에 이미지를 표시할 수 있으며, 이미지가 유사한지 여부를 알 수 있다. 대부분의 유사도 학습 과제에서는 양성 값과 음성 값이 쌍으로 훈련돼야 한다. 데이터셋이 유클리드 거리를 사용한다고 가정할 때, 모든 데이터셋은 분류 작업에 활용될 수 있다. 이전 장에서 이러한 알고리즘과 다른 알고리즘의 가장 큰 차이점은 인코더에서 특히 다른 알고리즘과 차별된다는 것이다.

## 대조 손실

대조 손실<sup>contrastive loss</sup>은 이미지의 유사도를 사용해 차별화한다. 특징 또는 잠재 레이어<sup>latent layer</sup>는 유사도 측정 항목을 사용해 비교되고, 유사도 점수에 대해 타깃과 함께 훈련된다. 양의 쌍의 경우 타깃 대상은 모두 0이므로 두 입력은 동일하다. 음의 쌍의 경우에는 코사인 거리 또는 정규화된 유클리드 거리라면 잠재 쌍 사이의 거리는 최대 0이 된다. 손실은 다음 코드에서 설명하는 `contrastive_loss`로 정의할 수 있다.

```
def contrastive_loss(model_1, model_2, label, margin=0.1):
    distance = tf.reduce_sum(tf.square(model_1 - model_2), 1)
    loss = label * tf.square(
        tf.maximum(0., margin - tf.sqrt(distance))) + (1 - label) *
    distance
```

```
loss = 0.5 * tf.reduce_mean(loss)
return loss
```

---

두 모델의 거리가 비교되고 손실이 계산될 수 있다. 이제 삼 네트워크를 정의하고 훈련을 시작할 것이다. 삼 네트워크의 경우 두 모델이 동일해야 한다. 다음 코드를 사용해 주어진 입력을 가진 간단한 CNN을 위한 함수를 정의해보자.

---

```
def get_model(input_):
    input_reshape = tf.reshape(input_, [-1, 28, 28, 1],
                                name='input_reshape')
    convolution_layer_1 = convolution_layer(input_reshape, 64)
    pooling_layer_1 = pooling_layer(convolution_layer_1)
    convolution_layer_2 = convolution_layer(pooling_layer_1, 128)
    pooling_layer_2 = pooling_layer(convolution_layer_2)
    flattened_pool = tf.reshape(pooling_layer_2, [-1, 5 * 5 * 128],
                                name='flattened_pool')
    dense_layer_bottleneck = dense_layer(flattened_pool, 1024)
    return dense_layer_bottleneck
```

---

정의된 모델은 삼 네트워크에 필요한 인코더를 정의하는 데 두 번 사용된다. 다음으로 두 모델의 자리 표시자가 정의된다. 모든 쌍에 대해 입력의 유사도도 입력으로 제공된다. 정의된 모델은 동일하다. 모델을 정의해 가중치를 공유할 수도 있다. 왼쪽과 오른쪽에 대한 두 가지 모델이 다음에 정의된다.

---

```
left_input = tf.placeholder(tf.float32, shape=[None, input_size])
right_input = tf.placeholder(tf.float32, shape=[None, input_size])
y_input = tf.placeholder(tf.float32, shape=[None, no_classes])
left_bottleneck = get_model(left_input)
right_bottleneck = get_model(right_input)
```

---

병목 현상 레이어(bottleneck layer)는 모델에서 가져와 연결된다. 이것은 유사도 학습 문제에 결정적인 요소다. 다음과 같이 여러 개의 모델을 만들 수 있으며, 마지막 레이어를 연결

할 수 있다.

---

```
dense_layer_bottleneck = tf.concat ([left_bottleneck, right_bottleneck], 1)
```

---

다음으로, 연결 레이어에서 계산된 로그와 함께 드롭아웃 레이어가 추가된다. 그리고 나면 절차가 다음과 같이 다른 네트워크와 유사하다.

---

```
dropout_bool = tf.placeholder(tf.bool)
dropout_layer = tf.layers.dropout(
    inputs=dense_layer_bottleneck,
    rate=0.4,
    training=dropout_bool
)
logits = dense_layer(dropout_layer, no_classes)
with tf.name_scope('loss'):
    softmax_cross_entropy = tf.nn.softmax_cross_entropy_with_logits(
        labels=y_input, logits=logits)
    loss_operation = tf.reduce_mean(softmax_cross_entropy, name='loss')
    tf.summary.scalar('loss', loss_operation)
with tf.name_scope('optimiser'):
    optimiser = tf.train.AdamOptimizer().minimize(loss_operation)
with tf.name_scope('accuracy'):
    with tf.name_scope('correct_prediction'):
        predictions = tf.argmax(logits, 1)
        correct_predictions = tf.equal(predictions, tf.argmax(y_input, 1))
    with tf.name_scope('accuracy'):
        accuracy_operation = tf.reduce_mean(
            tf.cast(correct_predictions, tf.float32))
    tf.summary.scalar('accuracy', accuracy_operation)
session = tf.Session()
session.run(tf.global_variables_initializer())
merged_summary_operation = tf.summary.merge_all()
train_summary_writer = tf.summary.FileWriter('/tmp/train', session.graph)
test_summary_writer = tf.summary.FileWriter('/tmp/test')
test_images, test_labels = mnist_data.test.images, mnist_data.test.labels
```

---



데이터는 다음과 같이 왼쪽 및 오른쪽 모델에 대해 별도로 공급되어야 한다.

---

```
for batch_no in range(total_batches):
    mnist_batch = mnist_data.train.next_batch(batch_size)
    train_images, train_labels = mnist_batch[0], mnist_batch[1]
    _, merged_summary = session.run([optimiser, merged_summary_operation],
                                     feed_dict={
                                         left_input: train_images,
                                         right_input: train_images,
                                         y_input: train_labels,
                                         dropout_bool: True
                                     })
    train_summary_writer.add_summary(merged_summary, batch_no)
    if batch_no % 10 == 0:
        merged_summary, _ = session.run([merged_summary_operation,
                                         accuracy_operation], feed_dict={
                                         left_input: test_images,
                                         right_input: test_images,
                                         y_input: test_labels,
                                         dropout_bool: False
                                     })
    test_summary_writer.add_summary(merged_summary, batch_no)
```

---

삼 네트워크를 정의하는 방법을 살펴보았다. 두 개의 인코더가 정의됐고, 잠재 공간이 언제든 훈련상에서의 손실이 발생할 수 있다. 왼쪽 및 오른쪽 모델에는 데이터가 별도로 제공된다. 다음으로 하나의 네트워크 내에서 유사도 학습이 어떻게 수행될 수 있는지 살펴본다.

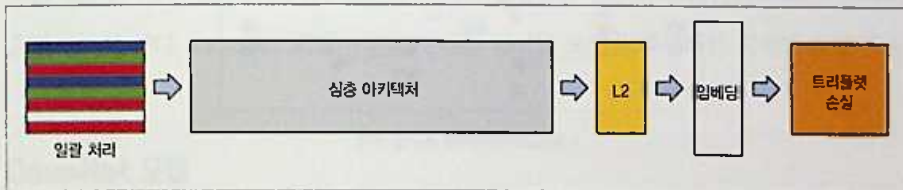
## FaceNet

슈뢰프<sup>Schroff</sup>가 제안한 FaceNet 모델(<https://arxiv.org/pdf/1503.03832.pdf>)은 얼굴 확인 문제를 해결한다. 하나의 심층 CNN을 학습한 후 얼굴 이미지를 임베딩하기 위해 변환한다. 얼굴 이미지 임베딩 작업은 얼굴을 비교해 얼마나 유사한지 확인하고자 사용하며, 다음 세 가지 방법을 이용할 수 있다.



- 얼굴 확인(face verification)은 두 얼굴을 고려해 유사 여부를 결정한다. 얼굴 확인은 거리 측정 항목을 계산해 수행할 수 있다.
- 얼굴 인식(face recognition)은 얼굴에 이름을 붙이기 위한 분류를 수행한다. 임베딩 벡터는 최종 레이블을 학습하는 데 사용할 수 있다.
- 얼굴 클러스터링(face clustering)은 사진 애플리케이션 프로그램에서 같은 사람의 사진을 함께 클러스터링하는 방법과 같이 비슷한 얼굴을 그룹화한다. K-평균(K-means)과 같은 클러스터링 알고리즘은 얼굴을 그룹화하는 데 사용된다.

다음 그림은 FaceNet 아키텍처를 보여준다.



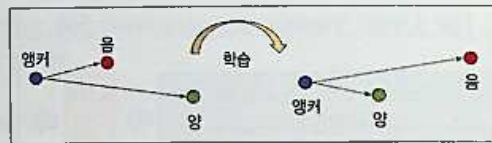
슈뢰프의 허락하에 재구성 게재

FaceNet은 일괄 처리를 위해 얼굴 이미지를 가져와서 훈련한다. FaceNet이 사용하는 일괄 처리 방법에는 몇 가지 양수 쌍이 있다. 손실을 계산하는 동안 양의 쌍과 가장 가까운 소수의 음의 쌍이 고려된다. 마이닝 선택 쌍(mining selective pair)은 원활한 훈련을 가능하게 한다. 항상 모든 음의 값이 없어지면, 훈련은 제대로 되지 않는다. 세 가지 데이터 포인트를 비교하는 것을 트리플렛 손실(triplet loss)이라 한다. 이미지는 손실 계산이 이뤄지는 동안에는 양수 및 음수 값이 일치되는 것으로 간주된다. 음의 값을 가지면 일정한 마진 차이가 발생함을 나타낸다. 트리플렛 손실은 다음 절에서 자세히 설명한다.

## 트리플렛 손실

트리플렛 손실은 이미지의 점수 벡터를 학습한다. 얼굴 기술자(face descriptor)의 점수 벡터를 사용해 유클리드 공간의 얼굴을 확인할 수 있다. 트리플렛 손실은 입력을 구별할 수 있도록 투영(projection) 방법을 학습하는 의미에서 미터법 학습과 유사하다. 이러한 투영, 기술자

또는 점수 벡터는 압축된 표현이므로 차원 감소 기술로 볼 수 있다. 트리플렛은 앵커, 양의 얼굴(positive face) 및 음의 얼굴(negative face)로 구성된다. 앵커는 어떤 얼굴이든 해당될 수 있으며, 양의 얼굴은 같은 사람의 이미지를 의미한다. 음의 이미지는 다른 사람이 올 수 있음을 나타낸다. 얼굴의 앵커에서 음의 값을 가지는 데이터가 많은 경우는 충분히 예상할 수 있다. 현재 앵커에 대해 음의 얼굴을 선택하면 인코더가 얼굴을 구별하기 어려워지므로 더 잘 학습할 수 있다. 이러한 과정을 최대한 가까운 음의 마이닝(hard negative mining)이라 한다. 더 음에 가까운 경우는 유클리드 공간에서 임계값으로 얻을 수 있다. 다음 그림은 트리플렛 손실 모델을 묘사한다.



슈뢰프의 허락하에 재구성 게재

텐서플로의 손실 계산은 다음과 같다.

---

```
def triplet_loss(anchor_face, positive_face, negative_face, margin):
    def get_distance(x, y):
        return tf.reduce_sum(tf.square(tf.subtract(x, y)), 1)
    positive_distance = get_distance(anchor_face, positive_face)
    negative_distance = get_distance(anchor_face, negative_face)
    total_distance = tf.add(tf.subtract(positive_distance,
negative_distance), margin)
    return tf.reduce_mean(tf.maximum(total_distance, 0.0), 0)
```

---

트리플렛 마이닝은 어려운 작업이다. 적절한 지점과 양의 쌍을 얻으려면 모든 지점을 다른 지점과 비교해야 한다. 트리플렛 마이닝은 다음과 같다.

---

```
def mine_triplets(anchor, targets, negative_samples):
    distances = cdist(anchor, targets, 'cosine')
    distances = cdist(anchor, targets, 'cosine').tolist()
```

---

```

QnQ_duplicated = [
    [target_index for target_index, dist in enumerate(QnQ_dist) if dist == QnQ_
dist[query_index]]
    for query_index, QnQ_dist in enumerate(distances)]
for i, QnT_dist in enumerate(QnT_dists):
    for j in QnQ_duplicated[i]:
        QnT_dist.itemset(j, np.inf)
QnT_dists_topk = QnT_dists.argsort(axis=1)[: , :negative_samples]
top_k_index = np.array([np.insert(QnT_dist, 0, i) for i, QnT_dist in
enumerate(QnT_dists_topk)])
return top_k_index

```

---

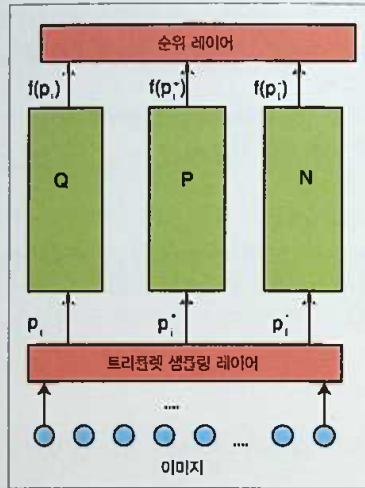
거리 계산은 CPU에서 이뤄지기 때문에 마이닝은 보조를 맞추기 위해 GPU에서는 천천히 훈련시켜야 한다. FaceNet 모델은 얼굴에 대한 유사도 모델을 훈련하는 최첨단 방법이다.

## DeepNet 모델

DeepNet 모델은 FaceNet과 같은 얼굴 확인 작업을 위한 얼굴 임베딩 동작을 학습하는 데 사용된다. 이것은 앞 절에서 논의된 FaceNet 방법의 결과 값을 향상시킨다. 동일한 얼굴 여러 개를 가져와서 더 나은 임베딩을 하기 위해 여러 인코더를 통과시킨다. 이는 FaceNet보다 더 나은 정확도를 달성하지만 많은 처리 시간이 소요된다. 잘라낸 얼굴은 동일 영역으로 만들어지고 각각의 인코더를 통과한다. 그런 다음 모든 레이어가 연결돼 트리플렛 손실을 방지한다.

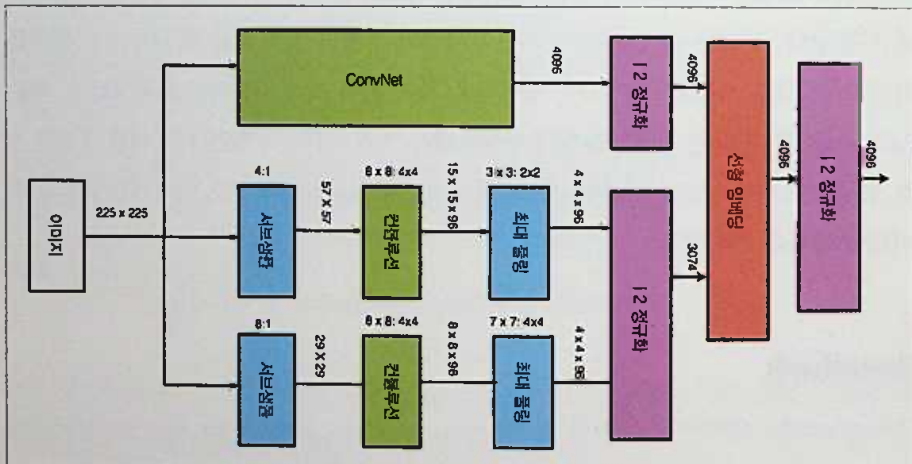
## DeepRank

DeepRank는 왕<sup>Wang</sup>이 제안한 방법([https://users.eecs.northwestern.edu/~jwa368/pdfs/deep\\_ranking.pdf](https://users.eecs.northwestern.edu/~jwa368/pdfs/deep_ranking.pdf))이며, 유사도를 기반으로 이미지의 순위를 매기는 데 사용된다. 이미지는 다음과 같이 서로 다른 모델을 통과한다.



왕의 허락하에 재구성 계재

여기서 트리플렛 손실이 계산되고 역전파가 더 원활하게 수행된다. 다음과 같이 이미지는 순위 지정이라는 목적을 위해 선형 임베딩으로 변환할 수 있다.

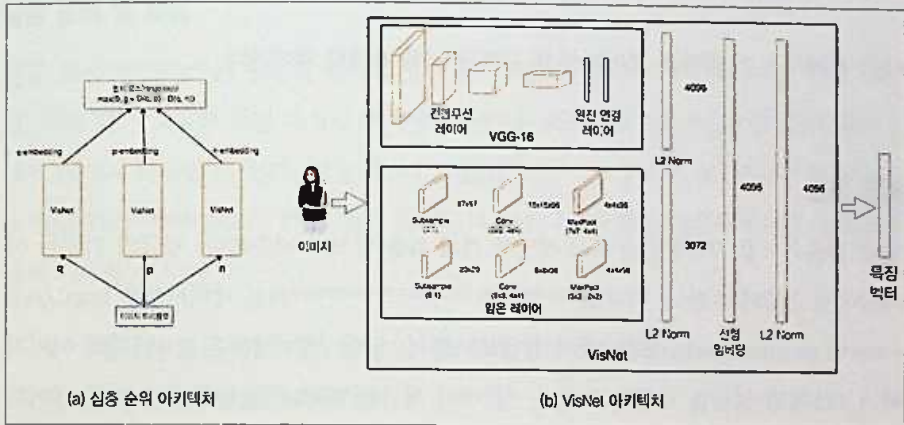


왕의 허락하에 재구성 계재

이 알고리즘은 순위 지정에 매우 유용하다.

## 시각적 추천 시스템

시각적 추천 시스템은 주어진 이미지에 대한 추천 사항을 얻는 데 적합한다. 추천 모델은 유사한 프로퍼티를 가진 이미지를 제공한다. 산카르Shankar가 제안한 다음 모델(<https://arxiv.org/pdf/1703.02344.pdf>)에서 비슷한 이미지가 포함됐는지 학습할 수 있으며, 다음과 같은 추천 사항들도 제공한다.



산카르의 허락하에 재구성 게재

위 그림은 유사도 학습에 사용되는 다양한 알고리즘을 보여준다. 다음 절에서는 이러한 기술을 얼굴에 적용하기 위한 방법을 살펴본다.

## ■ 인간 얼굴 분석

인간의 얼굴은 컴퓨터 비전을 사용해 여러 가지 방법으로 분석할 수 있으며, 여기서 나열된 몇 가지 요인을 고려해야 한다.

- **얼굴 검출:** 얼굴 위치의 경계 박스 찾기
- **얼굴 표식 검출:** 코, 입 등과 같은 얼굴 특징을 이루는 공간적 포인트 찾기



- 얼굴 정렬: 추가 분석을 위해 얼굴을 정면 얼굴로 변형
- 속성 인식: 성별, 웃음 등의 속성 찾기
- 감정 분석: 사람의 감정 분석
- 얼굴 인증: 두 이미지가 같은 사람의 것인지 확인
- 얼굴 인식: 얼굴의 신원 찾기<sup>identify</sup>
- 얼굴 클러스터링: 같은 사람의 얼굴을 함께 그룹화

다음 절에서는 이 작업의 데이터셋 및 구현에 대해 자세히 알아본다.

## 얼굴 검출

얼굴 검출은 4장, '객체 검출'에서 설명한 객체 검출 방식과 비슷하다. 얼굴의 위치는 이 이미지에서 검출돼야 한다. FDDB<sup>Face Detection Data Set and Benchmark</sup>라는 데이터셋은 <http://www.cs.umass.edu/fddb/>에서 다운로드할 수 있다. 데이터셋은 2,845개의 이미지와 5,171개의 얼굴을 포함하고 있다. 양<sup>Yang</sup>이 제안한 Wider face라는 또 다른 데이터셋은 <http://mmlab.ie.cuhk.edu.hk/projects/WIDERFace/>에서 다운로드할 수 있다. Wider face는 393,703개 얼굴을 가진 32,203개의 이미지를 가지고 있다. 다음은 Wider face 데이터셋의 이미지 샘플이다.



양(Yang)에 의해 제안되었으며, <http://mmlab.ie.cuhk.edu.hk/projects/WIDERFace/support/intro.jpg>를 통한 샘플 이미지다.



데이터셋에는 크기, 포즈, 폐색(오clusion), 표현, 메이크업, 조명의 변형이 있다. MALF<sup>Multi-Attribute Labeled Faces</sup>라는 다른 데이터셋에는 5,250개의 이미지와 11,931개의 얼굴이 있다. MALF는 <http://www.cbsr.ia.ac.cn/faceevaluation/> 링크에서 액세스할 수 있다. 객체 검출에 사용된 것과 동일한 기법을 얼굴 검출에 적용할 수 있다.

## 얼굴 표식 및 속성

얼굴 표식<sup>face landmark</sup>은 인간의 얼굴에 있는 공간적인 포인트다. 공간 점은 눈, 눈썹, 코, 입, 턱과 같은 다양한 안면 특징의 위치에 해당한다. 포인트의 수는 애노테이션에 따라 5에서 78까지 다양할 수 있다. 얼굴 표식은 기준점<sup>fiducial point</sup>, 얼굴 키 포인트<sup>facial key point</sup> 또는 얼굴 포즈<sup>face pose</sup>라고도 한다. 얼굴 표식은 다음에 나열된 많은 애플리케이션 프로그램에서 사용할 수 있다.

- 더 나은 얼굴 확인 또는 얼굴 인식을 위해 얼굴 정렬하기
- 동영상에서 얼굴 추적하기
- 얼굴 표정이나 감정 측정하기
- 건강 상태 진단에 도움 주기

지금부터는 기준점에 대한 애노테이션이 있는 데이터베이스를 살펴본다.

## MTFL 데이터셋

MTFL<sup>Multi-Task Facial Landmark</sup> 데이터셋은 장<sup>Zhang</sup>에 의해 제안됐으며 성별, 웃는 얼굴, 안경, 머리 포즈 애노테이션을 가지는 얼굴의 다섯 개 표식을 사용해 애노테이션 처리한다. 데이터베이스에는 12,995개의 얼굴이 있다. MTFL은 <http://mmlab.ie.cuhk.edu.hk/projects/TCDCN/data/MTFL.zip>에서 다운로드할 수 있다.

다음은 MTFE에 있는 이미지 샘플이다.



장에 의해 제한되었으며, <http://mmlab.ie.cuhk.edu.hk/projects/TCDCN/img/1.jpg>를 통한 샘플 이미지다.

나이, 조명, 감정 등과 관련해 얼굴에는 많은 변화가 있다. 머리 포즈는 도 단위로 표시된 얼굴 방향의 각도다. 미소 짓는 안경, 성별 속성 등은 이진 레이블로 애노테이션 처리된다.

## 캐글 키포인트 데이터셋

캐글<sup>Kaggle</sup> 키포인트 데이터셋에서는 15개의 얼굴 표식을 애노테이션 처리한다. 이 데이터셋에는 8,832개의 이미지가 있으며, <https://www.kaggle.com/c/facial-keypoints-detection/data> 링크에서 다운로드할 수 있다. 이미지는 96×96픽셀 크기를 가진다.

## MAFL 데이터셋

장이 제한한 MAFL<sup>Multi-Attribute Facial Landmark</sup> 데이터셋이다. 40가지 얼굴 속성을 가지는 다섯 개의 얼굴 표식을 애노테이션 처리하며, 데이터베이스에는 20,000개의 얼굴이 있다. MAFL은 <https://github.com/zhzhanp/TCDCN-face-alignment>에서 다운로드할 수

있다. 다음은 샘플 이미지들이다.



리우(Liu)에 의해 제안되었으며, <http://mmlab.ie.cuhk.edu.hk/projects/celeba/overview.png>를 통한 샘플 이미지다.

에노테이션의 속성에는 뾰족한 코, 콧수염, 물결 모양의 머리카락, 모자 착용 등이 포함된다. 이 이미지는 CelebA 데이터셋에도 포함돼 있다. 자세한 내용은 이후 설명한다.

## 얼굴 키 포인트 학습

이전에 설명한 것처럼 얼굴의 키 포인트를 계산하는 동안 정의해야 하는 몇 가지 매개변수가 있다. 다음의 코드를 사용해 이러한 매개변수를 정의한다.

```
image_size = 40
no_landmark = 10
no_gender_classes = 2
no_smile_classes = 2
no_glasses_classes = 2
```

```
no_headpose_classes = 5
batch_size = 100
total_batches = 300
```

---

다음으로는 다양한 입력에 대해 몇 가지 자리 표시자를 허용한다.

```
image_input = tf.placeholder(tf.float32, shape=[None, image_size, image_size])
landmark_input = tf.placeholder(tf.float32, shape=[None, no_landmark])
gender_input = tf.placeholder(tf.float32, shape=[None, no_gender_classes])
smile_input = tf.placeholder(tf.float32, shape=[None, no_smile_classes])
glasses_input = tf.placeholder(tf.float32, shape=[None, no_glasses_classes])
headpose_input = tf.placeholder(tf.float32, shape=[None, no_headpose_classes])
```

---

다음 코드에서 볼 수 있듯이 네 개의 컨볼루션 레이어를 사용해 기본 모델을 구성한다.

```
image_input_reshape = tf.reshape(image_input, [-1, image_size, image_size, 1],
name='input_reshape')
convolution_layer_1 = convolution_layer(image_input_reshape, 16)
pooling_layer_1 = pooling_layer(convolution_layer_1)
convolution_layer_2 = convolution_layer(pooling_layer_1, 48)
pooling_layer_2 = pooling_layer(convolution_layer_2)
convolution_layer_3 = convolution_layer(pooling_layer_2, 64)
pooling_layer_3 = pooling_layer(convolution_layer_3)
convolution_layer_4 = convolution_layer(pooling_layer_3, 64)
flattened_pool = tf.reshape(convolution_layer_4, [-1, 5 * 5 * 64],
name='flattened_pool')
dense_layer_bottleneck = dense_layer(flattened_pool, 1024)
dropout_bool = tf.placeholder(tf.bool)
dropout_layer = tf.layers.dropout(
    inputs=dense_layer_bottleneck,
    rate=0.4,
    training=dropout_bool
)
```

---

다음 코드를 사용해 모든 다른 작업에 대한 로짓<sup>logit</sup><sup>2</sup> 분기를 만든다.

---

```
landmark_logits = dense_layer(dropout_layer, 10)
smile_logits = dense_layer(dropout_layer, 2)
glass_logits = dense_layer(dropout_layer, 2)
gender_logits = dense_layer(dropout_layer, 2)
headpose_logits = dense_layer(dropout_layer, 5)
```

---

손실은 다음 코드와 같이 모든 얼굴 특징에 대해 개별적으로 계산된다.

---

```
landmark_loss = 0.5 * tf.reduce_mean (
    tf.square (landmark_input, landmark_logits))
gender_loss = tf.reduce_mean (
    tf.nn.softmax_cross_entropy_with_logits (
        labels = gender_input, logits = gender_logits))
smile_loss = tf.reduce_mean (
    tf.nn.softmax_cross_entropy_with_logits (
        labels = smile_input, logits = smile_logits))
glass_loss = tf.reduce_mean (
    tf.nn.softmax_cross_entropy_with_logits (
        labels = glasses_input, logits = glass_logits))
headpose_loss = tf.reduce_mean (
    tf.nn.softmax_cross_entropy_with_logits (
        labels = headpose_input, logits = headpose_logits))
loss_operation = landmark_loss + gender_loss + \
    smile_loss + glass_loss + headpose_loss
```

---

이제 다음 코드와 같이 최적화 프로그램을 초기화하고 훈련을 시작한다.

---

```
optimiser = tf.train.AdamOptimizer().minimize(loss_operation)
session = tf.Session()
session.run(tf.initialize_all_variables())
```

---

2 로지스틱 모형식을 활용함을 의미한다. - 옮김이



```

fiducial_test_data = fiducial_data.test
for batch_no in range(total_batches):
    fiducial_data_batch = fiducial_data.train.next_batch(batch_size)
    loss, _landmark_loss, _ = session.run(
        [loss_operation, landmark_loss, optimiser],
        feed_dict={
            image_input: fiducial_data_batch.images,
            landmark_input: fiducial_data_batch.landmarks,
            gender_input: fiducial_data_batch.gender,
            smile_input: fiducial_data_batch.smile,
            glasses_input: fiducial_data_batch.glasses,
            headpose_input: fiducial_data_batch.pose,
            dropout_bool: True
        })
    if batch_no % 10 == 0:
        loss, _landmark_loss, _ = session.run(
            [loss_operation, landmark_loss],
            feed_dict={
                image_input: fiducial_test_data.images,
                landmark_input: fiducial_test_data.landmarks,
                gender_input: fiducial_test_data.gender,
                smile_input: fiducial_test_data.smile,
                glasses_input: fiducial_test_data.glasses,
                headpose_input: fiducial_test_data.pose,
                dropout_bool: False
            })

```



이 프로세스는 표식뿐만 아니라 얼굴 특징을 검출하는 데 사용될 수 있다.

## 얼굴 인식

얼굴 인식(face recognition)은 디지털 이미지 또는 동영상에서 인물을 식별하는 과정이다. 다음 절에서 얼굴 인식에 사용할 수 있는 데이터셋을 알아본다.



## 와일드 데이터셋의 레이블이 부여된 얼굴

와일드<sup>wild</sup>(LFW) 데이터셋은 5,749명의 고유 사용자를 가진 13,233명의 얼굴을 포함하며, 얼굴 확인 데이터셋을 평가할 수 있는 표준 데이터셋이다. 정확도 측정 항목을 사용하면 알고리즘을 평가할 수 있다. 데이터셋은 <http://vis-www.cs.umass.edu/lfw/> 링크에서 액세스할 수 있다.

## 유튜브의 얼굴 데이터셋

유튜브<sup>YouTube</sup>의 얼굴 데이터셋에는 1,475개의 동영상 클립과 1,595명의 고유한 사람들이 있다. 동영상은 유튜브에서 수집되며, 데이터셋에는 1인당 최소 두 개의 동영상이 있다. 이 데이터셋은 동영상 내 얼굴 확인을 위한 표준 데이터셋이다. 데이터셋은 <https://www.cs.tau.ac.il/~wolf/ytfaces/> 링크에서 액세스할 수 있다.

## CelebA

CelebA<sup>CelebFaces Attributes dataset</sup> 데이터셋에는 다섯 개의 얼굴 표식, 40개의 속성과 함께 사람의 신원이 표시된다. 데이터베이스에는 202,599명의 얼굴 이미지를 가진 10,177명의 독특한 사람들이 있다. 얼굴 확인, 검출, 표식, 속성 인식 문제에 사용할 수 있는 대규모 데이터 집합 중 하나다. 이미지에는 다양한 애노테이션이 포함된 다양한 얼굴 모양이 있다. 데이터셋은 <http://mmlab.ie.cuhk.edu.hk/projects/CelebA.html> 링크에서 액세스할 수 있다.

## CASIA 웹 얼굴 데이터베이스

CASIA 데이터셋에는 총 494,414개 이미지에 대한 10,575명의 고유한 사람들이 있으며, 애노테이션이 달려 있다. 데이터셋은 <http://www.cbbsr.ia.ac.cn/english/CASIA-WebFace-Database.html>에서 얻을 수 있다. 이것은 얼굴 확인 및 인식 문제에 사용할 수 있는 두 번째로 큰 공용 데이터셋이다.

## VGGFace2 데이터셋

카오<sup>cao</sup>가 제안한 VGGFace2 데이터셋은 331만 개의 이미지에서 9,131명의 고유 사용자에게 애노테이션이 달려 있다. 이 데이터셋은 [http://www.robots.ox.ac.uk/~vgg/data/vgg\\_face2/](http://www.robots.ox.ac.uk/~vgg/data/vgg_face2/)에서 얻을 수 있으며 다양성에는 나이, 인종, 포즈, 직업, 조명이 포함된다. 이것은 얼굴 확인에 사용할 수 있는 가장 큰 데이터셋이다.

다음은 데이터셋에 있는 이미지 샘플이다.



카오에 의해 제안했으며, [http://www.robots.ox.ac.uk/~vgg/data/vgg\\_face2/web\\_page\\_img.png](http://www.robots.ox.ac.uk/~vgg/data/vgg_face2/web_page_img.png)를 통한 샘플 이미지다.

고유한 사람당 최소, 평균, 최대 이미지 수는 각각 87, 362.6, 843이다.

## 얼굴 간 유사도 계산하기

얼굴 유사도 계산은 다단계 문제를 가진다. 우선 얼굴을 검출하고 얼굴에 대한 기준점을 찾는다. 얼굴은 기준점과 정렬될 수 있다. 정렬된 얼굴을 비교에 사용할 수 있다. 이전에 언급했듯이 얼굴 인식은 객체 검출과 비슷하다. 따라서 얼굴 간의 유사점을 찾기 위해 먼저 다음 라이브러리를 사용해 필요한 라이브러리와 facenet 라이브러리를 가져온다.

---

```
from scipy import misc
import tensorflow as tf
import numpy as np
import os
import facenet
print facenet
from facenet import load_model, prewhiten
import align.detect_face
```

---

다음과 같이 이미지를 로드하고 정렬할 수 있다.

---

```
def load_and_align_data(image_paths, image_size=160, margin=44,
    gpu_memory_fraction=1.0):
    minsize = 20
    threshold = [0.6, 0.7, 0.7]
    factor = 0.709
    print('Creating networks and loading parameters')
    with tf.Graph().as_default():
        gpu_options =
            tf.GPUOptions(per_process_gpu_memory_fraction=gpu_memory_fraction)
        sess = tf.Session(config=tf.ConfigProto(gpu_options=gpu_options, log_device_
            placement=False))
        with sess.as_default():
            pnet, rnet, onet = align.detect_face.create_mtcnn(sess, None)
        nrof_samples = len(image_paths)
        img_list = [None] * nrof_samples
        for i in range(nrof_samples):
            img = misc.imread(os.path.expanduser(image_paths[i]), mode='RGB')
            img_size = np.asarray(img.shape)[0:2]
            bounding_boxes, _ = align.detect_face.detect_face(img, minsize, pnet, rnet,
            onet, threshold, factor)
            det = np.squeeze(bounding_boxes[0, 0:4])
            bb = np.zeros(4, dtype=np.int32)
            bb[0] = np.maximum(det[0] - margin / 2, 0)
            bb[1] = np.maximum(det[1] - margin / 2, 0)
            bb[2] = np.minimum(det[2] + margin / 2, img_size[1])
            bb[3] = np.minimum(det[3] + margin / 2, img_size[0])
```

```

cropped = img[bb[1]:bb[3], bb[0]:bb[2], :]
aligned = misc.imresize(cropped, (image_size, image_size), interp='bilinear')
prewhitened = prewhiten(aligned)
img_list[i] = prewhitened
images = np.stack(img_list)
return images

```

---

이제 임베딩을 수행하기 위해 이미지 경로를 처리할 것이다. 이에 대한 코드는 다음과 같다.

```

def get_face_embeddings(image_paths, model='/20170512-110547/'):
    images = load_and_align_data(image_paths)
    with tf.Graph().as_default():
        with tf.Session() as sess:
            load_model(model)
            images_placeholder = tf.get_default_graph().get_tensor_by_name("input:0")
            embeddings = tf.get_default_graph().get_tensor_by_name("embeddings:0")
            phase_train_placeholder = tf.get_default_graph().get_tensor_by_name("phase_train:0")
            feed_dict = {images_placeholder: images, phase_train_placeholder: False}
            emb = sess.run(embeddings, feed_dict=feed_dict)
    return emb

```

---

이제 다음 코드를 사용해 임베딩 간의 거리를 계산한다.

```

def compute_distance(embedding_1, embedding_2):
    dist = np.sqrt(np.sum(np.square(np.subtract(embedding_1, embedding_2))))
    return dist

```

---

이 함수는 임베딩 간의 유클리드 거리를 계산한다.

### 최적의 임곗값 찾기

앞의 함수를 사용해 이 시스템의 정확도를 계산할 수 있다. 다음 코드는 최적 임곗값을 계산하는 데 사용할 수 있다.

---

```
import sys
import argparse
import os
import re
from sklearn.metrics import classification_report
from sklearn.metrics import accuracy_score
```

---

이제 이미지 경로는 다음 코드를 사용해 폴더에서 가져온다.

---

```
def get_image_paths(image_directory):
    image_names = sorted(os.listdir(image_directory))
    image_paths = [os.path.join(image_directory, image_name) for image_name in
image_names]
    return image_paths
```

---

이미지의 거리는 다음 코드와 같이 임베딩을 통해 얻어진다.

---

```
def get_labels_distances(image_paths, embeddings):
    target_labels, distances = [], []
    for image_path_1, embedding_1 in zip(image_paths, embeddings):
        for image_path_2, embedding_2 in zip(image_paths, embeddings):
            if (re.sub(r'\d+', '', image_path_1)).lower() ==
(re.sub(r'\d+', '', image_path_2)).lower():
                target_labels.append(1)
            else:
                target_labels.append(0)
            distances.append(compute_distance(embedding_1, embedding_2)) # 여기서 거리 측정
항목 수행
    return target_labels, distances
```

---

임꺽값은 다음 코드와 같이 다양하며, 이에 따라 다양한 측정 항목이 출력된다.



---

```
def print_metrics(target_labels, distances):
    accuracies = []
    for threshold in range(50, 150, 1):
        threshold = threshold/100.
        predicted_labels = [1 if dist <= threshold else 0 for dist in distances]
        print("Threshold", threshold)
        print(classification_report(target_labels, predicted_labels, target_
names=['Different', 'Same']))
        accuracy = accuracy_score(target_labels, predicted_labels)
        print('Accuracy: ', accuracy)
        accuracies.append(accuracy)
    print(max(accuracies))
```

---

이미지 경로는 다음 코드를 사용해 임베딩 함수로 전달된다.

---

```
def main(args):
    image_paths = get_image_paths(args.image_directory)
    embeddings = get_face_embeddings(image_paths) #여기서 임베딩 계산 수행
    target_labels, distances = get_labels_distances(image_paths, embeddings)
    print_metrics(target_labels, distances)
```

---

마지막으로 이미지 디렉터리는 다음 코드에서 보는 것처럼 주요 인자로 메소드에 전달 된다.

---

```
if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('image_directory', type=str, help='Directory containing the
images to be compared')
    parsed_arguments = parser.parse_args(sys.argv[1:])
    main(parsed_arguments)
```

---



## 얼굴 클러스터링

얼굴 클러스터링은 같은 사람의 이미지를 앨범별로 그룹화하는 과정이다. 얼굴의 임베딩을 검출할 수 있으며 K-평균과 같은 클러스터링 알고리즘을 사용해 같은 사람의 얼굴을 모을 수 있다. 텐서플로는 K-평균 알고리즘을 위한 `tf.contrib.learn.KmeansClustering`이라는 API를 제공한다. K-평균 알고리즘은 데이터 포인트를 함께 그룹화한다. 이 K-평균 알고리즘의 도움으로 앨범의 삽입을 검출할 수 있고 개인의 얼굴을 함께 발견할 수 있다. 즉, 함께 구분 지어 모을 수 있다.

## I 요약

이 장에서는 유사도 학습의 기본 사항을 다뤘다. 측정 항목 학습, 삼 네트워크, FaceNet과 같은 알고리즘을 살펴보았다. 또한 대조 손실과 트리플렛 손실 같은 손실 함수도 다뤘다. 순위와 추천이라는 두 가지 영역도 다뤘다. 마지막으로 얼굴 인식에 대한 단계별 설명은 검출, 기준점 검출, 유사도 평가와 같은 여러 단계를 이해함으로써 수행된다.

다음 장에서는 반복적인 신경 회로망과 자연어 처리 문제에서의 사용을 다룰 것이다. 이어서 이미지 캡션을 위한 이미지 모델과 결합된 언어 모델을 사용할 것이다. 이 문제에 대한 여러 알고리즘을 소개하고, 두 가지 유형의 데이터 구현을 살펴본다.



## 이미지 캡션 처리

이 장에서는 이미지 캡션을 처리할 때의 문제점을 다룬다. 이 작업에서는 객체를 검색하고 이미지의 텍스트 캡션이 표시된다. 이미지 캡션은 이미지-텍스트 번역이라고도 한다. 이미지 캡션은 처리하기 어려운 작업이지만, 이번 캡션 처리 단계에서는 좋은 결과를 얻을 수 있을 것이다. 이 장의 경우 캡션이 있는 이미지의 데이터셋이 필요하다. 이 장에서는 이미지 캡션 기술과 애플리케이션 프로그램을 자세히 설명한다.

이 장에서는 다음 내용을 다룬다.

- 평가에 사용된 다양한 데이터셋 및 측정 항목의 이해
- 자연어 처리 문제에 사용되는 몇 가지 기술 이해
- 벡터 모델에 대한 다른 단어
- 이미지 캡션을 위한 몇 가지 알고리즘

- 불리한 결과와 개선 범위

## ■ 문제 및 데이터셋 이해하기

이미지 캡션을 자동으로 생성하는 과정은 언어와 비전, 이 두 가지가 결합된 중요한 학습 과제다. 이 문제가 가지고 있는 고유성 때문에 컴퓨터 비전에서 해결해야 할 주요 문제 중 하나가 된다. 이미지 캡션을 만들기 위한 딥러닝 모델을 통해 이미지에 있는 객체를 식별할 수 있어야 하며, 객체와 동작(action) 간의 관계를 표현하는 자연어로 텍스트를 생성할 수 있어야 한다. 이 문제를 해결하기 위해 사용할 데이터 집합은 많지 않다. 가장 유명한 데이터셋은 4장, '객체 검출'의 객체 검출에서 다루는 COCO 데이터셋의 확장 버전이다.

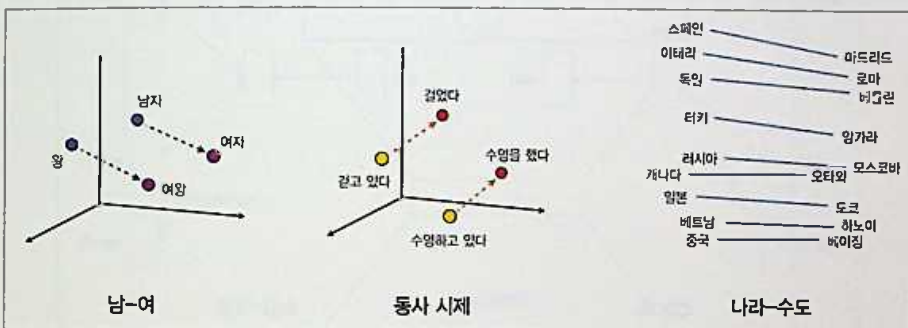
## ■ 이미지 캡션을 위한 자연어 처리 이해

이미지에서 자연어가 생성돼야 하므로 자연어 처리(NLP, Natural Language Processing)에 익숙해지는 것이 중요하다. NLP의 개념은 사실상 광범위한 주제이므로 이미지 캡션과 관련된 주제로 범위를 제한할 것이다. 자연어의 한 형태는 텍스트다. 텍스트는 일련의 단어 또는 문자며, 텍스트의 원자적 요소는 토큰이다. 토큰은 일련의 문자며, 문자는 텍스트의 원자적 요소다.

자연어를 텍스트 형식으로 처리하려면 구두점, 대괄호 등을 제거하는 텍스트 사전 처리 작업이 필요하다. 이어서 텍스트를 공백으로 분리해 단어로 토큰화해야 한다. 그리고 나서 단어를 벡터로 변환한다. 지금부터는 이 벡터 변환이 어떻게 도움이 되는지 살펴본다.

## 벡터 형태로 단어 표현하기

벡터 형식으로 표현된 단어는 자체적으로 산술 연산을 수행하는 데 도움이 될 수 있다. 벡터는 크기가 작고 컴팩트해야 한다. 동의어에는 유사한 벡터가 있어야 하고 반의어에는 다른 벡터가 있어야 한다. 단어를 벡터로 변환하면 다음과 같이 관계를 비교할 수 있다.



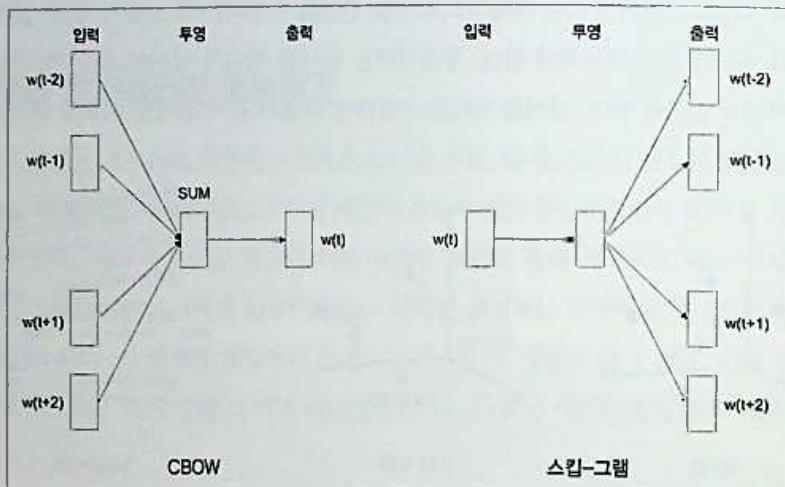
이 벡터 산술은 서로 다른 객체 사이의 의미 공간에서 비교할 수 있게 해준다. 이어서 단어를 벡터 표현으로 변환할 수 있는 딥러닝 모델을 만드는 방법을 살펴본다.

## 단어를 벡터로 변환

대형 텍스트 코퍼스(corpus)에서 모델을 학습해 단어를 벡터로 변환할 수 있다. 모델은 단어가 주어지면 근처 유사 단어(nearby word)를 예측할 수 있도록 훈련된다. 유사 단어를 사용해 원-핫 인코딩으로 예측하기 전에 숨겨진 레이어의 단어들이 먼저 원-핫 인코딩된다. 이런 식으로 훈련하면 단어를 간접하게 표현할 수 있다. 단어의 컨텍스트는 다음과 같이 두 가지 방법으로 얻을 수 있다.

- 스킵-그램(skip-gram): 하나의 단어가 주어지면 가까이 있는 단어 몇 개를 예측하려고 시도한다.
- 연속적 단어 가방(CBOW, Continuous Bag Of Words): 주어진 단어 그룹을 예측하는 스킵-그램

다음 그림은 이러한 동작 과정을 보여준다.

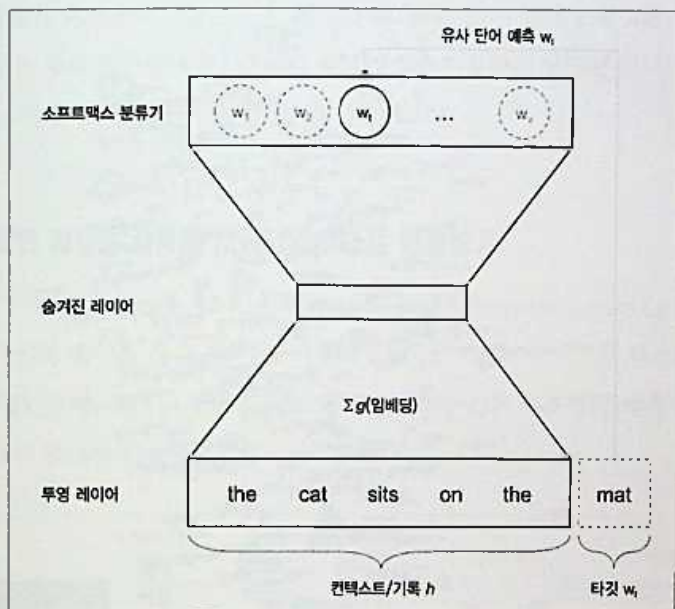


두 방법 모두 좋은 결과를 보여준다. 단어는 임베딩 공간에서 벡터로 변환된다. 다음으로 임베딩 공간을 훈련하기 위한 세부 사항을 볼 것이다.

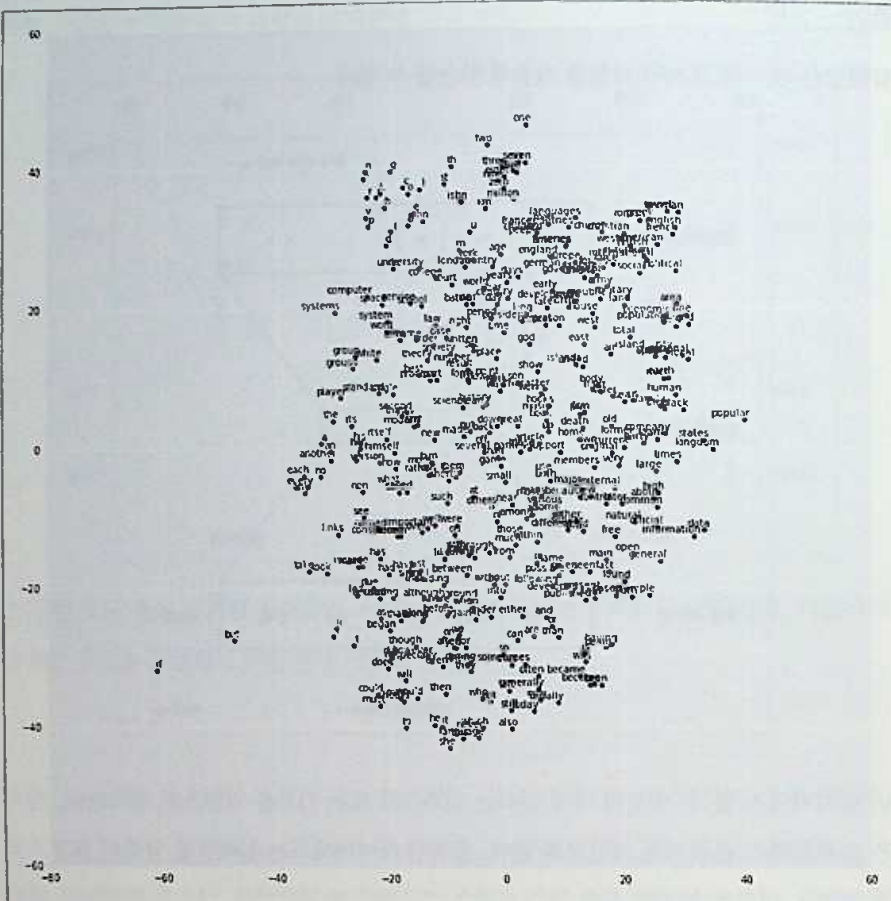


## 임베딩 훈련

임베딩은 여기에 표시된 모델을 사용해 학습할 수 있다.



위 이미지에서 볼 수 있듯이 타겟 단어는 컨텍스트 또는 기록을 기반으로 예측된다. 예측은 소프트맥스 분류기를 기반으로 한다. 숨겨진 레이어에서는 임베딩을 압축된 표현으로 학습한다. 이것은 완전한 심층 학습 모델은 아니지만 잘 작동한다. 다음은 임베딩을 차원을 낮춰서 시각화한 내용이다.



소프트맥스 분류기를 사용한 임베딩의 낮은 차원 시각화

여기서 처리한 시각화 방법은 텐서보드를 사용해 만들어진다. 비슷한 시맨틱 의미 또는 단어의 다른 일부분이 함께 나온다.

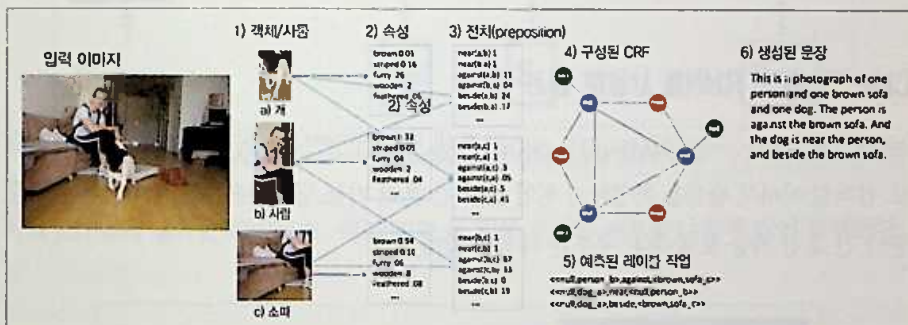
생성하기 위해 사용하는 텍스트에 대한 단순 표현 방법을 훈련시키는 법을 알아봤다. 이어서 이미지 캡션에 대한 접근법을 살펴본다.

## ■ 이미지 캡션 및 관련 문제에 대한 접근 방법

이미지에서 캡션을 얻는 방법은 여러 가지다. 직관적으로 이미지는 시각적 특징으로 변환되고, 그 특징이 텍스트로 만들어진다. 생성된 텍스트는 단어 임베딩 형식을 가진다. 텍스트를 생성하는 주요 접근법에는 LSTM과 관심 사항을 이용하는 방법이 있다. 먼저 텍스트를 생성하는 방법 중에서 오래된 방법부터 시작해보자.

### 조건부 무작위 필드를 사용해 이미지와 텍스트 연결하기

쿨카니 Kulkarni의 논문([http://www.tamaraberg.com/papers/generation\\_cvpr11.pdf](http://www.tamaraberg.com/papers/generation_cvpr11.pdf))에서는 이미지에서 객체와 속성을 찾은 후 이를 사용해 CRF(Conditional Random Field)로 텍스트를 생성하는 방법을 제안했다. CRF는 전통적으로 텍스트 생성과 같은 구조화된 예측에 사용된다. 텍스트 생성의 흐름은 다음과 같다.



CRF를 이용한 텍스트 생성 과정을 보여주는 그림(쿨카니의 논문을 토대로 재구성)

CRF를 사용하면 전치사를 적절하게 배치해 일관된 방식으로 텍스트를 생성하는 데 한계가 있다. 결과는 다음과 같다.

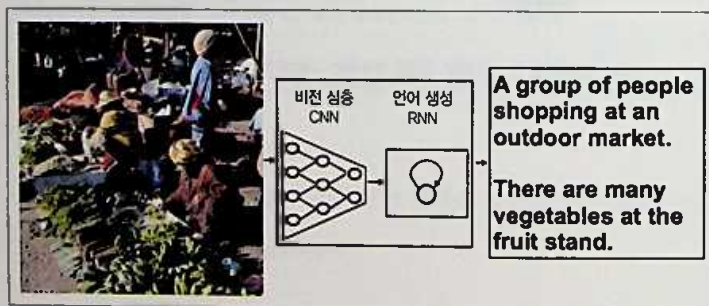


콜카니의 허락하에 재구성 게재

CRF를 사용한 결과 내에서는 객체와 속성에 대한 적절한 예측 결과를 포함하지만, 훌륭한 설명을 생성하는 것은 실패한다.

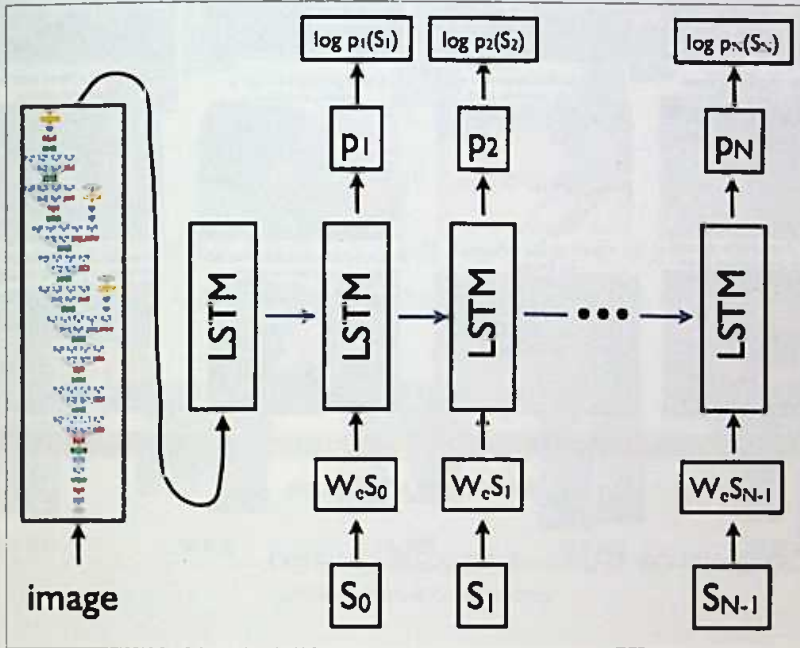
## CNN 기능에서 RNN을 사용해 캡션 생성

논문(<https://arxiv.org/pdf/1411.4555.pdf>)을 통해 빈알스Vinyals는 CNN과 RNN이 연속적으로 겹쳐진 이미지 캡션을 작성하기 위한 종단 간 훈련 가능 딥러닝을 제안했다. 이 방법은 종단 간 훈련 가능 모델이다. 구조는 다음과 같다.



빈알스의 허락하에 재구성 게재(2015)

이 모델은 자연어로 완성된 문장을 생성할 수 있다. 다음 그림은 CNN 및 LSTM의 확장 뷰를 보여준다.



CNN과 LSTM 아키텍처를 보여주는 그림(빈알스의 논문을 토대로 구성)

이제 LSTM의 결과를 확인하자. 여러 결과 중 일부 선택한 세트를 다음과 같이 보여준다.





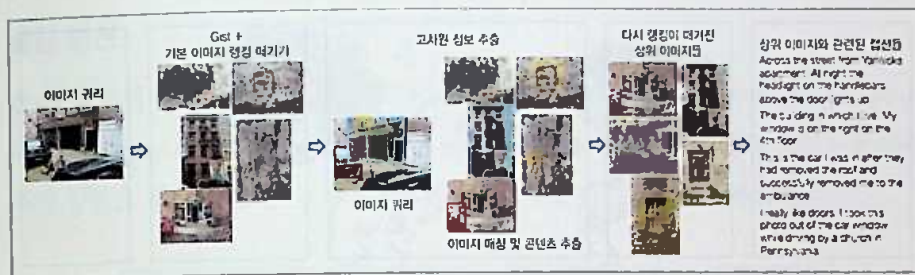
빈알스의 허락하에 재구성 게재(2015)

이때 CNN은 이미지를 특징(RNN 생성 문장)으로 인코딩한다.

## 이미지 순위를 사용해 캡션 만들기

오도네즈Ordonez는 논문(<http://papers.nips.cc/paper/4470-im2text-describing-images-using-1-million-captioned-photographs.pdf>)에서 캡션을 생성한 후 이미지의 순위를 매기는 방법을 제안했다. 이 프로세스의 흐름은 다음과 같다.





오도네즈의 허락하에 재구성 게재(2015)

등급이 매겨진 이미지에서 검출한 고급 정보를 사용해 텍스트를 생성할 수 있다. 다음 이미지는 순위 지정에 사용할 이미지가 많을수록 더 나은 결과를 얻을 수 있음을 보여준다.



오도네즈의 허락하에 재구성 게재(2015)

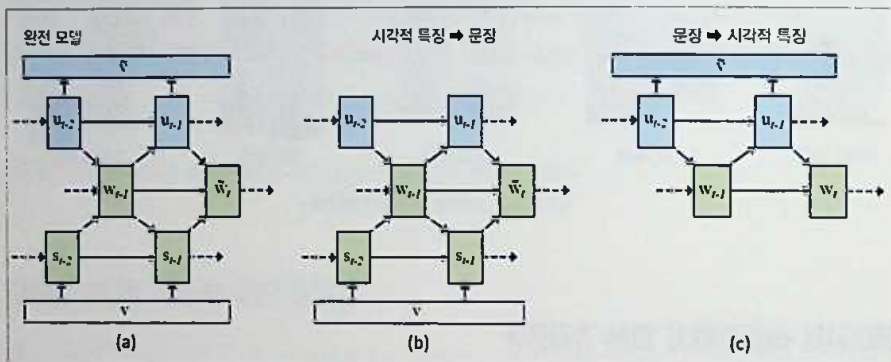
## 이미지와 이미지에서 캡션 가져오기

첸<sup>Chen</sup>은 논문([https://www.cs.cmu.edu/~xinleic/papers/cvpr15\\_rm.pdf](https://www.cs.cmu.edu/~xinleic/papers/cvpr15_rm.pdf))을 통해 이미지에서 텍스트를, 텍스트에서 이미지를 검색하는 방법을 제안했다. 이 방법은 양방향 매핑이다. 다음 그림은 자연어로 이미지를 설명하는 사람과 그것에 대해 시각적으로 생각하는 사람을 보여준다.



첸의 허락하에 재구성 게재(2015)

캡션을 검색하려면 다음과 같이 이미지와 텍스트의 인코더를 잠재적 공간을 이용해 연결한다.

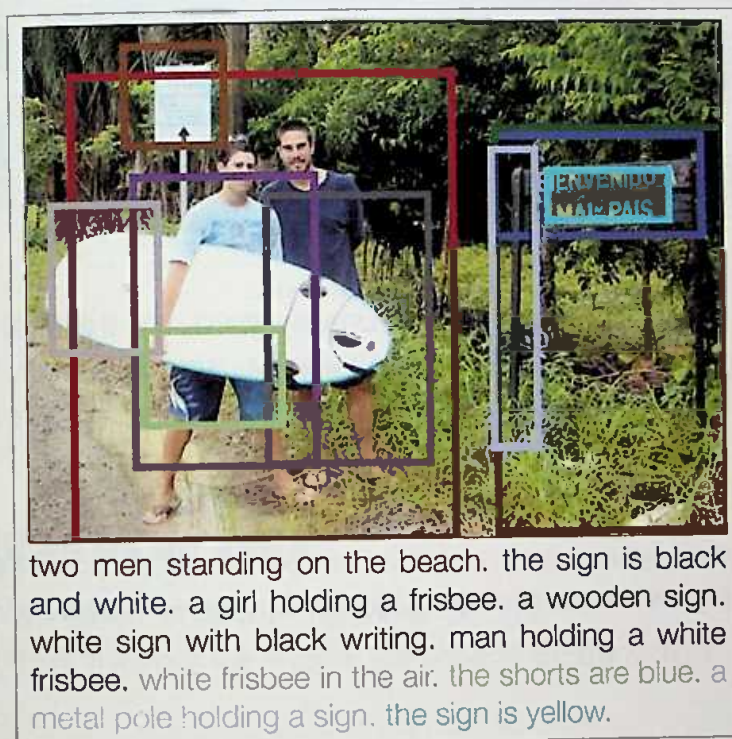


첸의 허락하에 재구성 게재(2015)

이미지의 첫 번째 모델은 훈련에 사용되는 전체 모델이다. 이미지에 표시된 것처럼 시각적 기능을 사용해 문장을 생성하거나 그 반대로도 사용할 수 있다.

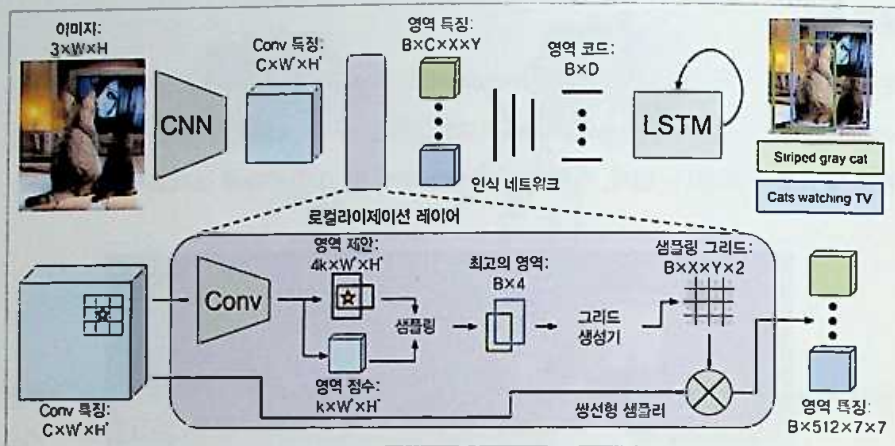
## 밀집 캡션

존슨<sup>Johnson</sup>의 논문([https://www.cv-foundation.org/openaccess/content\\_cvpr\\_2016/papers/Johnson\\_DenseCap\\_Fully\\_Convolutional\\_CVPR\\_2016\\_paper.pdf](https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/Johnson_DenseCap_Fully_Convolutional_CVPR_2016_paper.pdf))에서는 밀집 캡션을 붙이기 위한 방법을 제시했다. 먼저, 작업을 이해하기 위해 몇 가지 결과를 살펴보자.



존슨의 허락하에 재구성 게재

보다시피, 이미지의 객체와 액션에 대해 별도의 캡션이 생성된다. 그러므로 이름과 밀집 캡션을 얻을 수 있다. 존슨이 제안한 아키텍처는 다음과 같다.

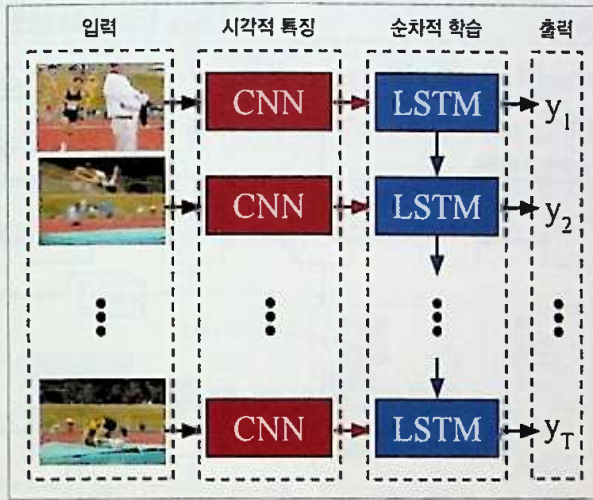


존슨의 허락하에 재구성 게재

이 아키텍처는 근본적으로 Faster R-CNN과 LSTM의 조합이다. 객체 검출 결과를 생성하는 영역이 생성되고, 영역의 시각적 특징이 캡션을 생성하는 데 사용된다.

## 캡션에 RNN 사용하기

도나휴<sup>Donahue</sup>는 논문(<https://arxiv.org/pdf/1411.4389.pdf>)에서 이미지 캡션 작업을 위한 LRCN(Long-term recurrent convolutional architecture)을 제안했다. 이 모델의 아키텍처는 다음과 같다.



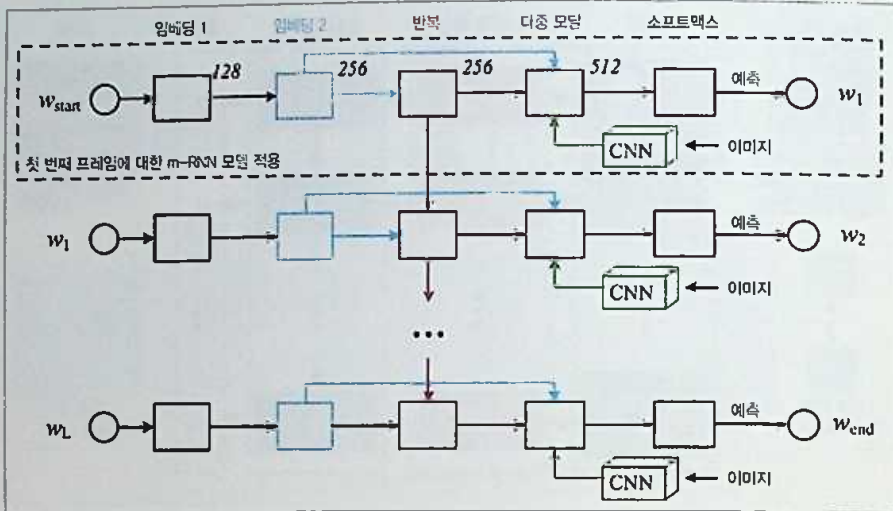
도나후의 허락하에 재구성 게재(2015)

CNN과 LSTM은 시간에 따른 그림 공유 가중치로 표시되므로, 이 방법을 임의로 긴 순차적 순서에 맞게 확장할 수 있다.

## 다중 모달 측정 항목 공간 사용하기

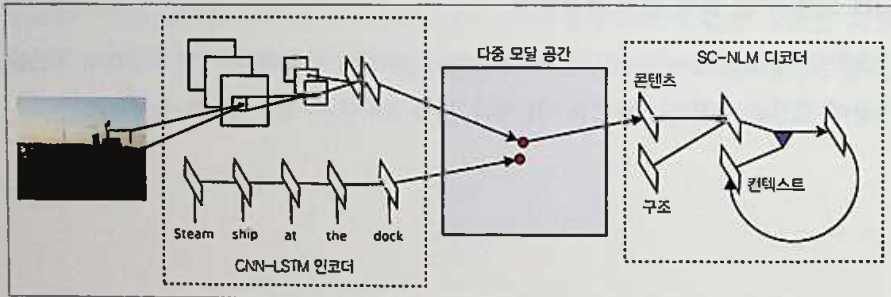
마오<sup>Mao</sup>는 논문(<https://arxiv.org/pdf/1412.6632.pdf>)에서 다중 모달<sup>multimodal</sup> 임베딩 공간을 사용해 캡션을 생성하는 방법을 제안했다. 다음 그림은 이 접근법을 보여준다.





마오의 허락하에 재구성 게재

키로스는 논문(<https://arxiv.org/pdf/1411.2539.pdf>)을 통해 캡션을 생성하기 위해 이미지와 텍스트를 동일한 다중 모달 공간에 임베딩할 수 있는 또 다른 다중 모달 방식을 제안했다. 다음 그림은 이 접근법을 보여준다.



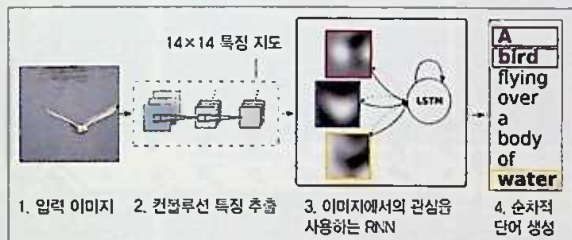
키로스의 허락하에 재구성 게재

다중 모달 접근법을 사용했을 때는 모두 좋은 결과를 얻을 수 있었다.



## 캡션 작성 시 관심 네트워크 사용하기

슈<sup>4</sup>는 논문(<https://arxiv.org/pdf/1502.03044.pdf>)에서 관심<sup>attention</sup> 메커니즘을 사용한 이미지 캡션 방법을 제안했다. 관심 메커니즘은 이미지의 특정 영역에 다른 부분보다 더 많은 가중치를 부여한다. 관심은 시각화를 가능하게 해서 다음 단어를 생성할 때 모델이 어느 곳에 초점을 맞추고 있는지 보여준다. 제안된 모델은 다음과 같다.

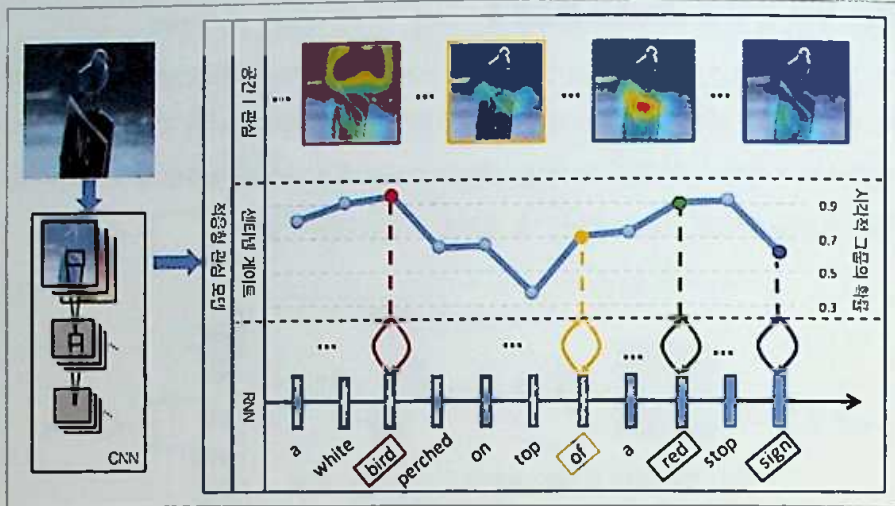


슈의 허락하에 재구성 게재

첫째, CNN 특징을 이미지에서 검출한다. 그런 다음 단어가 생성된 이미지에 관심을 기울인 RNN이 적용된다.

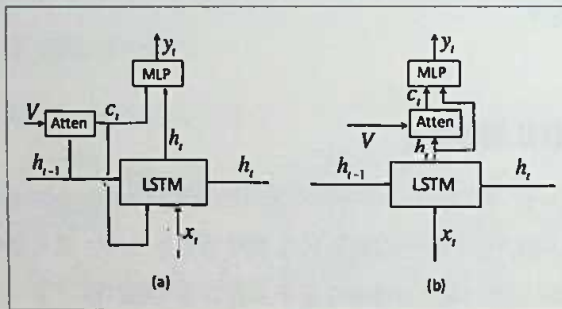
## 언제 살펴봐야 할지 파악하기

루<sup>5</sup>는 우수한 결과를 제공하면서 관심을 기울이는 방법(<https://arxiv.org/pdf/1612.01887.pdf>)을 제안했다. 이로써 더 나은 결과를 얻기 위해 관심을 가지는 캡처 영역을 언제 더 잘 볼 수 있는지 알 수 있다. 다음 그림에서 동작 흐름을 볼 수 있다.



루의 허락하에 재구성 게재

관심 메커니즘은 다음과 같다.



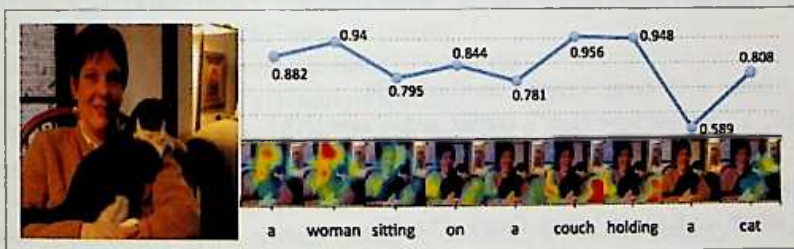
루의 허락하에 재구성 게재

주의를 기울인 영역의 결과는 다음 그림에 강조 표시돼 있다.



루의 허락하에 재구성 게재

캡션을 생성하는 동안 관심을 주지 않는 경우에는 다음과 같이 시각화된다.



루의 허락하에 재구성 게재

캡션 생성에 대한 여러 가지 방법을 소개했다. 지금부터는 구현 방법을 살펴본다.

## ■ 관심 기반 이미지 캡션 방법 구현하기

다음 코드를 사용해 VGG 및 LSTM 모델에서 CNN을 정의해본다.

---

```
vgg_model = tf.keras.applications.vgg16.VGG16(weights='imagenet',
                                                include_top=False,
                                                input_tensor=input_tensor,
                                                input_shape=input_shape)

word_embedding = tf.keras.layers.Embedding(
    vocabulary_size, embedding_dimension, input_length=sequence_length)
embedding = word_embedding(previous_words)
embedding = tf.keras.layers.Activation('relu')(embedding)
embedding = tf.keras.layers.Dropout(dropout_prob)(embedding)
cnn_features_flattened = tf.keras.layers.Reshape((height * height, shape))(cnn_
features)
net = tf.keras.layers.GlobalAveragePooling1D()(cnn_features_flattened)
net = tf.keras.layers.Dense(embedding_dimension, activation='relu')(net)
net = tf.keras.layers.Dropout(dropout_prob)(net)
net = tf.keras.layers.RepeatVector(sequence_length)(net)
net = tf.keras.layers.concatenate()(net, embedding)
net = tf.keras.layers.Dropout(dropout_prob)(net)
```

---

이제 CNN을 정의했으므로 다음 코드를 사용해 관심 레이어를 정의해본다.

---

```
h_out_linear = tf.keras.layers.Convolution1D(
    depth, 1, activation='tanh', border_mode='same')(h)
h_out_linear = tf.keras.layers.Dropout(
    dropout_prob)(h_out_linear)
h_out_embed = tf.keras.layers.Convolution1D(
    embedding_dimension, 1, border_mode='same')(h_out_linear)
z_h_embed = tf.keras.layers.TimeDistributed(
    tf.keras.layers.RepeatVector(num_v_feats))(h_out_embed)
Vi = tf.keras.layers.Convolution1D(
    depth, 1, border_mode='same', activation='relu')(V)
Vi = tf.keras.layers.Dropout(dropout_prob)(Vi)
Vi_emb = tf.keras.layers.Convolution1D(
```

```

embedding_dimension, 1, border_mode='same', activation='relu')(Vi)
z_v_linear = tf.keras.layers.TimeDistributed(
    tf.keras.layers.RepeatVector(sequence_length))(Vi)
z_v_embed = tf.keras.layers.TimeDistributed(
    tf.keras.layers.RepeatVector(sequence_length))(Vi_emb)
z_v_linear = tf.keras.layers.Permute((2, 1, 3))(z_v_linear)
z_v_embed = tf.keras.layers.Permute((2, 1, 3))(z_v_embed)
fake_feat = tf.keras.layers.Convolution1D(
    depth, 1, activation='relu', border_mode='same')(s)
fake_feat = tf.keras.layers.Dropout(dropout_prob)(fake_feat)
fake_feat_embed = tf.keras.layers.Convolution1D(
    embedding_dimension, 1, border_mode='same')(fake_feat)
z_s_linear = tf.keras.layers.Reshape((sequence_length, 1,
depth))(fake_feat)
z_s_embed = tf.keras.layers.Reshape(
    (sequence_length, 1, embedding_dimension))(fake_feat_embed)
z_v_linear = tf.keras.layers.concatenate(axis=-2)([z_v_linear, z_s_linear])
z_v_embed = tf.keras.layers.concatenate(axis=-2)([z_v_embed, z_s_embed])
z = tf.keras.layers.Merge(mode='sum')([z_h_embed, z_v_embed])
z = tf.keras.layers.Dropout(dropout_prob)(z)
z = tf.keras.layers.TimeDistributed(
    tf.keras.layers.Activation('tanh'))(z)
attention = tf.keras.layers.TimeDistributed(
    tf.keras.layers.Convolution1D(1, 1, border_mode='same'))(z)
attention = tf.keras.layers.Reshape((sequence_length, num_v_feats))(attention)
attention = tf.keras.layers.TimeDistributed(
    tf.keras.layers.Activation('softmax'))(attention)
attention = tf.keras.layers.TimeDistributed(
    tf.keras.layers.RepeatVector(depth))(attention)
attention = tf.keras.layers.Permute((1, 3, 2))(attention)
w_Vi = tf.keras.layers.Add()([attention, z_v_linear])
sumpool = tf.keras.layers.Lambda(lambda x: K.sum(x, axis=-2), output_
shape=(depth,))
c_vec = tf.keras.layers.TimeDistributed(sumpool)(w_Vi)
atten_out = tf.keras.layers.Merge(mode='sum')([h_out_linear, c_vec])
h = tf.keras.layers.TimeDistributed(
    tf.keras.layers.Dense(embedding_dimension, activation='tanh'))(atten_out)
h = tf.keras.layers.Dropout(dropout_prob)(h)
predictions = tf.keras.layers.TimeDistributed(

```



```
tf.keras.layers.Dense(vocabulary_size, activation='softmax'))(h)
```

---

앞 코드의 도움으로 관심 메커니즘을 통해 CNN 기능과 RNN을 결합한 딥러닝 모델을 정의했다. 이는 현재 캡션을 생성하는 가장 좋은 방법이다.

## I 요약

이 장에서는 이미지 캡션과 관련된 문제를 이해했다. 자연어 처리 및 GLOVE와 같은 다양한 word2vec 모델과 관련된 몇 가지 기술을 살펴봤고 CNN2RNN, 측정 항목 학습, 결합된 목표와 같은 여러 알고리즘을 이해했다. 이후 CNN과 LSTM을 결합한 모델을 구현했다.

다음 장에서는 생성 모델을 이해하게 될 것이다. 스타일<sup>style</sup> 알고리즘을 처음부터 배우고 구현하며 몇 가지 최상의 모델을 살펴본다. 또한 멋진 GAN과 다양한 애플리케이션 프로그램에 대해서도 다룬다.



## 생성 모델

생성 모델(generative model)은 컴퓨터 비전에서 중요한 애플리케이션 프로그램이다. 이전 장에서 설명한 애플리케이션 프로그램과 달리 생성 모델은 이미지로부터 예측하고, 특정 목적을 위한 이미지를 만들 수 있다. 이번 장에서는 다음 내용을 살펴본다.

- 생성 모델을 사용한 애플리케이션
- 스타일 전송 알고리즘
- 슈퍼 해상도 이미지를 위한 모델 훈련
- 생성 모델의 구현 및 훈련
- 현재 모델이 가지는 단점

이 장의 끝부분에서는 생성 모델에 대한 가능성과 어려움을 이해할 수 있으며, 스타일을 전달할 수 있는 애플리케이션 프로그램을 구현할 수 있다.

## ■ 생성 모델의 애플리케이션

이번 장에서는 생성 모델을 사용하는 애플리케이션 프로그램을 살펴본다. 여기서 제공하고자 하는 애플리케이션 프로그램은 엄청나다. 애플리케이션의 동기와 가능성을 이해하기 위해 몇 가지 프로그램을 살펴본다.

### 예술적 스타일 이전 방법

예술적 스타일 이전<sup>artistic style transfer</sup> 방법은 예술 스타일을 어떤 이미지를 대상으로 옮기는 과정이다. 예를 들어 이미지의 예술적 스타일과 다른 이미지의 내용을 결합해 새로운 이미지를 만들 수 있다. 여러 가지 다양한 스타일과 결합된 이미지의 예는 게이트스 Gatys가 만든 이미지([https://www.cv-foundation.org/openaccess/content\\_cvpr\\_2016/papers/Gatys\\_Image\\_Style\\_Transfer\\_CVPR\\_2016\\_paper.pdf](https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/Gatys_Image_Style_Transfer_CVPR_2016_paper.pdf))로 볼 수 있다. 이미지 A는 스타일이 적용된 사진이며, 다른 이미지에서 결과 표시를 볼 수 있다.

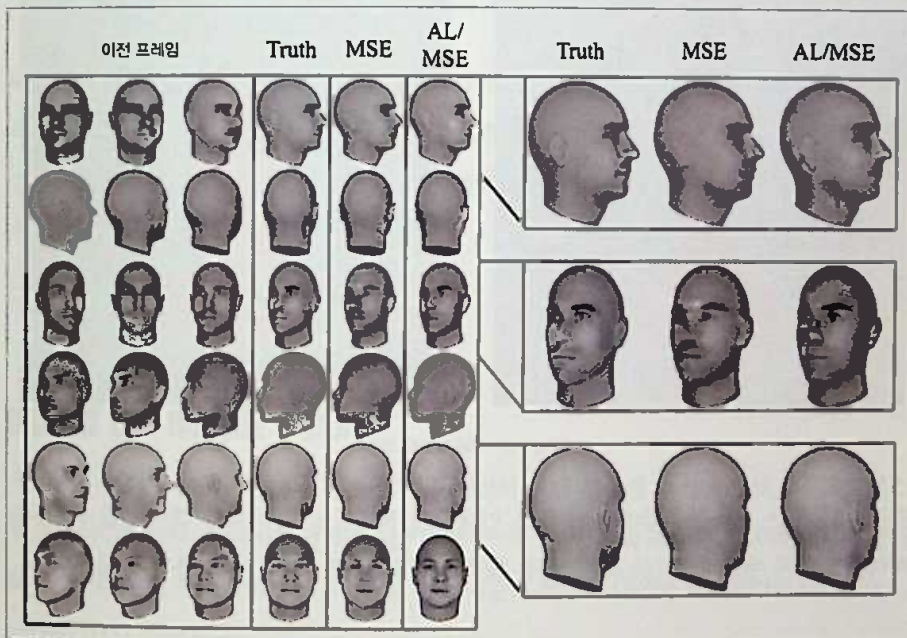


게이트스의 허락하에 재구성 게재

이 애플리케이션 프로그램은 대중의 관심을 끌었으며, 마켓에서 구할 수 있는 몇 가지 모바일 애플리케이션 프로그램이 있다.

## 동영상의 다음 프레임 예측 방법

생성 동영상 모델을 사용해 미래의 프레임을 예측할 수 있다. 로터<sup>Lotter</sup>가 제안한 다음 이미지(<https://arxiv.org/pdf/1511.06380.pdf>)에서 왼쪽의 이미지는 이전 프레임 모델이고, 오른쪽은 실측값이다. 여기서는 두 가지 알고리즘을 사용하며, 평가 비교를 수행한다.



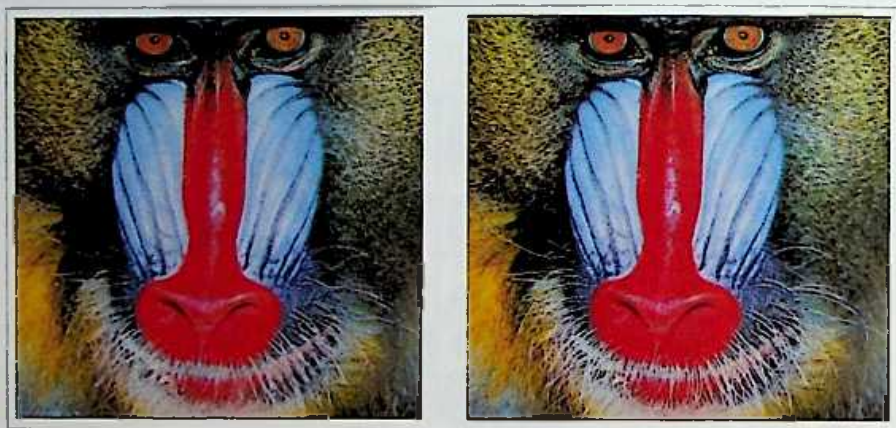
로터의 허락하에 재구성 게재



생성 모델에 의해 생성된 프레임도 현실적으로 보여준다.

## 슈퍼 해상도 이미지

슈퍼 해상도는 작은 이미지에서 고해상도 이미지를 만드는 과정이다. 전통적으로 보간법은 더 큰 이미지를 만드는 데 사용됐다. 그러나 보간법은 메끄러운 효과를 주기 때문에 고주파 세부 사항들을 놓치게 된다. 초고해상도라는 특정 목적을 위해 훈련된 생성 모델은 훌륭한 세부 묘사로 이미지를 만들 수 있다. 다음은 레딕<sup>Ledig</sup>이 제안한 모델의 예다 (<https://arxiv.org/pdf/1609.01802.pdf>). 왼쪽은 4배 배율로 만들어졌으며, 오른쪽 원본과 구별하기가 어렵다.



레딕의 허락하에 재구성 게재

슈퍼 해상도는 고품질 디스플레이 또는 인쇄물에 저해상도 이미지를 렌더링할 때 유용하다. 또 다른 애플리케이션 프로그램은 압축 이미지를 재구성해 좋은 품질을 제공한다.

## 대화형 이미지 생성하기

생성 모델은 상호작용을 사용해 이미지를 생성하는 데 사용할 수 있다. 사용자는 편집을 추가할 수 있으며, 이미지는 쉰<sup>Zhu</sup>가 제안한 것처럼 표시된 편집 내용을 반영해 생성될 수 있다.





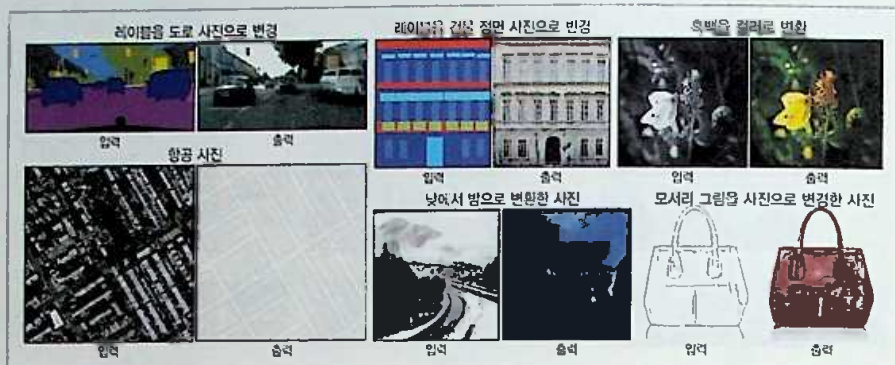
주의 허락하에 재구성 게재

위에서 나타낸 바와 같이, 이미지는 편집의 모양 및 색상에 기초해 생성된다. 아래쪽에 녹색 줄무늬가 초원을 만들고 직사각형이 초고층 건물을 만든다. 이미지는 사용자의 추가 입력으로 사용되고 미세 조정된다. 또한 생성된 이미지를 활용해 가장 유사한 실제 이미지를 검색할 수 있다.

대화형 이미지 생성 방법은 이미지를 직관적으로 검색할 수 있는 새로운 방식을 제공한다.

## 이미지를 다른 이미지로 변환하기

이미지는 특정 목적을 가진 다른 이미지를 생성하는 데 사용될 수 있으므로, 이 프로세스를 '이미지를 다른 이미지로 변환하기(an image to image translation)'라고 한다. 그러한 변환 작업의 몇 가지 예를 아이솔라<sup>sola</sup>가 제안한 해당 기준과 더불어 다음에 나타냈다(<https://arxiv.org/pdf/1611.07004.pdf>).



이미지들의 허락하에 재구성 게재

레이블로 구성된 그림은 창의적인 목적으로 실제 이미지로 변환할 수 있다. 또한 흑백 이미지를 컬러 이미지로 변환할 수 있다. 이러한 변환은 사진 편집 애플리케이션 프로그램, 오래된 영화에 컬러 색상 입히기, 패션 디자인 등에 유용하게 사용될 수 있다.

## 텍스트로 이미지 생성하기

텍스트 설명을 가지고 이미지를 생성할 수 있으며, 이 단계는 이미지를 다른 이미지로 변환하기와 유사하다. 다음은 리드<sup>Reed</sup>의 자연을 설명한 텍스트에서 만들어진 몇 가지 그림 예를 보여준다(<https://arxiv.org/pdf/1605.05396.pdf>).



이 작은 새는 분홍색 가슴, 연두색을 가지며 검은색을 기본색과 보조색으로 가진다.



이 꽃은 흰색 암송머리가 있으며 밝은 분홍색을 포함하는 자주색 꽃잎을 가진다.



이 웅장한 녀석은 붉은색 뺨과 란에 흰 패치가 있으며, 거의 대부분이 검은색을 가진다.



이 흰색과 노란색 꽃은 얇은 흰색 꽃잎과 둥근 황색 수술을 가진다.



리드의 허락하에 재구성 게재



현재 이 모델은 몇 가지 객체에 대해서만 작동한다. 텍스트를 가지고 이미지를 생성하는 것은 아직 애플리케이션 프로그램에서 사용하기에 그다지 현실적이지 않다.

## 불필요 제거

불필요 제거inpainting는 다음 그림과 같이 이미지 내의 일부 공간을 원하는 바대로 채워 넣는 과정이다.



출처: <https://www.flickr.com/photos/littleredell/4756059924/>

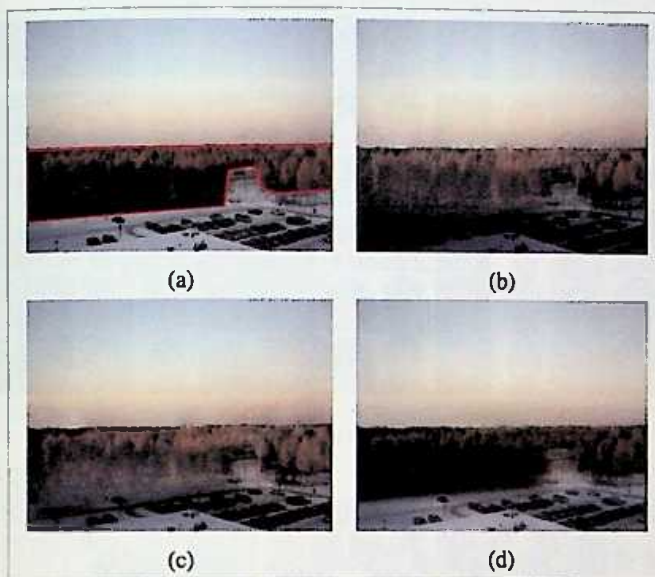
왼쪽 이미지는 정상 이미지고, 오른쪽 이미지는 처리된 이미지다.

이미지에서 볼 수 있듯이 원치 않는 것들이 그림에서 제거된다. 불필요 제거는 이미지에서 원치 않는 객체를 제거하고 스캔한 대상 공간을 채우는 데 유용하다.

## 블렌딩

블렌딩blending은 이미지의 일부를 다른 이미지 위에 부드럽게 붙여 넣는 프로세스다. 여기에 표시된 이미지 a는 하나의 이미지가 다른 이미지에 배치됐을 때 나쁜 인상을 주게 되는 경우를 보여준다. 이미지 b와 c는 각각 수정된 포아송 방법modified Poisson method과 다중 스플라인 방법multi-spline method 같은 종래의 블렌딩 기술을 나타낸다.

최종 이미지인 이미지 d는 블렌딩 생성 방법의 결과를 보여주며, 이는 Wu가 보여준 다른 방법보다 훨씬 나은 결과를 제공한다(<https://arxiv.org/pdf/1703.07195.pdf>).



우의 허락하에 재구성 게재



블렌딩은 사진 편집과 영화 산업의 특수 효과에 매우 유용하다.

## 속성 변환하기

이미지의 속성(attribute)은 생성 모델을 사용해 변경할 수 있다. 성격, 안경, 나이 등 다양한 속성을 반영하도록 사람의 얼굴을 수정할 수 있으며 램플(sample)의 논문을 통해 그 사례를 볼 수 있다([https://research.fb.com/wp-content/uploads/2017/11/fader\\_networks\\_conditional\\_attribute\\_based\\_image\\_generation\\_by\\_disentangling\\_in\\_latent\\_space.pdf](https://research.fb.com/wp-content/uploads/2017/11/fader_networks_conditional_attribute_based_image_generation_by_disentangling_in_latent_space.pdf)).



랭들의 허락하에 재구성 게재

속성을 변경하면 재미있기도 하고, 색다른 애플리케이션에서도 활용할 수 있으며, 유사 콘텐츠들을 사용해 더 많은 훈련 데이터를 생성하는 데도 사용할 수 있다.

## 훈련 데이터 생성

생성 모델은 대규모 스케일로 트레이닝을 생성하는 데 사용될 수 있으며, 트레이닝을 위해 만들어진 합성 이미지를 수정하는 데 사용될 수도 있다. 다음은 왕Wang이 제시한 생성 모델을 사용해 교통 신호를 인식하고자 만들어진 합성 이미지다(<https://arxiv.org/pdf/1707.03124.pdf>).



왕의 허락하에 재구성 게재



이러한 이미지를 사용하면 분류 작업이 좀 더 정확해질 수 있다.

## 새 애니메이션 캐릭터 만들기

생성 모델은 진<sup>jin</sup>에 의해 제시된 것처럼 표정, 헤어 스타일, 의상 등과 같은 다양한 조건을 사용해 새로운 애니메이션 캐릭터를 만드는 데 사용될 수 있다(<https://arxiv.org/pdf/1708.05509.pdf>).



진의 허락하에 재구성 게재

위의 사례처럼 속성이 다른 새로운 캐릭터를 만들 수 있게 되면, 애니메이션 업계에 혁명적인 변화가 일어날 수도 있다.



## 사진으로부터 3D 모델 생성

우<sup>Wu</sup>가 제시한 생성 모델을 사용해 2D 이미지로부터 3D 모델을 만들 수 있다.



우의 허락하에 재구성 게재

이미지에서 3D 모델을 만드는 것은 로봇 공학, 증강현실, 애니메이션 산업에 유용하다. 뒤에 나오는 절에서 3D 모델을 만드는 알고리즘을 살펴볼 것이다. 다음 절에서는 신경 예술 스타일 전송 방법을 구현해본다.

## ■ 신경 예술 스타일 전송

여기서 구현할 첫 번째 애플리케이션은 신경 예술 스타일 전송(neural artistic style transfer) 방법이다. 여기서는 반 고흐 미술의 스타일을 이미지로 옮길 것이다. 이미지는 스타일과 내용의 조합으로 간주될 수 있다. 예술 스타일 전송 기술은 특정 그림 스타일을 가진 그림처럼 보이도록 이미지를 변형한다. 여기서 이 아이디어를 구현하기 위한 코딩 방법을 알아본다. 손실 함수는 생성된 이미지를 그림 내용, 스타일과 비교한다. 따라서 이미지에 대한 최적화는 네트워크의 가중치가 아닌 이미지 픽셀에 대해 수행된다. 사진의 내용과 생성된 이미지를 비교하고, 그림의 스타일과 생성된 이미지를 비교해 두 값을 계산한다.



## 콘텐츠 손실

픽셀을 사용하는 것은 좋은 선택이 아니므로 다양한 레이어의 CNN 기능을 사용해야 한다. 다양한 레이어들은 콘텐츠보다 더 나은 표현이기 때문이다. 3장, '이미지 검색'에서 볼 수 있는 초기 레이어는 가장자리, 모서리, 텍스처와 같이 사용 빈도가 높지만, 그다음 레이어는 객체를 나타내므로 사용하기에 더 좋은 내용을 제공한다. 후자 레이어는 픽셀보다 객체 간 비교를 더 잘할 수 있다. 그러나 이를 위해서는 먼저 다음 코드를 사용해 필요한 라이브러리를 가져와야 한다.

---

```
import numpy as np
from PIL import Image
from scipy.optimize import fmin_l_bfgs_b
from scipy.misc import imsave
from vgg16_avg import VGG16_Avg
from keras import metrics
from keras.models import Model
from keras import backend as K
```

---

이제 다음 명령을 사용해 필요한 이미지를 불러온다.

---

```
content_image = Image.open (work_dir + 'bird_orig.png')
```

---

여기서 사용하는 인스턴스에 대해 다음 이미지가 제공된다.



가능 검출을 위해 VGG 아키텍처를 사용하므로, 다음 코드와 같이 ImageNet 이미지를 모든 이미지에서 빼야 한다.

---

```
imagenet_mean = np.array ([123.68, 116.779, 103.939], dtype = np.float32)
def subtract_imagenet_mean (image) :
    return (image - imagenet_mean)[:, :, :, :-1]
```

---

주의해야 할 사항은 채널이 다르다는 점이다. 전처리 함수는 생성된 이미지를 가져와서 평균값만큼 뺀 후 채널을 반전한다. deprocess 함수는 다음 코드와 같이 전처리 단계로 인한 효과를 그 이전으로 되돌린다.

---

```
def add_imagenet_mean(image, s):
    return np.clip(image.reshape(s)[: , : , : , :-1] + imagenet_mean, 0, 255)
```

---

먼저 다른 이미지의 내용을 사용해 이미지를 만드는 방법을 살펴본다. 이는 무작위 노이즈로 이미지를 만드는 과정이다. 여기에 사용된 콘텐츠는 일부 레이어를 활성화한 이후의 합계 결과다. 여기서 무작위 노이즈와 이미지 사이의 콘텐츠 손실을 최소화해야 한다. 이를 콘텐츠 손실이라 한다. 이 손실은 픽셀 단위 손실과 유사하지만, 레이어 활성화에 적용되므로 노이즈를 제거한 결과를 가지게 된다. 모든 CNN 아키텍처는 콘텐츠 이미지 및 임의 노이즈의 순방향 추론(forward inference)을 수행하는 데 사용될 수 있다. 활성화가 수행되고 평균 재곱 오차가 계산돼 두 출력의 활성화를 비교할 수 있다.

무작위 이미지의 픽셀은 CNN 가중치가 고정되는 동안 업데이트된다. 이 경우에 VGG 네트워크는 잠시 동작을 중지한다. 이제 VGG 모델을 불러올 수 있다. 생성 이미지는 최대 풀링과 같은 서브샘플링 기술에 따라 매우 민감하게 결과가 달라질 수 있다. 최대 풀링 작업을 한 이후에는 픽셀 값을 되돌리기가 불가능하다. 따라서 평균 풀링은 최대 풀링보다 더 나은 방법이다. 평균 풀링을 사용해 VGG 모델을 변환하는 함수는 다음과 같이 모델을 불러오는 데 사용된다.

---

```
vgg_model = VGG16_Avg (include_top = False)
```

---

풀링 유형이 변경됐더라도 원래대로 이 모델의 가중치는 동일하다. 다양한 추상화를 제공할 수 없기 때문에 ResNet 및 인셉션 모델은 적합하지 않다. 모델이 고정돼 있는 동안 VGG 모델의 마지막 컨볼루션 레이어인 block\_conv1에서 활성화를 가져온다. 이것은 VGG의 세 번째 레이어며, 넓은 수용 필드가 있다. 다음 코드를 참조할 수 있다.

---

```
content_layer = vgg_model.get_layer ( 'block5_conv1' ).output
```

---

이제는 레이어에서 좋은 특징을 제공할 때까지 잘려진 VGG로 새로운 모델을 만든다. 따라서 이미지를 지금 불러올 수 있으며, 실제 추론을 수행하고 활성화 레이어를 가져오는 데 사용될 수 있다. 텐서플로 변수는 다음 코드를 사용해 활성화 결과를 캡처할 수 있다.

---

```
content_model = Model(vgg_model.input, content_layer)
content_image_array =
subtract_imagenet_mean(np.expand_dims(np.array(content_image), 0))
content_image_shape = content_image_array.shape
target = K.variable(content_model.predict(content_image_array))
```

---

이미지 손실과 그래디언트를 계산하기 위해 평가자<sup>evaluator</sup> 클래스를 정의한다. 다음 클래스는 모든 반복 지점에서 손실 및 그래디언트 값을 반환한다.

---

```
class ConvexOptimiser(object):
    def __init__(self, cost_function, tensor_shape):
        self.cost_function = cost_function
        self.tensor_shape = tensor_shape
        self.gradient_values = None
    def loss(self, point):
        loss_value, self.gradient_values = self.cost_function([point.reshape(self.
tensor_shape)])
```

---

```
return loss_value.astype(np.float64)
def gradients(self, point):
    return self.gradient_values.flatten().astype(np.float64)
```

---

손실 함수는 특정 컨볼루션 레이어에서 활성화 값 사이의 평균 제곱 오류로 정의할 수 있다. 손실은 생성된 이미지와 원래 내용 사진 사이에서 다음과 같이 계산된다.

---

```
mse_loss = metrics.mean_squared_error (content_layer, target)
```

---

손실에서의 그래디언트 값은 다음과 같이 모델 입력 값을 고려해 계산할 수 있다.

---

```
grads = K.gradients (mse_loss, vgg_model.input)
```

---

함수에 대한 입력은 모델의 입력이며, 출력은 다음과 같이 손실 및 그래디언트 값의 배열 값이다.

---

```
cost_function = K.function ([vgg_model.input], [mse_loss] + grads)
```

---

이 함수는 최적화할 때 결정론적인<sup>deterministic</sup> 방법을 사용하므로, SGD<sup>Stochastic Gradient Descent</sup>는 필요하지 않다.

---

```
optimiser = ConvexOptimiser (cost_function, content_image_shape)
```

---

이 함수는 볼록형<sup>convex</sup>이고 결정론적이므로, 간단한 옵티마이저를 사용해 최적화할 수 있다. 그리고 모든 반복 단계에서 이미지를 저장할 수도 있다. 최종 최적화를 위해 scikit-learn의 최적화 도구를 사용할 때, 그래디언트에 액세스 가능하게 할 수 있다. 여기서 손실 함수는 볼록형이므로 간단한 옵티마이저가 계산하기에 충분하다는 점을 유의해야 한다. 최적화 프로그램은 다음 코드를 사용해 정의할 수 있다.

---

```
def optimise(optimiser, iterations, point, tensor_shape, file_name):
    for i in range(iterations):
        point, min_val, info = fmin_l_bfgs_b(optimiser.loss, point.flatten(),
            fprime=optimiser.gradients, maxfun=20)
        point = np.clip(point, -127, 127)
        print('Loss:', min_val)
        imsave(work_dir + 'gen_'+file_name+'_{i}.png', add_imagenet_mean(point.
copy(), tensor_shape)[0])
    return point
```

---

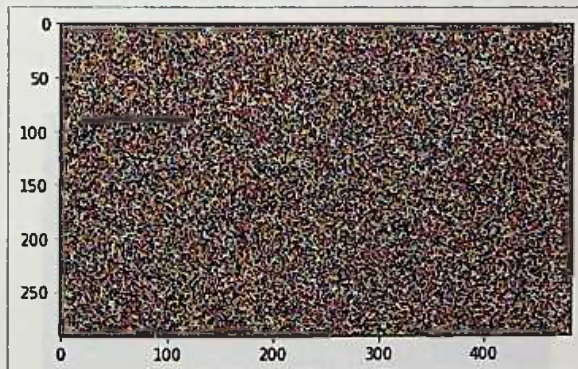
옵티마이저는 손실 함수, 점, 그레디언트를 사용하고, 업데이트 결과를 반환한다. 다음 코드를 사용해 콘텐츠 손실을 최소화할 수 있도록 임의의 이미지를 만들어야 한다.

---

```
def generate_rand_img(shape):
    return np.random.uniform(-2.5, 2.5, shape)/1
generated_image = generate_rand_img(content_image_shape)
```

---

다음은 생성된 임의의 이미지다.





다음과 같이 최적화를 10회 반복해 결과를 볼 수 있다.

---

```
iterations = 10
generated_image = optimise(optimiser, iterations, generated_image, content_image_shape, 'content')
```

---

모든 것이 잘 진행된다면, 반복을 통해 다음과 같이 손실 값이 출력된다.

---

```
Current loss value: 73.2010421753
Current loss value: 22.7840042114
Current loss value: 12.6585302353
Current loss value: 8.53817081451
Current loss value: 6.64649534225
Current loss value: 5.56395864487
Current loss value: 4.83072710037
Current loss value: 4.32800722122
Current loss value: 3.94804215431
Current loss value: 3.66387653351
```

---

다음은 생성된 이미지다. 이제 거의 새처럼 보이기 시작한다. 이 작업을 수행하기 위한 추가 반복 작업을 할 때 최적화를 실행할 수 있다.





옵티마이저는 이미지를 가져오고 픽셀을 업데이트해 내용이 동일하게 나오도록 만들었다. 결과는 더 나쁠 수도 있지만, 내용상으로는 어느 정도 이미지를 재현할 수 있었다. 이미지 생성 방법을 사용할 때 반복된 작업을 사용하면, 이미지는 좋은 결과를 얻을 수 있다. 이 프로세스에서는 배치 작업이 필요하지 않다. 다음 절에서는 그림의 스타일을 사용해 이미지를 만드는 방법을 살펴본다.

## 그림 행렬을 사용한 스타일 손실

여기서는 원본 이미지의 내용을 사용해 이미지를 만든 후에는 스타일만 적용해 다른 이미지를 만드는 방법을 보여준다. 스타일은 이미지의 색상과 질감의 혼합으로 생각할 수 있다. 이를 위해 스타일 손실을 정의할 것이다. 먼저 다음 코드와 같이 이미지를 불러오고 배열로 변환한다.

---

```
style_image = image.open (work_dir + 'starry_night.png')
style_image = style_image.resize (np.divide (style_image.size, 3.5) .astype (
    'int32'))
```

---

다음은 불러온 스타일 이미지다.



이제 다음 코드를 사용해 채널을 변경함으로써 이미지를 사전 처리한다.

```
style_image_array = subtract_imagenet_mean (np.expand_dims (style_image, 0)[:, :, :, : 3])
style_image_shape = style_image_array.shape
```

이를 위해 다음 코드에서 처리한 것처럼 여러 레이어를 고려한다.

```
모델 = VGG16_Avg (include_top = False, input_shape = shp [1 :])
outputs = {l.name : l.output for model.layers}
```

이제 다음 코드를 사용해 여러 개의 레이어를 첫 번째 네 블록의 배열 출력으로 가져온다.

```
layers = [outputs['block{}_conv1'.format(o)] for o in range(1,3)]
```

다음 코드를 사용해 모든 레이어를 출력하고 대상 변수를 할당할 수 있는 새 모델을 만든다.

```
layers_model = Model(model.input, layers)
targs = [K.variable(o) for o in layers_model.predict(style_arr)]
```

스타일 손실은 그램 행렬(Gram matrix)을 사용해 계산된다. 그램 행렬은 행렬과 행렬의 곱이다. 활성화 값은 전환되고 곱해진다. 그러고 나서 이 행렬을 스타일과 임의 이미지 간의 오류를 계산하는 데 사용한다. 그램 행렬은 위치 정보를 잃어버리지만 텍스처 정보는 보존한다. 다음 코드를 사용해 그램 행렬을 정의한다.

---

```
def gramian_matrix(matrix):  
    flattened_matrix = K.batch_flatten(K.permute_dimensions(matrix, (2, 0, 1)))  
    matrix_transpose_dot = K.dot(flattened_matrix, K.transpose(flattened_matrix))  
    element_count = matrix.get_shape().num_elements()  
    return matrix_transpose_dot / element_count
```

---

이제 알겠지만, 이 값은 한 쌍의 열 간 상관관계를 측정한 것이며 높이 및 너비 값은 납작해질 것이다. 여기서는 좌표 정보가 무시되고 로컬 정보가 포함되지 않는다. 스타일 손실은 다음 코드와 같이 입력 이미지의 그래프 행렬과 타깃 사이의 평균 제곱 오류 값으로 계산된다.

---

```
def style_mse_loss (x, y) :  
    return metrics.mse (gramian_matrix (x), gramian_matrix (y))
```

---

이제 다음 코드를 사용해 다양한 레이어의 모든 활성화를 요약해 손실을 계산해본다.

---

```
style_loss = sum(style_mse_loss(l1[0], l2[0]) for l1, l2 in  
zip(style_features, style_targets))  
grads = K.gradients(style_loss, vgg_model.input)  
style_fn = K.function([vgg_model.input], [style_loss]+grads)  
optimiser = ConvexOptimiser(style_fn, style_image_shape)
```

---

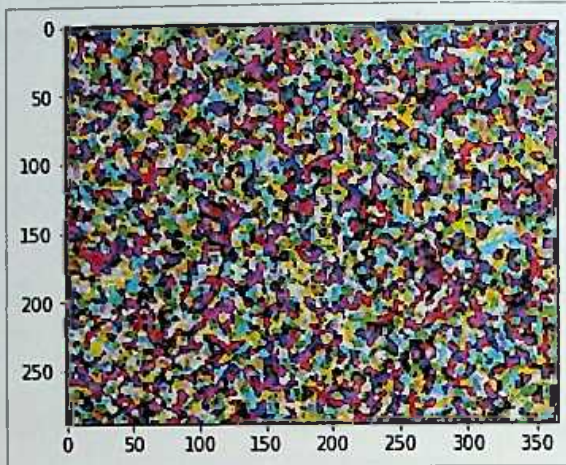
그리고 임의의 이미지를 만들어 이전과 같은 방법으로 해결한다. 그러나 이번에는 다음 코드와 같이 가우시안<sup>Gaussian</sup> 필터도 적용할 것이다.

---

```
generated_image = generate_rand_img (style_image_shape)
```

---

생성된 입의의 이미지는 다음과 같다.



다음과 같은 최적화 방법을 10회 반복해 결과를 확인할 수 있다.

---

```
generated_image = optimise(optimiser, iterations, generated_image,  
style_image_shape)
```

---

모든 것이 잘 동작하면, 결과 값으로 다음과 비슷한 손실 값을 출력한다.

---

```
Current loss value: 5462.45556641  
Current loss value: 189.738555908  
Current loss value: 82.4192581177  
Current loss value: 55.6530838013  
Current loss value: 37.215713501  
Current loss value: 24.4533748627  
Current loss value: 15.5914745331  
Current loss value: 10.9425945282  
Current loss value: 7.66888141632  
Current loss value: 5.84042310715
```

---



생성된 이미지는 다음과 같다.



여기서는 임의의 노이즈로부터 어떤 위치 정보 없이 특정한 그림 스타일로 이미지를 만들었다. 다음 절에서는 두 가지, 즉 내용과 스타일 손실을 결합하는 방법을 살펴본다.

## 스타일 전송

이제 이미지를 재구성하는 방법과 원본 이미지의 스타일을 캡처하는 이미지를 만드는 방법을 알았다. 확실한 아이디어는 다음 코드와 같이 두 가지 손실 함수를 가중치로 추가해, 이 두 가지 방법을 결합하는 것이다.

---

```
w, h = style.size
src = img_arr[:, :, h, : w]
```

---

이전과 마찬가지로 스타일 손실을 계산하기 위해 일련의 레이어 출력을 가져온다. 그러나 콘텐츠 손실을 계산하기 위해서는 하나의 레이어 출력만 있으면 된다. 어떤 레이어를 잡을지 어떻게 알 수 있을까? 앞에서 설명한 것처럼, 레이어가 낮을수록 내용 재구성이 정

확해진다. 콘텐츠 재구성과 스타일을 병합할 때 콘텐츠의 느슨한 재구성으로 스타일에 영향을 줄 수 있는 공간이 늘어날 수 있다. 또한 이후 레이어를 사용하면 동일한 세부 정보가 없더라도 이미지가 동일한 제목을 가지는 것처럼 보일 수 있다. 다음 코드는 이러한 과정 처리를 위해 사용된다.

---

```
style_layers = [outputs['block{}_conv2'.format(o)] for o in range(1,6)]
content_name = 'block4_conv2'
content_layer = outputs[content_name]
```

---

이제 다음 코드로 필요한 출력 레이어를 사용해 스타일에 대한 별도의 모델을 만든다.

---

```
style_model = Model(model.input, style_layers)
style_targs = [K.variable(o) for o in style_model.predict(style_arr)]
```

---

또한 다음 코드로 콘텐츠 레이어를 사용해 콘텐츠에 대한 다른 모델을 만든다.

---

```
content_model = Model(model.input, content_layer)
content_targ = K.variable(content_model.predict(src))
```

---

이제 두 가지 접근 방식의 병합은 각각의 손실 기능을 병합하는 것처럼 간단히 이뤄진다. 이전 함수와 달리 이 함수는 세 가지 유형의 출력을 생성한다.

- 원래 이미지용
- 스타일을 모방하고 있는 이미지용
- 픽셀을 훈련하고 있는 무작위 이미지용

재구성 혼합 방법을 조정할 수 있는 한 가지 방법은 콘텐츠 손실에 대한 요소를 변경하는 것이다. 여기서는 요소 값으로 1/10을 사용한다. 분모 값을 증가시키면 스타일은 이미지에 더 큰 영향을 준다. 하지만 너무 크면 이미지의 원래 내용이 구조화되지 않은 스타일에



의해 가려진다. 마찬가지로 분모 값이 너무 작으면 이미지의 스타일은 충분하지 않다. 이 프로세스에는 다음 코드가 사용된다.

---

```
style_wgts = [0.05,0.2,0.2,0.25,0.3]
```

---

손실 함수는 다음과 같이 스타일 및 내용 레이어를 모두 사용한다.

---

```
loss = sum(style_loss(l1[0], l2[0]))*w
    for l1,l2,w in zip(style_layers, style_targs, style_wgts))
loss += metrics.mse(content_layer, content_targ)/10
grads = K.gradients(loss, model.input)
transfer_fn = K.function([model.input], [loss]+grads)
evaluator = Evaluator(transfer_fn, shp)
```

---

다음 코드를 사용해 이전과 같이 10회 반복한다.

---

```
iterations=10
x = rand_img(shp)
x = solve_image(evaluator, iterations, x)
```

---

손실 값은 다음과 같이 출력된다.

---

```
Current loss value: 2557.953125
Current loss value: 732.533630371
Current loss value: 488.321166992
Current loss value: 385.827178955
Current loss value: 330.915924072
Current loss value: 293.238189697
Current loss value: 262.066864014
Current loss value: 239.34185791
Current loss value: 218.086700439
Current loss value: 203.045211792
```

---

이 결과는 매우 놀랍다. 예술가 스타일로 원본 이미지를 재현하는 환상적인 일을 수행했다. 생성된 이미지는 다음과 같다.



이제 스타일 전송에 대한 내용을 마무리할 것이다. 이 작업은 정말 느리지만 모든 이미지 작업이 가능하다. 다음 절에서는 유사한 아이디어를 사용해 슈퍼 해상도 네트워크를 만드는 방법을 살펴본다. 이를 개선하기 위한 몇 가지 방법이 있다.

- 무작위 이미지에 가우스 필터 추가하기
- 레이어에 다른 가중치 추가하기
- 다양한 레이어 및 가중치를 콘텐츠에 사용하기
- 무작위 이미지를 사용하기보다는 이미지를 초기화하기
- 색상 보존하기
- 필요한 것을 지정하기 위해 마스크 사용하기
- 모든 스케치를 그림으로 변환하기
- 스케치 그리기 및 이미지 만들기



모든 이미지는 해당 이미지를 출력하도록 CNN으로 훈련함으로써 예술적 스타일로 변환될 수 있다.

## I GAN

생성적 적대 신경망(GAN, Generative Adversarial Network)은 2014년에 이안 굿펠로우(Ian Goodfellow)가 만들었으며, 두 개의 신경 회로망이 동시에 판별자와 생성기로 훈련되는 비지도 알고리즘이다. 이 기술은 무작위 노이즈로부터 이미지를 생성할 수 있으며 판별기(discriminator)가 원본 이미지인지 여부를 평가할 수 있다. 추가적인 훈련을 거친 후 생성기 네트워크는 사실적인 이미지를 생성할 수 있다. 생성기 네트워크는 일반적으로 디컨볼루션적(deconvolutional) 신경망이며, 판별기는 컨볼루션 신경망(convolution neural network)이다.

이것을 이해하기 위한 훌륭한 예를 들어보면, 생성기는 가짜 돈을 인쇄하는 사람으로 생각하고 판별기는 돈이 가짜인지 아닌지 결정하는 경찰관으로 생각하는 것이다. 생성기는 경찰이 원래의 돈과 가짜 돈을 구별할 수 없을 때까지 경찰의 피드백을 기반으로 가짜 돈의 질을 향상시킨다. 이제 구현을 시작해보자.

## 바닐라 GAN

원래 GAN은 바닐라 GAN이라 한다. 모델을 만들기 전에 이 장의 나머지 부분에서 유용할 몇 가지 레이어를 정의해보자. 다음은 누설 활성화(leaky activation) 및 정규화(regularization)가 추가된 convolutional\_layers다.

---

```
def convolution_layer(input_layer,
                      filters,
                      kernel_size=[4, 4],
```

```

        activation=tf.nn.leaky_relu):
layer = tf.layers.conv2d(
    inputs=input_layer,
    filters=filters,
    kernel_size=kernel_size,
    activation=activation,
    kernel_regularizer=tf.nn.l2_loss,
    bias_regularizer=tf.nn.l2_loss,
)
add_variable_summary(layer, 'convolution')
return layer

```

---

이어서 다음 코드를 사용해 정규화된 convolution\_layer와 반대되는 transpose\_convolution\_layer를 정의한다.

```

def transpose_convolution_layer(input_layer,
                                filters,
                                kernel_size=[4, 4],
                                activation=tf.nn.relu,
                                strides=2):
    layer = tf.layers.conv2d_transpose(
        inputs=input_layer,
        filters=filters,
        kernel_size=kernel_size,
        activation=activation,
        strides=strides,
        kernel_regularizer=tf.nn.l2_loss,
        bias_regularizer=tf.nn.l2_loss,
    )
    add_variable_summary(layer, 'convolution')
    return layer

```

---

다음 코드를 사용해 비선형 활성화를 사용함으로써 밀집 레이어를 정의한다.

---

```
def dense_layer(input_layer,
                units,
                activation=tf.nn.relu):
    layer = tf.layers.dense(
        inputs=input_layer,
        units=units,
        activation=activation
    )
    add_variable_summary(layer, 'dense')
    return layer
```

---

이제 노이즈를 입력으로 받고 이미지로 변경하는 생성기를 정의한다. 생성기는 완전 연결 레이어 두 개로 구성되며, 그다음에 노이즈를 업샘플링하기 위해 전치(트랜스포즈<sup>transpose</sup>) 컨볼루션 레이어가 구성된다. 마지막으로, 노이즈를 단일 채널로 만들기 위해 컨볼루션 레이어가 제공된다. 그래디언트가 원활하게 흐르도록 모든 레이어 간에 배치 정규화 레이어가 존재한다. 다음 코드를 사용해 생성기를 정의한다.

---

```
def get_generator(input_noise, is_training=True):
    generator = dense_layer(input_noise, 1024)
    generator = tf.layers.batch_normalization(generator,
        training=is_training)
    generator = dense_layer(generator, 7 * 7 * 256)
    generator = tf.layers.batch_normalization(generator,
        training=is_training)
    generator = tf.reshape(generator, [-1, 7, 7, 256])
    generator = transpose_convolution_layer(generator, 64)
    generator = tf.layers.batch_normalization(generator,
        training=is_training)
    generator = transpose_convolution_layer(generator, 32)
    generator = tf.layers.batch_normalization(generator,
        training=is_training)
    generator = convolution_layer(generator, 3)
    generator = convolution_layer(generator, 1, activation=tf.nn.tanh)
    return generator
```

---



이제 이미지를 가지고 실제 이미지와 가짜를 구별하는 GAN의 판별기 부분을 정의할 것이다. 판별기는 컨볼루션 레이어(convolutional\_layers)가 뒤따르며 밀집 레이어가 뒤를 잇는 규칙적인 컨볼루션 네트워크(convolutional net)다. 또한 일괄 정규화(batch normalization) 레이어가 레이어 사이에 존재한다. 판별기를 정의하기 위해 다음 코드를 사용할 것이다.

---

```
def get_discriminator(image, is_training=True):
    x_input_reshape = tf.reshape(image, [-1, 28, 28, 1], name='input_reshape')
    discriminator = convolution_layer(x_input_reshape, 64)
    discriminator = convolution_layer(discriminator, 128)
    discriminator = tf.layers.flatten(discriminator)
    discriminator = dense_layer(discriminator, 1024)
    discriminator = tf.layers.batch_normalization(discriminator, training=is_training)
    discriminator = dense_layer(discriminator, 2)
    return discriminator
```

---

판별기가 생성된 후, 다음 코드를 사용해 생성기에 대한 입력이 될 노이즈 벡터를 생성한다.

---

```
input_noise = tf.random_normal ([batch_size, input_dimension])
```

---

GAN 모델은 텐서플로의 tf.contrib.gan 모듈을 사용해 만들 수 있다. 다음과 같이 생성기 및 판별기 메소드와 해당 입력을 가져온다.

---

```
gan = tf.contrib.gan.gan_model (
    get_generator,
    get_discriminator,
    real_images,
    input_noise)
```

---

이제 학습은 다음 코드를 사용해 생성기와 판별기에 대한 손실 및 최적화 프로그램과 함께 gan\_train\_ops 메소드를 사용하는 gan\_train 메소드로 시작된다.



---

```
tf.contrib.gan.gan_train(  
    tf.contrib.gan.gan_train_ops(gan,  
        tf.contrib.gan.gan_loss(gan),  
        tf.train.AdamOptimizer(0.001),  
        tf.train.AdamOptimizer(0.0001)))
```

---

gan\_train 메소드를 실행해서 임의의 벡터로부터 이미지를 출력할 수 있는 GAN 모델을 만든다. 생성된 이미지는 제약이 없으며 모든 레이블에서 가져올 수 있다. 다음 절에서는 조건부 GAN을 사용해 원하는 출력을 생성하는 방법을 살펴본다.

## 조건부 GAN

조건부 GAN은 사용자가 원하는 레이블로 이미지를 생성한다. 예를 들어, 숫자 8을 생성하도록 모델에 요청할 수 있고 모델은 8을 생성한다. 이 기능을 사용하고자 다음과 같이 모델에 대한 훈련을 받으려면 노이즈와 레이블이 필요하다.

---

```
gan = tf.contrib.gan.gan_model (  
    get_generator, get_discriminator, real_images, (input_noise, labels))
```

---

훈련의 나머지 부분은 바닐라 GAN의 훈련과 비슷하다. 다음에서는 GAN을 사용해 이미지를 압축한다.

## 적대적 손실

적대적 손실(adversarial loss)은 생성기의 손실을 의미한다. 이 손실은 가짜 및 실제 이미지 간의 픽셀 단위 손실과 결합해 결합된 적자 손실을 형성할 수 있다. GAN 모델은 생성자와 판별기 모두에게 real\_images를 함께 제공한다.

---

```
gan = tf.contrib.gan.gan_model(  
    get_autoencoder,  
    get_discriminator,  
    real_images,  
    real_images)
```

---

생성기는 자동 인코딩을 수행한다. 구현 방법은 3장. '이미지 검색'에서 찾을 수 있다. 손실은 그 후에 다음 코드를 사용해 정의할 수 있다.

---

```
loss = tf.contrib.gan.gan_loss(gan, gradient_penalty=1.0)  
l1_pixel_loss = tf.norm(gan.real_data - gan.generated_data, ord=1)  
loss = tf.contrib.gan.losses.combine_adversarial_loss(  
    loss, gan, l1_pixel_loss, weight_factor=1)
```

---

GAN 손실 기울기(그래디언트) 값에는 손실 값도 반영된다. 그리고 페널티 처리된 손실에 픽셀 단위 손실이 계산돼 추가된다. 이 모델을 훈련해 이미지 압축에 사용할 수 있는 강력한 자동 인코딩 프로그램을 만들 수 있다.

## 이미지 변환

애플리케이션 프로그램을 다루면서 이미 살펴본 것처럼 이미지를 다른 이미지로 변환할 수 있다. 입력 이미지는 판별기에 제공되지만, 대상 이미지는 GAN 모델을 생성하는 동안 다음과 같이 생성기에 제공된다.

---

```
gan = tf.contrib.gan.gan_model(  
    get_generator,  
    get_discriminator,  
    real_images,  
    input_images)
```

---

최소 제곱 손실은 모델을 훈련하기 위한 픽셀 단위 손실 외에 훈련에도 사용된다. 다음 코드를 사용해 계산할 수 있다.

---

```
loss = tf.contrib.gan.gan_loss(
    gan,
    tf.contrib.gan.losses.least_squares_generator_loss,
    tf.contrib.gan.losses.least_squares_discriminator_loss)
l1_loss = tf.norm(gan.real_data - gan.generated_data, ord=1)
gan_loss = tf.contrib.gan.losses.combine_adversarial_loss(
    loss, gan, l1_loss, weight_factor=1)
```

---

이 기술을 사용하면 이미지를 다른 이미지로 변환할 수 있다.

## InfoGAN

InfoGAN은 명시적 지도 훈련 없이도 필요한 레이블의 이미지를 생성할 수 있다. `infogan_model`은 다음 코드와 같이 비구조화된 입력과 구조화된 입력을 모두 다 사용한다.

---

```
info_gan = tf.contrib.gan.infogan_model(
    get_generator,
    get_discriminator,
    real_images,
    unstructured_input,
    structured_input)
loss = tf.contrib.gan.gan_loss(
    info_gan,
    gradient_penalty_weight=1,
    gradient_penalty_epsilon=1e-10,
    mutual_information_penalty_weight=1)
```

---

손실은 훈련이 불안정할 때 부과될 수 있는 벌점(페널티)으로 정의된다. 벌칙이 추가되면 훈련을 진행하면서 안정성이 향상될 수 있다.

## GAN의 단점

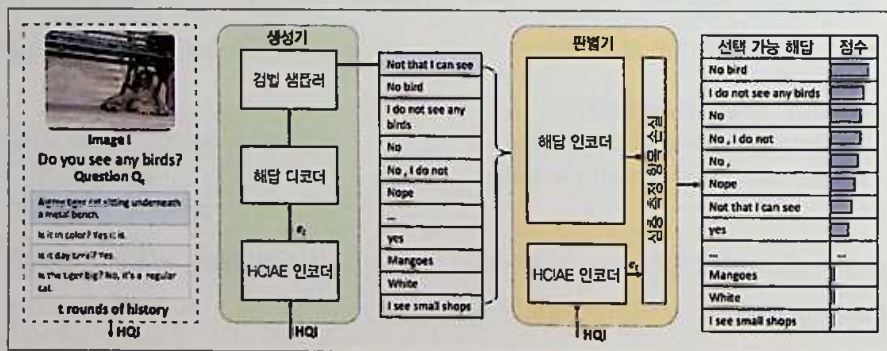
GAN이 생성한 이미지에는 계산, 원근감, 글로벌 구조와 같은 몇 가지 단점이 존재한다. 현재 모델을 개선하기 위해 이 단점들은 광범위하게 연구되고 있다.

## I VDM

시각적 대화 모델 VDM, Visual Dialogue Model은 이미지를 기반으로 한 채팅이 가능하도록 지원한다. VDM에는 컴퓨터 비전, NLP Natural Language Processing, 채팅 봇 기술이 적용된다. VDM을 사용해 시각 장애인에게 이미지를 설명해주는 기능, 의사에게 의료 스캔 결과를 제공하는 기능, 가상 동반자 기능 등을 다양한 애플리케이션 프로그램에 적용할 수 있다. 지금부터는 애플리케이션의 기능을 제공하기 위한 알고리즘을 살펴본다.

### VDM 알고리즘

여기에 설명된 알고리즘은 루나가 제안한 알고리즘이다([https://research.fb.com/wp-content/uploads/2017/11/camera\\_ready\\_nips2017.pdf](https://research.fb.com/wp-content/uploads/2017/11/camera_ready_nips2017.pdf)). 루는 GAN 기반 VDM을 제안했다. 생성기는 해답을 생성하고, 판별기는 해답에 대한 순위를 매긴다. 다음은 프로세스의 개략적인 표현 내용이다.

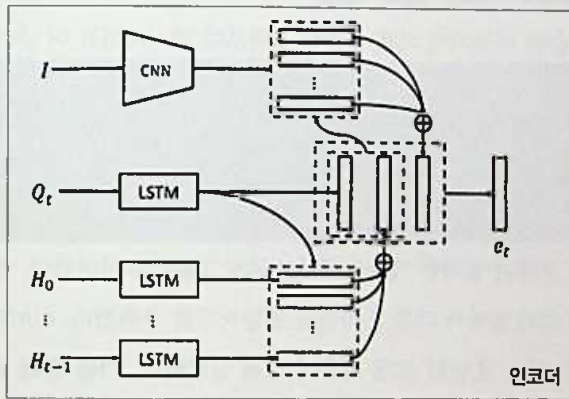


GAN 기법에 기반한 VDM 아키텍처(루의 허락하에 재구성 개제)

채팅 기록, 현재 질문, 사용된 이미지는 생성기에 대한 입력으로 제공된다. 지금부터는 생성기가 어떻게 작동하는지 살펴보자.

## 생성기

생성기는 인코더와 디코더를 가진다. 인코더는 이미지, 질문, 기록 값을 입력으로 사용한다. 인코더 내부에서는 먼저 LSTM에 기록을 넣고, 이미지에 대한 결과 값을 얻는다. 다음 그림에서 동작 흐름을 볼 수 있다.



루의 허락하에 재구성 게재

전체 기록 값을 사용할 수 있으며 LSTM은 채팅 기록을 사용한다. 출력할 때는 이미지 임베딩이 수행된다. 인코더에서 임베딩이 사용되고, 디코더에서는 해답을 생성하는 동작이 이뤄진다. 디코더는 RNN으로 구성된다. 인코더와 디코더를 모두 사용해 해답을 만드는 생성기를 구성한다. 이어서 판별자가 어떻게 작동하는지 살펴본다.

## 판별기

판별기는 발생된 시퀀스를 생성기에서 가져와 순위를 지정한다. 랭킹은  $n$ 쌍의 손실을 가지며, 임베딩이 학습돼 수행된다.  $n$ 쌍의 손실은 여러 양/음 쌍이 비교를 위해 사용되는 점을 제외하고는 트리플렛 손실과 유사하다. 다음은 모델에 의해 생성된 결과 중 일부다.



|                                                                                                                                                                                                                       |                                                                                                                                                                                    |                                                                                                                                                                    |                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                      |                                                                                                   |                                                                                   |                                                                                                                                                               |
| <p>Q: Is it a home or restaurant?<br/>A: I think restaurant.<br/>G-MLE: I can't tell.<br/>Ours (z1): Hard to say.<br/>Ours (z2): It looks like a restaurant.<br/>Ours (z3): I can't tell because it is too close.</p> | <p>Q: Can you see his face?<br/>A: I am not sure.<br/>G-MLE: Yes.<br/>Ours (z1): I can only see the back of his body.<br/>Ours (z2): No.<br/>Ours (z3): No, he's too far away.</p> | <p>Q: How old does the man seem to be?<br/>A: 20's.<br/>G-MLE: Late teens.<br/>Ours (z1): On his 20s.<br/>Ours (z2): In his twenties.<br/>Ours (z3): Mid 20's.</p> | <p>Q: Can you see broccoli?<br/>A: Yes, 3 larger pieces and some small.<br/>G-MLE: No, just the broccoli.<br/>Ours (z1): I can see broccoli, slightly better.<br/>Ours (z2): Yes, there is broccoli.<br/>Ours (z3): Yes, broccoli is green.</p> |

루의 허락하에 재구성 게재

이러한 결과는 단순한 판별기에 의해 산출된 결과보다 합리적이고 더 낫다.

## I 요약

이 장에서는 생성 모델과 방대한 애플리케이션 프로그램들을 살펴봤다. 콘텐츠를 보존하면서 스타일을 한 스타일에서 다른 스타일로 변환하도록 구현했다. GAN부터는 직감 같은 결과를 얻을 수 있는 훈련된 모델 사용에 대해 살펴봤다. 이를 통해 VDM에 대해 알게 됐다.

다음 장에서는 동영상 분석을 위한 딥러닝 방법을 살펴본다. 카메라, 파일 등을 통해 동영상 콘텐츠에 액세스하는 방법을 살펴본다. 프레임 레벨과 동영상 전체에 분류를 적용해 동영상 분류를 구현한다. 뒷부분에서는 동영상에서 객체를 추적하는 방법을 살펴본다.



## 동영상 분류

9장에서는 동영상 데이터에 대한 딥러닝 모델을 훈련하는 방법을 살펴본다. 프레임별로 동영상 분류를 수행하고 나서 더 나은 정확도를 위해 시간 정보를 사용한다. 이후 포즈 추정, 캡션 작성, 동영상 생성과 같은 이미지 적용 내용을 동영상으로 확장한다.

이 장에서는 다음 내용을 다룬다.

- 동영상 분류를 위한 데이터셋 및 알고리즘
- 동영상을 프레임으로 분할하고 분류하기
- 개별 프레임 수준에서 시각적 기능을 위한 모델 훈련
- 3D 컨볼루션 및 동영상에서의 사용 방법
- 동영상에 모션 벡터 통합
- 시간 정보를 이용한 객체 추적

- 사람의 포즈 추정 및 동영상 캡션과 같은 애플리케이션 프로그램

## I 동영상의 이해 및 분류

동영상은 일련의 이미지들의 구성이다. 동영상은 시간 방향에 따라 이미지에 새로운 차원을 추가한다. 이미지의 공간적 특징과 동영상의 시간적 특징을 결합해 이미지보다 더 나은 결과를 제공할 수 있다. 여분의 차원은 많은 공간을 사용하므로, 훈련과 추론의 복잡성을 증가시킨다. 동영상을 처리할 때는 많은 계산량이 필요하다. 동영상은 또한 시간적 특징을 고려해야 하므로 딥러닝 모델의 아키텍처를 변경해야 한다.

동영상 분류는 범주를 사용해 동영상에 레이블을 지정하는 작업이다. 카테고리리는 프레임 레벨 또는 전체 동영상에 해당한다. 레이블을 지정할 때는 동영상을 가지고 사용하는 작업이 있을 수 있다. 따라서 동영상 분류는 동영상에 있는 객체에 레이블을 붙이거나 동영상에서 확인 가능한 동작에 레이블을 붙일 수 있다. 다음 절에서는 동영상 분류 작업에 사용할 수 있는 데이터셋을 살펴본다.

### 동영상 분류 데이터셋 탐색

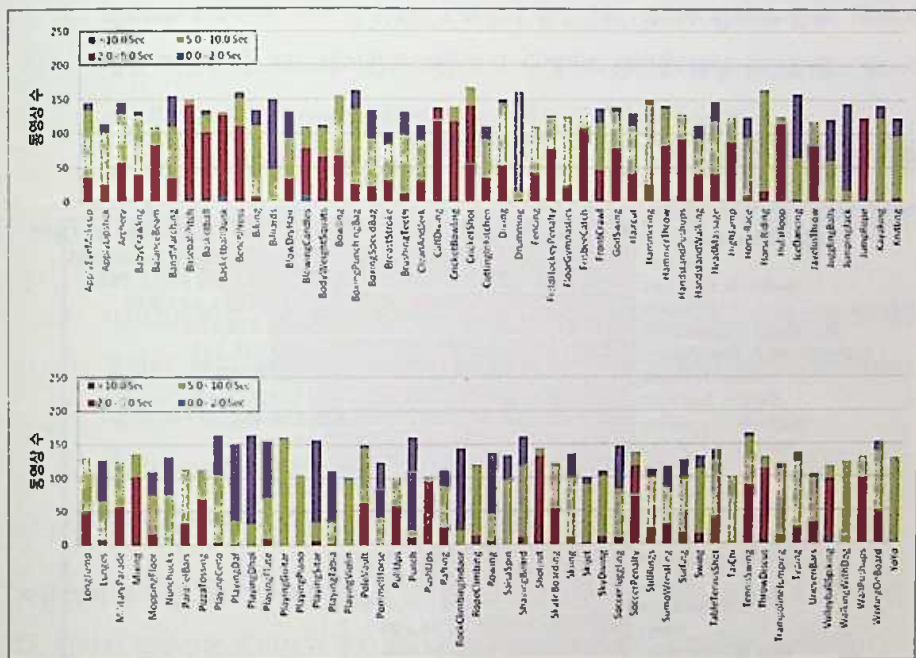
동영상 분류는 동영상 데이터를 사용해 연구가 이뤄지는 주요 분야다. 여러 동영상이 찍히고 데이터와 관련된 다양한 객체 또는 작업으로 레이블이 지정된다. 데이터셋은 크기, 품질, 레이블 유형에 따라 다르다. 일부는 동영상용 다중 레이블을 포함한다. 동영상의 길이는 다소 짧다. 긴 동영상은 다양한 동작을 수행할 수 있으므로 잘라낸 동영상 세그먼트 또는 조각을 개별적으로 분류하기 전에 시간적으로 세그먼트화할 수 있다. 다음 절에서는 특정 데이터셋의 세부 사항을 살펴본다.

#### UCF101

UCF101<sup>1</sup>University of Central Florida은 행동 인식을 위한 데이터셋이다. 동영상은 유튜브에서 수

집되며 실제 만들어 사용하기도 한다. 이 데이터셋에는 101개의 액션 카테고리가 있다. 50개의 카테고리가 있는 UCF50이라는 또 다른 데이터셋도 있다. 이 데이터셋에는 작업 전체에서 13,320개의 동영상상이 있다. 이 동영상상에서는 배경, 스케일, 포즈, 교합, 조명 조건이 다양하다. 액션 카테고리는 배경, 포즈, 스케일, 시점, 조명 등과 같은 비슷한 변화를 공유하는 25개 그룹으로 분류된다.

액션과 액션당 동영상 수는 다음과 같다.



출처: <http://crcv.ucf.edu/data/UCF101/Number%20of%20Videos%20.jpg>

101개의 모든 동작은 인간-객체 상호작용, 몸 동작, 인간-인간 상호작용, 악기 연주, 스포츠와 같은 다섯 가지 유형 동작으로 그룹화된다. 데이터셋 및 애노테이션은 <http://crcv.ucf.edu/data/UCF101.php>에서 다운로드할 수 있다.

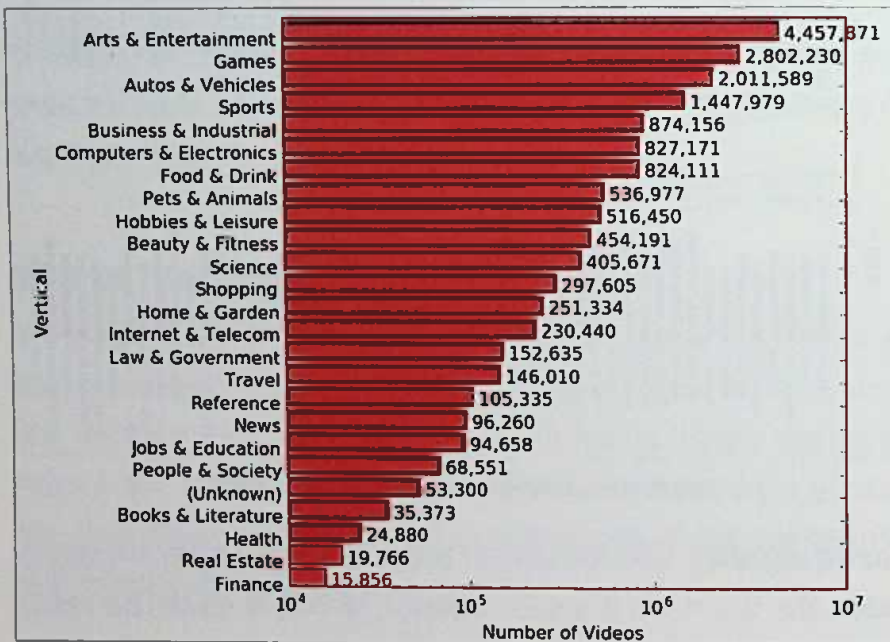
다음 절에서는 YouTube-8M 데이터셋에 대해 알아본다.

## YouTube-8M

YouTube-8M 데이터셋은 동영상 분류 문제에 사용된다. 데이터셋에는 레이블 및 시각적 기능이 있는 동영상 URL이 포함되어 있다. 다음은 데이터셋에 대한 몇 가지 통계 값을 보여준다.

- 동영상 URL 수: 700만
- 동영상 클립의 시간: 450,000
- 학급 레이블 수: 4,716
- 동영상당 평균 레이블 수: 3.4

다음은 다양한 장르에 걸친 데이터 집합의 요약이다.



출처: <https://research.google.com/youtube8m/vertical-videos.png>

이 그림은 데이터셋에서 사용할 수 있는 레이블 유형을 보여준다. 동영상 데이터는 크기가

크고 시각적 특징 값은 계산돼 데이터셋과 함께 제공된다. 데이터셋은 <https://research.google.com/youtube8m/> 링크를 통해 확인할 수 있다.

## 기타 데이터셋

동영상 분류 문제에 데이터셋을 추가적으로 더 사용할 수 있다. 몇 가지 데이터셋에 대한 자세한 내용은 다음과 같다.

- Sports-1M<sup>Sports - 1 Million</sup>: 1,133,158개의 동영상을 보유하고 있으며 487개의 수업을 제공한다. 애노테이션은 자동으로 수행된다. 데이터셋은 <http://cs.stanford.edu/people/karpathy/deepvideo/>에서 다운로드할 수 있다.
- UCF-11(센트럴 플로리다 대학교 - 11가지 행동 제공): 1,600개의 동영상과 11개의 행동을 제공한다. 동영상은 29.97fps(초당 프레임 수)를 가진다. 데이터셋은 UCF101과 함께 다운로드할 수 있다.
- HMDB-51(휴먼 모션 데이터베이스 - 51개 동작 제공): 5,100개의 동영상과 51개의 동작이 있다. 데이터셋 링크는 <http://serre-lab.clps.brown.edu/resource/hmdb-large-human-motion-database>다.
- Hollywood2: 1,707개의 동영상과 12개의 동작을 제공한다. 데이터셋 링크는 <http://www.di.ens.fr/~laptev/actions/hollywood2>다.

지금까지 동영상 분류 작업에 사용할 수 있는 데이터셋을 설명, 접근 링크와 함께 살펴보았다. 이어서 동영상을 불러오고 추가 처리를 위해 프레임 분할하는 방법을 살펴본다.

## 동영상을 프레임으로 분할하기

동영상은 프레임 단위로 변환돼 추가 사용을 위해 디렉터리에 저장될 수 있다. 프레임으로 분할하면 훈련 과정 전에 동영상의 압축을 풀기 때문에 시간을 절약할 수 있다. 먼저 동영상을 프레임으로 변환하기 위한 코드를 살펴보자.



---

```

import cv2
video_handle = cv2.VideoCapture (video_path)
frame_no = 0
while True:
    eof, frame = video_handle.read ()
    if not : eof :
        break
    cv2.imwrite ( "frame % d.jpg"% frame_no, frame)
    frame_no += 1

```

---



이 코드를 사용하면 위의 모든 데이터셋을 프레임으로 변환할 수 있다. 하지만 하드디스크 공간이 많이 필요해진다.

## 동영상 분류 접근법

동영상은 여러 애플리케이션별로 분류돼야 한다. 동영상은 많은 양의 데이터이기 때문에 훈련 및 추론 계산도 고려해야 한다. 모든 동영상 분류 접근법은 이미지 분류 알고리즘에서 영감을 얻었다. VGG, 인셉션 등과 같은 표준 아키텍처는 프레임 수준에서 특징 계산을 위해 사용되며, 그 외에 다른 용도로도 사용된다. 이전 장에서 배운 CNN, LSTM과 같은 개념이 여기서 유용하게 사용될 수 있다. 직관적으로 다음과 같은 접근 방식을 동영상 분류에 사용할 수 있다.

- 프레임을 검출하고 2장, '이미지 분류'에서 배운 모델을 사용해 프레임 단위로 분류한다.
- 3장, '이미지 검색'에서 배운 이미지 특징을 검출하고, 7장, '이미지 캡션 작성'에서 설명한 대로 특징을 사용해 RNN을 학습한다.
- 전체 동영상을 사용해 3D 컨볼루션 네트워크를 훈련한다. 3D 컨볼루션은 2D 컨볼루션의 확장이다. 다음 절에서 3D 컨볼루션의 동작을 자세히 살펴볼 것이다.



- 동영상의 광학 흐름 방법을 사용해 정확도를 더욱 향상시킨다. 광학 흐름 방법은 객체의 움직임 패턴을 사용한다. 자세한 내용은 다음 절에서 설명한다.

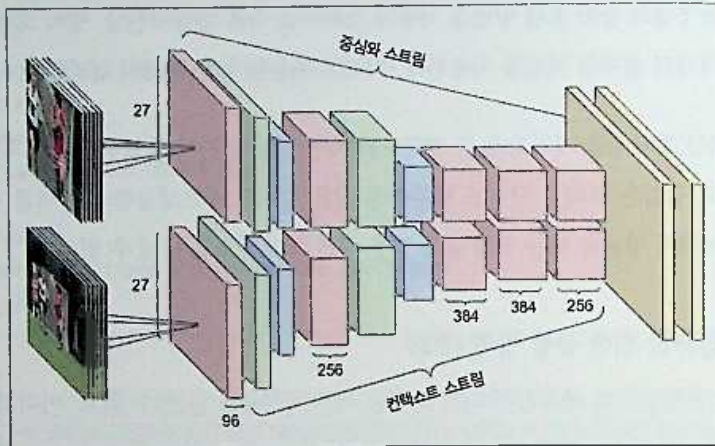
다양한 계산 복잡성을 가지고 높은 정확도를 제공하는 몇 가지 알고리즘을 보게 될 것이다. 데이터 집합은 프레임 단위로 변환하고 같은 길이로 서브샘플링해 준비할 수 있다. ImageNet에서 평균을 빼는 것과 같은 일부 사전 처리가 도움이 될 수 있다.

## 동영상 분류를 위한 융합 병렬 CNN

동영상을 프레임으로 예측하게 되면 미세한 세부 사항들을 잃을 수 있고, 이미지의 다운샘플링으로 인해 좋지 않은 결과를 얻을 수도 있다. 고해상도를 지원하는 CNN 방법을 사용하면 추론 시간이 길어지게 된다. 따라서 카파시<sup>Karpathy</sup>가 제안한 두 개의 스트림을 융합<sup>fusion</sup>하는 방법(<https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/42455.pdf>)을 사용해 동영상 분류를 할 수 있다. 프레임 단위 예측에는 다음과 같은 두 가지 문제점이 존재한다.

- 더 큰 CNN 아키텍처를 가지는 경우에는 예측이 오래 걸릴 수 있다.
- 독립적인 예측을 수행하면 시간 차원을 기준으로 정보를 잃어버릴 수 있다.

병렬로 실행되는 두 개의 작은 인코더를 사용하면 더 적은 수의 매개변수로 아키텍처를 단순화할 수 있다. 동영상은 두 개의 CNN 인코더를 통해 동시에 전달된다. 하나의 인코더는 저해상도 동영상을 가지고 고해상도로 처리한다. 인코더는 컨볼루션, 정규화<sup>normalization</sup>, 풀링 레이어<sup>pooling layer</sup>를 번갈아 사용한다. 두 인코더에서의 최종 레이어는 완전 연결 레이어와 연결돼 있다. 다른 인코더는 크기가 동일하지만, 여기에 표시된 것과 같이 중앙 자르기<sup>central crop</sup> 동작만 수행한다.

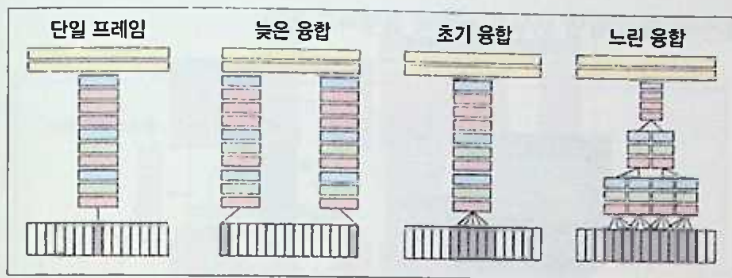


카파시의 허락하에 재구성 게재

프레임을 병렬 처리하면 동영상상을 다운샘플링해 동작 시간을 더 빠르게 만들 수 있다. CNN 아키텍처는 동일한 정확도를 유지하면서 매개변수가 절반으로 줄어들게 된다. 두 스트림을 중심화<sup>focus</sup> 스트림과 컨텍스트 스트림이라 한다. 스트림은 다음 코드를 통해 표시될 수 있다.

```
high_resolution_input = tf.placeholder(tf.float32, shape=[None, input_size])
low_resolution_input = tf.placeholder(tf.float32, shape=[None, input_size])
y_input = tf.placeholder(tf.float32, shape=[None, no_classes])
high_resolution_cnn = get_model(high_resolution_input)
low_resolution_cnn = get_model(low_resolution_input)
dense_layer_1 = tf.concat([high_resolution_cnn, low_resolution_cnn], 1)
dense_layer_bottleneck = dense_layer(dense_layer_1, 1024)
logits = dense_layer(dense_layer_bottleneck, no_classes)
```

동영상 프레임은 다음 그림과 같이 임의의 차원<sup>temporal dimension</sup>으로 구분해 처리한다.



카파시의 허락하에 재구성 게재

동영상 클립은 고정된 크기를 가지지 않으며, 다른 시간에 볼 수도 있다. 위의 그림에서는 임의의 정보를 연결하는 세 가지 방법을 보여준다. 늦은 융합(late fusion)에는 더 긴 시간 프레임이 필요하고, 초기 융합(early fusion)에는 몇 개의 프레임이 함께 표시된다.<sup>1</sup> 느린 융합(slow fusion)은 늦은 융합과 초기 융합이 결합돼 좋은 결과를 얻을 수 있다. 이 모델은 Sports-1M 데이터셋을 이용해 학습하고 487개의 클래스를 가지며 50%의 정확도를 달성한다. 동일한 모델을 UCF101에 적용할 경우 정확도는 60%가 된다.

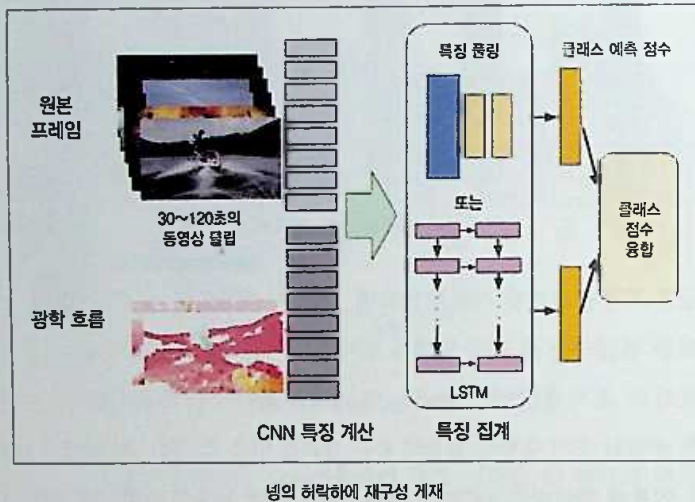
## 장시간에 걸친 동영상 분류

짧은 동영상에는 융합 방법이 적합하다. 많은 프레임을 계산하고 기억해야 하므로 아주 긴 동영상을 분류하기는 어렵다. Ng<sup>9</sup>은 매우 긴 동영상을 분류하기 위한 두 가지 방법([https://www.cvfoundation.org/openaccess/content\\_cvpr\\_2015/papers/Ng\\_Beyond\\_Short\\_Snippets\\_2015\\_CVPR\\_paper.pdf](https://www.cvfoundation.org/openaccess/content_cvpr_2015/papers/Ng_Beyond_Short_Snippets_2015_CVPR_paper.pdf))을 제안했다.

- 첫 번째는 컨볼루션 특징을 일시적으로 풀링하는 방법이다. 최대 풀링(max-pooling)은 특징에 대한 집계(aggregation) 방법으로 사용된다.
- 두 번째 접근법에서는 가변 길이 동영상을 처리할 수 있는 컨볼루션 특징을 연결하는 LSTM을 사용할 수 있다.

1 '초기 융합'은 특징(feature) 단계에서의 융합을 말하고, '늦은 융합'은 결정(decision) 단계에서의 융합을 말한다([https://bi.snu.ac.kr/Publications/Conferences/Domestic/KIASE2014W\\_KWPark.pdf](https://bi.snu.ac.kr/Publications/Conferences/Domestic/KIASE2014W_KWPark.pdf)). - 옮긴이

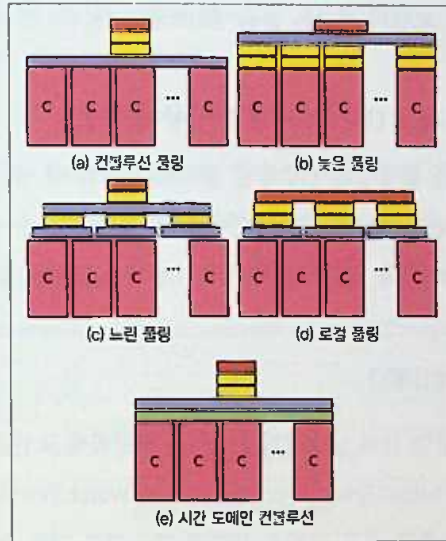
다음 그림은 두 가지 방법 모두를 간략히 설명하고 있다.



다음 코드에서는 CNN 특징을 검출해 작은 LSTM 네트워크에 공급할 수 있다.

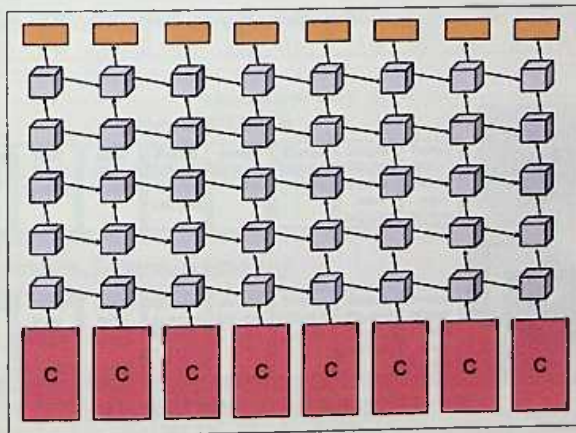
```
net = tf.keras.models.Sequential()
net.add(tf.keras.layers.LSTM(2048,
    return_sequences=False,
    input_shape=input_shape,
    dropout=0.5))
net.add(tf.keras.layers.Dense(512, activation='relu'))
net.add(tf.keras.layers.Dropout(0.5))
net.add(tf.keras.layers.Dense(no_classes, activation='softmax'))
```

대신 특징 풀링에 LSTM을 추가하면 성능이 향상될 수 있다. 특징은 다음 그림과 같이 다양한 방법으로 풀링할 수 있다.



넷의 허락하에 재구성 게재

그림에서 표시한 것처럼, 컨볼루션 특징은 몇 가지 다른 방식으로도 집계될 수 있다. 풀링은 완전 연결 레이어 다음 단계에서 수행된다. 이 방법을 사용해 각각 Sports-1M 데이터셋 및 UCF101 데이터셋에서 73.1% 및 88.6%의 정확도를 달성했다. LSTM 접근 방식은 다음 그림에서 확인할 수 있다.



넷의 허락하에 재구성 게재



여러 LSTM을 사용해서 모델의 계산은 높은 정확도를 가질 수 있다.

## 액션 인식을 위한 두 개의 CNN 스트리밍 사용하기

동영상에서 객체 동작은 동영상에서 수행된 동작을 분석할 때 필요한 좋은 정보들을 제공한다. 객체의 움직임은 광학 흐름(optical flow)에 의해 정량화할 수 있다. 사이먼안(Simonyan)과 지저만(Zisserman)은 이미지와 광학 흐름에서 두 개의 흐름을 사용하는 행동 인식 방법(<http://papers.nips.cc/paper/5353-two-stream-convolutional-networks-for-action-recognition-in-videos.pdf>)을 제안했다.

광학 흐름은 관찰자와 장면 간의 상대적인 움직임을 정량화해 모션을 측정한다. 광학 흐름에 대한 자세한 내용은 <https://www.youtube.com/watch?v=5VyLAH8BhF8>을 통해 확인할 수 있다. 광학 흐름은 다음 명령을 실행해 결과 값을 얻을 수 있다.

```
p1, st, err = cv2.calcOpticalFlowPyrLK(old_gray, frame_gray, p0, None, **lk_params)
```

하나의 스트림은 개별 프레임을 가져오고 일반 CNN을 적용해 작업 결과를 예측한다. 다른 스트림은 여러 프레임을 사용해 광학 흐름을 계산한 경우다. 광학 흐름은 예측하기 위해 CNN을 사용한다. 다음 그림은 두 예측 동작 모두를 보여준다.



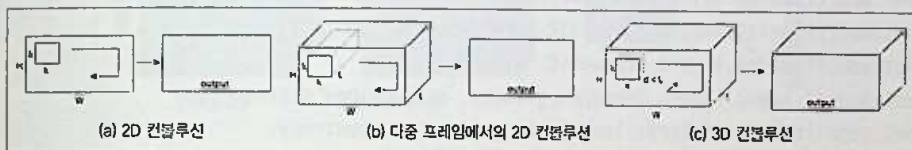
그림 1: 동영상 분류를 위한 두 개의 스트림을 사용한 구조

사이먼안과 지저만의 허락하에 재구성 게재

두 예측 방법 모두가 최종 예측과 결합될 수 있다.

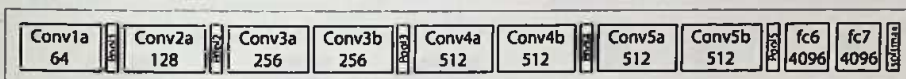
## 시간적 학습을 위한 3D 컨볼루션 사용하기

동영상은 3D 컨볼루션을 사용해 분류할 수 있다. 3D 컨볼루션 연산은 입력과 출력의 크기를 동일하게 하나로 사용하고, 2D 컨볼루션 동작은 2D 크기를 사용하거나 여러 2D 이미지로 출력할 수 있다. 차이점은 다음의 그림에서 확인할 수 있다.



트랜의 허락하에 재구성 게재

처음 두 이미지는 2D 컨볼루션에 속한다. 출력은 항상 이미지 형태다. 하지만 3D 컨볼루션은 볼륨 형태로 출력한다. 이 둘 간의 차이점은 커널을 사용한 세 가지 방향의 컨볼루션 연산이 사용되는지 여부다. 트랜Tran은 동영상 분류를 위해 3D 컨볼루션 방법([https://www.cv-foundation.org/openaccess/content\\_iccv\\_2015/papers/Tran\\_Learning\\_Spatiotemporal\\_Features\\_ICCV\\_2015\\_paper.pdf](https://www.cv-foundation.org/openaccess/content_iccv_2015/papers/Tran_Learning_Spatiotemporal_Features_ICCV_2015_paper.pdf))을 사용했다. 3D 컨볼루션 모델은 다음과 같다.



트랜의 허락하에 재구성 게재

다음은 3D 컨볼루션을 사용하는 모델의 예제 코드다.

```
net = tf.keras.models.Sequential()
net.add(tf.keras.layers.Conv3D(32,
                                kernel_size=(3, 3, 3),
                                input_shape=(input_shape)))
net.add(tf.keras.layers.Activation('relu'))
net.add(tf.keras.layers.Conv3D(32, (3, 3, 3)))
```

```

net.add(tf.keras.layers.Activation('softmax'))
net.add(tf.keras.layers.MaxPooling3D())
net.add(tf.keras.layers.Dropout(0.25))
net.add(tf.keras.layers.Conv3D(64, (3, 3, 3)))
net.add(tf.keras.layers.Activation('relu'))
net.add(tf.keras.layers.Conv3D(64, (3, 3, 3)))
net.add(tf.keras.layers.Activation('softmax'))
net.add(tf.keras.layers.MaxPool3D())
net.add(tf.keras.layers.Dropout(0.25))
net.add(tf.keras.layers.Flatten())
net.add(tf.keras.layers.Dense(512, activation='sigmoid'))
net.add(tf.keras.layers.Dropout(0.5))
net.add(tf.keras.layers.Dense(no_classes, activation='softmax'))
net.compile(loss=tf.keras.losses.categorical_crossentropy,
            optimizer=tf.keras.optimizers.Adam(), metrics=['accuracy'])

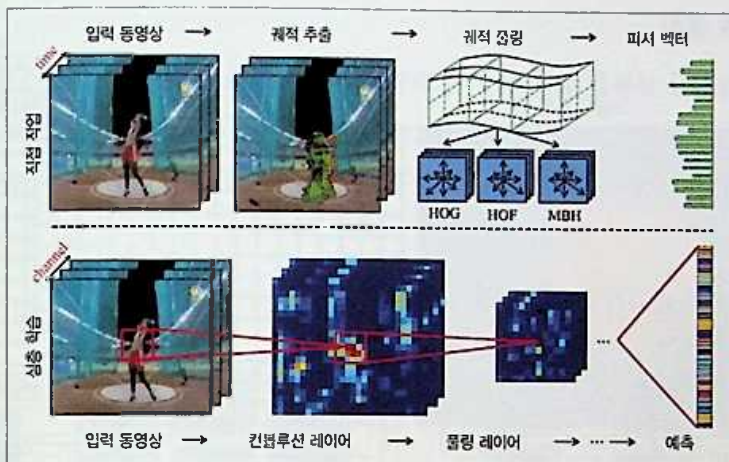
```



3D 컨볼루션에는 많은 컴퓨팅 연산 능력이 필요하다. 3D 컨볼루션은 Sports-1M 데이터셋에서 90.2%의 정확도를 나타낸다.

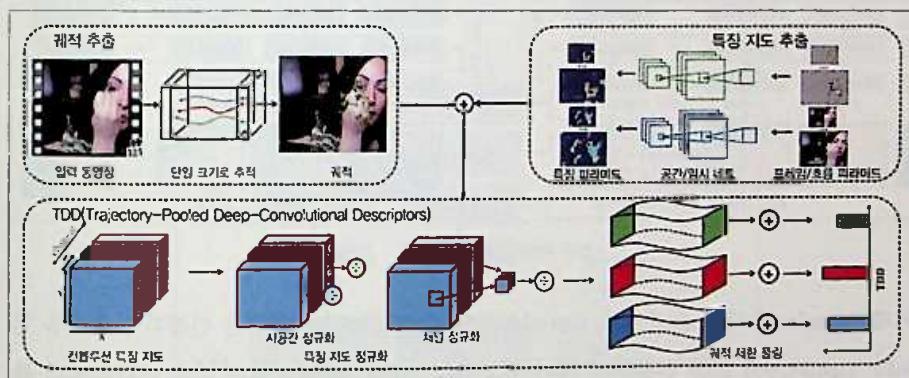
## 분류 동작에 궤적 사용하기

왕<sup>Wang</sup>은 수행된 행동을 분류하기 위해 사물의 일부 궤적<sup>trajectory</sup>을 사용하는 방법([https://www.cv-foundation.org/openaccess/content\\_cvpr\\_2015/papers/Wang\\_Action\\_Recognition\\_With\\_2015\\_CVPR\\_paper.pdf](https://www.cv-foundation.org/openaccess/content_cvpr_2015/papers/Wang_Action_Recognition_With_2015_CVPR_paper.pdf))을 제안했다. 이 방법에서는 직접 작업한 특징 값과 심층 학습으로 얻은 특징 값들을 결합해 최종 예측 결과를 얻는다. 다음 그림은 분류 방법을 보여준다.



왕의 허락하에 재구성 게재

직접 작업해 얻은 특징은 피쳐 벡터며, 이 특징들은 CNN으로부터 얻는다. 다음 그림은 궤적과 특징 지도를 검출하는 방법을 보여준다.



왕의 허락하에 재구성 게재

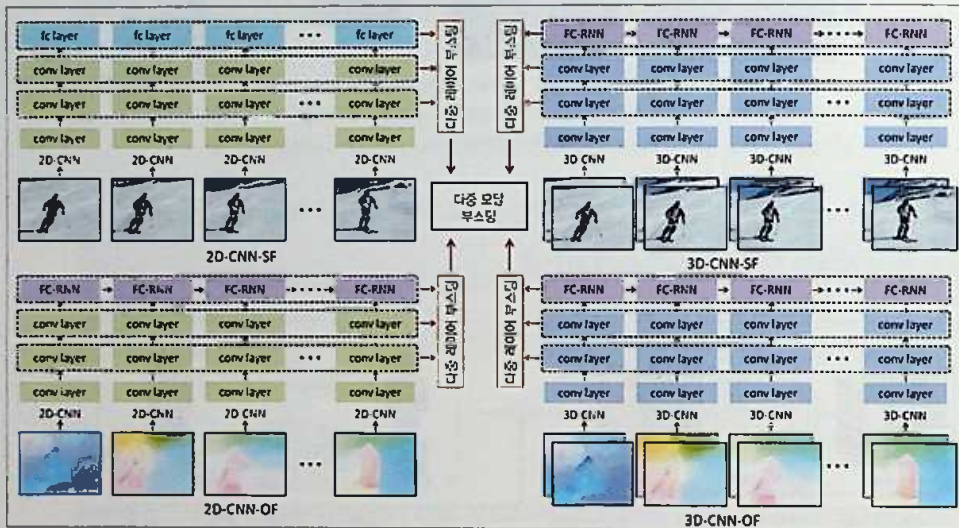
궤적과 특징 지도는 최종 예측 결과를 만들기 위해 일시적으로 합쳐질 수 있다.



## 다중 모드 융합

양Yang은 동영상 분류를 위한 네 가지 모델을 가진 다중 모드 융합 방법([http://research.nvidia.com/sites/default/files/pubs/2016-10\\_Multilayer-and-Multimodal/MM16.pdf](http://research.nvidia.com/sites/default/files/pubs/2016-10_Multilayer-and-Multimodal/MM16.pdf))을 제안했다. 네 가지 모델은 3D 컨볼루션 기능, 2D 광학 흐름, 3D 광학 흐름, 2D 컨볼루션 기능이다.

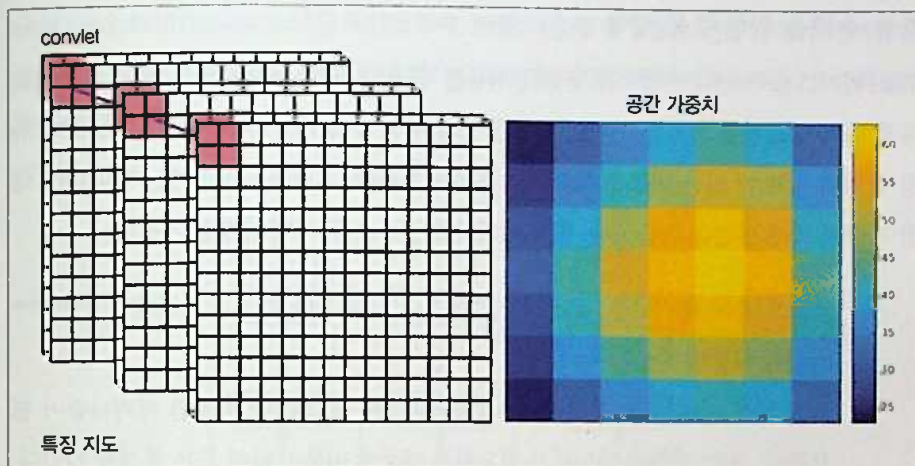
이 방법에서의 데이터 흐름은 다음과 같다.



양의 허락하에 재구성 게재

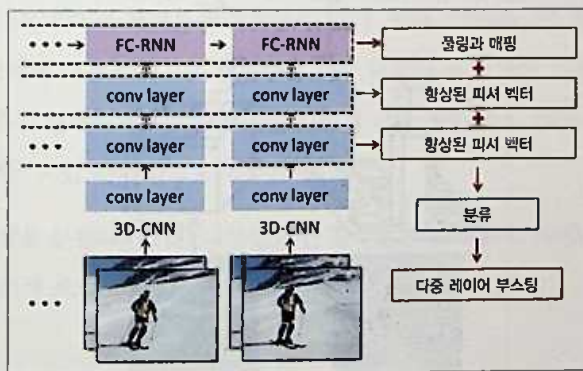
이제 Convlet에 대해 살펴보자. Convlet은 단일 커널을 사용해 작은 컨볼루션 출력을 얻는다. Convlet에 의한 컨볼루션 레이어의 공간 가중치 학습 방법은 다음 이미지에서 볼 수 있다.





양의 허락하에 재구성 계재

공간 가중치는 로컬 공간 영역이 컨볼루션 레이어에서 얼마나 구별될 수 있는지 또는 중요한지 나타낸다. 다음 그림은 다양한 컨볼루션 레이어와 완전 연결 레이어에서 수행되는 다중 레이어 표현의 융합 과정을 보여준다.



양의 허락하에 재구성 계재

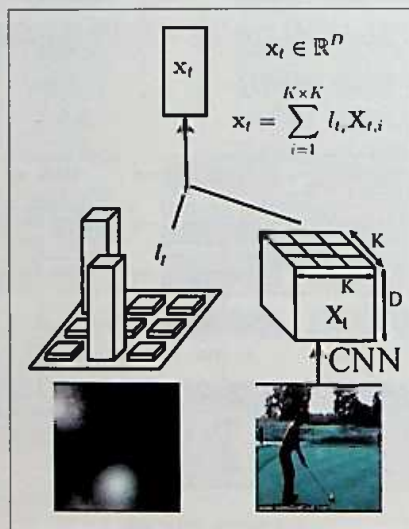
부스팅 메커니즘은 예측 결과를 결합하는 데 사용된다. 부스팅은 여러 모델 예측을 최종 예측으로 결합할 수 있는 메커니즘이다.

## 분류 수행을 위한 주목 영역

주목 메커니즘(attention mechanism)을 분류에 사용할 수 있다. 주목 메커니즘은 인식 활동에서 특정 영역에 초점을 맞추는 인간의 행동을 모방한다. 주목 메커니즘은 다른 영역보다 특정 영역에 대해 더 많은 비중을 부여하는 특성을 가진다. 훈련하는 동안에는 데이터에 대한 가중치 측정 방법을 학습한다. 주목 메커니즘은 주로 두 가지 유형을 가진다.

- 소프트 주목 메커니즘: 결정적 특성을 가지기 때문에 이것은 역전파(backpropagation) 방법을 사용해 학습할 수 있다.
- 하드 주목 메커니즘: 확률적 특성을 가지므로, 학습할 때 복잡한 메커니즘이 필요하다. 또한 샘플링 데이터가 필요하기 때문에 비용이 많이 드는 특성을 가진다.

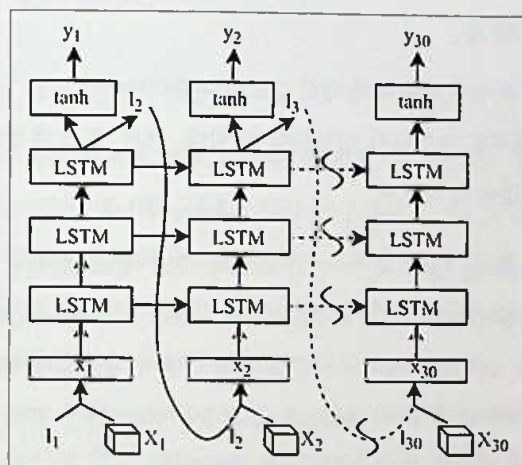
다음은 소프트 주목 방식을 시각화해 나타낸다.



샤르마의 허락하에 재구성 제재

주목 방식을 사용해 CNN 특징을 계산하고, 가중치도 적용된다. 특정 영역에 주어진 주목 또는 가중치 값은 시각화를 위해 사용될 수 있다. 샤르마(Sharma)는 이러한 아이디어(<https://>

arxiv.org/pdf/1511.04119.pdf)를 제안했으며, 이를 사용해 동영상상을 분류한다. LSTM은 컨볼루션 기능을 수행하고자 할 때 사용된다. LSTM은 다음 그림과 같이 다음 번 프레임에 주목해 영역을 예측한다.



사르마의 허락하에 재구성 게재

LSTM의 각 스택은 위치와 레이블을 예측할 수 있다. 모든 스택에는 세 개의 LSTM이 존재한다. LSTM 스택에 대한 입력으로 컨볼루션 특징 큐브와 위치 값이 사용된다. 위치에 대한 확률 값은 주목 영역(attention region)에 대한 가중치 값이 사용된다. 주목 영역을 사용하면 예측을 시각화할 수 있고, 정확도도 향상된다.

지금까지 동영상을 분류하기 위한 다양한 접근 방식을 살펴보았다. 이어서 동영상에 대한 다른 애플리케이션 프로그램들을 알아본다.

## Ⅰ 동영상에 대한 이미지 기반 접근법 확장

이미지는 포즈 추정(pose estimation), 스타일 전송, 이미지 생성, 분할, 캡션 등에서 사용될 수 있다. 마찬가지로, 이러한 애플리케이션 프로그램에서는 동영상도 사용할 수 있다. 임시

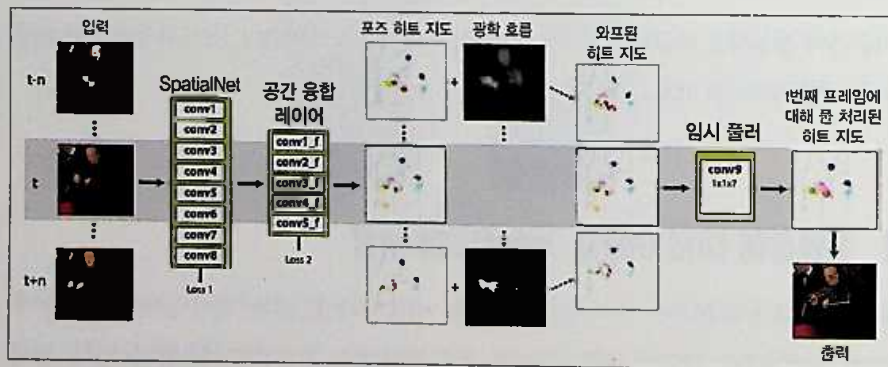
정보들을 사용하면 이미지의 예측을 향상시킬 수 있고, 그 반대의 경우도 발생할 수 있다. 이번 절에서는 이러한 애플리케이션 프로그램을 동영상으로 확장하는 방법을 살펴본다.

## 사람의 포즈도 적용하기

사람의 포즈 추정은 동영상 데이터에 대한 중요한 애플리케이션 프로그램이며, 동작 인식과 같은 다른 작업에서도 정확도를 향상시킬 수 있다. 먼저, 포즈 추정에 사용할 수 있는 데이터셋에 대한 설명을 살펴보자.

- 와일드 데이터셋 내의 포즈: 인간 포즈에 대해 애노테이션된 30개의 동영상을 포함한다. 데이터셋의 링크 주소는 <https://lear.inrialpes.fr/research/posesinthewild/>다. 데이터셋에서는 사람의 상체 관절이 애노테이션으로 표시된다.
- FLICFrames Labeled In Cinema: 30개의 영화에서 얻은 인간 포즈 데이터셋(<https://bensapp.github.io/flic-dataset.html>)을 포함한다.

프피스터Pfister는 동영상에서 사람이 어떤 포즈를 취했는지 예측하는 방법([https://www.cv-foundation.org/openaccess/content\\_iccv\\_2015/papers/Pfister\\_Flowing\\_ConvNets\\_for\\_ICCV\\_2015\\_paper.pdf](https://www.cv-foundation.org/openaccess/content_iccv_2015/papers/Pfister_Flowing_ConvNets_for_ICCV_2015_paper.pdf))을 제안했다. 다음 그림은 사람의 포즈를 회귀 분석하는 파이프라인을 보여준다.



프피스터의 허락하에 재구성 게재

동영상의 프레임들을 컨볼루션 네트워크에 통과시킨다. 레이어들은 융합되고, 포즈 히트 지도가 생성된다. 포즈에 대한 히트 지도는 광학 흐름과 결합돼 와프된(그림을 뒤집은) 히트 지도를 얻게 된다. 일정 시간에 걸쳐서 뒤틀린 히트 지도를 폴링하면 풀<sup>pool</sup>된 히트 지도를 얻을 수 있고 최종 포즈가 나온다.

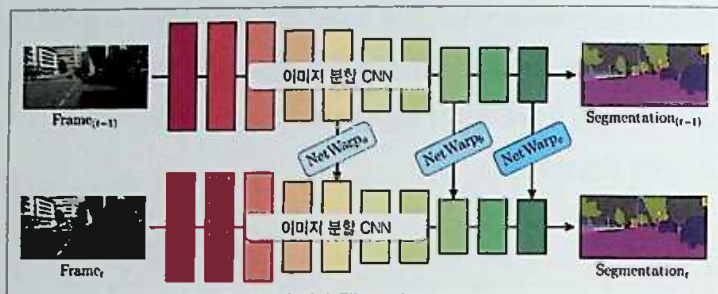
## 얼굴 표식 추적

동영상에서 얼굴 분석을 하려면 얼굴 검출, 표식 검출, 포즈 추정, 검증 등이 필요하다. 얼굴의 표식을 잡는 것은 애니메이션에서 얼굴을 구분하기, 인간-컴퓨터 상호작용, 인간 활동 인식을 수행하는 데 특히 중요하다. 여러 프레임들을 사용해 계산하는 대신에 동영상을 사용해 계산할 수 있다. 구<sup>Gu</sup>는 RNN을 사용해 동영상에서 얼굴 표식을 검출하고 추적하기 위해 공동 추정을 사용하는 방법([http://research.nvidia.com/sites/default/files/pubs/2017-07\\_Dynamic-Facial-Analysis/rnnface.pdf](http://research.nvidia.com/sites/default/files/pubs/2017-07_Dynamic-Facial-Analysis/rnnface.pdf))을 제안했다. 이에 따라 프레임에서 정확한 예측을 할 수 있게 됐고, 다른 이전 모델보다 우수한 결과를 얻을 수 있다. 표식은 CNN에 의해 계산되고 임의의 특성<sup>temporal aspect</sup> 값은 RNN으로 인코딩된다. 합성 데이터가 훈련에서 사용된다.

## 동영상 분할

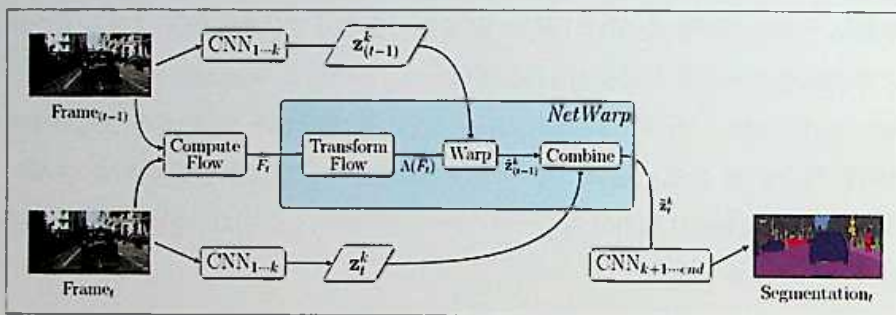
시간적 정보를 사용하면, 동영상을 더 나은 방법으로 분할할 수 있다. 개드<sup>Gadde</sup>는 워핑을 사용해 시간 정보를 결합하는 방법([https://ps.is.tuebingen.mpg.de/uploads\\_file/attachment/attachment/386/gadde2017videocnns.pdf](https://ps.is.tuebingen.mpg.de/uploads_file/attachment/attachment/386/gadde2017videocnns.pdf))을 제안했다. 다음 그림은 2 프레임을 분할하고 워핑을 결합하는 솔루션을 보여준다.





개드의 허락하에 재구성 게재

워핑 네트는 다음 그림에서 볼 수 있다.



개드의 허락하에 재구성 게재

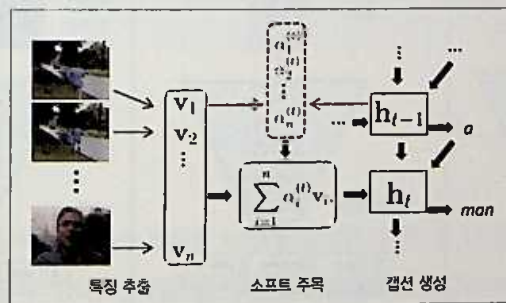
광학 흐름은 워핑과 결합된 두 프레임 사이에서 계산된다. 워핑 모듈은 광학 흐름을 가져와서 변환하고 warped 표현 값과 결합한다.

## 동영상 캡션

7장. '이미지 캡션'에서는 텍스트와 이미지를 결합하는 몇 가지 방법을 설명했다. 마찬가지로 동영상 캡션을 생성해 어떤 내용을 가지고 있는지 설명할 수 있다. 캡션을 가지는 동영상 데이터셋 목록은 다음과 같다.

- MSR-VTT<sup>Microsoft Research - Video To Text</sup>는 200,000개의 동영상 클립과 문장 쌍을 갖고 있다. 자세한 내용은 <https://www.microsoft.com/en-us/research/publication/msr-vtt-a-large-video-description-dataset-for-bridging-video-and-language/>에서 확인할 수 있다.
- MPII-MD<sup>MPII Movie Description Corpus</sup>는 <https://www.mpi-inf.mpg.de/departments/computer-vision-and-multimodalcomputing/research/vision-and-language/mpii-movie-description-dataset>에서 구할 수 있다. 여기에는 94개의 영화와 68,000개의 문장이 있다.
- M-VAD<sup>Montreal Video Annotation Dataset</sup>는 <https://mila.quebec/en/publications/public-datasets/m-vad/>에서 얻을 수 있으며 49,000개의 짧은 동영상이다.
- YouTube2Text에는 1,970개의 동영상과 80,000개의 설명이 있다.

야호<sup>Yao</sup>는 동영상 캡션 작성 방법([https://www.cv-foundation.org/openaccess/content\\_iccv\\_2015/papers/Yao\\_Describing\\_Videos\\_by\\_ICCV\\_2015\\_paper.pdf](https://www.cv-foundation.org/openaccess/content_iccv_2015/papers/Yao_Describing_Videos_by_ICCV_2015_paper.pdf))을 제안했다. 행동 인식을 위해 훈련된 3D 컨볼루션 네트워크가 로컬 시간 특징을 검출하는 데 사용된다. 그리고 나서 RNN을 사용해 텍스트를 생성하기 위해 주목 메커니즘을 사용한다. 이와 관련된 프로세스는 다음 그림과 같다.

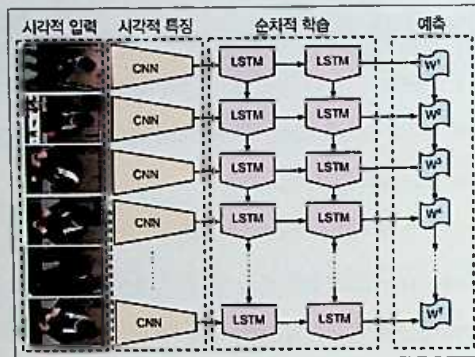


야호의 허락하에 재구성 게재

도나휴<sup>Donahue</sup>는 컨볼루션 기능을 가진 LSTM을 사용하는 동영상 캡션 또는 서술을 위한

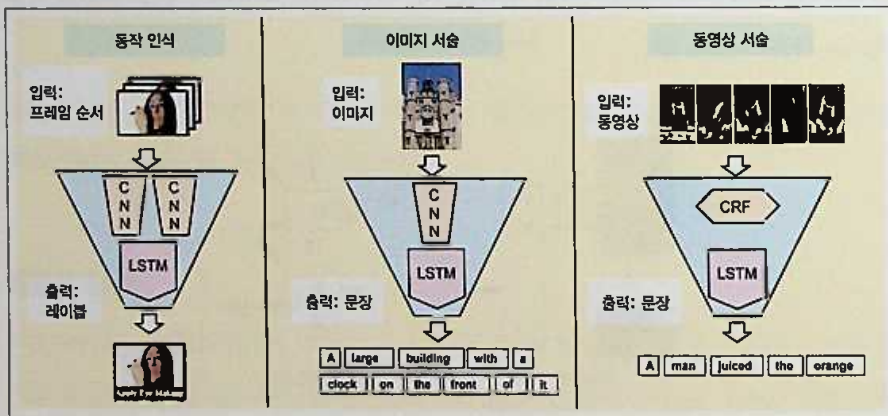
또 다른 방법([https://www.cv-foundation.org/openaccess/content\\_cvpr\\_2015/papers/Donahue\\_Long-Term\\_Recurrent\\_Convolutional\\_2015\\_CVPR\\_paper.pdf](https://www.cv-foundation.org/openaccess/content_cvpr_2015/papers/Donahue_Long-Term_Recurrent_Convolutional_2015_CVPR_paper.pdf))을 제안했다.

이는 다음과 같이 2D 컨볼루션 기능을 사용하는 점만 제외하면 이전 방법과 유사하다.



도나휴의 허락하에 재구성 게재

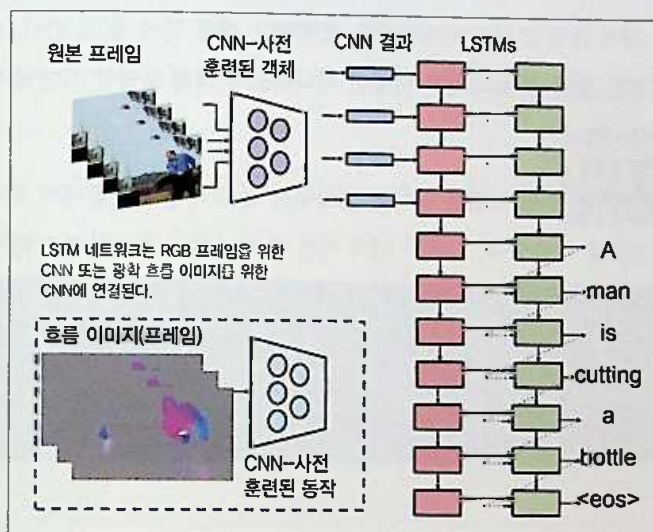
구글은 동작 인식, 이미지 서술, 동영상 서술 기법 등 텍스트와 이미지를 결합하는 여러 가지 방법을 보유했다. 다음은 이러한 기술을 나타낸다.



도나휴의 허락하에 재구성 게재

베뉴고팔란Venugopalan은 인코더-디코더 방식을 이용한 동영상 캡션 방법(<https://www>.

cv-foundation.org/openaccess/content\_iccv\_2015/papers/Venugopalan\_Sequence\_to\_Sequence\_ICCV\_2015\_paper.pdf)을 제안했다. 다음 그림은 그가 제안한 기법을 시각화한 것이다.



베누고팔란의 허락하에 재구성 게재



이러한 방법을 위해 CNN은 이미지의 광학 흐름이나 프레임에 대해 계산한다.

## 동영상 생성

동영상은 비지도 방식으로 생성 모델을 사용해 생성할 수 있다. 미래 프레임은 현재 프레임을 사용해 예측할 수 있다. 란자초 Ranzato는 언어 모델에서 영감을 받아 동영상을 생성하는 방법(<https://arxiv.org/pdf/1412.6604.pdf>)을 제안했다.

RNN 모델은 이미지의 패치를 가져와서 다음 패치를 예측하는 데 사용된다.

## ■ 요약

이 장에서는 동영상 분류와 관련된 다양한 주제를 다뤘다. 동영상을 프레임으로 분할하고 다양한 작업을 위해 이미지에 있는 딥러닝 모델을 사용하는 방법을 살펴봤고, 추적 객체와 같은 몇 가지 동영상 관련 알고리즘을 설명했다. 행동 인식, 몸짓 인식, 보안 애플리케이션 프로그램, 침입 검출과 같은 다양한 시나리오에 대해 동영상 기반 솔루션을 적용하는 방법을 살펴봤다.

10장에서는 9장의 숙련된 모델을 다양한 클라우드 및 모바일 플랫폼에서 프로덕션 형태로 배포하는 방법을 학습한다. 그리고 대기 시간 및 처리량과 관련해 하드웨어가 서로 다른 성능에 어떤 영향을 미치는지 살펴본다.



## 배포

이번 장에서는 최대 처리량과 최소 대기 시간을 달성하기 위해 다양한 플랫폼에 훈련된 모델을 배치하는 방법을 살펴본다. 이를 위해 GPU와 CPU 같은 다양한 하드웨어의 성능을 이해하고, 아마존 웹 서비스<sup>AWS</sup>, 구글 클라우드 플랫폼<sup>GCP</sup>과 안드로이드, iOS, 테그라<sup>Tegra</sup> 등의 모바일 플랫폼에 텐서플로를 배치하는 과정을 살펴본다.

이번 장에서 다루는 내용은 다음과 같다.

- 딥러닝 모델 훈련 및 추론<sup>inference</sup>의 성능에 영향을 미치는 요인 이해
- 성능 향상을 위한 다양한 방법
- 다양한 하드웨어의 벤치마크 결과를 보고 성능을 극대화하기 위해 조정하는 과정
- 배포를 위한 다양한 클라우드 플랫폼 사용
- 다양한 모바일 플랫폼을 사용한 배포

## I 모델 성능

성능은 딥러닝 모델을 훈련하거나 배포할 때 모두 중요하다. 훈련은 대용량 데이터 또는 대형 모델 아키텍처로 인해 더 많은 시간이 일반적으로 소요된다. 결과 모델(resultant model)은 훈련한 데이터보다 더 큰 크기일 수 있고, RAM 메모리에 제약이 있는 모바일 장치에서 사용할 때 문제가 될 수 있다. 컴퓨팅 시간이 늘어날수록 인프라 비용은 증가한다. 추론 시간은 동영상 애플리케이션에서 매우 중요하다. 앞서 언급한 성능의 중요성 때문에 이번 절에서는 성능을 향상시키는 방법을 주로 살펴본다. 모델의 복잡성을 낮추기는 쉽지만, 정확도도 함께 떨어지게 된다. 여기서는 정확도를 심각하게 떨어뜨리지 않으면서 성능을 향상시키는 방법에 중점을 둔다. 다음 절에서는 양자화(quantization) 옵션을 설명한다.

### 모델 양자화

딥러닝 모델에서 사용되는 가중치 값은 32비트 부동 소수점 값이다. 가중치가 8비트로 양자화될 때 정확도가 덜 떨어지기 때문에 배포할 때 눈치채지 못할 수 있다. 이러한 사항은 결과 값에서 가중치의 정밀도가 딥러닝 모델의 정확도 성능에 미치는 영향이 적음을 나타낸다. 이러한 내용은 딥러닝을 사용할 때 고려해볼 만하며, 모델 크기가 결정적일 때 유용할 수 있다. 32비트 부동 소수점 값을 8비트 값으로 바꾸면 모델 크기가 크게 줄어들고 추론 속도가 빨라질 수 있다. 따라서 모델을 양자화할 때 많은 옵션을 사용할 수 있다. 가중치는 8비트로 저장할 수 있지만, 추론 연산은 32비트 부동 소수점 값으로 수행할 수 있다. 아키텍처의 모든 구성 요소는 양자화 크기에 따라 다르게 동작할 수 있으므로 레이어에 따라 32비트, 16비트, 8비트 값 중에서 선택할 수 있다.

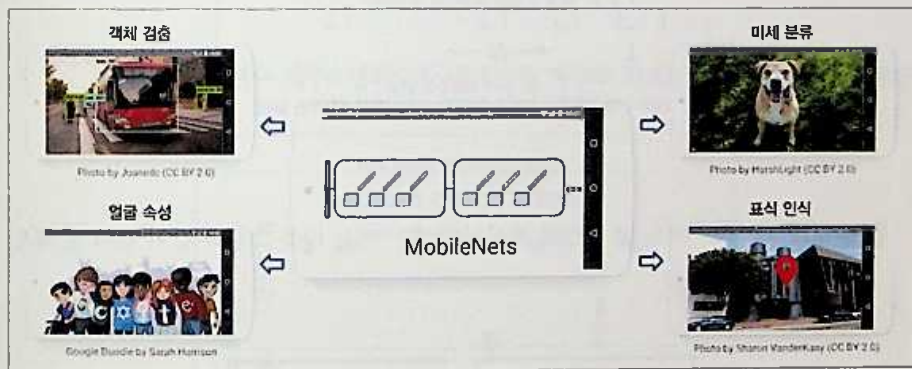
양자화는 여러 가지 이유로 사용된다. 일반적으로 딥러닝 모델은 이미지가 가지고 있는 노이즈를 줄이기 위해 훈련되므로 강건한 특성을 가진다. 추론 계산에는 중복 정보가 있을 수 있으며, 중복 정보는 양자화로 인해 제거될 수 있다.

최신 CPU 및 RAM 하드웨어는 부동 소수점 계산을 위해 이미 최적화돼 있으므로, 이러한

하드웨어에서는 양자화 효과가 눈에 띄게 나타나지 않을 수 있다. 따라서 이러한 시나리오 오는 점점 더 많은 하드웨어가 도입됨에 따라 변화하고 있다. 메모리와 속도에서의 차이점은 GPU에서 정밀도가 낮은 부동 소수점 작업을 할 때 확실히 알 수 있다. 정밀도가 낮은 부동 소수점 작업을 할 때는 다른 특별한 하드웨어를 사용할 수 있다.

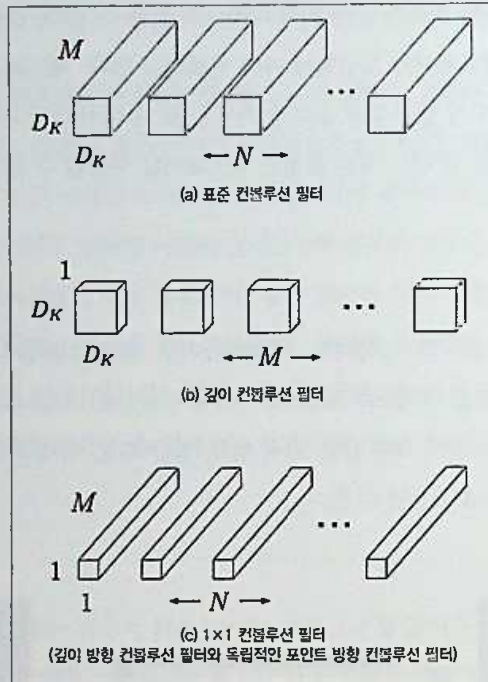
## MobileNets

하워드Howard는 모바일 및 임베디드 애플리케이션 프로그램에서 사용할 수 있는 MobileNets라는 새로운 클래스의 모델(<https://arxiv.org/pdf/1704.04861.pdf>)을 도입했다. MobileNets는 다음과 같이 객체 검출, 표식 인식, 얼굴 속성, 미세 분류와 같은 다양한 애플리케이션 프로그램에 사용할 수 있다.



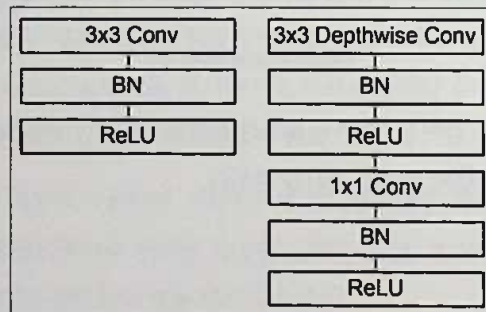
하워드의 허락하에 재구성 게재

MobileNets는 다음과 같이 (a) 표준 컨볼루션 필터를 (b) 깊이 방향 및 (c) 포인트 방향 컨볼루션으로 대체해 모델의 크기와 계산을 줄인다.



하위드의 허락하에 재구성 게재

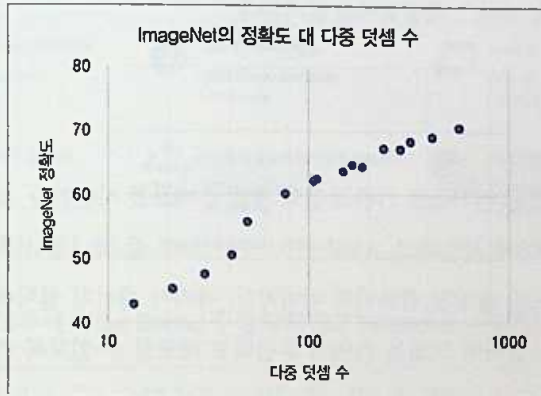
일괄 정규화(batch normalization) 및 활성화 레이어(activation layer)는 다음 그림과 같이 깊이 및 포인트 컨볼루션에 추가된다.



하위드의 허락하에 재구성 게재

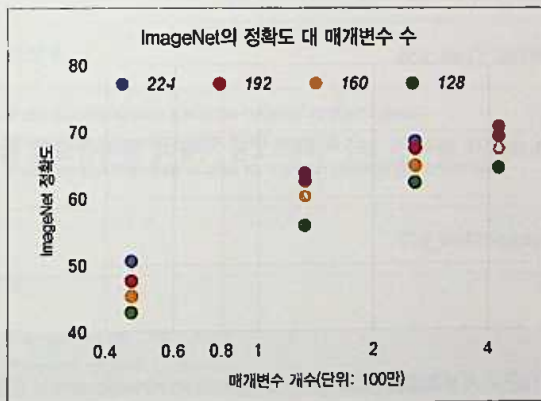
모델 선택에 영향을 주는 매개변수는 두 가지다.

- 곱셈과 덧셈의 개수: 정확도와 다중 덧셈 사이의 절충 값은 다음과 같다.



하위드의 허락하에 재구성 계재

- 모델의 매개변수 개수: 정확도와 매개변수 개수의 트레이드오프 관계는 다음과 같다.



하위드의 허락하에 재구성 계재

MobileNets는 모바일 및 임베디드 장치에 사용되는 정확성을 약간 낮춤으로써 모델 계산 및 크기를 줄일 수 있음을 보여준다. 모델과 정확도 간의 트레이드오프 관계는 하위드가 보여준 그림에서 확인할 수 있다.



## ■ 클라우드에서 배포하기

여러 애플리케이션 프로그램에서 사용할 모델들은 클라우드를 통해 배포해야 한다. 이를 위해 주요 클라우드 서비스 제공업체를 살펴본다.

### AWS

아마존 웹 서비스AWS는 텐서플로 기반 모델의 개발 및 배포를 지원한다. <https://aws.amazon.com/>에서 AWS에 가입하고, AMIAmazon Machine Images 중 하나를 선택한다. AMI는 필수 소프트웨어가 모두 설치된 컴퓨터의 이미지다. 따라서 패키지 설치에 대해 걱정할 필요가 없다. AWS는 딥러닝 모델을 손쉽게 훈련하고 배포할 수 있도록 DLAMIDeep Learning AMI를 제공한다. 여기서 선택할 수 있는 몇 가지 옵션이 있다. Conda는 텐서플로를 실행하는 데 필요한 여러 패키지를 함께 제공한다. 파이썬의 경우 버전 2와 3이라는 두 가지 옵션이 있다. 다음 코드는 CUDA 8에서 파이썬 3와 케라스Keras 2를 사용해 텐서플로를 활성화한다.

```
source activate tensorflow_p36
```

다음 코드는 CUDA 8에서 파이썬 2와 케라스 2를 사용해 텐서플로를 활성화한다.

```
source activate tensorflow_p27
```



더 자세한 내용과 튜토리얼은 <https://aws.amazon.com/tensorflow/>에서 얻을 수 있다.

가상 머신VM은 다음 단계를 통해 시작할 수 있다.

1. [aws.amazon.com](https://aws.amazon.com/)으로 이동한 후 아마존Amazon 계정을 사용해 로그인한다.

## 2. 로그인 페이지에서 Launch a virtual machine을 선택한다.

### Build a solution

Get started with simple wizards and automated workflows.

|                                                                                                                                                        |                                                                                                                                                 |                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <br>Launch a virtual machine<br>With EC2 or Lightsail<br>~1-2 minutes | <br>Build a web app<br>With Elastic Beanstalk<br>~6 minutes    | <br>Host a static website<br>With S3, CloudFront, Route 53<br>~5 minutes |
| <br>Connect an IoT device<br>With AWS IoT<br>~5 minutes               | <br>Start a development project<br>With CodeStar<br>~5 minutes | <br>Register a domain<br>With Route 53<br>~3 minutes                     |

## 3. 다음 윈도우에서 Get Started를 클릭해 EC2 Instance를 선택한다.



### EC2 Instance

For users with cloud experience who need a flexible and scalable virtual machine\*

#### Why EC2 ?

- Fully customizable Instance tailored to your needs.
- Seamless integration with AWS services.
- Flexible solution that scales to support changing workloads.

#### Pricing

- Pay as you go. [Learn more](#)
- Free-tier eligible. [Learn more](#)

[Get started](#)

\*This wizard creates an EC2 t2.micro instance with default configurations. For more options, use the EC2 launch instance wizard.

4. EC2 Instance의 이름을 지정한다.

Name your EC2 instance

This is how you will identify your instance in AWS console. Choose a name that is easy for you to remember.

tensorflow

Use this name

5. 운영체제(Operating System) 타입을 선택한다.

Select an Operating System



Ubuntu Server 16.04 LTS

Don't see the OS you are looking for? AWS offers additional options through the advanced EC2 Launch Instance wizard or you can explore the AWS Marketplace.

Next

6. 인스턴스<sup>Instance</sup> 타입을 선택한다. 인스턴스 타입은 다양한 RAM 크기와 CPU의 구성 방법을 나타낸다. GPU에 대해서도 선택할 수 있는 몇 가지 옵션이 있다. 인스턴스 유형을 선택하고 Next 버튼을 클릭한다.

Select an instance type

t2  
t2.micro

...  
more options

t2.micro

1 Core vCPU (up to 3.3 GHz), 1 GiB Memory RAM, 8 GB Storage **FREE TIER ELIGIBLE**

Need a different instance type? AWS offers additional options through the advanced EC2 Launch Instance wizard.

Next

7. 다음과 같이 로그인하는 데 사용되는 개인정보 보호 PEM<sup>Privacy Enhanced Mail Security</sup> Certificate 파일을 만든다.

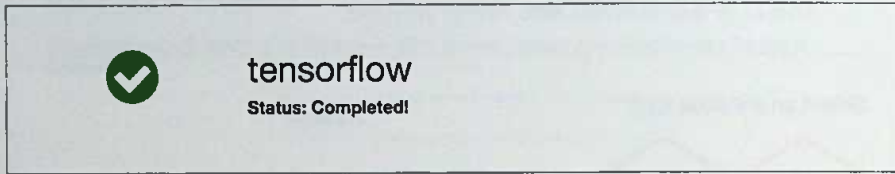
Create a key pair

Amazon EC2 secures your instance using a key pair. In this step you will download the private key to your computer.

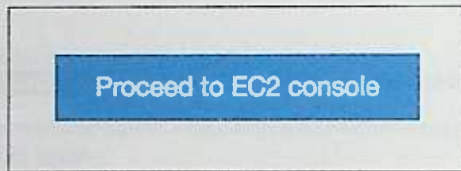
Save it in a safe place and use it when you connect to your instance.

tensorflow|

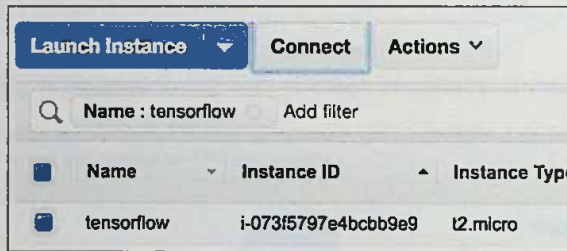
8. 인스턴스를 생성하는 데 다소 시간이 걸릴 것이며, 일정 시간이 지나면 완료 상태가 표시된다.



9. 이어서 Proceed to EC2 console 버튼을 클릭한다.



10. 이제 인스턴스가 생성됐다. 다음 그림과 같이 Connect 단추를 클릭한다.



11. 그리고 나서 인스턴스를 가상 시스템의 명령 프롬프트에 연결한다. 연결하는 데 필요한 지시 사항이 이 단계에서 제공된다. 이전 단계에서 다운로드한 pem 파일이 필요하다. 표시된 지침에 따라 시스템에 연결한다.



## Connect To Your Instance

I would like to connect with

A standalone SSH client

A Java SSH Client directly from my browser (Java required)

To access your instance:

1. Open an SSH client. (find out how to connect using PuTTY)
2. Locate your private key file (tensorflow1.pem). The wizard automatically detects the key you used to launch the instance.
3. Your key must not be publicly viewable for SSH to work. Use this command if needed:
 

```
chmod 400 tensorflow1.pem
```
4. Connect to your instance using its Public DNS:
 

```
ec2-35-165-164-121.us-west-2.compute.amazonaws.com
```

**Example:**

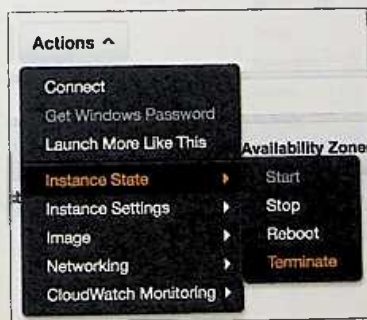
```
ssh -i "tensorflow1.pem" ubuntu@ec2-35-165-164-121.us-west-2.compute.amazonaws.com
```

Please note that in most cases the username above will be correct, however please ensure that you read your AMI usage instructions to ensure that the AMI owner has not changed the default AMI username.

If you need any assistance connecting to your instance, please see our connection documentation.

Close

12. 작업을 마치면 Action > Instance State > Terminate를 클릭해 인스턴스를 완료한다.



설치 및 실행 단계를 수행하려면 '1장, 시작하기'를 참조한다.

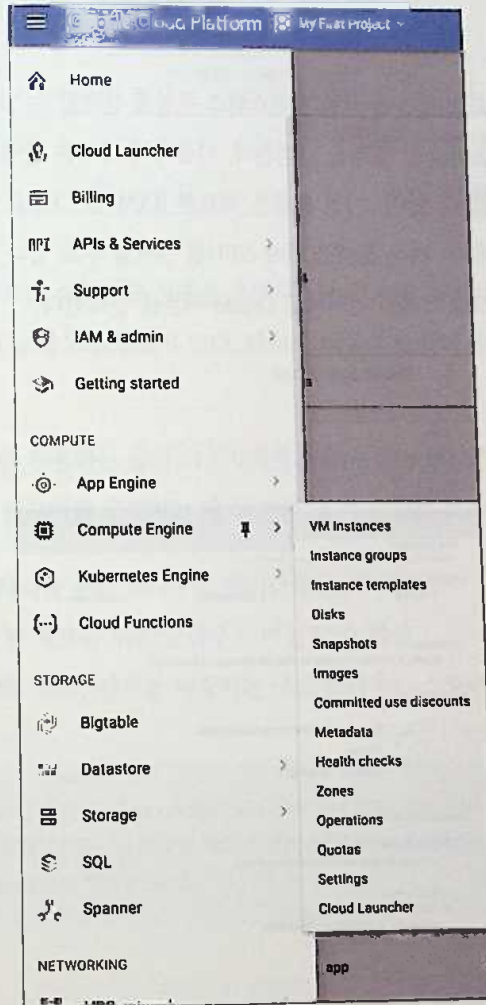
## 구글 클라우드 플랫폼

구글 클라우드 플랫폼<sup>GCP</sup>은 구글에서 제공하는 클라우드 플랫폼이며 AWS와 비슷한 기능을 제공한다. 간단한 가상 머신을 사용하면 AWS와 같은 모델을 다음 단계에 따라 훈련할 수 있다.

1. cloud.google.com을 사용해 구글 클라우드 플랫폼으로 이동한 후 지메일<sup>Gmail</sup> 계정을 사용해 플랫폼에 로그인한다.
2. 이제 GO TO CONSOLE 버튼을 클릭해 콘솔로 이동한다.



3. 콘솔로 이동한 후 다음 스크린샷과 같이 오른쪽 상단 메뉴에서 Compute Engine > VM Instance를 클릭해 VM 만들기<sup>creation</sup> 페이지로 이동한다.



4. 그리고 나서 CREATE INSTANCE 버튼을 클릭해 필요한 인스턴스를 만든다.



CREATE INSTANCE

5. 이후에는 설정 메뉴를 사용해 인스턴스 유형을 선택할 수 있다. 영역 매개변수는 인스턴스가 배포될 영역을 알려준다. 사용자에게 가까운 영역을 선택하면 대기 시간을 절감할 수 있다. 머신 유형은 필요한 RAM 및 CPU를 의미하며, 사용자가 정의할 수 있다. 빠른 훈련을 위해 GPU를 선택할 수도 있다. 다음 스크린샷과 같이 인스턴스의 크기를 선택하고 Create 버튼을 클릭한다.

← Create an instance

Name ⓘ  
instance-2

Zone ⓘ  
us-east1-b

Machine type ⓘ  
Customize to select cores, memory and GPUs.  
2 vCPUs 7.5 GB memory Customize

Container ⓘ  
Deploy a container image to this VM instance. Learn more

Boot disk ⓘ  
New 10 GB standard persistent disk  
image  
Ubuntu 16.04 LTS Change

Identity and API access ⓘ  
Service account ⓘ  
Compute Engine default service account

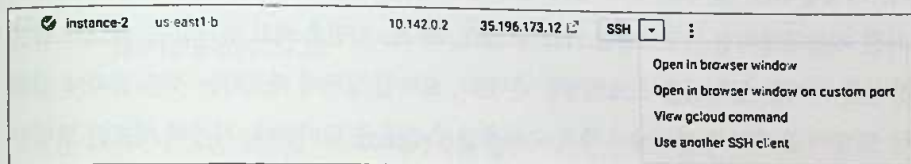
Access scopes ⓘ  
● Allow default access  
✓ Allow full access to all Cloud APIs  
Set access for each API

Firewall ⓘ  
Add tags and firewall rules to allow specific network traffic from the Internet  
✓ Allow HTTP traffic  
☑ Allow HTTPS traffic  
Management, disks, networking, SSH keys

You will be billed for this instance. Learn more

Create Cancel

6. 인스턴스를 만드는 데는 몇 분이 걸린다. 이어서 인스턴스에 대한 SSH 드롭다운 목록을 클릭하고, 여기에 표시된 대로 Open in browser window 옵션을 선택해 브라우저에서 콘솔을 연다.



셀을 사용해 텐서플로를 설치하고 모델을 훈련하거나 배포할 수 있다. 가상 시스템 구성 상에서 선택할 수 있는 많은 옵션들이 있다. 따라서 비용과 시간의 절충에 따라 구성을 선택할 수 있다.

GCP에는 텐서플로를 사용하는 동안 사용자에게 도움이 되는 클라우드 머신 학습 엔진이 있다. 훈련 및 배포 인프라를 구성하는 데 GCP의 세 가지 구성 요소를 사용할 수 있다.

1. 이미지 전처리를 위한 클라우드 데이터플로우(Cloud DataFlow)
2. 모델 훈련 및 배포를 위한 클라우드 머신 학습 엔진
3. 훈련 데이터, 코드, 결과를 저장하는 구글 클라우드 스토리지(Google Cloud Storage)



클라우드 머신 학습 엔진(Cloud Machine Learning Engine)을 사용해 사용자 지정된 이미지 분류 모델을 만드는 데 유용한 학습 자료는 <https://cloud.google.com/ml-engine/docs/flowers-tutorial>에서 찾을 수 있다.



## ■ 장치에 모델 배포하기

텐서플로 모델은 모바일 장치에도 배포할 수 있다. 모바일 장치에는 스마트폰, 무인 항공기, 가정용 로봇 등이 있다. 수십억 대의 스마트폰은 심층적인 학습을 할 수 있는 컴퓨터 비전 애플리케이션 프로그램을 가질 수 있다. 하나는 사진을 찍고 검색하며, 태그가 붙은 장면을 가지는 동영상상을 스트리밍할 수 있다. 모바일 장치에 배포하는 것은 장치에 딥러닝 모델이 존재하고 장치에서 추론이 이뤄질 수 있음을 의미한다. 기기에 배포된 모델은 개인정보 보호 문제를 해결하는 데 도움이 된다. 다음 주제에서는 다양한 모바일 플랫폼에 배포하는 방법을 설명한다.

### Jetson TX2

Jetson TX2는 AI 컴퓨팅에 매우 효과적인 내장 장치며, 엔비디아에서 공급한다. Jetson TX2는 가볍고 소형이므로 드론, 공공 장소 등에 배치하기에 적합하며, 사전 설치된 TensorRT를 제공한다. TensorRT는 텐서플로를 위한 런타임이다. 텐서플로를 설치하기 전에 Jetson과 플래시Flash를 설치하고 우분투Ubuntu, CUDA, CUDNN을 설치할 수 있다.

<https://github.com/jetsonhacks/installTensorFlowTX2>를 복사하고, 명령 프롬프트에서 다음과 같이 명령을 입력한다.

1. 먼저 다음 코드를 사용해 필수 구성 요소를 설치한다.

```
./installPrerequisites.sh
```

2. 이제 다음 코드를 사용해 텐서플로를 복제한다.

```
./cloneTensorFlow.sh
```

3. 다음 코드를 사용해 필요한 환경 변수를 설정한다.

```
./setTensorFlowEV.sh
```

4. 다음 코드를 사용해 텐서플로를 빌드한다.

```
./buildTensorFlow.sh
```

5. 이어서 다음 코드를 사용해 패키지를 <sup>wheel</sup> 파일로 처리한다.

```
./packageTensorFlow.sh
```

6. 이제 다음 코드를 사용해 텐서플로를 설치한다.

```
pip install $HOME / tensorflow-1.0.1-cp27-cp27mu-linux_aarch64.whl
```

이러한 단계들을 통해 Jetson TX2에 텐서플로를 설치할 수 있다.

## 안드로이드

모든 안드로이드 앱에서 텐서플로를 사용할 수 있으며 빌드 세부 정보를 알려주는 링크는 [https://www.tensorflow.org/mobile/android\\_build](https://www.tensorflow.org/mobile/android_build)다. 안드로이드에 대한 공식 예제는 <https://github.com/tensorflow/tensorflow/tree/master/tensorflow/examples/android>에서 찾을 수 있다. 안드로이드 기기에서 텐서플로를 구현하는 단계는 안드로이드 프로그래밍 경험이 있다고 가정하면 다음과 같다.

1. 3장, '이미지 검색'에 설명된 단계를 사용해 텐서플로 모델을 .pb 파일로 내보낸다.
2. .so 및 .jar 바이너리 파일을 빌드한다.
3. 라이브러리를 불러올 수 있도록 gradle 파일을 편집한다.

#### 4. 안드로이드 앱을 불러와서 실행한다.

### 아이폰

애플은 CoreML 프레임워크를 사용해 아이폰 애플리케이션 프로그램에서 사용하는 머신러닝을 통합했으며, 애플리케이션 프로그램에 직접 통합할 수 있는 표준 모델 목록을 제공한다. 텐서플로를 사용해 맞춤형 딥러닝 모델을 훈련하고, 이를 아이폰에서 사용할 수 있다. 커스텀 모델을 배치하려면 텐서플로를 CoreML 프레임워크 모델로 변환해야 한다. 구글은 텐서플로 모델을 CoreML 모델로 변환하기 위한 <https://github.com/tf-coreml/tf-coreml>을 릴리스했다. TFcoreML은 다음 코드를 사용해 설치할 수 있다.

---

```
pip install -U tfcoreml
```

---

다음 코드를 사용해 모델을 내보낼 수 있다.

---

```
import tfcoreml as tf_converter
tf_converter.convert(tf_model_path='tf_model_path.pb',
                    mlmodel_path='mlmodel_path.mlmodel',
                    output_feature_names=['softmax:0'],
                    input_name_shape_dict={'input:0': [1, 227, 227, 3]})
```

---

모델을 내보내는 동작은 아이폰에서 예측하기 위해 사용할 수 있다.

## ■ 요약

이 장에서는 다양한 플랫폼과 장치에서 훈련된 딥러닝 모델을 배포하는 방법을 살펴보았다. 그리고 이 플랫폼을 위한 최고의 성능을 얻는 방법에 관련된 지침과 단계를 설명했다. 정확도와 추론 시간은 약간의 트레이드오프 관계를 가지고 있음을 확인했으며, MobileNets의 장점도 살펴보았다.





## | 찾아보기 |

### ㄱ

가상 머신 318  
 가우스 필터 276  
 가우시안 필터 271  
 간격 컨볼루션 183  
 감소 차원 컨볼루션 192  
 개발 환경 설정 48  
 객체 검출 147, 160  
 객체 검출 API 163  
 객체 검출 모델 재훈련 167  
 객체 검출 훈련 파이프라인 168  
 거칠기 186  
 검색 파이프라인 133  
 검출 44, 148  
 결과 모델 314  
 결합된 목표 250  
 경계 박스 156  
 경계 박스 회귀 변수 162  
 경계 분할 네트워크 190  
 경사하강법 36  
 곱셈과 덧셈의 개수 317  
 공간 가중치 303  
 공간 변환 네트워크 95  
 공간 불변성 95  
 관심 네트워크 245  
 관심 메커니즘 245  
 관심 사항 235  
 관심을 기울이는 방법 245  
 관심 풀링 영역 161

광학 흐름 298  
 광학 흐름 방법 293  
 교차 엔트로피 34, 65  
 교차 엔트로피 계산 72  
 구글 클라우드 스토리지 327  
 구글 클라우드 플랫폼 324  
 구멍 컨볼루션 181, 183  
 궤적 300  
 그림 행렬 269  
 근사 최근접 이웃 145  
 근처 유사 단어 231  
 기준점 215  
 기준점 검출 157  
 기하학 검증 132

### ㄴ

노이즈 제거 자동 인코더 145  
 누설 합성화 277  
 느린 융합 295  
 느슨한 재구성 274  
 늦은 융합 295

### ㄷ

다중 레이어 컨볼루션 신경망 68  
 다중 모달 공간 244  
 다중 모달 측정 항목 공간 243  
 다중 모드 융합 302  
 다중 스플라인 방법 268  
 다중 인덱스 해싱 132  
 다중 클래스 81

다층 퍼셉트론 33  
 대조 손실 205  
 대형 커널 190  
 대화형 이미지 254  
 더 매끄러운 기울기 96  
 데이터셋 179  
 데이터 확장 100  
 동영상 분류 287  
 동영상 분류 데이터셋 탐색 288  
 동영상 분류 접근법 292  
 동영상 분할 307  
 동영상 생성 311  
 동영상의 다음 프레임 예측 방법 253  
 동영상 캡션 308  
 동영상 캡션 작성 방법 309  
 드롭아웃 34  
 드롭아웃 레이어 72, 207  
 디컨볼루션 181, 183  
 딥러닝 28

## ㄹ

로지스틱 분류 모델 102  
 로짓 65, 72  
 로컬라이제이션 44, 147

## ㄴ

마이크로소프트 ResNet-50 모델 93  
 모델 32  
 모델 구축하기 81  
 모델 내보내기 127  
 모델 양자화 314  
 모델의 매개변수 개수 317  
 모델 추론 127  
 모델 훈련하기 169

모션 벡터 통합 287  
 몸짓 인식 312  
 무작위 잘라내기 100  
 무차별 대입 방법 137  
 미세 조정 104  
 밑집 레이어 70, 155, 278  
 밑집 캡션 241

## ㄷ

바닐라 GAN 277  
 바닐라 확률적 경사하강법 88  
 바이오메트릭스 204  
 박스 병합 159  
 박스 회귀 변수 162  
 반려 동물 이미지 데이터 97  
 배치 정규화 35  
 배치 정규화 레이어 279  
 배포 313  
 벡터 모델 229  
 벡터 산술 231  
 변수 요약 함수 69  
 병목 특징 102, 134  
 병목 현상 레이어 139, 206  
 보간 187  
 보안 애플리케이션 프로그램 312  
 보행자 검출 모델 173  
 부분적 간격 컨볼루션 183  
 부스팅 메커니즘 303  
 분류 동작 300  
 분류 수행 304  
 분류 시그니처 128  
 분할 45, 305  
 분할 데이터셋 175

불필요 제거 257  
 브랜드 안정성 108  
 블렌딩 258  
 블루 160

## ㅅ

사전 훈련된 모델 165  
 새 애니메이션 캐릭터 만들기 261  
 생성기 285  
 생성 모델 46, 251, 259  
 생성 모델의 애플리케이션 252  
 삼 네트워크 204, 206, 227  
 서버샘플링 264  
 선택 검색 162  
 선형 스캔 145  
 소프트맥스 34, 66, 81  
 소프트맥스 분류기 233  
 소프트맥스 활성화 방법 199  
 소프트 주목 메커니즘 304  
 속성 변환하기 259  
 손실 169  
 손실 모델 203  
 손실 함수 72, 82, 117, 203, 275  
 수정된 포아송 방법 258  
 순방향 추론 264  
 순위 지정 212  
 순차 모델 81  
 순환 신경망 41  
 숨겨진 유닛 71  
 슈퍼 레이블 84  
 슈퍼 해상도 이미지 254  
 스칼라 요약 데이터 73  
 스케일-공간 개념 154

스케일 지터링 89  
 스킵 192  
 스킵 연결 198  
 스킵-그램 231  
 스타일 손실 269, 273  
 스타일 전송 273, 305  
 스타일 전송 알고리즘 251  
 스티칭 119  
 슬라이딩 윈도우 153, 155, 158  
 시각적 추천 시스템 213  
 시각적 특징 112  
 시각화의 계층 119  
 시간 정보 287  
 시그모이드 143  
 시그모이드 함수 29  
 시맨틱 분할 175  
 시맨틱 분할을 위한 알고리즘 180  
 시어링 100, 105  
 신경망 학습 36  
 신경 예술 스타일 전송 262  
 실제 사례 126  
 싱글 샷 다중 박스 검출기 163  
 쌍곡탄젠트 함수 30  
 쌍선형 보간법 198

## ㅇ

아이폰 330  
 아핀 변환 95  
 안드로이드 329  
 알고리즘 로컬라이제이션 153  
 압축 패턴 141  
 액션 인식 298  
 앵커 210

양방향 매핑 239  
 양의 얼굴 210  
 언더피팅 107  
 언어 모델 311  
 얼굴 간 유사도 222  
 얼굴 검출 214  
 얼굴 기술자 209  
 얼굴 인식 209, 220  
 얼굴 인식 프로세스 203  
 얼굴 클러스터링 209, 227  
 얼굴 키 포인트 215  
 얼굴 키 포인트 학습 217  
 얼굴 포즈 215  
 얼굴 표식 및 속성 215  
 얼굴 표식 추적 307  
 얼굴 확인 209  
 업샘플링 181  
 업컨볼루션 183  
 에지 검출 120  
 연결 건너뛰기 186  
 연속적 단어 가방 231  
 예술가 스타일 276  
 예술적 스타일 이전 방법 252  
 예측 시그니처 128  
 오류 복원 139  
 오버피팅 104, 107  
 오차역전파법 36  
 옥타브 이미지 123  
 옵티마이저 267  
 와일드 데이터셋 221  
 와일드 데이터셋 내의 포즈 306  
 완전 연결 레이어 33, 65, 71, 154  
 완전 컨볼루션 그물 155

완전 컨볼루션 네트워크 180  
 울트라-신경 분할 191  
 워드넷 계층 86  
 워핑 307  
 워핑 네트 308  
 원-핫 인코딩 33  
 위성 이미지 분할 196  
 유도된 역전파 117  
 유사도 136  
 유사도 학습 45, 203  
 유클리드 거리 136  
 융합 병렬 CNN 293  
 은닉 유닛 32  
 음의 얼굴 210  
 의료 이미지 진단 177  
 의류 모델 108  
 이미지 변환 282  
 이미지 분류 43, 61  
 이미지 생성 305  
 이미지 순위 238  
 이미지의 패치 311  
 이미지 캡셔닝 45  
 이미지 캡션 229, 250  
 이미지 캡션 처리 229  
 이중 선형 보간 122  
 인간 얼굴 분석 213  
 인공 신경망 32  
 인셉션-V3 90  
 인셉션 모델 133  
 인셉션 모델 파일 121  
 인스턴스 분할 175, 177, 200  
 인코더-디코더 방식 310  
 일괄 정규화 191, 316

일괄 정규화 레이어 280

임베딩 시각화 114

임베딩 훈련 233

## 지

자동 인코더 139, 145

자연어 처리 230, 250

자연어 처리 문제 229

자율주행용 보행자 검출 훈련 170

잔여 블록 93

잠재적 공간 240

장단기 메모리 42

장시간에 걸친 동영상 분류 295

재구성 혼합 방법 274

적대적 사례 125

적대적 손실 281

적합성 피드백 133

전이 학습 101

전치 컨볼루션 181

전치 컨볼루션 레이어 279

절단 정규 분포 171

정규화 101

정규화된 유클리드 거리 205

정확화 277

정확성 모니터링 169

정확하게 이미지 자르기 153

조건부 GAN 281

조건부 무작위 필드 235

중단 훈련 161

중단 간 훈련 가능 딥러닝 236

증황비 156

주성분 분석 84

중심와 스트림과 컨텍스트 스트림 294

지역 민감 해싱 132

## 초

차원 축소 112

채널 시프팅 101

초기 융합 295

초기화 방법 170

최근접 이웃 112

최대 풀링 40

최댓값 매치 113

최상위 모델 105

최적화 알고리즘 82

추론 313

추적 객체 312

축소된 차원 91

측정 항목 82

측정 항목 학습 227, 250

침입 검출 312

## 캐

캐글 97

캐글 키포인트 데이터셋 216

캡션 305

커널 39

컨볼루션 신경망 38, 184

컨볼루션 회귀 159

케라스 79

케라스 라이브러리 60

케라스 유틸리티 80

코너 검출 120

코사인 거리 205

콘텐츠 기반 이미지 검색 131

콘텐츠 손실 263

쿼리 확장 133

클리핑 144

## ㄷ

텍스트로 이미지 생성하기 256

텐서보드 57, 68, 74

텐서플로 66

텐서플로 라이브러리 53

텐서플로 서빙 133

텐서플로 서빙 도구 59

텐서플로 플레이그라운드 38

트리플렛 마이닝 210

트리플렛 손실 209, 212

특징 변환 96

특징 지도 301

## ㅍ

파스칼 VOC 챌린지 149

파이썬 51

파인 레이블 84

판별기 285

퍼셉트론 28

평가자 클래스 265

평균 풀링 264

페색 113

포즈 306

포즈 검출 157

포즈 추정 305

표현 학습 111

풀링 182

물링 레이어 40, 189, 293

프로토콜 버퍼 134, 164

플레이스홀더 55

플립핑 100, 105

피셔 벡터 301

픽셀 예측 176

## ㅎ

하드 주목 메커니즘 304

합성 이미지 125

해시 145

행동 인식 312

화이트닝 101

확대/축소 101, 105

확률적 경사하강법 37

확장 거리 187

확장 컨볼루션 183, 186

활성화 레이어 171, 265, 316

활성화 함수 29, 87

회귀 문제 156

회귀 방법 158

회귀 분석 157

회전 101

효율적 검색 137

훈련 데이터 생성 260

훈련된 모델 사용 129

훈련 오류 93

훈련 프로세스 77

휴먼 애노테이션 150

## A

activation layer 171, 316

adversarial loss 281

Affine transformation 95

AlexNet 87

ANN 32

ANNOY 137

Approximate nearest neighbour oh yeah 137

Artificial Neural Network 32



aspect ratio 156  
atrous convolution 181  
autoencoder 139  
AWS 318

## **B**

backpropagation 36  
batch normalization 35, 191, 316  
biometrics 204  
blending 258  
blob 160  
block\_conv1 265  
bottleneck feature 102  
bottleneck layer 206  
box merging 159

## **C**

CASIA 웹 얼굴 데이터베이스 221  
CBIR 145  
CBOW 231  
CelebA 221  
CelebFaces Attributes dataset 221  
channel shifting 101  
CIFAR 데이터셋 83  
classification signature 128  
clipping 144  
close cropping 153  
CNN 38  
CNN2RNN 250  
coarseness 186  
COCO 객체 검출 챌린지 150  
COCO 데이터셋 163  
Conditional Random Field 187, 235  
Continuous Bag Of Words 231

contrastive loss 205  
contrastive\_loss 205  
convolutional\_layers 277  
Convolutional Neural Network 38  
convolutional regression 159  
convolution\_layer 155  
CoreML 모델 330  
CRF 187, 235  
CUDA 49  
CUDA 8 318  
CUDNN 50

## **D**

data augmentation 100  
deconvolution 181, 183  
DeepDream 120  
DeepLab 187  
DeepLab v3 191  
DeepNet 모델 211  
DeepRank 211  
dense layer 155  
dense\_layer 142  
DenseNet 모델 95  
dilated convolution 183  
dilation distance 187  
dimensionality reduction 112  
dropout 34

## **F**

face clustering 209  
face descriptor 209  
FaceNet 208, 227  
face pose 215  
face recognition 209, 220

face verification 209  
facial key point 215  
Fashion-MNIST 데이터셋 84  
Faster R-CNN 162, 242  
Fast R-CNN 161  
FCN 180  
FCN 기반 모델 185  
FCN 레이어 199  
FCN 모델링 197  
fiducial point 215  
fine label 84  
fit\_generator 메소드 99  
FLIC 306  
flipping 100  
forward inference 264  
fractionally strided convolution 183  
Frames Labeled In Cinema 306  
Fully Convolutional Network 180

## G

GAN 277  
gan\_train\_ops 메소드 280  
gan\_train 메소드 280  
GAN 기반 VDM 284  
GAN 손실 기울기 값 282  
GCP 324, 327  
generative model 251  
GLOVE 250  
Google Cloud Storage 327  
gradient descent 36

## H

hSpy 52  
hidden unit 71

HMDB-51 291  
Hollywood2 291  
host 130

## I

ImageDataGenerator 99  
ImageNet 86  
ImageNet 데이터셋 85, 121, 149  
inception model 133  
Inception-V3 90  
InfoGAN 283  
infogan\_model 283  
initializer 170  
inpainting 257  
Intersection over Union 151  
IoU 151  
ISPRS 196

## J

Jetson TX2 328

## K

kaggle 97  
kernel 39  
K-평균 227

## L

L1 및 L2 정규화 35  
LabelMe 모바일 애플리케이션 179  
leaky activation 277  
leaky\_relu 171  
Locality Sensitive Hashing 132  
Local Response Normalization 88  
Long Short-Term Memory 42  
Long-term recurrent convolutional architecture 242

loss 143  
loss function 203  
LRCN 242  
LRN 88  
LSH 132  
LSTM 42, 235, 242, 309  
LSTM 네트워크 296  
LSTM 셀 구조 42

## M

MAFL 데이터셋 216  
mAP 151  
Mean Precision Average 151  
MNIST 데이터셋 62, 83  
MNIST 모델 79  
Mobilenet 166  
MobileNets 315  
model\_version 127  
modified Poisson method 258  
MPII-MD 309  
MSR-VTT 309  
MTFL 데이터셋 215  
multi-spline method 258  
M-VAD 309

## N

Natural Language Processing 230  
nearby word 231  
nearest neighbour 112  
negative face 210  
neural artistic style transfer 262  
NLP 230  
numpy 52

## O

objective 함수 123  
occlusion 113  
OpenCV 53  
optical flow 298  
optimise 143

## P

PCA 84  
perceptron 28  
Pet 데이터셋 167  
pillow 52  
placeholder 55  
pooling layer 293  
port 130  
pose estimation 305  
positive face 210  
prediction signature 128  
Principal Component Analysis 84  
protobul 164  
Protocol Buffers 134, 164  
PSPnet 189  
Python 51

## R

random cropping 100  
R-CNN 160, 173  
R-CNN 알고리즘 200  
real\_images 281  
Recurrent Neural Network 41  
ReliNet 188  
Region of Interest pooling 161  
regularization 277  
ReLU 87

ReLU 함수 31  
resize\_image 함수 122  
ResNet 모델 198  
resultant model 314  
RNN 41, 236  
RoI 161

## S

Scalar 페이지 76  
scale jittering 89  
scikit-learn 52  
scipy 52  
segmentation 45  
SegNet 아키텍처 181  
selective search 162  
semantic segmentation 175  
SGD 37, 88  
shearing 100  
similarity learning 203  
simple\_cnn 모델 98  
skip-gram 231  
slow fusion 295  
softmax 34  
spatial invariance 95  
Sports-1M 291  
SqueezeNet 모델 94  
SSD 모델 166, 173  
stitching 119  
Stochastic Gradient Descent 37  
strided convolution 183  
super label 84

## T

tanh 환성화 142

TensorBoard 57  
TensorFlow playground 38  
TensorFlow Serving 133  
TensorRT 328  
TFcoreML 330  
transfer learning 101  
transpose\_convolution\_layer 278  
transpose convolution 181  
t-SNE 141

## U

UCF-11 291  
UCF101 288  
UNET 모델 192  
up-convolution 183  
upsampling 181

## V

VDM 284, 286  
VGG 91  
VGG-16 모델 88  
VGFace2 데이터셋 222  
VGG 모델 103, 264  
VM 318

## W

whitening 101  
word2vec 모델 250

## Y

YOLO 객체 검출 알고리즘 170  
YOLO 네트워크 172  
YOLO 알고리즘 147  
YouTube-8M 290

## 기호

\_callback 메소드 130

## 숫자

2D 광학 흐름 302

2D 컨볼루션 299, 302

2D 컨볼루션 기능 310

3D 광학 흐름 302

3D 모델 생성 262

3D 컨볼루션 287, 292

3D 컨볼루션 기능 302



에이콘출판의 기쁨을 마련하신 故 정완재 선생님 (1935-2004)

## 컴퓨터 비전과 딥러닝

텐서플로와 케라스를 사용한 전문 가이드

---

발행 | 2018년 8월 31일

지은이 | 라자링가파 산우가마니

옮긴이 | 테크 트랜스 그룹 T4

펴낸이 | 권성준

편집장 | 황영주

편집 | 조유나

디자인 | 박주란

에이콘출판주식회사

서울특별시 양천구 국회대로 287 (목동)

전화 02-2653-7600, 팩스 02-2653-0433

[www.acornpub.co.kr](http://www.acornpub.co.kr) / [editor@acornpub.co.kr](mailto:editor@acornpub.co.kr)

한국어판 © 에이콘출판주식회사, 2018, Printed in Korea.

ISBN 979-11-6175-200-6

ISBN 978-89-6077-210-6 (세트)

<http://www.acornpub.co.kr/book/deep-learning-computer-vision>

이 도서의 국립중앙도서관 출판시도서목록(CIP)은 서지정보유통지원시스템 홈페이지(<http://seoji.nl.go.kr>)와  
국가자료공동목록시스템(<http://www.nl.go.kr/kolisnet>)에서 이용하실 수 있습니다. (CIP제어번호: CIP2018026544)

책값은 뒷표지에 있습니다.



## acorn+PACKT Technical Book 시리즈

서버리스 웹 애플리케이션 구축  
머신 러닝과 통계  
실전 예제로 배우는 AWS Lambda  
유니티로 배우는 안드로이드 게임 개발  
리눅스에서 하는 스윙프트 개발  
Node.js로 봇 만들기  
IPython 6.0  
R로 마스터하는 머신 러닝 2/e  
유니티 2D 디펜스 게임은 이렇게 만든다  
타입스크립트 마스터 2/e  
React 16 핵심 정리 2/e  
모던 C++로 배우는 함수형 프로그래밍  
Pandas Cookbook  
쿠버네티스 마스터  
파이썬 GUI 프로그래밍 콰북 2/e  
엔터프라이즈 데이터 레이크 구축  
디지털 포렌식과 사고 대응  
아파치 카프카로 데이터 스트리밍 애플리케이션 제작  
AWS를 이용한 데브옵스 완벽 구축  
젠킨스 마스터  
사물인터넷 자바스크립트 프로그래밍  
딥러닝 데이터 전처리 입문  
안드로이드 스튜디오 3 통달  
파이썬 네트워크 마스터  
움직이는 증강 현실 게임 개발  
Scikit-learn으로 머신 러닝 마스터 2/e  
코틀린을 이용한 안드로이드 개발  
일렉스틱 스택 6 입문  
Akka 콰북  
R 딥러닝 콰북  
GNU/Linux 패속 임베디드 프로그래밍  
유니티 2017 게임 개발 에센셜 3/e  
쿠버네티스 기초 다지기 2/e  
파이썬 자연어 처리의 이론과 실제  
Neo4j로 시작하는 그래프 데이터베이스 2/e  
대용량 머신 러닝과 스파크  
유니티 3D RPG 게임은 이렇게 만든다  
텐서플로 1.x로 배우는 머신 러닝  
금융공학으로 R 마스터하기  
Go 언어를 활용한 마이크로서비스 개발  
홀로렌즈 블루프린트  
동형 자바스크립트 웹 개발  
쿠버네티스 마스터 2/e  
스포링 5.0 마스터  
트러블슈팅 도커  
Windows Server 2016 Hyper-V 콰북 2/e  
실전 게임 기초 AI 프로그래밍  
JVM 언어 입문  
Yocto 프로젝트를 활용한 임베디드 리눅스 개발 2/e  
컴퓨터 버전과 딥러닝